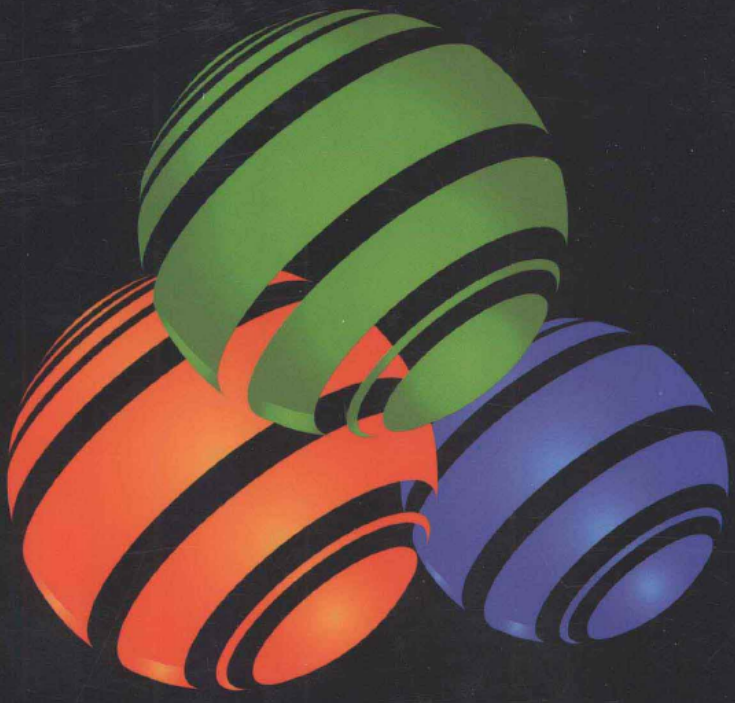
HZ BOOKS
华章科技

CUDA开发者社区技术总监亲自撰写，英伟达中国首批CUDA官方认证工程师翻译，译著双馨

全面、详实地讲解了CUDA并行程序设计的技术知识点和编程方法，包含大量实用代码示例，是目前学习CUDA编程最权威的著作之一

ELSEVIER
爱思唯尔

高性能计算系列丛书



CUDA Programming
A Developer's Guide to Parallel Computing with GPUs

CUDA并行程序设计
GPU编程指南

（美）Shane Cook 著
苏统华 李东 李松泽 魏通 译 马培军 审校

机械工业出版社
China Machine Press

CUDA并行程序设计
GPU编程指南

CUDA Programming
A Developer's Guide to Parallel Computing with GPUs

本书旨在为读者学习CUDA打下坚实基础。涵盖如何理解串行程序并把它转化到单指令流多数据流编程模型下，以及如何基于CUDA实现高效的并行程序。本书除了提供多个实例，还深入讨论了GPU程序的优化，同时对共享内存、内存预读和线程的使用均有涉及。

—— Nagarajan Kandasamy博士，德雷塞尔大学电子与计算机工程系副教授

CUDA是一种专门为提高并行程序开发效率而设计的计算架构。在构建高性能应用程序时，CUDA架构可充分发挥GPU的强大计算功能，颇受广大开发者拥趸。本书以并行编程实践者视角，展示了全面、快速提升CUDA程序效能的途径。从并行机制到CUDA开发环境搭建，从GPU高性能计算相关硬件知识到并行计算和CUDA编程技巧，从核心概念到多个热点技术和高级主题，无所不包。本书侧重于CUDA实践应用，特别在CUDA程序优化上下足了工夫，例如发挥多GPU效力。难能可贵的是，作者结合多年实践经验，精心总结出开发者在使用CUDA过程中易犯的错误和有效的规避方法。

本书亮点：

- 全面介绍并行机制和CUDA编程，即便没有CUDA编程经验也能够轻松掌握；
- 细致指导，以帮助读者优化CUDA应用程序；
- 以实用技术演示如何在并行程序中管理内存、线程、算法和各种资源；
- 涵盖英伟达硬件上三大操作系统的CUDA开发；
- 每章附有练习，方便检验读者的掌握程度或为课堂提供讨论要点。

本书译自原版CUDA Programming: A Developer's Guide to Parallel Computing with GPUs并由Elsevier授权出版

ELSEVIER 爱思唯尔

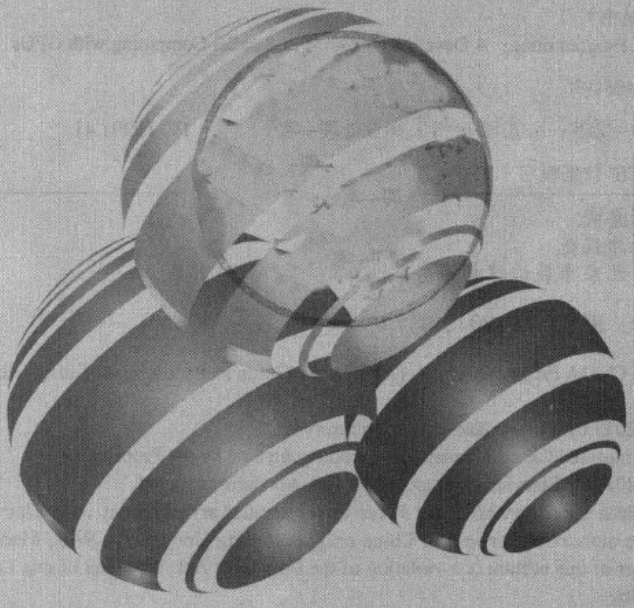
ELSEVIER

投稿热线：(010) 88379604
客服热线：(010) 88378991 88361066
购书热线：(010) 68326294 88379649 68995259

华章网站：www.hzbook.com
网上购书：www.china-pub.com
数字阅读：www.hzmedia.com.cn

上架指导：计算机/程序设计
ISBN 978-7-111-44861-7
9 787111 448617 >
定价：99.00元

算系列丛书



CUDA Programming
A Developer's Guide to Parallel Computing with GPUs

CUDA并行程序设计

GPU编程指南

（美）Shane Cook 著
苏统华 李东 李松泽 魏通 译 马培军 审校

机械工业出版社
China Machine Press

图书在版编目（CIP）数据

CUDA 并行程序设计：GPU 编程指南 /（美）库克（Cook, S.）著；苏统华等译；马培军审校 . —北京：机械工业出版社，2014.1
（高性能计算系列丛书）
书名原文：CUDA Programming：A Developer's Guide to Parallel Computing with GPUs

ISBN 978-7-111-44861-7

I. C… II. ①库… ②苏… ③马… III. 图像处理—程序设计 IV. TP391.41

中国版本图书馆 CIP 数据核字（2013）第 276497 号

版权所有 · 侵权必究
封底无防伪标均为盗版
本书法律顾问 北京市展达律师事务所

本书版权登记号：图字：01-2013-0171

Shane Cook : CUDA Programming: A Developer's Guide to Parallel Computing with GPUs (ISBN 978-0-12-415933-4).

Copyright © 2013 by Elsevier Inc. All rights reserved.

Authorized Simplified Chinese translation edition published by the Proprietor.

Copyright © 2014 by Elsevier (Singapore) Pte Ltd. All rights reserved.

Printed in China by China Machine Press under special arrangement with Elsevier (Singapore) Pte Ltd. This edition is authorized for sale in China only, excluding Hong Kong SAR, Macau SAR and Taiwan. Unauthorized export of this edition is a violation of the Copyright Act. Violation of this Law is subject to Civil and Criminal Penalties.

本书简体中文版由 Elsevier(Singapore)Pte Ltd. 授权机械工业出版社在中国大陆境内独家出版和发行。本版仅限在中国境内（不包括香港特别行政区、澳门特别行政区及台湾地区）出版及标价销售。未经许可之出口，视为违反著作权法，将受法律之制裁。

本书封底贴有 Elsevier 防伪标签，无标签者不得销售。

本书是 CUDA 并行程序设计领域最全面、最详实和最具权威性的著作之一，由 CUDA 开发者社区技术总监亲自撰写，英伟达中国首批 CUDA 官方认证工程师翻译，详实地讲解了 CUDA 并行程序设计的技术知识点（平台、架构、硬件知识、开发工具和热点技术）和编程方法，包含大量实用代码示例，实践性非常强。

全书共分为 12 章。第 1 章从宏观上介绍流处理器演变历史。第 2 章详解 GPU 并行机制，深入理解串行与并行程序，以辩证地求解问题。第 3 章讲解 CUDA 设备及相关的硬件和体系结构，以实现最优 CUDA 程序性能。第 4 章介绍 CUDA 开发环境搭建和可用调试环境。第 5 章介绍与 CUDA 编程紧密相关的核心概念——网格、线程块与线程，并通过示例说明线程模型与性能的关系。第 6 章借助实例详细讲解了不同类型内存的工作机制，并指出实践中容易出现的误区。第 7 章细述多任务的 CPU 和 GPU 协同，并介绍多个 CPU/GPU 编程秘技。第 8 章介绍如何在应用程序中编写和使用多 GPU。第 9 章详述 CUDA 编程性能限制因素、分析 CUDA 代码的工具和技术。第 10 章介绍编程实践中的库与软件开发工具包。第 11 章讲解如何设计基于 GPU 的系统。第 12 章总结 CUDA 应用中易犯错误以及应对建议。

机械工业出版社（北京市西城区百万庄大街 22 号 邮政编码 100037）
责任编辑：肖晓慧
藁城市京瑞印刷有限公司印刷
2014 年 1 月第 1 版第 1 次印刷
186mm × 240mm • 33.5 印张
标准书号：ISBN 978-7-111-44861-7
定 价：99.00 元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换
客服热线：（010）88378991 88361066 投稿热线：（010）88379604
购书热线：（010）68326294 88379649 68995259 读者信箱：hzjsj@hzbook.com

致中国读者

获悉本书将译为中文，传播于华夏大地，我欣喜不已。中国已跃居高性能计算的超级大国行列。中国的经济发展与GPU市场的发展有些相似：它没有被过去的束缚所羁绊，而是开启了一个新世界。中国建造高性能计算机的实力，足以问鼎美国大型计算机公司。中国的高性能计算明显优于其他对手的原因在于大量采用了GPU技术。GPU技术在中国广泛传播，主要得益于英伟达硬件和CUDA语言，而后者正是本书竭诚呈现的主题。

与很多国家一样，中国的计算领域也经历了变革。陈旧的串行编程模型正在被抛弃，计算领域正在拥抱并行体系结构。GPU拥有大规模并行的处理器核心，这是这一变革的重要组成部分。CPU领域的编程模型必然发生变化，但仅是微小的调整，以便利用GPU加速器的潜力。GPU目前应用到了各行各业。不论是学生还是专业人士，通过阅读本书均会收获良多。本书将帮助你在GPU和大规模并行处理器编程方面一路前行。

Shane Cook

译 者 序

我们正在由单核时代进入多核时代和众核时代。在单核时代，软件行业一直享用着“免费的午餐”。受益于CPU主频的指数级提速，开发软件无须任何代码修改，只要换上新一代的处理器，即可获得性能上的飞速提升。随着汹涌而来的众核时代，这里已经“不再有免费午餐”[⊖]。随着计算架构的不断演进，编程模型也发生着深刻的变化。计算机软件行业面临着最大的变迁问题——从串行、单线程的问题求解方式切换到大规模线程同时执行的问题求解方式。而CUDA提供了非常优秀的可扩展架构，以支持这种大规模并行程序设计需求。

本书是一本很出众的CUDA书籍，内容全面而又不落窠臼。全书可以分成四个部分。第一部分为背景篇，包括前4章。其中前两章简述流处理器历史和并行计算基本原理，第3～4章分别介绍了CUDA的硬件架构与计算能力和软件开发配置。第二部分为CUDA基本篇，包括第5～7章。第5、6章依次介绍了CUDA线程抽象模型和内存抽象模型，在此过程中，紧密结合直方图统计实例和样本排序实例进行讨论。为了更好地增进读者的实践经验，第7章全方位剖析了AES加密算法的CUDA实现过程。第三部分为CUDA扩展篇，包括第8～10章。其中第8、9章面向优化执行性能，而第10章为提升开发生产效率。第8章从充分利用多个硬件设备的角度，讲述了流的使用。相反的，第9章从程序优化角度，给出了CUDA性能调优的全方位指导。第10章介绍了一些常用的函数库和CUDA开发包中提供的优质SDK，为大型软件的快速发布提供了支持。第四部分为CUDA经验篇，包括最后的两章。这两章分别针对硬件系统搭建和软件生产过程中的共性问题提供建议，是作者多年CUDA开发经验的总结。

本书特色鲜明。作者在介绍CUDA时，仿佛在跟朋友聊天论道，谈论家常，讲着故事，娓娓道来。论到关键之处，却又语重心长，体贴备至。在不知不觉中，把CUDA的魅力展示得淋漓尽致，同时把CUDA程序设计的功力传授于你。

本书的翻译工作经过精心的组织，整个过程得到大批专业人士的帮助。在交付出版社之前，译者团队经过了全书讨论、初译、初核、再译、再核、审校等六个环节。很荣幸地邀请到在哈尔滨工

⊖ Herb Sutter在2005年发表的论文“The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software”中对这一问题作了前瞻性讨论。——译者注

业大学长年承担“并行程序设计”和“计算机体系结构”等课程教学工作的李东教授，加盟本书的翻译团队。李东教授翻译了第 1 ~ 3 章，对保证本书的翻译质量起到了重要作用。本书第 5 ~ 7 章的初译以及第 9 章部分小节的初译和再译由李松泽负责。本书第 10 章和第 12 章的初译以及第 9 章部分小节的初译和再译由魏通负责。苏统华负责了本书前言、第 4 章、第 8 章、第 9 章部分小节和第 11 章的初译和再译任务，除此之外，他还负责了全书的初核和再核任务。特别感谢哈尔滨工业大学软件学院院长马培军教授，他应邀审校了全部译文，提出了很多中肯的改进意见。本书在交到出版社之后，又得到了机械工业出版社编辑团队的大力帮助，他们的工作专业而细致，让人钦佩。另外还要感谢哈尔滨工业大学软件学院 2012 级数字媒体方向的硕士研究生，参与了部分内容的初译，特别是王烁行同学做了不少工作。如果没有这么多人的辛勤奉献，这本中译本很难如期呈现。另外，国家自然科学基金（资助号：61203260）对本书的翻译提供了部分资助，哈尔滨工业大学创新实验课《CUDA 高性能并行程序设计》也对本书的翻译提供了大力支持。

由于本书涉及面广，很多术语较新，目前尚无固定译法，翻译难度很大。有时，为一个术语选择一个恰当的中文译法，译者经常反复推敲、讨论。但由于译者水平有限，译文中难免存在一些问题，真诚地希望读者朋友们将您的意见发往 cudabook@gmail.com。

苏统华

哈尔滨工业大学软件学院

前　言

过去的五年中，计算领域目睹了英伟达（NVIDIA）公司带来的变革。随后的几年，英伟达公司异军突起，逐渐成长为最知名的游戏硬件制造商之一。计算统一设备架构（Compute Unified Device Architecture，CUDA）编程语言的引入，第一次使这些非常强大的图形协处理器为 C 程序员日常所用，以应对日益复杂的计算工作。从嵌入式设备行业到家庭用户，再到超级计算机，所有的一切都因此而改变。

计算机软件界最大的变迁是从串行编程转向并行编程。其中，CUDA 起到了重要的作用。究其本质，图形处理单元（Graphics Processor Unit，GPU）是为高速图形处理而设计的，它具有天然的并行性。CUDA 采用一种简单的数据并行模型，再结合编程模型，从而无须操纵复杂的图形基元。

实际上，CUDA 与之前的架构不同。它不要求程序员对图形或者图形基元有所了解，也不用程序员有任何这方面的知识。你也不一定要成为游戏开发人员。CUDA 语言使得 GPU 看起来与别的可编程设备一样。

本书并不假定读者有 CUDA 或者并行编程的任何经验，仅假定读者有一定的 C/C++ 语言编程知识。随着本书的不断深入，读者将越来越胜任 CUDA 的编程工作。本书包含更高级的主题，帮助你从不知晓并行编程的程序员成长为能够全方位发掘 CUDA 潜力的专家。

对已经熟悉并行编程概念和 CUDA 的程序员来说，本书包含丰富的学习资料。专设章节详细讨论 GPU 的架构，包括最新的费米（Fermi）和开普勒（Kepler）硬件，以及如何将它们的效能发挥到极致。任何可以编写 C 或 C++ 程序的程序员都可以在经过几个小时的简单训练后编写 CUDA 程序。通过对本书的完整学习，你将从仅能得到数倍程序加速的 CUDA 编程新手成长为能得到数十倍程序加速的高手。

本书特别针对 CUDA 学习者而写。在保证程序正确性的前提下，侧重于程序性能的调优。本书将大大扩展你的技能水平和对编写高性能代码的认识，特别是 GPU 方面。

本书是实践者在实际应用程序中使用 CUDA 编程的实用指南。同时我们将提供所需的理论知识和背景介绍。因此，任何人（不管有无基础）都可以使用本书，从中学习如何进行 CUDA 编程。综上，本书是专业人士和 GPU 或并行编程学习者的理想之选。

本书编排如下：

第 1 章 从宏观上介绍流处理器（streaming processor）的演变历史，涉及几个重要的发展历程，正是它们把我们带入今天的 GPU 处理世界。

第 2 章 介绍并行编程的概念。例如，串行与并行程序的区别，以及如何采用不同的策略寻找求解问题之道。本章意在为既有串行程序员建立一个基本的认识，这里的概念将在后面进一步展开。

第 3 章 详尽地讲解 CUDA 设备及与其紧密相关的硬件和架构。为了编写最优性能的 CUDA 程序，适当了解设备硬件的相关知识是必要的。

第 4 章 介绍了如何在 Windows、Mac 和 Linux 等不同操作系统上安装和配置 CUDA 软件开发工具包，另外介绍可用于 CUDA 的主要调试环境。

第 5 章 介绍 CUDA 线程模型，并通过一些示例来说明线程模型是如何影响程序性能的。

第 6 章 我们需要了解不同的内存类型，它们在 CUDA 中的使用方式是影响性能的最大因素。本章借助实例详细讲解了不同类型内存的工作机制，并指出实践中容易出现的误区。

第 7 章 主要详述了如何在若干任务中恰当地协同 CPU 和 GPU，并讨论了几个有关 CPU/GPU 编程的议题。

第 8 章 介绍如何在应用程序中编写和使用多 GPU。

第 9 章 对 CUDA 编程中限制性能的主要因素予以详解，考察可以用来分析 CUDA 代码的工具和技术。

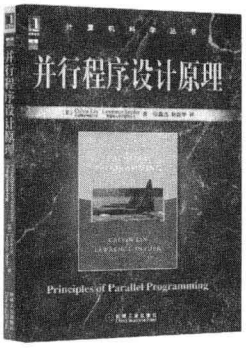
第 10 章 介绍了 CUDA 软件开发工具包的示例和 CUDA 提供的库文件，并介绍如何在应用程序中使用它们。

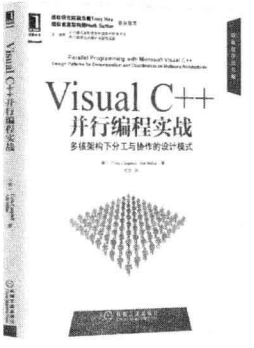
第 11 章 关注构建自己的 GPU 服务器或者 GPU 集群时的几个相关议题。

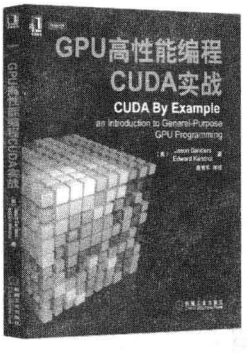
第 12 章 检视多数程序员在开发 CUDA 应用程序时易犯的错误类型，并对如何检测和避免这些错误给出了建议。

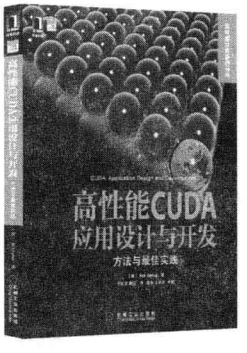
推荐阅读

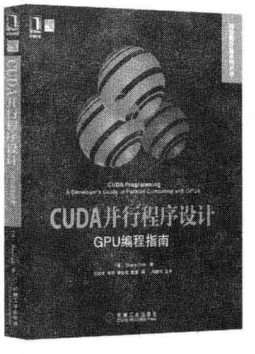
并行程序设计导论
An Introduction to Parallel Programming


并行程序设计原理
Principles of Parallel Programming

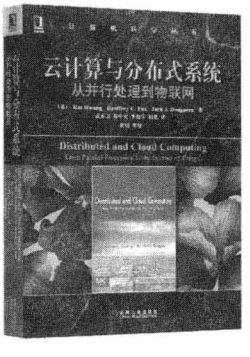

Visual C++
并行编程实战
多核架构下分工与协作的设计模式


GPU高性能编程
CUDA实战
CUDA By Example
an Introduction to General-Purpose
GPU Programming


高性能CUDA
应用设计与开发
方法与最佳实践

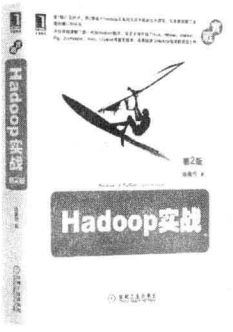

CUDA并行程序设计
GPU编程指南

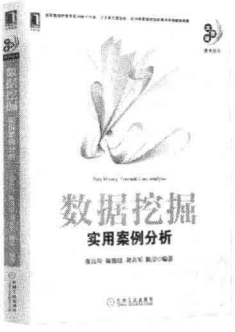
OpenCL编程指南

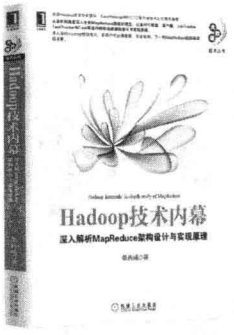

云计算与分布式系统
从并行处理到物联网
Distributed and Cloud Computing

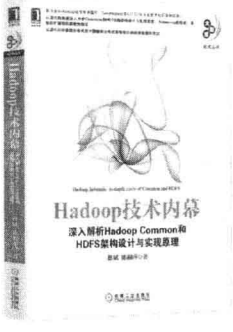
大规模分布式存储系统
原理解析与架构实战

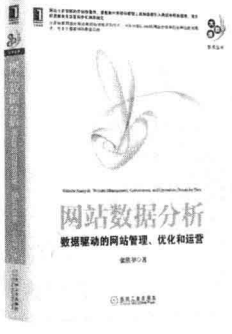
推荐阅读











Hadoop实战（第2版）
作者：陆嘉恒
ISBN：978-7-111-39583-6
定价：79.00元

数据挖掘：实用案例分析
作者：张良均
ISBN：978-7-111-42591-5
定价：79.00元

Hadoop技术内幕：
深入解析MapReduce架构设计与实现原理
作者：董西成
ISBN：978-7-111-42226-6
定价：69.00元

Hadoop技术内幕：
深入解析Hadoop Common和HDFS架构设计与实现原理
作者：蔡斌 等
ISBN：978-7-111-41766-8
定价：89.00元

数据挖掘与数据化运营实战：
思路、方法、技巧与应用
作者：卢辉
ISBN：978-7-111-42650-9
定价：59.00元

网站数据分析：
数据驱动的网站管理、优化和运营
作者：张洪举
ISBN：978-7-111-43514-3
定价：69.00元

目　录

致中国读者

译者序

前　言

第 1 章　超级计算简史 …… 1

1.1　简介 …… 1
1.2　冯·诺依曼计算机架构 …… 2
1.3　克雷 …… 4
1.4　连接机 …… 5
1.5　Cell 处理器 …… 6
1.6　多点计算 …… 8
1.7　早期的 GPGPU 编程 …… 10
1.8　单核解决方案的消亡 …… 11
1.9　英伟达和 CUDA …… 12
1.10　GPU 硬件 …… 13
1.11　CUDA 的替代选择 …… 15
1.11.1　OpenCL …… 15
1.11.2　DirectCompute …… 16
1.11.3　CPU 的替代选择 …… 16
1.11.4　编译指令和库 …… 17
1.12　本章小结 …… 18

第 2 章　使用 GPU 理解并行计算 …… 19

2.1　简介 …… 19
2.2　传统的串行代码 …… 19
2.3　串行 / 并行问题 …… 21
2.4　并发性 …… 22
2.5　并行处理的类型 …… 25
2.5.1　基于任务的并行处理 …… 25
2.5.2　基于数据的并行处理 …… 27
2.6　弗林分类法 …… 29
2.7　常用的并行模式 …… 30
2.7.1　基于循环的模式 …… 30
2.7.2　派生 / 汇集模式 …… 31
2.7.3　分条 / 分块 …… 33
2.7.4　分而治之 …… 34
2.8　本章小结 …… 34

第 3 章　CUDA 硬件概述 …… 35

3.1　PC 架构 …… 35
3.2　GPU 硬件结构 …… 39
3.3　CPU 与 GPU …… 41
3.4　GPU 计算能力 …… 42
3.4.1　计算能力 1.0 …… 42
3.4.2　计算能力 1.1 …… 43
3.4.3　计算能力 1.2 …… 44
3.4.4　计算能力 1.3 …… 44

3.4.5 计算能力 2.0 …… 44
3.4.6 计算能力 2.1 …… 46

第 4 章 CUDA 环境搭建 …… 48

4.1 简介 …… 48
4.2 在 Windows 下安装软件开发工具包 … 48
4.3 Visual Studio …… 49
4.3.1 工程 …… 49
4.3.2 64 位用户 …… 49
4.3.3 创建工程 …… 51
4.4 Linux …… 52
4.5 Mac …… 55
4.6 安装调试器 …… 56
4.7 编译模型 …… 58
4.8 错误处理 …… 59
4.9 本章小结 …… 60

第 5 章 线程网格、线程块以及线程 … 61

5.1 简介 …… 61
5.2 线程 …… 61
5.2.1 问题分解 …… 62
5.2.2 CPU 与 GPU 的不同 …… 63
5.2.3 任务执行模式 …… 64
5.2.4 GPU 线程 …… 64
5.2.5 硬件初窥 …… 66
5.2.6 CUDA 内核 …… 69
5.3 线程块 …… 70
5.4 线程网格 …… 74
5.4.1 跨幅与偏移 …… 76
5.4.2 X 与 Y 方向的线程索引 …… 77
5.5 线程束 …… 83
5.5.1 分支 …… 83
5.5.2 GPU 的利用率 …… 85
5.6 线程块的调度 …… 88
5.7 一个实例——统计直方图 …… 89
5.8 本章小结 …… 96

第 6 章 CUDA 内存处理 …… 99

6.1 简介 …… 99
6.2 高速缓存 …… 100
6.3 寄存器的用法 …… 103
6.4 共享内存 …… 112
6.4.1 使用共享内存排序 …… 113
6.4.2 基数排序 …… 117
6.4.3 合并列表 …… 123
6.4.4 并行合并 …… 128
6.4.5 并行归约 …… 131
6.4.6 混合算法 …… 134
6.4.7 不同 GPU 上的共享内存 … 138
6.4.8 共享内存小结 …… 139
6.5 常量内存 …… 140
6.5.1 常量内存高速缓存 …… 140
6.5.2 常量内存广播机制 …… 142
6.5.3 运行时进行常量内存更新 … 152
6.6 全局内存 …… 157
6.6.1 记分牌 …… 165
6.6.2 全局内存排序 …… 165
6.6.3 样本排序 …… 168
6.7 纹理内存 …… 188
6.7.1 纹理缓存 …… 188

6.7.2 基于硬件的内存获取操作 … 189
6.7.3 使用纹理的限制 … 190
6.8 本章小结 … 190

第 7 章 CUDA 实践之道 … 191

7.1 简介 … 191
7.2 串行编码与并行编码 … 191
7.2.1 CPU 与 GPU 的设计目标 … 191
7.2.2 CPU 与 GPU 上的最佳算法对比 … 194
7.3 数据集处理 … 197
7.4 性能分析 … 206
7.5 一个使用 AES 的示例 … 218
7.5.1 算法 … 219
7.5.2 AES 的串行实现 … 223
7.5.3 初始内核函数 … 224
7.5.4 内核函数性能 … 229
7.5.5 传输性能 … 233
7.5.6 单个执行流版本 … 234
7.5.7 如何与 CPU 比较 … 235
7.5.8 考虑在其他 GPU 上运行 … 244
7.5.9 使用多个流 … 248
7.5.10 AES 总结 … 249
7.6 本章小结 … 249

第 8 章 多 CPU 和多 GPU 解决方案 … 252

8.1 简介 … 252
8.2 局部性 … 252
8.3 多 CPU 系统 … 252
8.4 多 GPU 系统 … 253
8.5 多 GPU 算法 … 254
8.6 按需选用 GPU … 255
8.7 单节点系统 … 258
8.8 流 … 259
8.9 多节点系统 … 273
8.10 本章小结 … 284

第 9 章 应用程序性能优化 … 286

9.1 策略 1：并行 / 串行在 GPU/CPU 上的问题分解 … 286
9.1.1 分析问题 … 286
9.1.2 时间 … 286
9.1.3 问题分解 … 288
9.1.4 依赖性 … 289
9.1.5 数据集大小 … 292
9.1.6 分辨率 … 293
9.1.7 识别瓶颈 … 294
9.1.8 CPU 和 GPU 的任务分组 … 297
9.1.9 本节小结 … 299
9.2 策略 2：内存因素 … 299
9.2.1 内存带宽 … 299
9.2.2 限制的来源 … 300
9.2.3 内存组织 … 302
9.2.4 内存访问以计算比率 … 303
9.2.5 循环融合和内核融合 … 308
9.2.6 共享内存和高速缓存的使用 … 309
9.2.7 本节小结 … 311
9.3 策略 3：传输 … 311
9.3.1 锁页内存 … 311
9.3.2 零复制内存 … 315
9.3.3 带宽限制 … 322

9.3.4 GPU 计时 …… 327
9.3.5 重叠 GPU 传输 …… 330
9.3.6 本节小结 …… 334
9.4 策略 4：线程使用、计算和分支 …… 335
9.4.1 线程内存模式 …… 335
9.4.2 非活动线程 …… 337
9.4.3 算术运算密度 …… 338
9.4.4 一些常见的编译器优化 …… 342
9.4.5 分支 …… 347
9.4.6 理解底层汇编代码 …… 351
9.4.7 寄存器的使用 …… 355
9.4.8 本节小结 …… 357
9.5 策略 5：算法 …… 357
9.5.1 排序 …… 358
9.5.2 归约 …… 363
9.5.3 本节小结 …… 384
9.6 策略 6：资源竞争 …… 384
9.6.1 识别瓶颈 …… 384
9.6.2 解析瓶颈 …… 396
9.6.3 本节小结 …… 403
9.7 策略 7：自调优应用程序 …… 403
9.7.1 识别硬件 …… 404
9.7.2 设备的利用 …… 405
9.7.3 性能采样 …… 407
9.7.4 本节小结 …… 407
9.8 本章小结 …… 408

第 10 章 函数库和 SDK …… 410

10.1 简介 …… 410
10.2 函数库 …… 410
10.2.1 函数库通用规范 …… 411
10.2.2 NPP …… 411
10.2.3 Thrust …… 419
10.2.4 CuRAND …… 434
10.2.5 CuBLAS 库 …… 438
10.3 CUDA 运算 SDK …… 442
10.3.1 设备查询 …… 443
10.3.2 带宽测试 …… 445
10.3.3 SimpleP2P …… 446
10.3.4 asyncAPI 和 cudaOpenMP … 448
10.3.5 对齐类型 …… 455
10.4 基于指令的编程 …… 457
10.5 编写自己的内核 …… 464
10.6 本章小结 …… 466

第 11 章 规划 GPU 硬件系统 …… 467

11.1 简介 …… 467
11.2 CPU 处理器 …… 469
11.3 GPU 设备 …… 470
11.3.1 大容量内存的支持 …… 471
11.3.2 ECC 内存的支持 …… 471
11.3.3 Tesla 计算集群驱动程序 … 471
11.3.4 更高双精度数学运算 …… 472
11.3.5 大内存总线带宽 …… 472
11.3.6 系统管理中断 …… 472
11.3.7 状态指示灯 …… 472
11.4 PCI-E 总线 …… 472
11.5 GeForce 板卡 …… 473
11.6 CPU 内存 …… 474
11.7 风冷 …… 475
11.8 液冷 …… 477
11.9 机箱与主板 …… 479

11.10 大容量存储 …… 481
11.10.1 主板上的输入/输出接口 … 481
11.10.2 专用 RAID 控制器 …… 481
11.10.3 HDSL …… 483
11.10.4 大容量存储需求 …… 483
11.10.5 联网 …… 483
11.11 电源选择 …… 484
11.12 操作系统 …… 487
11.12.1 Windows …… 487
11.12.2 Linux …… 488
11.13 本章小结 …… 488

第 12 章 常见问题、原因及解决方案 … 489

12.1 简介 …… 489
12.2 CUDA 指令错误 …… 489
12.2.1 CUDA 错误处理 …… 489
12.2.2 内核启动和边界检查 …… 490
12.2.3 无效的设备操作 …… 491
12.2.4 volatile 限定符 …… 492
12.2.5 计算能力依赖函数 …… 494
12.2.6 设备函数、全局函数和主机函数 …… 495
12.2.7 内核中的流 …… 496
12.3 并行编程问题 …… 497
12.3.1 竞争冒险 …… 497
12.3.2 同步 …… 498
12.3.3 原子操作 …… 502
12.4 算法问题 …… 504
12.4.1 对比测试 …… 504
12.4.2 内存泄漏 …… 506
12.4.3 耗时的内核程序 …… 506
12.5 查找并避免错误 …… 507
12.5.1 你的 GPU 程序有多少错误 …… 507
12.5.2 分而治之 …… 508
12.5.3 断言和防御型编程 …… 509
12.5.4 调试级别和打印 …… 511
12.5.5 版本控制 …… 514
12.6 为未来的 GPU 进行开发 …… 515
12.6.1 开普勒架构 …… 515
12.6.2 思考 …… 518
12.7 后续学习资源 …… 519
12.7.1 介绍 …… 519
12.7.2 在线课程 …… 519
12.7.3 教学课程 …… 520
12.7.4 书籍 …… 521
12.7.5 英伟达 CUDA 资格认证 … 521
12.8 本章小结 …… 522

第 1 章

超级计算简史

1.1 简介

为什么我们会在一本关于 CUDA 的书籍中谈论超级计算机呢？超级计算机通常走在技术发展的最前沿。我们在这里看到的技术，在未来的 5 ~ 10 年内，将是桌面计算机中很普通的技术。2010 年，在德国汉堡举行的一年一度的国际超级计算机大会上宣布，根据 500 强名单（http://www.top500.org），英伟达基于 GPU 的机器在世界最强大的计算机列表中位列第二。从理论上讲，它的峰值性能比强大的 IBM Roadrunner 和当时的第一名 Cray Jaguar 的性能还要高。当时 Cray Jaguar 的性能峰值接近 3 千万亿次。2011 年，采用 CUDA 技术的英伟达 GPU 仍然是世界上最快的超级计算机。这时大家突然清楚地认识到，与简陋的桌面 PC 一起，GPU 已经在高性能计算领域达到了很高的地位。

超级计算是我们在现代处理器中看到的许多技术的发展动力。由于对用更快的处理器来处理更大数据集的需求，工业界不断生产出更快的计算机。正是在这些发展变化中，GPU 的 CUDA 技术走到了今天。

超级计算机和桌面计算正在向着异构计算发展——人们试图通过将中央处理器（Central Processor Unit，CPU）和图形处理器（Graphics Processor Unit，GPU）技术混合在一起来实现更高的性能。使用 GPU 的两个最大的国际项目是 BOINC 和 Folding @ Home，它们都是分布式计算的项目。这两个项目使得普通人也能为具体的科学项目做出真正的贡献。在项目中，采用 GPU 加速器的 CPU/GPU 主机的贡献远远超过了仅装备 CPU 主机的贡献。截至 2011 年 11 月，大约 550 万台主机提供了约 5.3 千万亿次的计算性能，这将近是 2011 年世界上最快的超级计算机（日本富士通的“京（K）计算机”）计算性能的一半。

作为美国最快的超级计算机 Jaguar 的升级换代产品，命名为 Titan 的超级计算机计划于 2013 年问世。它将用近 30 万个 CPU 核和高达 18 000 个 GPU 板卡达到每秒 10 ~ 20 千万亿次的性能。正是由于有像 Titan 这样的来自世界各地的大力支持，无论是在 HPC（高性能计算）行业，还是在桌面电脑领域，GPU 编程已经成为主流。

现在，你可以自己“攒”或者购买一台具有数万亿次运算性能的桌面超级计算机了。在 21 世纪初期，这将会使你跻身 500 强的首位，击败拥有 9632 奔腾处理器的 IBM ASCI Red。

这不仅部分地展现了过去十几年计算机技术取得的巨大进步，更向我们提出了从现在开始的未来十几年，计算机技术将发展到何种水平这个问题。你可以完全相信在未来一段时间内，GPU 将位于技术发展的前沿。因此，掌握 GPU 编程将是任何一个优秀开发人员必备的重要技能。

1.2 冯·诺依曼计算机架构

几乎所有处理器都以冯·诺伊曼提出的处理结构为工作基础，冯·诺伊曼被认为是计算之父之一。在该结构中，处理器从存储器中取出指令、解码，然后执行该指令。

现代处理器的运行速度通常高达 4GHz。现代 DDR-3 内存，与标准的英特尔 I7 设备配合使用时，可以在运行任何程序时最高达到 2GHz 的速度。然而，在一个 I7 设备中至少具有四个处理器或内核。如果你认为超线程能力只能作为一个真正的处理器，那么一个 I7 设备中也是两个处理器。

在一个 I7 Nehalem 系统中，三通道 DDR-3 内存具有的理论带宽如表 1-1 所示。受主板和确切的内存模式影响，实际带宽可能要小很多。

表 1-1 I7 Nehalem 处理器带宽

QPI 时钟频率	理论带宽	单核带宽
4.8GT/s（标准配置）	19.2GB/s	4.8GB/s
6.4GT/s（最大配置）	25.6GB/s	6.4GB/s

注意：QPI 指快速通道互联（Quick Path Interconnect）

当考虑处理器的时钟速度时，你遇到的第一个问题是关于内存带宽的。用速度为 4GHz 的处理器，你可能每个时钟周期需要取来一条指令（操作码）和某个数据（操作数）。

通常，每个指令长 32 位。所以，假设你在每个核上只执行一组不带数据的、顺序执行的指令，则每秒钟你需要取来 4.8GB/s ÷ 4=1.2GB 条指令。这是假设处理器平均每个时钟周期执行一条指令的情况[⊖]。不过，通常你还需要读取和写回数据，这里假设数据与指令的比例是 1：1，那么这意味着我们的实际吞吐量将减少一半。

内存速度和时钟速度的比率是限制 CPU 和 GPU 吞吐量的一个重要因素，这一点我们将在后续的章节中讨论。深入分析你就会发现，除了 CPU 和 GPU 中的一些例外，大多数应用程序属于“内存受限型”[⊜]，而不是“处理器时钟周期或负载受限型”。

CPU 厂商试图通过使用缓存和突发内存访问来解决这个问题，这利用了程序的局部性原理。下面是一个典型的 C 程序，请仔细观察该函数中相关操作的类型：

```
void some_function
{
 int array[100];
 int i = 0;
```

⊖ 实际达到的执行速度可能大于或小于 1，这里我们做了简化处理。

⊜ 即性能提高受到内存速度的限制。——译者注

```
for (i=0; i<100; i++)
{
 array[i] = i * 10;
}
}
```

让我们看看处理器是如何实现的。首先，数组 array 的地址被装入某个内存访问寄存器，然后参数 i 被装入到另一个寄存器。循环退出条件为 100，这个退出条件既可以装入一个寄存器，也可以编码的形式，成为指令流中的一个字面值。这时，计算机将重复执行这几条指令，循环 100 次。对于每一个计算出的值，我们需要取来并执行“控制指令”、“访存指令”和“计算指令”。

这显然是低效的，因为计算机执行的是相同的指令，只是处理的数据不同。因此，硬件设计者就为几乎所有的处理器设置了容量很小的缓存，并且在更复杂的处理器中，设置多级缓存（如图 1-1 所示）。当需要从内存中取数据或指令时，处理器首先查询缓存。如果数据或指令在缓存中，则高速缓存直接将其交给处理器。

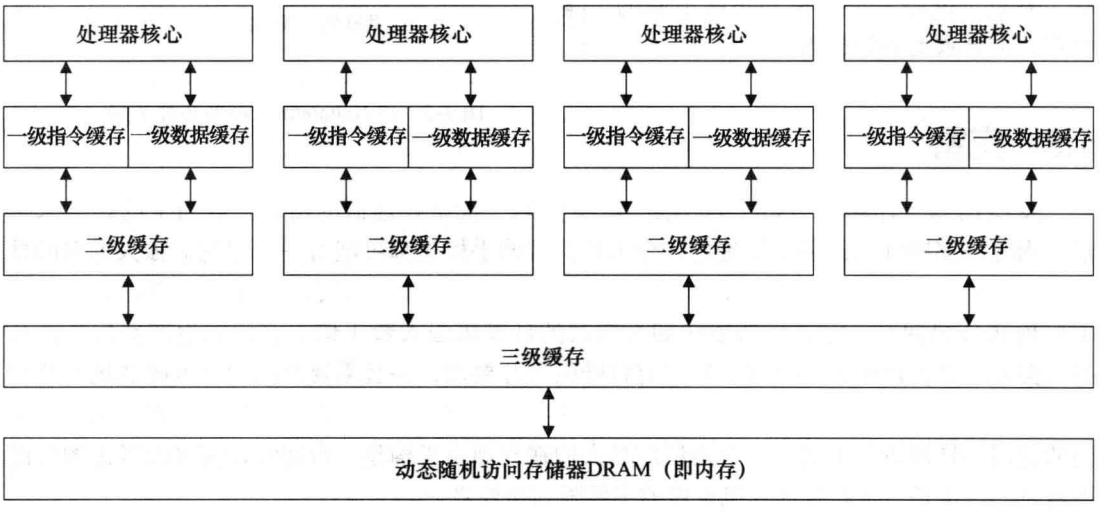


图 1-1 典型的现代 CPU 缓存组成结构

如果数据不在一级缓存（L1）中，则处理器向二级或三级（L2 或 L3）缓存发出读取请求。如果缓存中没有此数据，则需要从主存（即内存）中读取。一级缓存的工作速度通常能达到或接近处理器的时钟速度。因此，假设写入和读取都是在缓存中完成，则循环的执行就很有可能接近处理器全速。然而，这是有一定成本的：一级缓存的大小，通常只有 16K 或 32KB 大小。二级缓存就要慢些，但空间会大些，通常约为 256K。三级缓存则要大得多，通常几兆字节大小，但是比二级缓存要慢得多。

在实际程序中，循环程序往往非常大，可能达几兆字节大小。即便是程序可以存放在缓存中，但数据集往往就存不下了。所以尽管设置了缓存，处理器仍然会因为受到内存吞吐率或带宽的限制，而无法发挥其所具有的处理能力。

当处理器从缓存而不是主存中取来一条指令或一个数据时，称为缓存命中（cache hit）。但是随着缓存容量的不断增大，使用更大容量缓存所带来的增速收益却迅速下降。这意味着，现代处理器中使用大容量缓存并不是提高性能的一个有效办法，除非有办法将问题的整个数据集都装入缓存。

英特尔I7-920处理器有8MB的片内三级缓存。这个缓存的设计是有代价的，看一下英特尔I7处理器的模型，我们会知道三级缓存占用了约30%左右的芯片面积（如图1-2所示）。

随着缓存容量的增长，用于制造处理器的硅片的物理尺寸也逐渐变大。芯片越大，制造成本越昂贵，在制造过程中因发现差错而将芯片丢弃的可能性也就越高。尽管通过屏蔽掉有缺陷的核，这些有生产缺陷的芯片可以当作三核或双核芯片低价售出。但是对最终用户而言，容量不断增大而效率却不断下降的缓存，会导致更高的成本。

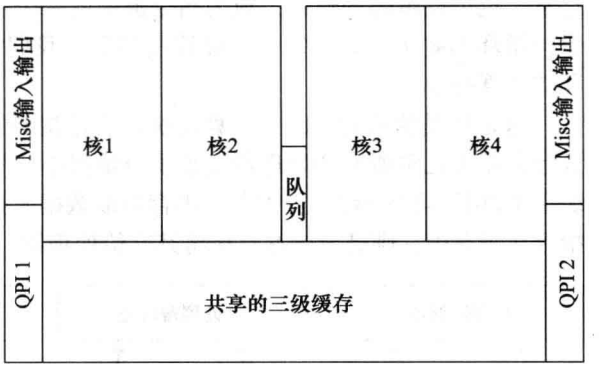


图1-2 I7 Nehalem处理器芯片布局

1.3 克雷

众所周知，计算革命始于20世纪50年代第一批微处理器的出现。以当下的标准来衡量，当时的那些计算设备是很慢的，今天你的智能手机里都可能有一个比它们强大得多的处理器。然而，正是它们导致了超级计算机的诞生。这些超级计算机通常为政府部门、大型学术机构或公司拥有。它们的功能比如今普通的计算机强大数千倍。它们的生产费用高达几百万美元，会占用庞大的空间，通常有特殊的冷却要求，并且需要专门的技术团队进行管理和维护。它们的运行需要消耗大量的电能，从某种程度上说，它们每年的运行费用和生产它们的费用一样昂贵。事实上，电力供应是人们在规划是否构建一台超级计算机时考虑的关键因素之一，也是当下超级计算机发展的主要限制条件之一。

现代超级计算机的创始人之一是西蒙·克雷（Seymour Cray），他主持设计的Cray-1超级计算机由克雷研究所（Cray Research）于1976年制造完成。在Cray-1的内部，所有的器件都是通过成千上万条独立的电线连接在一起。为此，他们聘请了许多女工，因为女人的手要比大多数男人的手小些，便于在狭小的空间里将成千上万条电线连接起来。

通常用以小时为单位的正常运行时间（两个停机故障之间的实际运行时间）来评价一台超级计算机。如果能让超级计算机持续运行一整天而不出任何问题，那就是一个了不起的成就。按照今天的标准，这似乎很落后。然而，我们今天拥有的很多产品和技术都要归功于克雷和那个时代的其他人所开展的研究。

克雷持续生产了很多以Cray命名的、极具突破性的超级计算机。最初的Cray-1超级计算机耗资880万美元，计算速度达到了160 MFLOPS（每秒百万次浮点运算）。而今天衡量计

算速度的标准是 TFLOPS（每秒万亿次浮点运算），是原来指标 MFLOPS 的一百万倍（10^{12} 与 10^6 对比）。今天，一个费米（Fermi）型 GPU 卡性能的理论峰值已经超过了 1 TFLOPS。

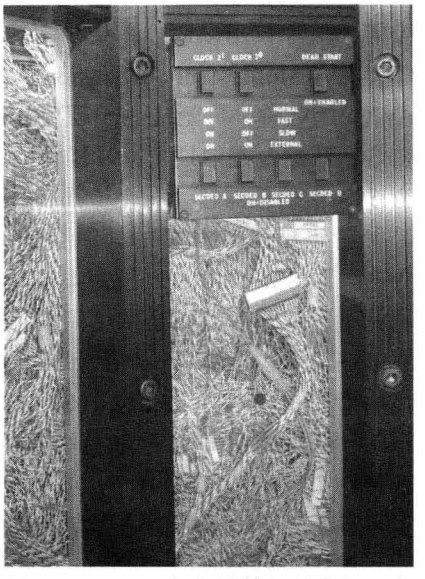


图 1-3 Cray-2 超级计算机的内部连线

Cray-2 在 Cray-1 的基础上有了明显的改进。它采用分成若干内存条的共享内存架构，这些内存条可以与一个、两个或四个处理器相连，进而发展成为今天基于服务器的对称多处理器（Symmetrical MultiProcessor，SMP）系统。在该系统中，多 CPU 共享同一个内存空间。跟那个时代的很多计算机一样，Cray-2 是一台基于向量的计算机（如图 1-3 所示）。在向量机中，一个操作同时处理多个操作数。诸如 MMX、SSE 和 AVX 这样的处理器扩展部分，以及 GPU 设备，它们的核心都是向量处理器。而时至今日，这些向量处理器仍然与早期的超级计算机在设计上有很多相似之处。

Cray 系列超级计算机通过硬件来支持“散布”（scatter）类和“收集”（gather）类操作原语，这些原语在并行计算中非常有用，我们将在后面的章节中介绍。

今天，Cray 系列超级计算机仍然活跃于超级计算机市场。由 Cray 公司研发的、安装在美国橡树岭国家实验室（The Oak Ridge National Laboratory）的超级计算机 Jaguar（http://www.nccs.gov/computingresources/jaguar/），依然位于 2010 年年底发布的世界超级计算机 500 强排行榜中。建议你登录 Cray 公司的网站（http://www.cray.com），了解一下这个伟大公司的历史，它将告诉你计算机发展的内在规律以及我们今天所处的位置。

1.4 连接机

早在 1982 年，一家名为 Thinking Machines 的公司提出了一个非常有趣的设计方案，即连接机（Connection Machine，CM）。

这是一个比较简单的概念，却引发了一场现代并行计算机的革命。他们通过一遍又一遍地使用一些简单的零部件创造了一个 16 核的 CPU，并在一台机器上安装了 4096 个这样的设备。这是一个完全不同的概念。它们并不是通过一个高速处理器处理数据集，而是通过 64K 个处理器来完成一个任务。

举一个简单的例子，假如我们要处理一个 RGB（红 Red，绿 Green，蓝 Blue）图像的颜色。其中，每种颜色用单字节表示，用 3 个字节表示 1 个像素的颜色。现在假设我们的问题是要将蓝色值降为零。

假设内存不是交替地存储各个像素点的颜色值，而是被分成红、绿、蓝三条。在传统的处理器中，我们会用一个循环来将蓝色内存条中每个像素的颜色值减 1。这个操作对每个数

据项都是相同的，即每次循环迭代，我们都要对指令流进行取指、译码和执行三个操作。

连接机采用的是单指令多数据（Single Instruction，Multiple Data，SIMD）型并行处理。今天，这种技术以诸如单指令多数据流扩展指令（Streaming SIMD Extensions，SSE）、多媒体扩展（Multi-Media eXtension，MMX）以及高级矢量扩展（Advanced Vector eXtensions，AVX）的名称，广泛应用于现代处理器中。这个技术先定义好一个数据范围，然后让处理器在这个数据范围内进行某种操作。尽管 SSE 和 MMX 是基于一个处理器核的，但连接机却拥有 64K 个处理器核，每个核都在其数据集上执行 SIMD 指令。

诸如 Intel I7 这样的处理器是 64 位的处理器，一次最多可以处理 64 位（即 8 个字节）的数据。而 SSE 的 SIMD 指令集已经扩展到了 128 位。在这样的处理器上运行 SIMD 指令，我们就可以消除所有多余的访问内存取指令的操作，并将内存读 / 写周期变为原来的 1/16，而原来是每次取出和写入 1 个字节。AVX 将这种技术扩展到了 256 位，使其效率更高。

对于分辨率为 1920 × 1080 的高清（High-Definition，HD）视频图像，它的数据大小为 2 073 600 字节，即每种颜色平面约为 2MB。如果使用常规的 SSE/MMX 处理器来处理，将需要约 260 000 个 SIMD 周期。对于 SIMD 周期，我们仅需要一个“读、计算和写”周期。实际需要的处理器时钟数可能会差别很大，这取决于实际的处理器架构。

连接机有 64K 个处理器。因此对于每个处理器，2MB 大小的帧将需要约 32 个 SIMD 的处理周期。显然，这种方法要远远优于现代处理器的 SIMD 方法。不过，有一点需要注意，如果将现在 CPU 采用的粗线程方案迁移至这种采用大规模并行方式处理的机器，处理器之间的同步和通信将是很大的问题。

1.5 Cell 处理器

超级计算机的另一个有趣的发展，源于 IBM 公司发明的 Cell 处理器（如图 1-4 所示）。

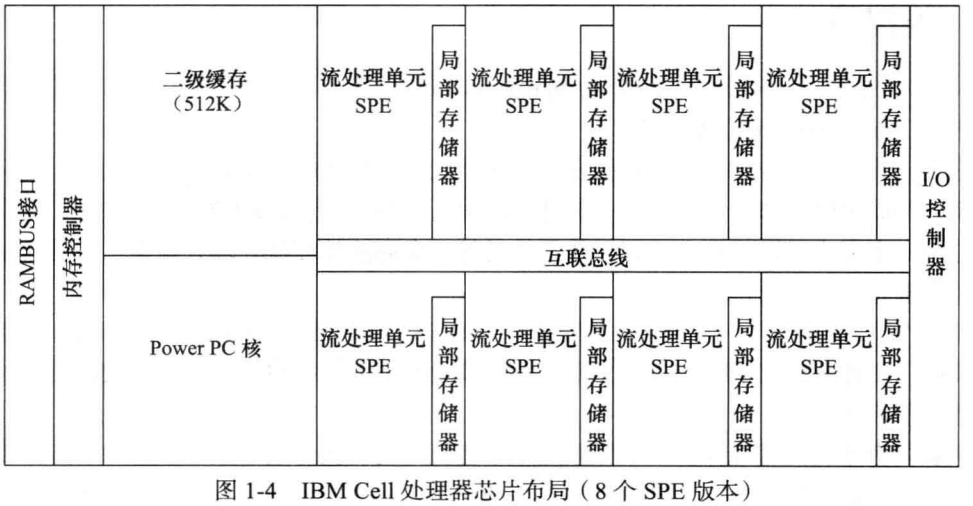


图 1-4 IBM Cell 处理器芯片布局（8 个 SPE 版本）

它的思想是用一个常规处理器作为监管处理器，该处理器与大量的高速流处理器相连。在 Cell 处理器中，常规的 PowerPC（PPC）处理器担任与流处理器和外部世界的接口。而 SIMD 流处理器，IBM 称其为 SPE，则在常规处理器的管理下，处理数据集。

对我们来说，Cell 是一个非常有趣的处理器，因为它与英伟达公司的 G80 和随后生产的 GPU 在设计上很相似。索尼公司也将其应用到游戏产业中的 PS3 控制台机器上，游戏产业是与 GPU 的主要应用领域非常类似的一个领域。

要想为 Cell 处理器编程，你需要写一个在 PowerPC 核心处理器上运行的程序。该程序会用一个互不相同的二进制码，在每个流处理单元 SPE（Stream Processing Element）上，调用执行一个程序。实际上，每个 SPE 本身就是一个核。它可以从自己的本地内存取出一个独立的程序来执行，这个程序与旁边的 SPE 执行的程序是不同的。另外，通过一个共享的互连网络，SPE 之间、SPE 与 PowerPC 核之间可以相互通信。当然，针对这种混合架构进行编程是很困难的。程序员必须从程序和数据两个方面，明确管理 8 个 SPE，以及在 PowerPC 核上运行的串行程序。

由于具有和相应处理器直接对话的能力，因此很多问题的求解就可以通过一系列简单的步骤来实现。例如在前述的 RGB 示例中，PPC 核可以取来一组待处理的数据，然后将其分配给 8 个 SPE 处理。每个 SPE 所做的处理是相同的，即读取一个字节，将其减一，然后存回本地内存。当所有 SPE 完成工作后，PC 核再从每个 SPE 中取回数据，然后将这组数据（或数据条）写入内存区域，整个图像在内存中被组合起来。Cell 处理器被设计为以组的方式工作，这样就重复了我们前面介绍过的连接机的设计。

当连接到一个高速的环形网络时，SPE 还可以按顺序排列起来，以执行一个包括多个步骤的流式操作（如图 1-5 所示）。

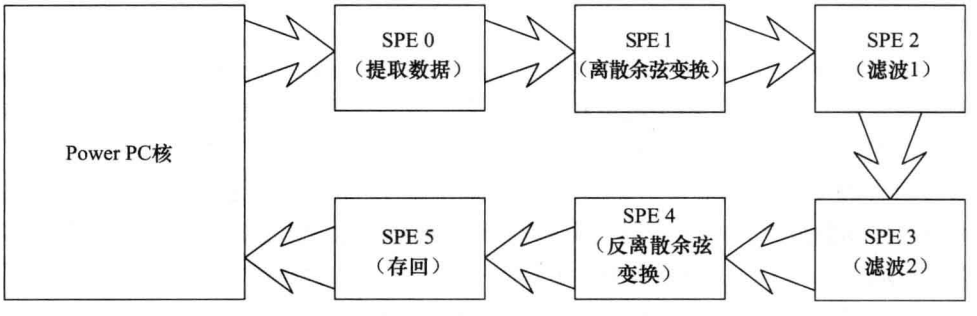


图 1-5　Cell 处理器的流处理器连接路径示例

这种流或流水线处理方法的问题是，系统运行的最快速度等于最慢节点的运行速度。它和工厂里生产线的实际运行情况是完全相同的，整条生产线的运行速度和最慢那个点的运行速度一样。就像装配生产线上的工人一样，每一个 SPE（工人）只执行一小部分任务，所以它可以非常迅速和高效地完成这部分任务。然而，跟其他处理器一样，SPE 也存在带宽限制以及数据传递至下一阶段的开销问题。因此，当通过在每个 SPE 上执行一个一致的程序来提高效率时，我们也会在处理器间通信上有所损失，并最终受到最慢的那个处理步骤的限制。

这是任何基于流水线模型的工作都会遇到的一个共性问题。

还有一种效率更高的方法是，将所有待处理的数据放在一个 SPE 上，然后让其余 SPE 分别处理一小块数据。这相当于培训所有装配线上的工人去装配一个完整的部件。对于简单的任务，这是很容易的，但是每个 SPE 受限于可用的程序和数据内存。这时，PowerPC 核就必须面向 8 个 SPE 进行数据的分发和收集，而不是原先的两个，因此主机和 SPE 间的管理开销及通信量就增加了。

在 2010 年超级计算机 500 强排名中位列第三的 Roadrunner 超级计算机中，IBM 使用了高功率版本的 Cell 处理器。该计算机由 12 960 块 PowerPC 核加上总共 103 680 块流处理器组成。每个 PowerPC 板由一个双核 AMD（Advanced Micro Device）Opteron 处理器监控，总共有 6912 个 PowerPC 板。Opteron 处理器的作用是协调各个节点工作。建造 Roadrunner 超级计算机耗资 1.25 亿美元，占地 560 平方米，理论吞吐率为 1.71 petaflops。工作时需要消耗 2.35 兆瓦的电能！

1.6 多点计算

随着对单台计算机硬件（如 CPU、主存、辅存）要求的提升，所需成本快速增加。若购买一个主频为 2.6GHz 的处理器需要花费 250 美元，而购买一个时钟速度增加不到 1GHz 的、主频为 3.4GHz 的类似处理器则需要 1400 美元。在运算速度、主存容量与存储容量上，也能看到类似的关系。

计算性能要求的提高，不仅会带来成本的上升，而且对电力的需求以及相应的散热方面的要求也随之增加。在足够的电力供应和充分的散热条件下，处理器能达到 4 ~ 5GHz 的时钟频率。

在计算领域里，你经常能遇见“收益递减规律”（The law of diminishing returns）。即便你在一个单一方面投入再多，结果也没有太大改变，因为它受限于成本、空间、电力供应、散热等因素。解决办法是在各个影响因素之间选择一个平衡点，然后多次地复制它。

随着时钟频率的不断增长，集群计算在 20 世纪 90 年代开始流行起来，其原理非常简单。用一些成套的或是自己从市场上购买零部件组装的 PC 机，把它们连接到市场上买来的 8 端口、16 端口、24 端口或者 32 端口的以太网交换机上，你得到的性能就比单一主机的性能高 32 倍。与其花费 1600 美元购买一个高性能处理器，不如花 250 美元购买 6 个中等性能的处理器。如果你的应用程序需要巨大的存储容量，则可以先把多台机器上的 DIMM 内存条扩充满，然后互联在一起，这样就可以获得足够的存储容量了。同时工作时，多台机器联合起来达到的计算能力要远远大于花同样的钱购买到的任何一台单机。

一夜之间，大学、中 / 小学校、政府机关和计算机系都能够制造出计算性能比以前强很多的计算机，不再因资金短缺而被排斥在高速计算市场为外。如今天的 GPU 计算一样，集群计算在当年就是一种能改变计算机世界面貌的革命性技术。与不断增加的单核处理器时钟速度结合在一起，这种技术为使用多个单核 CPU 来实现并行处理，提供了一种比较省钱的方式。

PC 集群通常运行 Linux 操作系统的集群版本。在这个版本中，它的每个节点从中央主控节点上获取引导指令和操作系统（Operating System，OS）。例如，在 CudaDeveloper 环境中，我们有一个由低功耗、带嵌入式 CUDA GPU 的独立 PC 机组成的小型集群，很便宜就能买来建立一个集群。有时也可以使用已经被替换掉的旧 PC 机来组建集群，硬件就几乎不花钱了。

然而，集群计算存在的问题是它的速度受到节点之间通信总量的限制，而这些通信是求解问题所必需的。如果你有 32 个运算节点，问题正好被划分成 32 个子任务，并且各个节点之间无须通信，那么你的问题求解是最适合集群计算的。如果每个数据点都要从所有节点上获取数据，那么你就是把一个很糟糕的问题交给了集群。

在现代的 CPU 和 GPU 中都能看到集群技术，比如前面的图 1-1。如果我们把图中的每个 CPU 核作为一个节点，二级缓存作为内存（Dynamic Random Access Memory，DRAM），三级缓存作为网络开关，将内存作为大容量的辅存，我们就得到一个集群的微缩模型（如图 1-6 所示）。

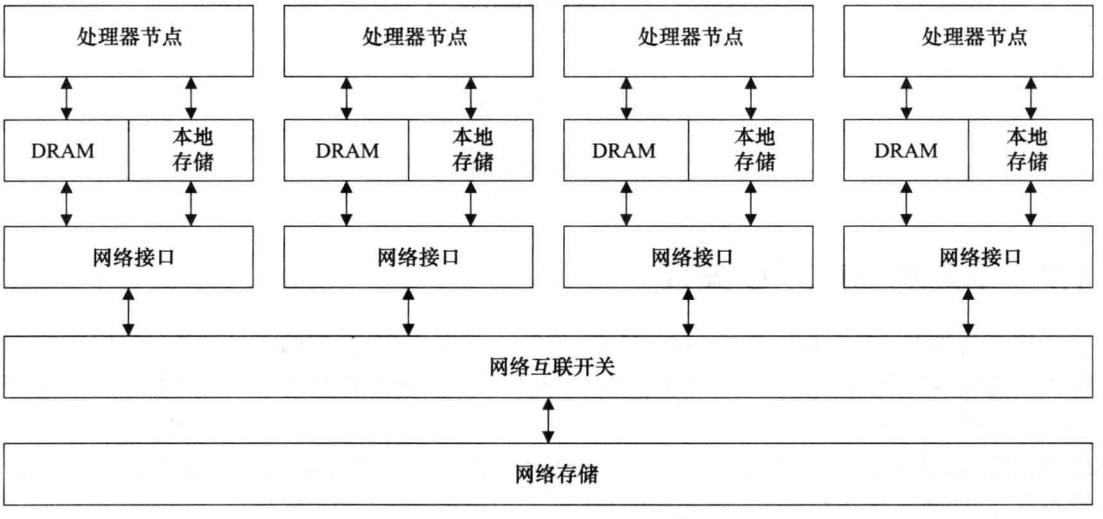


图 1-6 典型的集群层次结构

现代 GPU 的体系结构也完全相同。一个 GPU 内有许多流处理器簇（Streaming Multiprocessor，SM），它们就类似 CPU 的核。这些 SM 与共享存储（一级缓存）连接在一起，然后又与相当于 SM 间互联开关的二级缓存相连。数据先是存储在全局存储中，然后被主机取出并使用。除留一部分自己处理外，主机将剩余的数据通过 PCI-E 互联开关直接送往另一个 GPU 的存储空间。PCI-E 互联开关的传输速度比任何一个互联网络快好多倍。

如图 1-7 所示，节点可以在集群中重复设置。通过在一个可控的环境下，重复设置节点就可以构造一个集群。集群设计的一个进步是分布式应用程序。分布式应用程序可以运行在许多节点上，而每个节点又由包含多个 GPU 的许多处理单元组成。分布式应用程序可以运行在一个受集中控制的集群平台上，也可以运行在随机连接在一起、自主控制的机器上，来求解一个共同的问题。BOINC 和 Folding@Home 是此类应用的两个最大实例。在这两个实例

中，求解问题的计算机是通过因特网（Internet）连接起来的。

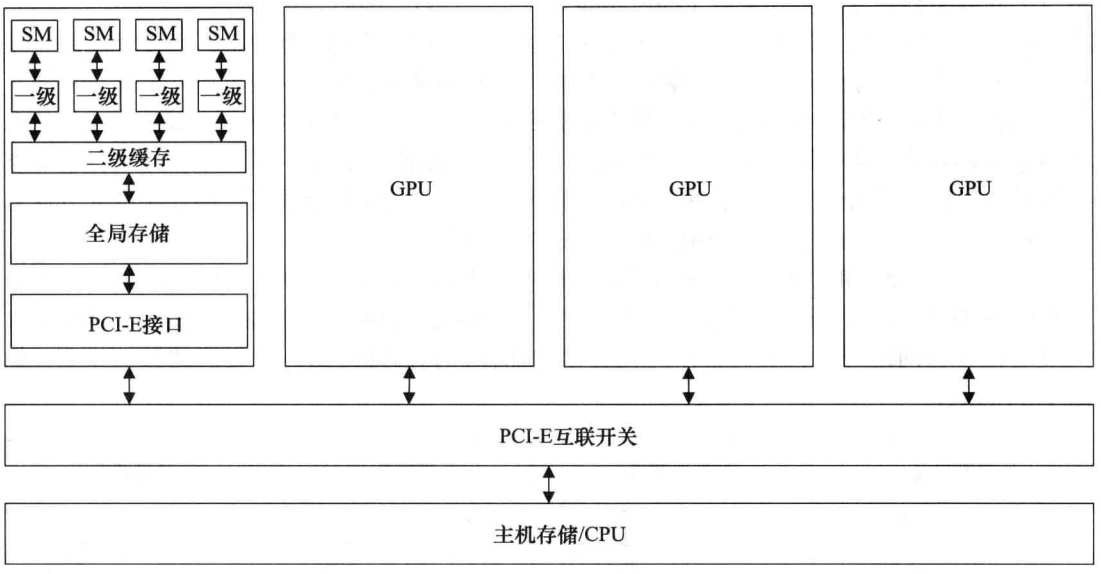


图 1-7 类似集群的 GPU 簇

1.7 早期的 GPGPU 编程

图形处理器（Graphics Processing Unit，GPU）是现代 PC 机中的常见设备。它们向 CPU 提供一些基本的操作，比如，对内存中的图像进行着色，然后将其显示在屏幕上。一个 GPU 通常会处理一个复杂的多边形集合，即需要着色的图片映像，然后给这些多边形涂上图片的纹理，进而再做阴影和光照处理。英伟达 5000 系列显卡第一次给我们带来逼真的图片效果，请查看 2003 年英伟达发布的“晨光中的花仙子”（Dawn Fairy）图片示例。

请浏览 http://www.nvidia.com/object/cool_stuff.html#/demos，并下载一些旧的图片示例，从中你可以看出在过去的十几年里 GPU 的发展变化有多大，详见表 1-2。

表 1-2 GPU 技术发展历程表

演示	显卡	年份
Dawn	GeForce FX	2003
Dusk Ultra	GeForce FX	2003
Nalu	GeForce 6	2004
Luna	GeForce 7	2005
Froggy	GeForce 8	2006
Human Head	GeForce 8	2007
Medusa	GeForce 200	2008
Supersonic Sled	GeForce 400	2010
A New Dawn	GeForce 600	2012

重要的进步之一就是可编程着色器（programmable shader）的出现。它们是 GPU 运行的一些用来计算各种图片效果的小程序。这样，着色就不必固定在 GPU 中进行。通过可下载的着色器，就可以完成这些操作。这就是最初的通用图形处理器（General-Purpose Graphical Processor Unit，GPGPU）编程。这表示 GPU 设计朝着“处理单元功能不再是固定的”方向迈出了第一步。

然而，这些着色器很自然地取来那些表示一个多边形图像的三维（3D）点集进行处理。着色器以一种高度并行的方式，对许多这样的数据集进行相同的操作，从而提供了巨大的计算能力。

现在即使多边形是用三个点的集合来表示的，其他的一些数据集，比如 RGB 图片，也能够用三个点的集合来表示，但还有许多数据集不是这样表示。因此，有一些勇敢的研究人员就尝试着用 GPU 技术来提高通用计算的速度。这导致开发出了许多新产品（如 Brook GPU、Cg 及 CTM 等），研发这些产品的目标都是将 GPU 变成一个具有和 CPU 一样运行模式的可编程设备。遗憾的是，它们既有优点也存在不足，这些产品都不容易学习和编程使用，而且愿意学习它们的人并不多。一句话，一项不容易学习的技术不可能获得程序员的追捧，也不可能激起程序员广泛的兴趣。这些产品一直没有在市场上获得成功。而 CUDA 可能是首次实现了上述目标，同时向程序员提供了一个真正通用的 GPU 编程语言。

1.8 单核解决方案的消亡

现代处理器的问题之一是它们已经达到了 4GHz 左右的时钟速度极限。就目前的技术而言，处理器在这个极限点上工作会产生太多的热量，从而导致需要特殊的、昂贵的冷却措施。产生热量的原因是随着时钟频率的提升，电力功耗增大了。事实上，在电压不变的情况下，一个 CPU 的电力功耗大约是它时钟频率的三次方。更糟糕的是，如果 CPU 产生的热量增加，那么即使时钟频率不变，根据硅材料的性质，CPU 的功耗也会进一步增加。这个电 / 热转换是对能源的一个浪费。这个不断增加的无效的电能消耗，意味着你要么不能充分为处理器提供电力，要么不能够有效地冷却处理器，已经达到了电子设备或芯片封装的散热极限，即所谓的“功耗墙”（power wall）。

面对不能再提高时钟频率的挑战，却还需要制造更快的处理器，处理器制造商只好另辟蹊径。两个主要的 PC 机处理器制造商，Intel 和 AMD，已经采取了另外一种方案。他们已经从持续地提高时钟频率或者通过指令级并行处理技术提高每个时钟周期内执行的指令条数的旧的发展道路，转移到向处理器里添加更多核的新的发展道路。现在已经有了双核、3 核、4 核、6 核、8 核，12 核甚至 16 核和 32 核的处理器，这就是以 CPU 和 GPU 为核心的计算技术未来的发展方向。从 CPU 的角度说，费米（Fermi）型 GPU 已经是一个真正的 16 核处理器了。

然而，这种方法有一个很大的问题——它需要编程者从以前的串行、单线程的问题求解方法切换到多线程同时执行的问题求解方法。现在，编程人员必须考虑 2 个、4 个、6 个或 8 个线程以及线程之间的交互和通信。当双核 CPU 出现时，还相对简单，因为正好有一些程

序需要在后台运行，将这些程序迁移到第二个核上运行即可。当4核CPU出现时，并没有修改这么多的程序来支持4核运算，还是会买4核CPU以运行单线程的应用程序。甚至游戏厂商也不想很快地转向4核编程，而我们通常认为这个领域是最希望采用最新技术的。

在一定程度上，处理器制造商也应该为此负责，因为单核程序可以在4核处理器的一个核中运行得很好。当只有一个核在工作时，一些设备甚至会动态地提升时钟频率来提高性能，这就导致编程人员变得懒惰，不愿充分使用已经可用的硬件。

当然，这也有经济上的原因，软件开发公司需要尽早把产品推向市场。开发出一个更好的4核程序固然是很好的，但如果竞争对手抢先一步占领市场就不好了。当硬件制造商仍然继续生产单核和双核设备的时候，软件市场自然就稳定在最低配置上，因为这样的软件销售范围是最广的。只有当4核CPU变成市场上产品的最低配置时，市场才会迫使软件开发朝着多核编程的方向发展。

1.9 英伟达和CUDA

如果留意GPU和CPU的计算能力，可以得到一张有意思的图（见图1-8）。我们看到，最初CPU和GPU计算能力的差距不是很大。但是在2009年之后，当GPU的性能最终突破了1万亿每秒大关后，它们的差距就越来越大了，而在2009年，GPU从G80硬件设备发展为G200硬件设备。然后在2010年，发展到革命性的费米型GPU。这其中的发展动力是大规模并行硬件的引入。G80是一个具备128个CUDA核的设备，G200是一个具备256个CUDA核的设备，而Fermi则是一个具备512个CUDA核的设备。

我们看到英伟达的GPU，从G200架构到费米型架构，浮点计算性能实现了每秒3千亿次（300 gigaflops）的飞跃，在吞吐量方面提高了接近30%。相比之下，英特尔公司从Core 2（酷睿2）架构升级到Nehalem架构仅有小幅的改进。只有改为Sandy Bridge架构后，CPU性能才显著地提升。这并不意味着孰优孰劣，传统CPU的目标是执行串行代码，在这方面它们还是做得非常好的。它们包含了一些特殊硬件，例如，分支预测单元、多级缓存等，所有这些都是针对串行代码的执行。但GPU并不是为执行串行代码而设计的，且只有完全按照并行模式运行时才能发挥它的峰值性能。

2007年，英伟达发现了一个能使GPU进入主流的契机，那就是为GPU增加一个易用的编程接口，也就是所谓的统一计算架构（Compute Unified Device Architecture，CUDA）。这为无须学习复杂的着色语言或者图形处理原语，就能进行GPU编程提供了可能。

CUDA是C语言的一种扩展，它允许使用标准C来进行GPU代码编程。这个代码既适用于主机处理器（CPU），也适用于设备处理器（GPU）。主机处理器负责派生出运行在GPU设备处理器上的多线程任务（CUDA称其为内核程序）。GPU设有内部调度器来把这些内核程序分配到相应的GPU硬件上。调度方法将在本书的后面详细介绍。假设这些任务有足够的并行度，随着GPU中流处理器簇数量的增加，程序的运算速度就会提升。

但是，这里仍然有一个大问题：程序中能够并行运行的代码占多大比例呢？可能达到的

最大加速比受限于程序中串行代码的数量。即便你有无限的计算能力，并且使得并行任务的运行时间趋近于零，但你还需要付出运行串行代码的时间。因此，我们必须从一开始就考虑是否能够把大量的工作并行化。

英伟达一直致力于为 CUDA 开发提供支持。在其官方网站（http://www.nvidia.com）上的 CudaZone 栏目中，英伟达提供了大量有助于 CUDA 开发的信息、示例和工具。

与之前的处理器不同，CUDA 已经开始进入发展的快车道，并且成为首个有可能发展成为 GPU 开发的候选编程语言。假如有数以百万计的 GPU 支持 CUDA，那么基于 CUDA 的应用程序将成为一个巨大的市场。

现在有很多基于 CUDA 的应用，并且数量还在逐月递增。英伟达在它的社区网站 http://www.nvidia.com/object/cuda_apps_flash_new.html 中列出了其中的诸多应用。

在程序需要做大量计算工作的领域，例如，从家庭录影带翻录成一个 DVD 盘（视频转码），我们发现大部分主流的视频开发包现在都已支持 CUDA。这个领域的平均加速比达到 5 ~ 10 倍。

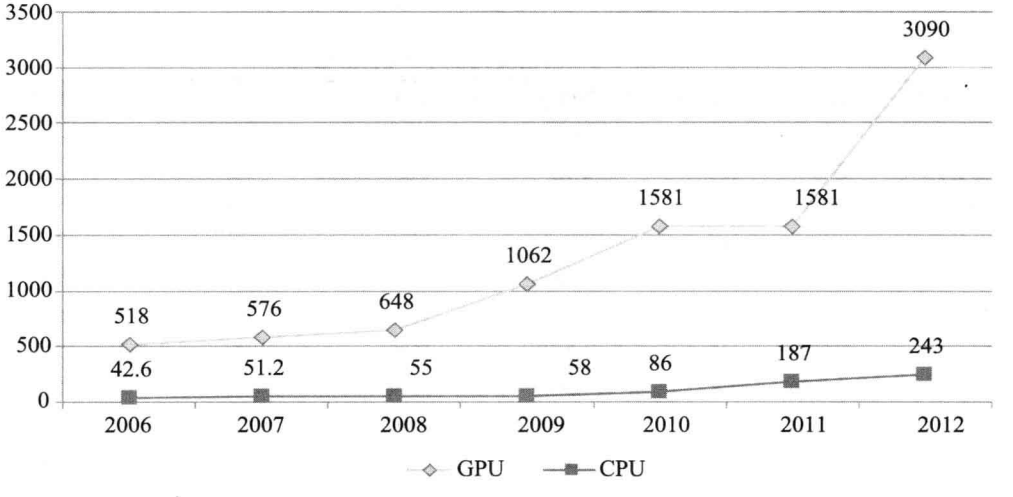


图 1-8 CPU 和 GPU 峰值性能（单位：十亿次浮点操作每秒（gigaflops））

随着 CUDA 一起引入的，是 Tesla 系列板卡。这些并不是图形卡，事实上这些卡既没有 DVI 接口，也没有 VGA 接口。它们是专用于科学计算的计算卡。使用它们可以为科学计算提供很大的加速比。这些卡既可以安装在常规的桌面 PC 上，也可以安装在专用的服务器机架上。在 http://www.nvidia.com/object/preconfigured_clusters.html 网站上，英伟达提供了一个示例系统，据称该系统的计算能力相当于普通集群系统的 30 倍。CUDA 和 GPU 正在改变着高性能计算领域的形式。

1.10 GPU 硬件

英伟达 G80 系列处理器及其后续产品是采用类似连接机和 IBM 的 Cell 处理器的设计方

案来实现的。每个图形卡由若干个流处理器簇（SM）组成，每个 SM 配备 8 个或更多的流处理器（Stream Processor，SP）。先前的 9800 GTX 卡有 8 个 SM，总共有 128 个 SP。但是与 IBM 的超级计算机 Roadrunner 不同，购买一块 GPU 板卡只需要几百美元，并且还不需要 2.35 兆瓦的电力去驱动它。后面当我们讲到建立 GPU 服务器时就会看到，电能的问题是不能忽视的。

GPU 卡大致可以看成是一种加速卡或协处理器卡。目前，一个 GPU 卡必须与基于 CPU 的主机相连才能工作。在这方面，它完全遵循 Cell 处理器搭配一个常规的串行处理核和 N 个 SIMD SPE 核的方法。每个 GPU 设备包括一组 SM，每个 SM 又包含一组 SP 或者 CUDA 核。这些 SP 最高可以实现 32 个单元并行工作。它们消除了 CPU 上为实现高速串行执行而设计的大量复杂的指令级并行处理电路。取而代之的是由程序员指定的显式的并行模型，使得同样大小的硅片可以容纳更强的计算能力。

GPU 的总体性能主要由其所具有的 SP 的数量、全局内存的带宽和程序员利用并行架构的充分程度等因素决定。表 1-3 是当前英伟达生产的 GPU 卡的列表。

表 1-3 当前英伟达生产的 GPU 卡

GPU 系列	设备	SP 数目	最大存储空间	GFlops（FMAD）	带宽（GB/s）	能耗（瓦）
9800 GT	G92	96	2GB	504	57	125
9800 GTX	G92	128	2GB	648	70	140
9800 GX2	G92	256	1GB	1152	2 × 64	197
260	G200	216	2GB	804	110	182
285	G200	240	2GB	1062	159	204
295	G200	480	1.8GB	1788	2 × 110	289
470	GF100	448	1.2GB	1088	134	215
480	GF100	448	1.5GB	1344	177	250
580	GF110	512	1.5GB	1581	152	244
590	GF110	1024	3GB	2488	2 × 164	365
680	GK104	1536	2GB	3090	192	195
690	GK104	3072	4GB	5620	2 × 192	300
Tesla C870	G80	128	1.5GB	518	77	171
Tesla C1060	G200	240	4GB	933	102	188
Tesla C2070	GF100	448	6GB	1288	144	247
Tesla K10	GK104	3072	8GB	5184	2 × 160	250

对于一个指定的应用程序，如何选择一块合适的板卡，其实就是在指定的应用程序的内存能耗和 GPU 处理能耗之间找到一个平衡。注意，9800 GX2、295、590、690 和 K10 卡其实是双卡，所以如果要完全利用这些卡，就要把它们当作两个设备来编程而非单个设备。另外需要说明的是，以上描述 GPU 的数据是针对单精度浮点数（32 位）而不是双精度浮点数（64 位）。同样需要注意的是，GF100（Fermi 型）系列，作为 Tesla 的衍生型，双精度运算单元的数量是标准桌面单元的两倍，所以获得了明显更好的双精度输出。即将发布的开普勒

（Kepler）型 GPU K20，与已经发布的兄弟款 K10 相比，双精度运算性能也将有显著改进。

虽然没有提及，但仍应注意到在新一代产品中，每个 SM 的时钟和能耗已经降低了。但是总体能耗却显著增加，这是任何一个基于多 GPU 的解决方案都要考虑的关键问题之一。一个典型的例子是，由于使用了共享电路和降低了时钟频率，基于双 GPU 的卡（9800 GX2、295、590、690）的能耗，比等价的两个单卡的总能耗稍低一些。

为了实现高密度运算，英伟达提供了采用共享 PCI-E 总线相连的、由 2 ~ 4 块 Tesla 卡组成的多种机架（M 系列计算模块）。这样就可以利用标准 PC 部件来建立你自己的 GPU 集群或者微型超级计算机。在本书的后面，我们将介绍如何实现这些想法。

对于 CUDA 来说，一件了不起的事情是，无论硬件设备如何变动，为早期的 CUDA 设备编写的程序依然能够运行在如今的 CUDA 设备上。CUDA 编译模型使用了和 Java 语言一样的编译原则——基于虚拟指令集的运行时编译。这允许现代的 GPU 可以运行即便是最老的 GPU 上的程序代码。当针对新 GPU 而需要修改程序的某些特性时，原先的程序常常可以为程序员的工作提供很多便利。事实上，针对不同硬件版本的更新，可以对程序做很多的优化调整，这些将在本书的最后部分介绍。

1.11 CUDA 的替代选择

1.11.1 OpenCL

那么其他的 GPU 制造商，如 ATI（现在是 AMD）能够成为主要的厂商吗？从计算能力上看，AMD 的产品和英伟达的产品是旗鼓相当的。但是，在英伟达引入 CUDA 很长时间之后，AMD 才将流计算技术引入市场。从而导致英伟达针对 CUDA 可用的应用程序要远远多于 AMD/ATI 在其技术框架上的应用程序。

OpenCL（Open Computing Language）和“直接计算”（Direct Compute）不是本书详细讨论的内容，但是作为 CUDA 的替代选择，应当提及。目前，CUDA 仅仅能够正式运行于英伟达的硬件产品上。虽然英伟达在 GPU 市场上占有很大的份额，但是其他竞争者所拥有的份额也不小。作为开发者，我们希望开发出的产品能够面向的市场越大越好，尤其是消费者市场。同样的，人们也关心是否有能够同时支持英伟达和其他厂商硬件产品的 CUDA 的替代品。

OpenCL 是一个开放的、免版税的标准，由英伟达、AMD 和其他厂商所支持。OpenCL 的商标持有者是苹果公司，它制定出一个允许使用多种计算设备的开放标准。计算设备可以是 GPU、CPU 或者其他存在 OpenCL 驱动程序的专业设备。截至 2012 年，OpenCL 支持绝大多数品牌的 GPU 设备，包括那些至少支持 SSE3[⊖]的 CPU。

任何熟悉 CUDA 的程序员都可以相对轻松地使用 OpenCL，因为它们的基础概念十分相似。但是，与 CUDA 相比，使用 OpenCL 会复杂一些，因为很多由 CUDA 运行时 API（应用

⊖ SSE3 是 Streaming SIMD Extensions 3 的缩写，表示“单指令多数据流扩展指令 3”。——译者注

程序编程接口）所完成的功能，在 OpenCL 中需要由程序员显式地编程实现。

在 http://www.khronos.org/opencl/ 网站上有更多关于 OpenCL 的内容。而且也有很多关于 OpenCL 的书籍。我个人推荐：在学习 OpenCL 之前，先学习 CUDA。因为在某种意义上讲，CUDA 是一种比 OpenCL 更高级的语言扩展。

1.11.2 DirectCompute

DirectCompute 是微软开发的可替代 CUDA 和 OpenCL 的产品。它是集成在 Windows 操作系统，特别是 DirectX 11 API 上的专用产品。对于之前从事显卡编程的人来说，DirectX API 是一个巨大的飞跃。这种产品使得开发者只需掌握一个 API 库，就可以对所有的显卡进行编程，而不必为每个主要的显卡生产商编写或发布驱动程序。

DirectX 11 是最新的标准并且 Windows 7 操作系统能够支持它。由于标准背后的支持厂商是微软，你可以想象得到它会被开发者群体迅速接受。对于已经熟悉 DirectX API 的开发人员，情况更是这样的。如果你熟悉 CUDA 和 DirectCompute，那么将一个 CUDA 应用程序移植到 OpenCL 的确是一项轻松的任务。据微软所言，对同时熟悉两种产品的人来说，这通常仅仅是一个下午的工作量。但是，由于以 Windows 操作系统为核心，DirectCompute 技术被排除在各种版本的 UNIX 占主导地位的高端系统之外。

微软还推出了基于 C++ 的 AMP（加速大规模并行计算）库，它是标准模板库（Standard Template Library，STL）的补充部分。对于熟悉 C++ 风格的 STL 的程序员，它更具有吸引力。

1.11.3 CPU 的替代选择

主要的并行程序设计扩展语言有 MPI 和 OpenMP，在 Linux 下开发时，还有 Pthreads。Windows 操作系统下有 Windows 线程模型和 OpenMP。MPI 和 Pthreads 被用作与 UNIX 进行联系的接口。

MPI（Message Passing Interface）可能是目前使用最广泛的消息传递接口。它是基于进程的，通常在各个大规模计算实验室中得到应用。它需要一个系统管理员来正确地安装配置，并且它适合于可控的计算环境。它实现的并行处理表现为，在集群的各个节点上，派生出成百上千个进程，通常这些进程通过基于网络的高速通信链路（如，以太网或 InfiniBand）显式地交换消息，以协同完成一个大的任务。MPI 被广泛使用和学习。在可控的集群环境下，它是一个很好的解决方案。

OpenMP（Open Multi-Processing）是专门面向单个节点或单个计算机系统而设计的并行计算平台，它的工作方式是完全不同的。在使用 OpenMP 时，程序员需要利用编译器指令精确写出并行运算指令。然后编译器根据可用的处理器核数，自动将问题分为 N 部分。很多编译器对 OpenMP 的支持都是内嵌的，包括用于 CUDA 的 NVCC 编译器。OpenMP 希望根据底层的 CPU 架构，实现对问题的可扩展并行处理。但是，CPU 内的访存带宽常常不够大，满足不了所有核连续将数据写入内存或者从内存中取出数据的要求。

pthreads 是一个主要应用于 Linux 上的多线程应用程序库。同 OpenMP 一样，pthreads

使用线程而不是进程，因为它是设计用来在单个节点内实行并行处理的。和 OpenMP 不同的是，线程管理和线程同步由程序员来负责。这提供了更多的灵活性，因此精心设计的程序会带来很好的性能。

ZeroMQ（0MQ）也值得一提，这是一个你可以链接的简单的库。在本书的后面，我们将使用它来开发一个多节点、多 GPU 的例子。ZeroMQ 使用一个跨平台的 API 来支持基于线程、基于进程和基于网络的通信模型。Linux 和 Windows 平台都支持 ZeroMQ。它是针对分布式计算而设计的，所以连接是动态的，节点失效不会影响它的工作。

Hadoop 也是你应当考虑的技术。Hadoop 是谷歌 MapReduce 框架的一个开源版本。它针对的是 Linux 平台。其概念是你取来一个大数据集然后将其切割或映射（map）成很多小的数据块。然而，并不是将数据发送到各个节点，取而代之的是数据集通过并行文件系统已经被划分给上百个或者上千个节点。因此，归约（reduce）步骤是把程序发送到已经包含数据的节点上。输出结果写入本地节点并保存在那里。后续 MapReduce 程序取得之前的输出并以某种方式对其进行转换。由于数据实际上映射到了多个节点上，因此它可以应用于高容错和高吞吐率系统。

1.11.4 编译指令和库

很多编译器厂商，如 PGI、CAPS 以及最著名的 Cray，都支持最近发布的针对 GPU 的 OpenACC 编译器指令集。在本质上，这些是 OpenMP 方法的复制。它们都要求程序员在程序中插入编译器指令来标注出“应该在 GPU 上执行”的代码区域。然后编译器就做些简单的工作，即从 GPU 上移入或移出数据、调用内核程序等。

若使用 pthreads 替代 OpenMP，由于 pthreads 提供更底层的控制，所以你可以获得更高的性能。使用 CUDA 替代 OpenACC 也是这样。但是，对额外层面的控制，要求程序员掌握更高水平的编程知识，也会带来更高的出错风险，进而对开发进度产生影响。目前，OpenACC 不仅要求用指令标注出哪些区域的代码需要运行在 GPU 上，而且还要指明数据要存储在哪种类型的内存上。英伟达宣称使用这类指令可以获得 5 倍以上的速度提升。对于那些想要程序跑得更快的程序员来说，这是一个很好的选择。对于那些把程序设计放在次要位置，仅仅考虑在合理的时间内完成任务的人来说，这也是一个很棒的选择。

库的使用也是很重要的，因为它可以让你的生产效率以及执行程序时间的加速比提高。一些库提供的通用函数的执行效率很高，例如，SDK 提供的 Thrust。诸如 CUBLAS 这样的库是针对线性代数最好的库。很多诸如 Matlab 和 Mathematica 这样著名的应用程序中都有库的存在。在某些语言中，如 Python、Perl、Java 和许多其他的语言中，库是以语言绑定的形式存在的。CUDA 甚至还被集成进 Excel 里。

在现代化软件开发的各个方面，很多你准备开发的东西别人已经做好了。在你准备花费数周时间开发一个库之前，请先去互联网上搜索一下，看看哪些是已经存在的。除非你是一个 CUDA 专家，否则你自己开发不大可能比使用已有的更快。

1.12 本章小结

也许你在想，为什么要使用 CUDA 进行开发？答案是，在技术支持、调试工具和驱动程序等方面，CUDA 是目前最容易进行开发的语言。CUDA 在各个方面都领先一步，在成熟性方面更是遥遥领先。即便你的应用程序需要支持非“英伟达”的硬件，但最好的方式也是先在 CUDA 上开发，然后将其移植为其他 API 的应用程序。同理，我们应专注于 CUDA，因为当你成为一个 CUDA 专家后，使用开发所需的其他 API 也是很容易的。理解 CUDA 的工作原理，可以帮助你更好地利用任何一个高层 API，并有助于理解它们的局限性。

我衷心地希望，你能从由单线程 CPU 程序员转变成 GPU 上的并行程序员的旅程中，找到乐趣。即使将来不做 GPU 开发，你获得的知识也会对设计多线程程序有极大帮助。如果你，像我们一样，看到世界正在向并行编程模型转变，那么你将会渴望能够站在技术挑战与创新浪潮的最前沿。单线程工业正在慢慢衰落。要想保持自身价值、成为公司渴望的人才，你需要掌握那些代表计算机世界发展方向的技术，而不是那些日趋没落的技术。

GPU 正在改变计算机世界的面貌。突然之间，十几年前超级计算机的计算能力现在已经能够出现在你的办公桌上。你不再需要：排队、分批提交任务，然后用几个月的时间等待管理员（committee）批准你的请求——在一个超负荷运行的计算系统上使用有限的计算机资源。现在，最多花 5000 ~ 10 000 美元，你就可以在你的办公桌上添置一台超级计算机，或者花一小部分钱买来一台运行 CUDA 的机器。GPU 是一项翻天覆地的技术革新，它使每个人拥有超级计算机级别的计算能力成为可能。

第 2 章

使用 GPU 理解并行计算

2.1 简介

本章旨在对并行程序设计的基本概念及其与 GPU 技术的联系做一个宽泛的介绍。本章主要面向具有串行程序设计经验，但对并行处理概念缺乏了解的读者。我们将用 GPU 的基本知识来讲解并行程序设计的基本概念。

2.2 传统的串行代码

绝大多数程序员是在串行程序占据主导地位而并行程序设计仅吸引极少数技术狂的年代里成长起来的。大多数人是因为对技术感兴趣才到大学去攻读与 IT 有关的学位的。同时他们也会意识到将来还需要一个能够获得可观收入的工作岗位。因此，在选择专业时，他们肯定会考虑毕业后，社会上是否有足够多的就业岗位。即便在并行程序设计最火的年代，除了研究所或学术机构的岗位外，适合并行程序设计的商业岗位总是很少的。绝大多数程序员都是用简单的串行模式来开发应用软件，而这个串行模式主要是基于大学程序设计课程的教学内容，且受市场需求的驱动。

并行程序设计的发展态势一直不太明朗，与它有关的各种技术及程序设计语言一直没能将它变成主流。市场上从来没有出现过对并行硬件的大规模需求。相应地，也没有出现对并行程序员的大规模需求。每一到两年，各个 CPU 厂商都会推出新一代的处理器。相比之前的处理器，新一代处理器执行代码的速度更快。无须改进运行方式就可以有更快的运行效果，因此串行程序的地位稳如泰山。

并行程序通常与硬件的联系更紧密。引入并行程序的目的是为了获得更高的性能，但是这往往需要付出降低可移植性的代价。在新一代并行计算机中，原先计算机的某种特性可能会换种方式实现，甚至被取消。每隔一定的时间，就会出现一种新的革命性的并行计算机架构，从而导致所有的程序代码都要重新编写。作为程序员，如果你的知识仅局限于某一个处理器，那么从商业角度看，这些知识会随着这款处理器的淘汰而失去价值。由此可见，学习 x86 类型架构的程序设计比学习某种并行计算机架构的程序设计具有更大的商业意义，因为

后者很可能仅有几年的使用寿命。

然而有趣的是，多年以来，两个并行程序设计标准，OpenMP 和 MPI，却通过不断地修改、完善而得以始终采用。面向包含多核处理器的共享存储并行计算机而设计的 OpenMP 标准，强调的是在单个节点内部实现并行处理。它不涉及节点间并行处理的任何概念。因此，你所能解决的问题只受到单个节点的处理能力、内存容量和辅存空间的限制。但是对于程序员而言，采用 OpenMP，程序设计相对容易，因为大多数低层级的线程代码（需要采用 Windows 线程库函数或 POSIX 线程库函数编制）都由 OpenMP 替你完成了。

MPI（Message Passing Interface）标准用于解决节点间的并行处理，常用于定义良好的网络内的计算机集群。它常用于由几千个节点组成的超级计算机系统。每个节点只分担问题的一小部分。因此，公共资源（CPU、缓存、内存、辅存等）的大小就等于单个节点的资源量乘以网络中节点的个数。任何网络的阿喀琉斯之踵（Achilles' heel，要害）就是它的互连结构，即把机器连接在一起的网络结构。通常，节点间通信是任何基于集群的解决方案中决定最大速率的关键因素。

当然，OpenMP 和 MPI 也可以联合使用来实现节点内部的并行处理和集群中的并行处理。但是由于应用程序编程接口（API）库和编程方法完全不同，所以这种情况并不常见。OpenMP 的并行指令允许程序员通过定义并行区，从而从一个较高的层次分析算法中的并行性；而 MPI 却需要程序员做大量的工作来显式地定义节点间的通信模型。

既然已经花很长时间才掌握一种 API 库，程序员都不愿意再学习另外一种。因此，适合于一台计算机就能解决的问题通常采用 OpenMP，而需要用集群来解决的大问题就采用 MPI。

本书将要探讨的 GPU 编程语言 CUDA，却能够很好地将 OpenMP 和 MPI 联系在一起。例如，CUDA 提供的类 OpenMP 指令（OpenACC）就很方便地供熟悉 OpenMP 的程序员采用。OpenMP、MPI 和 CUDA 正在越来越多地出现在大学计算机专业本科生或研究生的课堂教学内容中。

然而，绝大多数串行程序设计者第一次接触并行程序是在介绍多核处理器的时候。除了少数技术狂之外，他们往往对眼前的并行处理环境视而不见，因为多核处理器主要用于实现“任务并行”（task parallelism），这是一种基于操作系统的并行处理，后面我们再详细介绍。

显而易见，今天的计算机技术正在向着多核的方向前进。越来越多的程序员开始关注多核处理器。几乎所有商场销售的台式机要么采用双核处理器，要么采用四核处理器。因此，程序员开始利用多线程来发挥处理器中多核的作用。

线程是程序中一个独立的执行流，它可以在主执行流的要求下分出或聚合。通常情况下，CPU 程序拥有的活动线程数量，不超过其包含的物理处理核数量的两倍。就像在单核处理器中，操作系统的任务是分时、轮流运行的，每个任务只能运行一小段时间，从而实现数量多于物理 CPU 核数的众多任务同时运行。

然而，随着线程数量的增加，终端用户也开始感觉到线程的存在。因为在后台，操作系统需要频繁进行“上下文切换”（即对于一组寄存器进行内容的换入、换出）。而“上下文切换”是一项很费时的操作，通常需要几千个时钟周期，所以 CPU 的应用程序的线程数要尽量

比 GPU 的少。

2.3 串行 / 并行问题

线程会引起并行程序设计中的许多问题，例如，资源的共享[⊖]。通常，这个问题用信号灯（semaphore）来解决，而最简单的信号灯就是一个锁或者令牌（token）。只有拥有令牌的那个线程可以使用资源，其他线程只能等待，直到这个线程释放令牌。因为只有一个令牌，所以所有工作都有条不紊地进行。

当同一线程必须拥有两个或者更多的令牌时，就会出现问题。例如，在线程 0 拥有令牌 0 而线程 1 拥有令牌 1 的情况下，如果线程 0 想得到令牌 1 而线程 1 想得到令牌 0，想得到的令牌没有了，线程 0 和线程 1 就只好休眠，直到它们期待的令牌出现。由于没有一个线程会释放掉已经拥有的令牌，所以所有的线程将永远等待下去。这就是所谓的“死锁”（deadlock），如果程序设计不当，死锁将不可避免地发生。

当然，在极偶然的情况下，程序可以共享资源而不发生死锁。无论哪种加锁系统，每一个线程都必须正确地使用资源。也就是说，它们必须先申请令牌，如果没成功则等待。获得令牌后，才执行相应的操作。这就需要程序员定义好共享的资源，并制定恰当的机制来协调多线程对该资源的更新操作[⊜]。然而，在任何一个团队里都有若干个程序员，即便有一个程序员不遵守规则，或者并不知道它是一个共享资源，你的程序多数情况下不会正常工作。

我曾经为一个大公司开发的项目里就出现过这个现象。所有的线程都申请一个锁，如果没成功则等待。获得令牌后，就更新共享资源。一切都运行正常，所有的代码都通过了质量保证检查（Quality Assurance，QA）和所有的测试。然而投入运行后，现场的用户偶尔会报告说某个参数被重置为 0 了，看上去像随机发生的。由于跟踪发现 Bug 的前提往往是它能够持续地引发同一个问题，所以随机的 Bug 总是很难发现的。

最后是公司里的一个实习生发现了问题的原因。在代码中一个与其完全无关的区域里，一个指针在特定的条件下没有被初始化。在程序的运行过程中，当线程以某种特定的顺序执行时，这个指针就会碰巧指向受我们保护的数据。程序中的其他代码会将这个指针所指向变量的值初始化成 0。这样，受我们保护的并被线程共享的参数的值就被清除了。

这是基于线程操作的一个让人为难的地方。线程操作的是一个共享的内存空间，这既可以带来不借助消息就可以完成数据交换的便利，也会引起缺乏对共享数据保护的问题。

可以用进程来替换线程。不过，因为代码和数据的上下文都必须由操作系统保存，所以操作系统装入进程就要吃力得多。相比之下，只有线程代码的上下文（一个程序或指令计数器加上一组寄存器）由操作系统保存，而数据空间是共享的。总之，进程和线程可以在任意时刻、在程序中的不同区域分别执行。

⊖ 即对一个公共变量或内存地址进行读 / 写操作。——译者注

⊜ 即对公共变量或内存地址进行写操作。——译者注

默认情况下，进程在一个独立的内存空间内运行。这样就可以确保一个进程不会影响其他进程的数据。因此，一个错误的指针访问将引发“越界访问”异常，或者很容易在特定的进程中找到 Bug。不过，数据传递只能通过进程间消息的发送和接收来完成。

一般说来，线程模型较适合于 OpenMP，而进程模型较适合于 MPI。在 GPU 的环境下，就需要将它们混合在一起。CUDA 使用一个线程块（block）构成的网格（grid）。这可以看成是一个进程（即线程块）组成的队列（即网格），而进程间没有通信。每一个线程块内部有很多线程，这些线程以批处理的方式运行，称为线程束（warp）。后续的章节中我们将进一步介绍。

2.4 并发性

并发性的首要内涵是，对于一个特定的问题，无须考虑用哪种并行计算机来求解，而只需关注求解方法中哪些操作是可以并行执行的。

如果可能，请设计出一个公式来把每一个输出数据表示成输入数据的函数。不过，对于某些算法而言，这显得很麻烦，例如，迭代次数很大的算法。对于这些算法，可以单独考虑每一步或每一次迭代。能否将对应每一步的数据表示成输入数据集的一个变换？如果能，则你只需拥有顺序执行的一组内核函数（迭代步）即可解决问题。只需将这些操作压入队列（或者处理流），计算机将依次执行该队列的操作。

很多问题属于“易并行”（embarrassingly parallel）问题，其实这个名称还是相当保守的。如果能够设计出一个公式把每一个输出数据都表示成相互无关的，例如，矩阵乘法，那将是很令人开心的结局。这类问题可以在 GPU 上得到很好的解决而且编程很简单。

尽管算法中可能有一个阶段不是“易并行”，但某一步或某几步还是能够用这种方式表达，这也不错。尽管该阶段会成为一个“瓶颈”（bottleneck），还会让程序员很劳神。但对于问题的其他部分，程序员还是很容易就可以编写出在 GPU 上求解的代码。

如果求解问题的算法在计算每一个点的值的时候，必须知道与其相邻的其他点的值，那么算法的加速比（speedup）最终将很难提高。在这种情况下，对一个点的计算就需要投入多个处理器。在这一点上，计算将会变得很慢，因为处理器（或者线程）需要花费更多的时间来进行通信以实现数据共享，而不做任何有意义的计算。至于你到底会遇到怎样糟糕的情况，则取决于通信的次数和每次通信的开销。

由于“易并行”不需要或者只需少许线程间或线程块间通信，所以 CUDA 是很理想的并行求解平台。它用基于片上资源的、显式的通信原语来支持线程间通信。但是块间通信只有通过按顺序调用多个内核程序才能实现，而且内核间通信需要用到片外的全局内存。块间通信还可以通过对全局内存的原子操作来实现，当然使用这种方法会受到一定的限制。

CUDA 将问题分解成线程块的网格，每块包含多个线程。块可以按任意顺序执行。不过在某个时间点上，只有一部分块处于执行中。一旦被调度到 GPU 包含的 N 个“流处理器簇”（Streaming Multiprocessors，SM）[⊖]中的一个上执行，一个块必须从开始执行到结束。网格中

⊖ 原文误为 symmetrical multiprocessors。——译者注

的块可以被分配到任意一个有空闲槽的 SM 上。起初，可以采用“轮询调度”（round-robin）策略，以确保分配到每个 SM 上的块数基本相同。对绝大多数内核程序而言，分块的数量应该是 GPU 中物理 SM 数量的八倍或者更多倍。

以一个军队比喻，假设有一支由士兵（线程）组成的部队（网格）。部队被分成若干个连队（块），每个连队由一位连长来指挥。按照 32 名士兵一个班（一个线程束），连队又进一步分成若干个班，每个班由一位班长来指挥（参见图 2-1）。

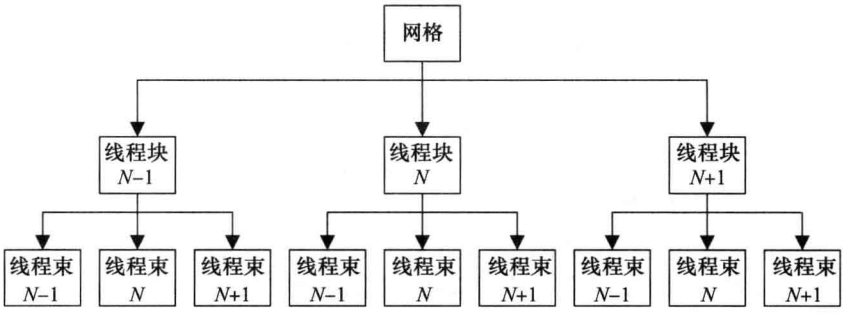


图 2-1 基于 GPU 的线程视图

要执行某个操作，总司令（内核程序 / 主机程序）必须提供操作名称及相应的数据。每个士兵（线程）只处理分配给他的问题中的一小块。在连长（负责一个块）或班长（负责一束）的控制下，束与束之间的线程或者一束内部的线程之间，要经常地交换数据。但是，连队（块）之间的协同就得由总司令（内核函数 / 主机程序）来控制了。

因此，当思考 CUDA 程序如何实现并发处理时，你应该用这种非常层次化的结构来协调几千个线程的工作。尽管一开始听上去很复杂，但是对绝大多数“易并行”程序而言，仅仅需要针对一个线程思考它计算输出数据的那一点。在一个典型的 GPU 上可以运行 24K[⊖]个‘活动”线程。在费米架构 GPU 上，你总共可以定义 65 535 × 65 535 × 1536 个线程，其中 24K 个线程随时都是活动的。这表明一个节点就足够满足单点中绝大多数问题的求解要求了。

局部性

在过去的十几年间，计算性能的提高已经从受限于处理器的运算吞吐率的阶段，发展到迁移数据成为首要限制因素的阶段。从设计处理器过程中的成本来讲，计算单元（Algorithmic Logic Unit，ALU）是很便宜的。它们能够以很高的速度运行，而消耗很小的电能并占用很少的物理硅片空间。然而，ALU 的工作却离不开操作数。将操作数送入计算单元，然后从计算单元中取出结果，耗费了大量的电能和时间。

在现代计算机设计中，这个问题是通过使用多级缓存来解决的。缓存的工作基础是空间局部性（地址空间相对簇集）和时间局部性（访问时间的簇集）。因此，之前被访问过的数据，很可能还要被再次访问（时间局部性）；刚刚被访问过的数据附近的数据很可能马上就会被访

⊖ 1K=1024。——译者注

问（空间局部性）。

当任务被多次重复执行时，缓存的作用会充分地发挥。假设一个工人带着一个装有 4 件工具的工具箱（缓存），当分配给他的大量工作都是相同的时候，这 4 件工具将被反复使用（缓存命中）。

反之，大量的工作都需要不同的工具，情况就不一样了。例如，工人来上班时，并不知道今天会分配给他什么工作。简单来说，今天的工作需要一件不同的工具。由于它不在工具箱（一级缓存）中，那么他就到工具柜（二级缓存）里找。

偶尔，工人可能需要一个工具箱和工具柜里都没有的、特殊的工具或零件，这时他就得停下手中的工作，跑到附近的硬件仓库（全局内存）里，取回他想要的东西。由于公路上可能发生拥堵，或者五金商店门前排起了长队（其他进程争相访问主存），因此，无论是工人还是客户都不知道为了拿到工具需要花费多少时间（延迟）。

显然，工人的时间效率很低。因此，每项工作都需要一个新的、不同的工具或零件，工人就只得去工具柜或者五金商店去取。而这个取的过程，工人并不是忙于手里的工作。

与到硬件仓库里去取工具类似，到硬盘或者固态硬盘（Solid-State Drive，SSD）里取数据，也是很糟糕的。尽管固态硬盘比硬盘要快得多，好比一个普通的送货员需要几天才能把硬盘里的数据送到，而送货员一个晚上就能把固态硬盘里的数据送到，但是与访问全局内存相比，它还是很慢的。

某些现代处理器已经支持多线程。如某些英特尔处理器中，每个处理器核支持 2 个硬件线程即“超线程”(hyperthreading)。接着上面的比喻，“超线程”相当于给工人配了一名助手，并分配给他两项任务。在处理某项任务时，每当需要一个新的工具或零件，工人就派他的助手去取。然后工人就切换去处理另一项任务。假定这个助手总是能够在另一项任务也需要新的工具或零件前就能返回，那么工人就始终处于忙碌的工作状态。

尽管改进后时间效率提高了，但是从硬件仓库（全局内存）里取回新的工具或零件到底有多少延迟还是不清楚。通常，访问全局内存的延迟大概有几百个时钟周期。对于传统的处理器设计，这个问题的答案是不断增大缓存的容量。实际上，为了降低访问仓库的次数，必须采用一个更大的工具柜。

然而，这种办法会增加成本。一则更大的工具柜需要投入更多的资金，二则在一个更大的工具柜里查找工具或零件需要更多的时间。因此，在当前绝大多数处理器设计中，这种方法表现为一个工具柜（二级缓存）加一个货车（三级缓存）。在服务器处理器中，这种情况尤为突出，就如同工厂购买了一辆 18 轮大货车来保证工人总是处于忙碌状态。

因为一个基本的原因（局部性），所以上述工作是必须的。CPU 是设计以运行软件的，而编制软件的程序员并不一定关心局部性。无论处理器是否试图向程序员隐藏局部性，局部性是客观存在的。为了降低访存延迟而需要引入大量的硬件并不能否认局部性是客观存在的这个事实。

GPU 的设计则采用一个不同的方法。它让 GPU 的程序员负责处理局部性，并给程序员提供很多小的工具柜而不是一辆 18 轮大货车，同时给他配备很多工人。

对于GPU程序设计，程序员必须处理局部性。对于一个给定的工作，他需要事先思考需要哪些工具或零件（即存储地址或据结构），然后一次性地把它们从硬件仓库（全局内存）取来，在工作开始时就把它们放在正确的工具柜（片内存储器）里。一旦数据被取来，就尽可能把与这些数据相关的不同工作都执行了，避免发生“取来——存回——为了下一个工作再取”。

因此，“工作——等待——从全局内存取”、“工作—等待—从全局内存取……这样连续的周期就被打破了。我们可以用生产流水线来比拟，一次性提供给工人一筐零件，而不是工人需要一个零件就到仓库管理员那里去取。后者导致工人的大多数时间被浪费了。

这个简单的计划使得程序员能够在需要数据前就把它们装入片内存储器。这项工作既适合于诸如GPU内共享内存的显式局部存储模型，也适合于基于CPU的缓存。在共享内存的情况下，你可以通知存储管理单元去取所需的数据，然后就回来处理关于已有的其他数据的实际工作。在缓存的情况下，你可以用特殊的缓存指令来把你认为程序将要使用的数据，先行装入缓存。

与共享内存相比，缓存的麻烦是替换和对“脏”数据的处理。所谓“脏”数据是指缓存中被程序写过的数据。为了获得缓存空间以接纳新的有用数据，“脏”数据必须在新数据装入之前写回到全局内存。这就意味着，对于延迟未知的全局内存访问，我们需要做两次，而不是一次——第一次是写旧的数据，第二次是取新的数据。

引入受程序员控制的片上内存，带来的好处是程序员可以控制发生写操作的时间。如果你正在进行数据的局部变换，可能就没有必要将变换的中间结果写回全局内存。反之用缓存的话，缓存控制器就不知道哪些数据应该写、哪些数据可以抛弃。因此，它全部写入。这势必增加了很多无用的访存操作，甚至会造成内存接口拥塞。

尽管做了很多工作，但是并不是每个算法都具备“事先预知的”内存访问模式，而程序员正是需要对此进行优化。同时，也不是每个程序员都想处理局部性的事务，所以程序的局部性要么是“与生俱有的”，要么就根本没有。要想深入理解局部性，最好、最有效的方法就是开发一个程序，验证概念，然后琢磨如何改进局部性。

为了使这种方法得到更好的应用，并改进那些没有很好定义数据或执行模式的算法的局部性，新一代GPU（计算能力2.x以上）同时设有一级和二级缓存。它们还可以根据需要配置成缓存或者共享内存，这样程序员就可以针对具体的问题灵活地配置硬件条件了。

2.5 并行处理的类型

2.5.1 基于任务的并行处理

如果仔细分析一个典型的操作系统，我们就会发现它实现的是一种所谓任务并行的并行处理，因为各个进程是不同的、无关的。用户可以在上网阅读文章的同时，在后台播发他音乐库中的音乐。多个CPU核运行不同的应用程序。

就并行程序设计而言，这可以通过编写一个程序来实现，这个程序由多个段组成，这些段将信息从一个应用程序“传递”（通过发送消息）到另一个应用。Linux 操作系统的管道操作符（|）就具有这个功能。一个程序（例如 grep）的输出是另一个程序（例如 sort）的输入。这样，就可以轻松地对一组输入文件进行扫描（通过 grep 程序），以查看是否包含特定的字符，然后输出排好序的结果（通过 sort 程序）。每个程序都分别调度到不同 CPU 核上运行。

一个程序的输出作为下一个程序的输入，这种并行处理称为流水线并行处理（pipeline parallelism）。借助一组不同的程序模块，例如，Linux 操作系统中各种基于文本的软件工具，用户就可以实现很多有用的功能。也许程序员并不知道每个人所需要的输出，但通过共同工作并可以轻松连接在一起的模块，程序员可以为广泛的、各种类型的用户服务。

这种并行处理向着“粗粒度并行处理”（coarse-grained parallelism）发展，即引入许多计算能力很强的处理器，让每个处理器完成一个庞大的任务。

就 GPU 而言，我们看到的“粗粒度并行处理”是由 GPU 卡和 GPU 内核程序来执行的。GPU 有两种方法来支持流水线并行处理模式。一是，若干个内核程序被依次排列成一个执行流，然后不同的执行流并发地执行；二是，多个 GPU 协同工作，要么通过主机来传递数据，要么直接通过 PCI-E 总线，以消息的形式在 GPU 之间直接传递数据。后一种方法，也叫“点对点”（Peer-to-Peer，P2P）机制，是在 CUDA 4.x SDK 中引入的，需要特定的操作系统 / 硬件 / 驱动程序级支持。

和任何生产流水线一样，基于流水线的并行处理模式的一个问题就是，它的运行速度等于其中最慢的部件。因此，若流水线包含 5 个部件，每个部件需要工作 1 秒，那么我们每 1 秒钟就可以产生一个结果。然而，若有一个部件需要工作 2 秒，则整个流水线的吞吐率就降至每 2 秒钟产生一个结果。

解决这个问题的方法是“加倍”(twofold)。让我们用工厂的流水线来打比方。由于工作复杂，Fred 负责的工段需要花费 2 秒钟。如果我们为 Fred 配一名助手 Tim，那么 Fred 就可以把工作一分为二，把一半分给 Tim。这样我们就又回到每个工段只需 1 秒的状态。现在我们拥有了一个 6 段流水线而不是 5 段流水线，但流水线的吞吐率又重新回到每秒钟一个结果。

出于某种考虑，经过精心设计（参见第 11 章），你可以将 4 个 GPU 加到一个桌面 PC 中。这样，若我们仅有一个 GPU，则处理一个特定的工作流花费的时间太长。若我们增加一个 GPU，则这个节点的整体处理能力就提高了。但是我们需要考虑在两个 GPU 之间如何划分工作。简单的 50/50 划分可能不可行。若仅能实现 70/30 划分，则最大收益是当前运行时间的 7/10（70%）。若我们再增加一个 GPU 并能够分给它占总时间 20% 的任务，即按 50/30/20 划分。这样与一个 GPU 相比，加速效果是原来时间的 1/2 或 50%。无论如何，整体的执行时间仍取决于最慢的时间。

为了加速而使用一个 CPU/GPU 组合时，也需要考虑上述问题。如果我们把 80% 的工作从 CPU 移到 GPU 上，而 GPU 计算这些任务仅需原来时间的 10%，那么加速比是多少呢？由于 CPU 花费原来时间的 20%，而 GPU 花费原来时间的 10%，但是它们是并行的，因此决定性因素仍然是 CPU。由于 GPU 是并行地与 CPU 一起工作，且工作时间少于 CPU，所以它

的工作时间就忽略不计了。因此，最大加速比就等于程序执行时间最长那部分占整个程序比例的倒数。

这称为“阿姆达尔法则”（Amdahl’s law），它表示任意加速比的上限。它让我们在一开始，一行代码都还没写的情况下，就知道可能达到的最大加速比。无论如何，你都要做串行操作。即便把所有的计算都移到 GPU 上，你也需要用 CPU 来访问辅存、装入和存回数据。你还需要与 GPU 交换数据以完成输入及输出（I/O）。因此，最大加速比取决于程序中计算或算术部分占整个程序的比例加上剩下的串行部分的比例。

2.5.2 基于数据的并行处理

在过去的二十多年间，计算能力不断增长。现在我们已经拥有了运算速度达到每秒万亿次浮点操作的 GPU。然而，跟不上计算能力增长步伐的是数据的访问时间。基于数据的并行处理的思路是首先关注数据及其所需的变换，而不是待执行的任务。

基于任务的并行处理更适合于粗粒度并行处理方法。让我们看一个对 4 个不同且无关的等长数组分别进行不同变换的例子。我们有 4 个 CPU 核以及 1 个带有 4 个 SM 的 GPU。若对这个问题采用基于任务的分解，则 4 个数组将被分别赋给 4 个 CPU 核或者 GPU 中的每个 SM。对问题的这种并行分解是在只考虑任务或变换，而不考虑数据的思路下进行的。

在 CPU 上，我们将产生 4 个线程或进程来完成任务。在 GPU 上，我们将使用 4 个线程块，并把每个数组的地址分别送给 1 个块。在更新的费米架构和开普勒架构 GPU 上，我们也可以产生 4 个并发运行的内核程序，每个内核程序处理 1 个数组。

基于数据的分解则是将第 1 个数组分成 4 数据块，CPU 中的 1 个核或者 GPU 中的 1 个 SM 分别处理数组中的 1 数据块。处理完毕后，再按照相同的方式处理剩下的 3 个数组。对采用 GPU 处理而言，将产生 4 个内核程序，每个内核程序包含 4 个或者更多的数组块。对问题的这种并行分解是在考虑数据后再考虑变换的思路下进行的。

由于我们的 CPU 仅有 4 个核，所以使我们想到将数据分成 4 数据块。我们既可以让线程 0 处理数组元素 0，线程 1 处理数组元素 1，线程 2 处理数组元素 2，线程 3 处理数组元素 3，以此类推。我们还可以把数组分成 4 数据块，每个线程处理数组中对应的一数据块。

在第一种情况下，线程 0 去取元素 0。由于 CPU 包含多级缓存，数据将存入缓存。通常，三级缓存是被所有的核心共享的。因此，第一次内存访问取来的数据需要分发给所有 CPU 核。相反在第二种情况下，需要进行 4 次不同的内存访问，取来的数据分别存入到三级缓存中的不同缓存中。因为 CPU 核心需要把数据写回内存，所以后一种方法通常更好些。第一种情况下，CPU 核交替地使用数据，这意味着，缓存必须协调和组合来自不同 CPU 核心的写操作，这是很糟糕的思路。

如果算法允许，我们还可以探讨另一种类型的数据并行处理——单指令多数据（Single Instruction，Multiple Data，SIMD）模型。这需要特殊的 SIMD 指令，例如，许多基于 x86 型 CPU 提供的 MMX、SSE、AVX 等。这样，线程 0 就可以取出多个相邻的数组元素，并用一条 SIMD 指令来进行处理。

同样的问题，如果我们用GPU来处理，每个数组则需要进行不同的变换，且每一个变换将映射以被看成一个单独的GPU内核程序。与CPU核不同，每个SM可以处理多个数据块，每个数据块的处理被分成多线程来执行。因此为了提高GPU的使用效率，我们需要对问题进行进一步的分解。通常，我们将块和线程做一个组合，使得一个线程处理一个数组元素。使用CPU，让每个线程处理多个数据，会有很多好处。相比之下，由于GPU仅支持“加载”(load)/“存储”(store)/“移动”(move)三条显式的SIMD原语，那么它的应用受到限制。但这反过来促进了GPU“指令级并行处理”(Instruction-Level Parallelism，ILP)功能的增强。在本书的后面，我们将看到ILP给我们带来的好处。

在费米架构和开普勒架构GPU上，我们有一个共享的二级缓存，它的功能与CPU上的三级缓存相同。因此，在CPU上，一个线程访存的结果可以从缓存直接分布给其他线程。早期的硬件处理器，没有缓存。但是在GPU上，相邻的内存单元是通过硬件合并(组合)在一起进行存取的。因此单次访存的效率就更高了。具体细节可查阅第6章关于内存的介绍。

GPU与CPU在缓存上的一个重要差别就是“缓存一致性”(cache coherency)问题。对于“缓存一致”的系统，一个内存的写操作需要通知所有核的各个级别的缓存。因此，无论何时，所有处理器核看到的内存视图是完全一样的。随着处理器中核数量的增多，这个“通知”的开销迅速增大，使得“缓存一致性”成为限制一个处理器中核数不能太多的一个重要因素。“缓存一致”系统中最坏的情况是，一个内存写操作会强迫每个核的缓存都进行更新，进而每个核都要对相邻的内存单元进行写操作。

相比之下，非“缓存一致”系统不会自动地更新其他核的缓存。它需要由程序员写清楚每个处理器核输出的各自不同的目标区域。从程序的视角看，这支持一个核仅负责一个输出或者一个小的输出集。通常，CPU遵循“缓存一致”原则，而GPU则不是。故GPU能够扩展到一个芯片内具有大数量的核心(流处理器簇)。

为了简单起见，我们假设4个线程块构成一个GPU内核程序。这样，GPU上就有4个内核程序，而CPU上有4个进程或线程。CPU也可能支持诸如“超线程”这样的机制，使得发生停顿事件(例如，访问缓存不命中)时，CPU能够处理其他的进程或线程。这样，我们就可以把CPU上进程的数量提高到8，从而得到性能的提升。然而，在某个时间点上，甚至在进程数小于核数的时候，CPU可能会遇到线程过多的情况。

这时，内存带宽就变得很拥挤，缓存的利用率急剧下降，导致性能降低而不是增高。

在GPU上，4个线程块无论如何也不能满足4个SM的处理能力。每个SM最大能处理8个线程块(开普勒架构中为16个线程块)。因此，我们需要$8 \times 4=32$个线程块才能填满4个SM。既然需要完成4个不同的操作，我们就可以借助其流处理功能(参见第8章关于使用多GPU的内容)，在费米架构GPU上同时启动4个内核程序。最终，我们总共可以启动16个线程块来并行处理4个数组。若采用CPU，则一次处理一个数组效率会更高些，因为这会提高缓存的利用率。总之，使用GPU时，我们必须确保总是有足够多的线程块(通常至少是GPU内SM数量的8 ~ 16倍)。

2.6 弗林分类法

前面我们用到一个词“SIMD”。它来源于划分不同计算机架构的弗林分类法（Flynn’s taxonomy）。根据弗林分类法，计算机的结构类型有：

- SIMD——单指令，多数据
- MIMD——多指令，多数据
- SISD——单指令，单数据
- MISD——多指令，单数据

绝大多数人熟悉的标准串行程序设计遵循的是SISD模型，即在任何时间点上只有一个指令流在处理一个数据项，这相当于一个单核CPU在一个时刻只能执行一个任务。当然，它也可以通过所谓的“分时”（time-slicing）机制，即在多个任务间迅速切换，达到“同时”执行多个任务的效果。

我们今天看到的双核或4核桌面计算机就是MIMD系统。它具有一个线程或进程的工作池，操作系统负责逐个取出线程或进程，将他们分配到N个CPU核中的一个上执行。每个线程或进程具有一个独立的指令流，CPU内部包含了对不同指令流同时进行解码所需的全部控制逻辑。

SIMD系统尽可能简化了它的实现方法，针对数据并行模型，在任何时间点，只有一个指令流。这样，在CPU内部就只需要一套逻辑来对这个指令流进行解码和执行，而无须多个指令解码通路。由于从芯片内部移除了部分硅实体，因此相比它的MIMD兄弟，SIMD系统就可以做得更小、更便宜、能耗更低，并能够在更高的时钟频率下工作。

很多算法只需要对很少量的数据点进行这样或那样的处理。很多数据点常常被交给一条SIMD指令。例如，所有的数据点可能都是加上一个固定的偏移量，再乘以一个数据，例如放大因子。这就很容易地用SIMD指令来实现。实际上就是你编写的程序，从“对一个数据点进行一个操作”改为“对一组数据进行一个操作”。既然这组数据中每一个元素需要进行的操作或变换是固定的，所以“访问程序存储区取指令然后译码”只需进行一次。由于数据区间是有界且连续的，所以数据可以全部从内存中取出，而不是一次只取一个字。

然而，如果算法是对一个元素进行A变换而对另一个元素进行B变换，而对其他元素进行C变换，那么就很难用SIMD来实现了。除非这个算法由于非常常用而采用硬编码在硬件中实现，例如“先进的加密标准”（Advanced Encryption Standard，AES）”和H.264（一种视频压缩标准）。

与SIMD稍有不同，GPU实现的是被英伟达称为“单指令多线程”（Single Instruction Multiple Thread，SIMT）的模型。在这种模型中，SIMD指令的操作码跟CPU中硬件实现的方式不同，它指示的并不是一个固定的功能。程序员需要通过一个内核程序，指定每个线程的工作内容。因此，内核程序将统一读入数据，程序代码根据需要执行A、B或C变换。实际上，A、B、C是通过重复指令流而顺序执行的，只不过每次执行时屏蔽掉无须参与的线程。与仅支持SIMD的模型相比，从理论上说，这个模型更容易掌握。

2.7 常用的并行模式

很多并行处理问题都可以按照某种模式来分析。在很多程序中，尽管并不是每个人都意识到它们的存在，但我们也可以看到不同的模式，按照模式来分析，使得我们能够对问题进行深入的解构或抽象，这样就很容易找到解决问题的办法。

2.7.1 基于循环的模式

几乎任何一个编写过程序的人都会对循环很熟悉。不同循环语句（如，for、do...while、while）的主要区别在于入口、退出条件以及两次循环迭代之间是否会产生依赖。

循环的迭代依赖是指循环的一次迭代依赖于之前的一次或多次先前迭代的结果。这些依赖将并行算法的实现变得十分困难，而这是我们希望消除的。如果消除不了，则通常将循环分解成若干个循环块，块内的迭代是可以并行执行的。循环块0执行完后将结果送给循环块1，然后送给循环块2，以此类推。本书的后面有一个例子就是采用这种方法来处理前缀求和（prefix-sum）算法。

基于循环的迭代是实现并行化的模式中最容易的一个。如果循环间的依赖被消除掉了，那么剩下的问题就是在可用的处理器上如何划分工作。划分的原则是让处理器间通信量尽可能少，片内资源（GPU上的寄存器和共享内存，CPU上的一级/二级/三级缓存）的利用率尽可能高。糟糕的是，通信开销通常会随着分块数目的增多而迅速增大，成为提高性能的瓶颈、系统设计的败笔。

对问题的宏观分解应该依据可用的逻辑处理单元的数量。对于CPU，就是可用的逻辑硬件线程的数量；对于GPU，就是流处理器簇（SM）的数量乘以每个SM的最大工作负载。依赖于资源利用率、最大工作负荷和GPU模型，SM的最大工作负载取值范围是1~16块。请注意，我们使用的词是逻辑硬件线程而不是物理硬件线程。某些英特尔CPU采用所谓的“超线程”技术，在一个物理CPU核上支持多个逻辑线程。由于GPU在一个SM内运行多个线程块，所以我们需要用SM的数量乘以每个SM支持的最大块数。

在一个物理设备上支持多个线程可以使设备的吞吐率最大化，也就是说在某线程等待访存或者I/O类型的操作时，设备可以处理其他线程的工作。这个倍数的选择有助于在GPU上实现负载平衡（load balancing），并可以应用于改进新一代GPU。当数据划分导致负载不均时，这一点表现得尤为明显——某些块花费的时间远远大于其他块。这时，可以用几倍于SM数目的数量作为划分数据的基础。当一个SM空闲下来后，它可以去存放待处理块的“池子”里取一个块来处理。

然而对于CPU，过多的线程数量却可能会导致性能下降，这主要是由于上下文切换时，操作系统以软件的形式来完成。对缓存和内存带宽竞争的增多，也要求降低线程的数量。因此对于一个基于多核CPU的解决方案，通常它划分问题的粒度要远大于面向GPU的划分粒度。如果在GPU上解决同一个问题，你则要对数据进行重新划分，把它们划分成更小的数据块。

当考虑采用循环并行来处理一个串行程序时，最关键的是发现隐藏的依赖关系。在循环

体中仔细地查找，确保每一次迭代的计算结果不会被后面的迭代使用。对于绝大多数循环而言，循环计数通常是从 0 ~设置的最大值。当遇到反过来采用递减计数的循环，你就应该小心些。为什么这个程序员采用相反的计数方法呢？很可能就是循环中存在某种依赖。如果不了解这一点就将循环并行化了，很可能把其中的依赖破坏掉。

我们还需要考虑的另外一种情况是有一个内循环和多个外循环。如何将它们并行化呢？对于 CPU，由于你只有有限的线程，所以只能将这些外循环并行化。不过，像前面提到的那样，可以这样处理的前提是不存在循环迭代依赖。

如果分配给 GPU 执行的内循环是很小的，通常用一个线程块内的线程来处理。由于循环迭代是成组进行的，所以相邻的线程通常访问相邻的内存地址，这就有助于我们利用访存的局部性，这一点对 CUDA 程序设计十分重要。外循环的并行处理都是用线程块来实现的，这部分内容将在第 5 章详细介绍。

考虑到大多数循环是可以展开的，因此把内循环和外循环合并成一个循环。例如，图像处理算法中，沿 X 轴的处理是内循环，而沿 Y 轴的处理是外循环。可以通过把所有像素点看成是一个一维数组来展开循环，这样迭代就是沿像素点而不是图像坐标进行。尽管编程时麻烦一些，但是在每次循环包含的迭代次数很小时，收效很大。因为这些小的循环带来的循环开销相对每次迭代完成的有效工作比较大，所以这些循环的效率很低。

2.7.2 派生 / 汇集模式

派生 / 汇聚模式是一个在串行程序设计中常见的模式，该模式中包含多个同步点而且仅有一部分内容是可以并行处理的，即首先运行串行代码，当运行到某一点时会遇到一个并行区，这个并行区内的工作可以按某种方式分布到 P 个处理器上。这时，程序就“派生”（fork）出 N 个线程或进程来并行地完成这些工作。N 个线程或进程的执行是独立的、互不相关的，当其工作完成后，则“汇聚”（join）起来。在 OpenMP 中常常可以看见这种处理方法——程序员用编译指令语句定义可并行区，并行区中的代码被分成 N 个线程，随后再汇聚成单个线程。

如图 2-2 所示，有一个输入的数据项队列和三个处理单元（即 CPU 核），输入数据队列被分成三个小的数据队列，一个处理单元处理一个小的数据队列，每个队列的处理是互不相关的，处理结果分别写在结果队列的相应位置。

通常，派生 / 汇聚模式是采用数据的静态划分来实现，即串行代码派生出 N 个线程并把数据集等分到这 N 个线程上。如果每个数据块的处理时间相同的话，这种划分方法是很好的。但是，由于总的执行时间等于最慢线程的执行时间，所

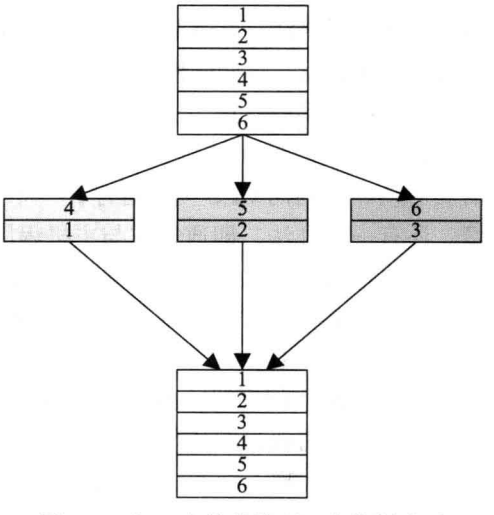


图 2-2 由 N 个线程处理一个数据队列

以如果分配给一个线程太多的工作，它将成为决定总时间的一个因素。

诸如OpenMP这样的系统跟GPU的方案类似，实现动态的调度分配。具体办法是，先创建一个“线程池”（对GPU而言是一个“块池”），然后池中的线程取一个任务执行，执行完后再取下一个。假设有1个任务需要10个单位时间才能完成，而其余20个任务需要1个单位时间就能完成，则它们只能分配到空闲的处理器核上执行。现在有一个双核处理器，则把那个需要10个单位时间的大任务和5个需要1个单位时间的小任务分配给核1，而把其余的15个需要1个单位时间的小任务分配给核2。这样，核1与核2就基本上可以同时完成任务了。

在图2-2的例子中，我们选择派生3个线程。既然队列中有6个数据，为什么不派生6个线程呢？这是因为在实际工作中，我们要处理的数据多达好几百万，无论用哪种方法，派生一百万个线程都会使任何一个操作系统以某种方式崩溃掉。

通常，操作系统执行的是一个“公平的”调度策略。因此，每个线程都需要按顺序，分配到4个可用的处理器核中的某一个上处理，每个线程都需要一个它自己的内存空间，例如，在Windows操作系统中，每个线程需要1MB的栈空间。这就意味着，在派生出足够多的线程前，我们已经迅速地用尽了全部的内存空间。

因此，对于CPU而言，程序员或者多数多线程库通常是按照处理器的个数来派生相同数目的逻辑处理器线程。由于CPU创建或删除一个线程的开销是很大的，而且线程过多也会降低处理器的利用率，所以常常使用一个“工人”线程池，池中的“工人”每次从待处理的任务队列中取一个任务来处理，处理完后再取下一个。

对于GPU则相反，我们的确需要成千上万个线程。我们还是使用在很多先进的CPU调度程序中使用过的线程池的概念，不过将“线程池”改为“线程块池”更好。GPU上可以并发执行的“线程块”的数目存在一个上限。每个线程块内包含若干个线程。每个线程块内包含的线程的数目和并发执行的“线程块”的数目会随着不同系列的GPU而不同。

派生/汇聚模式常常用于并发事件的数目事先并不确定的问题。遍历一个树形结构或者路径搜索这类算法，在遇到另一个节点或路径时，就很可能会派生出额外的线程。当所有的路径都被考查后，这些线程就汇聚回线程池中或者汇聚后又开始新一轮的派生。

由于在启动内核程序时，块/线程的数量是固定的，所以GPU并不是天生就支持这种模式。额外的块只能由主机程序而不是内核程序启动。因此，在GPU上实现这类算法一般都需要启动一系列的GPU内核程序，一个内核程序要产生启动下一个内核程序所需的工作环境。还有一种办法，即通知或与主机程序共同，启动额外的并发内核程序。因为GPU是被设计来执行固定数目的并发线程，所以无论哪种方法实际效果都不算太好。为了解决这个问题，开普勒架构GPU引入了“动态并行性”（dynamic parallelism）的概念。关于这个概念的更多内容，请参见第12章。

在求解某些问题时，内核程序内部的并发性会不断变化，内部也会出现一些问题。为此，线程之间需要进行通信与协调。在GPU的一个线程块内，线程之间通信与协调可以通过很多方法来实现。例如，假设你有一个8×8的块矩阵，很多块仅需要64个工作线程。然而，很可能其他块却需要使用256个线程。你可以在每个块上同时启动256个线程，这时多

数线程处于空闲状态直到需要它们进行工作。由于这些空闲进程占用了一定的资源，会限制整个系统的吞吐率，但它们在空闲时不会消耗 GPU 的任何执行时间。这样就允许线程使用靠近处理器的更快的共享内存，而不是创建一系列需要同步的操作步骤，而同步这些操作步骤需要使用较慢的全局内存并启动多个内核程序。内存的类型将在第 6 章介绍。

最后，新的 GPU 支持更快的原子操作和同步原语。除了可以实现同步外，这些同步原语还可以实现线程间通信，本书的后面部分将给出这方面的例子。

2.7.3 分条 / 分块

使用 CUDA 来解决问题，都要求程序员把问题分成若干个小块，即分条 / 分块。绝大多数并行处理方法也是以不同的形式来使用“条 / 块化”的概念。甚至像气候模型这样巨大的超级计算问题也必须分为成千上万个块，每个块送到计算机中的一个处理单元上去处理。这种并行处理方法在可扩展方面具有很大的优势。

在很多方面，GPU 与集成在单个芯片上的对称多处理器系统非常类似。每个流处理器簇（SM）就是一个自主的处理器，能够同时运行多个线程块，每个线程块通常有 256 或者 512 个线程。若干个 SM 集成在一个 GPU 上，共享一个公共的全局内存空间。它们同时工作时，一个 GPU（GTX680）的峰值性能可达 3 Tflops。

尽管峰值性能会给你留下深刻的印象，但是达到这个性能却需要一个精心设计的程序，因为这个峰值性能并不包括诸如访存这样的操作，而这些操作却是影响任何一个实际程序性能的关键因素。无论在什么平台上，为了达到高性能，就必须很好地了解硬件的知识并深刻理解两个重要的概念——并发性和局部性。

许多问题中都存在并发性。可能是由于先前串行程序的背景，你也许不能立刻就看出问题中的并发性。而“条 / 块模型”就很直观地展示了并发性的概念。在二维空间里想象一个问题——数据的一个平面组织，它可以理解为将一个网格覆盖在问题空间上。在三维空间里想象一个问题，就像一个魔方（Rubik's Cube），可以把它理解为把一组块映射到问题空间中。

CUDA 提供的是简单二维网格模型。对于很多问题，这样的模型就足够了。如果在一个块内，你的工作是线性分布的，那么你可以很好地将其分解成 CUDA 块。由于在一个 SM 内，最多可以分配 16 个块，而在一个 GPU 内有 16 个（有些是 32 个）SM，所以把问题分成 256 个甚至更多的块都可以。实际上，我们更倾向于把一个块内的元素总数限制为 128、256 或者 512，这样有助于在一个典型的数据集内划分出更多数量的块。

当考虑并发性时，还可以考虑是否可以采用指令级并行性（ILP）。通常，人们从理论上认为一个线程只提供一个数据输出。但是，如果 GPU 上已经充满了线程，同时还有很多数据需要处理，这时我们能够进一步提高吞吐量吗？答案是肯定的，但只能借助于 ILP。

实现 ILP 的基础是指令流可以在处理器内部以流水线的方式执行。因此，与“顺序执行 4 个加法操作”（压入—等待—压入—等待—压入—等待—压入—等待）相比，“把 4 个加法操作压入流水线队列、等待然后同时收到 4 个结果”（压入—压入—压入—压入—等待）的效率更高。对于绝大多数 GPU，你会发现每个线程采用 4 个 ILP 级操作是最佳的。第 9 章中有更

详细的研究和例子。如果可能的话，我们更愿意让每个线程只处理 N 个元素，这样就不会导致工作线程的总数变少了。

2.7.4 分而治之

分而治之模式也是一种把大问题分解成的小问题的模式，其中每个小问题都是可控制的。通过把这些小的、单独的计算汇集在一起，使得一个大的问题得到解决。

常见的分而治之的算法使用“递归”(recursion) 来实现，“快速排序”(quick sort) 就是一个典型的例子。该算法反复递归地把数据一分为二，一部分是位于支点 (pivot point) 之上的那些点，另一部分是位于支点之下的那些点。最后，当某部分仅包含两个数据时，则对它们做“比较和交换”处理。

绝大多数递归算法可以用迭代模型来表示。由于迭代模型较适合于 GPU 基本的条块划分模型，所以该模型易于映射到 GPU 上。

费米架构 GPU 也支持递归算法。尽管使用 CPU 时，你必须了解最大调用深度并将其转换成栈空间使用。所以你可以调用 API cudaDeviceGetLimit() 来查询可用的栈空间，也可以调用 API cudaDeviceSetLimit() 来设置需要的栈空间。如果没有申请到足够的栈空间，CPU 将产生一个软件故障。诸如 Parallel Nsight 和 CUDA-GDB 这样的调试工具可以检测出像“栈溢出”(stack overflow) 这样的错误。

在选择递归算法时，你必须在开发时间与程序性能之间做出一个折中的选择。递归算法较易于理解，与将其转换成一个迭代的方法相比，编码实现递归算法也比较容易。但是所有的递归调用都需要把所有的形参和全部的局部变量压入栈。GPU 和 CPU 实现栈的方法是相同的，都是从全局内存中划出一块存储区间作为栈。尽管 CPU 和费米架构 GPU 都用缓存栈，但与使用寄存器来传递数据相比，这还是很慢的。所以，在可能的情况下最好还是使用迭代的方法，这样可以获得更好的执行性能，并可以在更大范围的 GPU 硬件上运行。

2.8 本章小结

至此，我们已经对并行处理的概念及其如何应用到 GPU 工业领域中，有了一个全面的了解。本书的写作目的并不是全面深入地论述并行处理，因为这方面的书籍已经很多了。我们只是希望读者能够感受到并行程序设计的理念，不再按照串行程序设计的思路来考虑程序设计问题。

在后续的章节中，我们将通过分析实际例子，详细介绍上述基本概念。我们还将分析并行前缀求和算法。这个算法允许对一个共享的数组同时进行多个写操作，而不会发生“一个写操作写到另外一个写操作的数据上”这样的错误。这类问题在串行程序设计中不会出现，无须考虑。

随着并行处理带来的复杂度的提高，需要程序员把并发性和局部性当作关键问题事先考虑。在设计面向 GPU 的任何软件时，都应该时刻把这两个概念牢记于心。

第 3 章

CUDA 硬件概述

3.1 PC 架构

首先，我们看看当下许多 PC 中都使用的酷睿 2（Core 2）处理器的典型架构，然后分析一下它是如何影响我们使用 GPU 加速器的（如图 3-1 所示）。

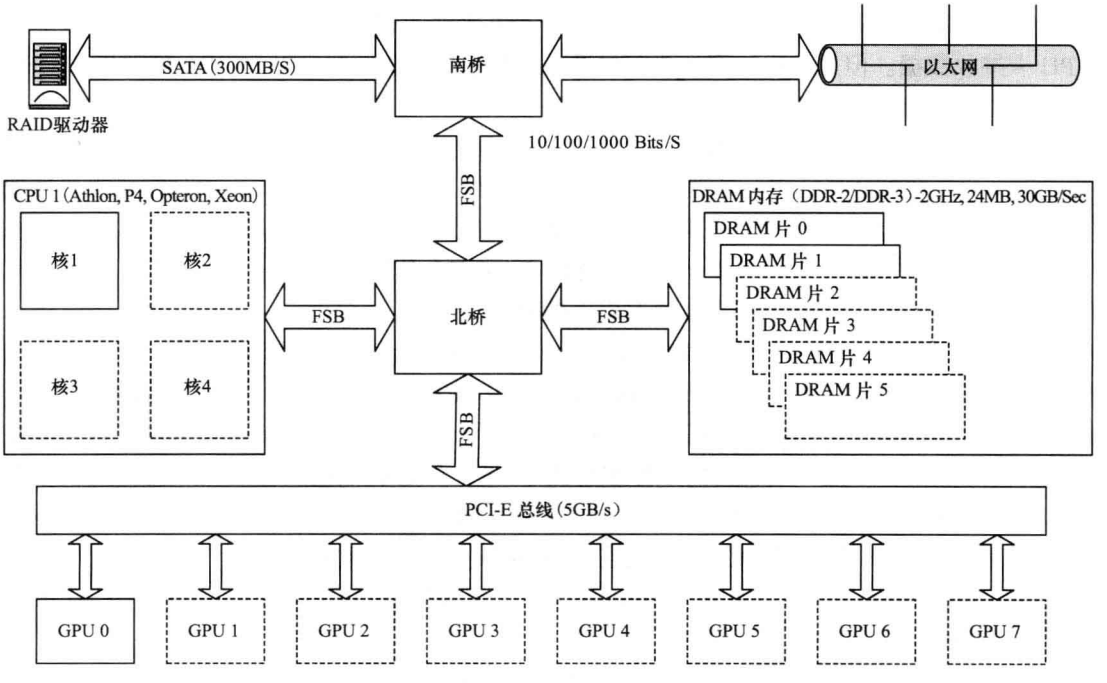


图 3-1　典型的酷睿 2（Core 2）系列处理器的结构图

由于所有的 GPU 设备都是通过 PCI-E（Peripheral Communications Interconnect Express）总线与处理器相连，所以我们以 PCI-E 2.0 总线标准来讨论本章内容。PCI-E 2.0 是目前最快的总线标准，它的传输速率为 5GB/s。在撰写本书的过程中，PCI-E 3.0 已经问世了，它的带

宽明显提高了。

然而，为了从处理器中获取数据，我们需要通过与低速前端总线（Front-Side Bus，FSB）连接的北桥（Northbridge）。理论上，FSB的时钟频率最高只能达到1600 MHz，而很多实际设计的产品就更低了。这通常只是一个高速处理器时钟频率的1/3。

访问内存也需要经过北桥，访问外设则需要经过北桥和南桥（Southbridge）。北桥服务于所有的高速设备，如内存、CPU、PCI-E总线接口等；而“南桥”则服务于低速设备，如硬盘、USB、键盘、网络接口，等等。当然，把硬盘控制器直接连接到PCI-E总线接口上也是可能的。实际上，在这样的系统中，这是要获得高速RAID数据访问的唯一正确方式。

PCI-E是一个很有意思的总线。与其上一代PCI（Peripheral Component Interconnect，外围设备互连）总线不同，PCI-E提供一个确定的带宽。在原先的PCI系统中，每一个设备都可以使用总线的全部带宽，但一次只能让一个设备使用。因此，你增加的PCI卡越多，每个卡能够获得的可用带宽就越少。PCI-E总线通过引入PCI-E通道（lane）解决了这个问题。这些通道是一些高速的串行链路，这些链路组合在一起构成了X1、X2、X4、X8或X16链路。目前，绝大多数GPU使用的至少是如图3-1所示的PCI-E 2.0的X16规范，此配置下提供5 GB/s的全双工总线。这意味着，数据的传入与传出可以同时进行并享有同样的速度。也就是说，我们在以5GB/s的速度向GPU卡传送数据的同时，还能够以5GB/s的速度从GPU卡接收数据。但是，这并不意味着如果不接收数据，我们就可以10GB/s的速度向GPU卡传送数据（即带宽是不可以累加的）。

在一个典型的超级计算机或者一个台式机应用程序中，我们常常需要处理一个很大的数据集。一个超级计算机需要处理上千万亿字节（PB）的数据，而一个桌面计算机也要处理数十亿字节（GB）的高分辨率视频图像。这两种情况都需要从外设取来大量的数据，单块100MB/s的硬盘每分钟只能上传6GB的数据。按照这个速度，读取一个标准的1万亿字节（TB）硬盘上的全部内容需要两个半小时以上的时间。

如果使用集群系统中常用的MPI（Message Passing Interface）作为通信软件，像图3-1这样将以太网（Ethernet）接口连接到南桥芯片而不是PCI-E总线，构成的通信延迟是很大的。因此，诸如InfiniBand这样的专用高速互连设备或者10千兆位（Gigabit）以太网卡常常连接到PCI-E总线上。不过，这就占用了原本可用于GPU的总线插槽。之前，并没有直接用于GPU的MPI接口函数。这类系统的所有通信都需要经过PCI-E总线连通到CPU，然后再原路返回。CUDA 4.0 SDK提供的GPU直连（GPU-Direct）技术就解决了这个问题。借助SDK的支持，InfiniBand卡就可以直接与GPU通信，而无须先经过CPU转发。SDK中的这项升级还支持GPU与GPU的直接通信。

Nehalem架构中有很多新的变化，其中最主要的变化就是用X58芯片组代替了“北桥”和“南桥”芯片。Nehalem架构引入了快速通道互联（Quick Path Interconnect，QPI）技术，该技术明显优于“前端总线”（Front Side Bus，FSB），达到了与AMD公司的超传输（HyperTransport）相当的水平。QPI是一种高速的、可用于与其他设备或CPU直接通信的互连结构。在一个标准的Nehalem系统中，QPI的作用是连接内存子系统，并通过X58芯片组

连接 PCI-E 子系统（如图 3-2 所示）。与 Extreme/Xeon 型号的处理器配合时，QPI 的工作速率要么是 4.8GT/s[⊖]，要么是 6.4GT/s。

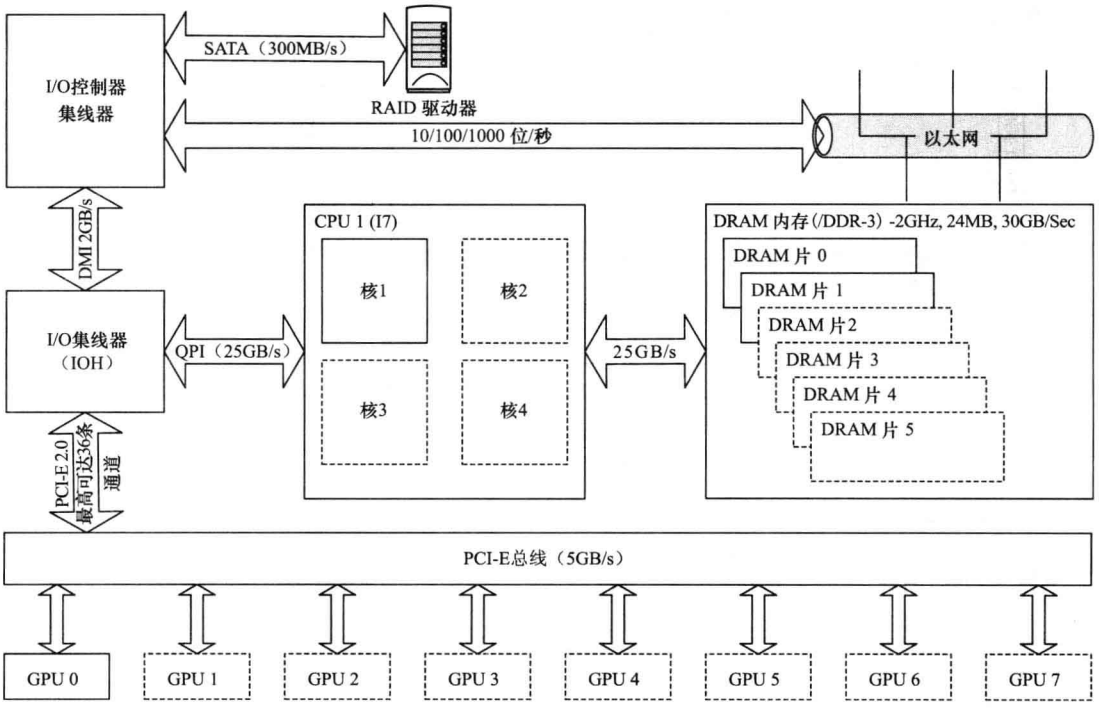


图 3-2 Nehalem/X58 系统

当使用 X58 芯片组和 LGA1366 处理器插槽时，共计有 36 个 PCI-E 通道。这就意味着，配置为 X16 时最多可以支持 2 个 GPU 卡，配置为 X8 时最多可以支持 4 个 GPU 卡。在 LGA2011 插槽出现之前，这是为 GPU 卡数据传输提供的最好的带宽解决方案。

在较小的 P55 芯片组中，也可以使用 X58 方案。不过，这时仅有 16 个 PCI-E 通道。这就意味着，配置为 X16 时仅支持 1 个 GPU 卡，配置为 X8 时最多可以支持 2 个 GPU 卡。

从 I7/X58 芯片组开始，英特尔公司引入了如图 3-3 所示的沙桥（Sandy Bridge）设计方案。其最引人注目的改进之一是支持传输速率可达 600MB/s 的 SATA-3 标准。通过与固态硬盘（SSD）结合，在存 / 取数据时，“沙桥”可以提供很高的输入 / 输出性能。

沙桥的另外一个主要进步是引入了 AVX（Advanced Vector Extensions，高级向量扩展）指令集，该指令集也同时被 AMD 的处理器支持。AVX 允许向量指令最多可以并行处理 4 个双精度浮点数（256 位或 32 字节）。这是一个很有趣的改进，可以使 CPU 上的计算密集型应用程序获得一个很高的加速比。

⊖ GT/s 指 QPI 总线速率，是实际时钟频率的 2 倍。

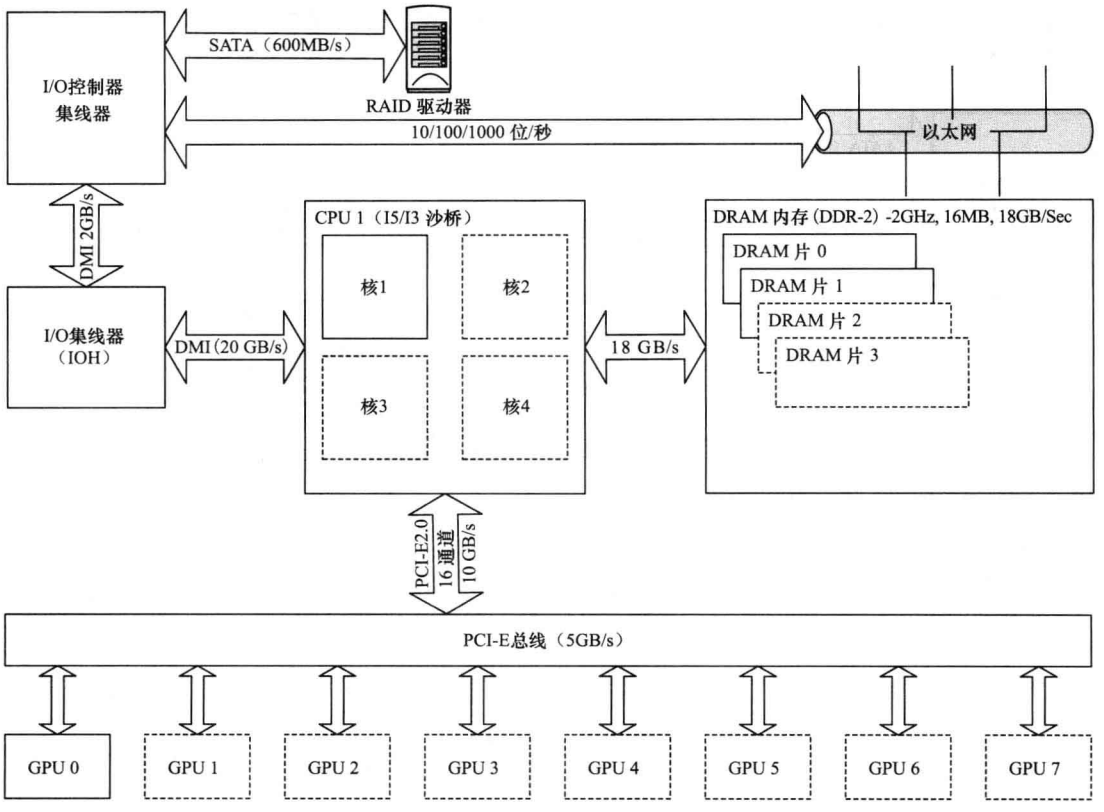


图 3-3 沙桥设计图

然而，需要注意的是，LGA 1155 插槽的沙桥设计方案存在一个很大的问题，就是仅支持 16 个 PCI-E 通道，这将 PCI-E 的理论带宽限制在 16GB/s 以内，而实际带宽为 10GB/s。在桌面处理器上，英特尔公司摒弃了向 CPU 中集成更多 PCI-E 通道的路线。仅仅是面向服务器的 LGA 2011 “沙桥 -E” 插槽，才拥有数量可观的 PCI-E 通道（40 个）。

相比英特尔公司的技术，AMD 的设计方案又是怎样的呢？与英特尔公司不断地减少 PCI-E 通道的数量（除服务器系列产品外）不同，AMD 始终保持一个固定不变的数量。AMD 的 FX 芯片组，要么支持 2 个 X16 设备，要么支持 4 个 X8 PCI-E 设备。AMD 3+ 插槽与 990FX 芯片组一起提供了强劲的工作动力，即 6GB/s 的 SATA 端口以及最多可达 4 个的 X16 PCI-E 插槽（通常以 X8 的速度运行）。

英特尔与 AMD 的主要差别之一是，同一价位对应的处理核数是不同的。如果你考虑的仅仅是实际处理器核而不是逻辑核（如超线程），那么对于同一价位，AMD 的处理器中通常会有较多的核。然而，英特尔处理器核的性能要高一些。因此，选择哪一种处理器主要取决你需要支持的 GPU 的数量以及分配给处理器核的工作负载水平。

在英特尔的设计方案中，你会发现，除了连接主存的带宽有差别外，系统周围的带宽都是基本相同的。在高端系统中，英特尔使用 3 个或者 4 个通道的内存；仅在低端系统中，英

特尔才使用双通道的内存。而 AMD 只使用双通道的内存，这就导致 CPU 与内存之间的可用带宽明显减少（如图 3-4 所示）。

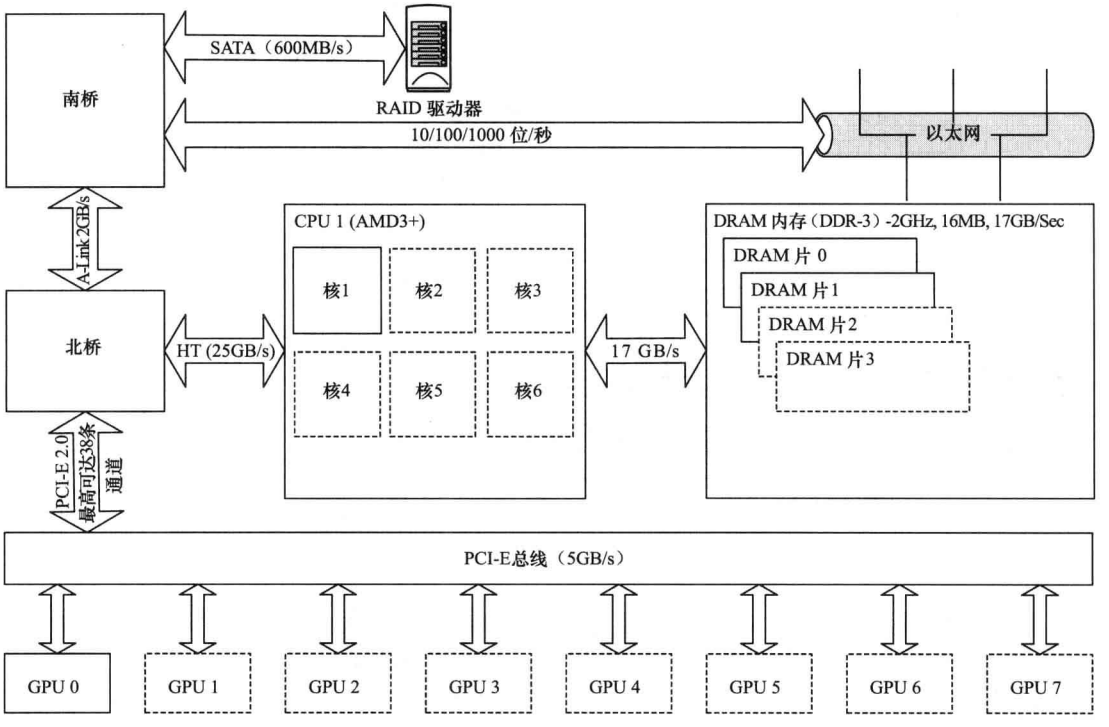


图 3-4 AMD 结构图

相比英特尔的产品，AMD 芯片组的一个重要优点是支持最高可达 6 个的 6GB/s 的 SATA（Serial ATA）端口。如果考虑到系统中最慢的部件通常会限制整个系统的吞吐率，你就需要在这方面好好考虑再做选择。如果系统使用了好几个固态硬盘，SATA3 将很快使“南桥”带宽超载。使用 PCI-E 总线也许是一个更好的选择，但是成本将会显著提高。

3.2 GPU 硬件结构

GPU 的硬件结构与 CPU 的硬件结构有着根本的不同，图 3-5 显示了一个位于 PCI-E 总线另一侧的多 GPU 系统。

从图中可以看出，GPU 的硬件由以下几个关键模块组成：

- 内存（全局的、常量的、共享的）
- 流处理器簇
- 流处理器

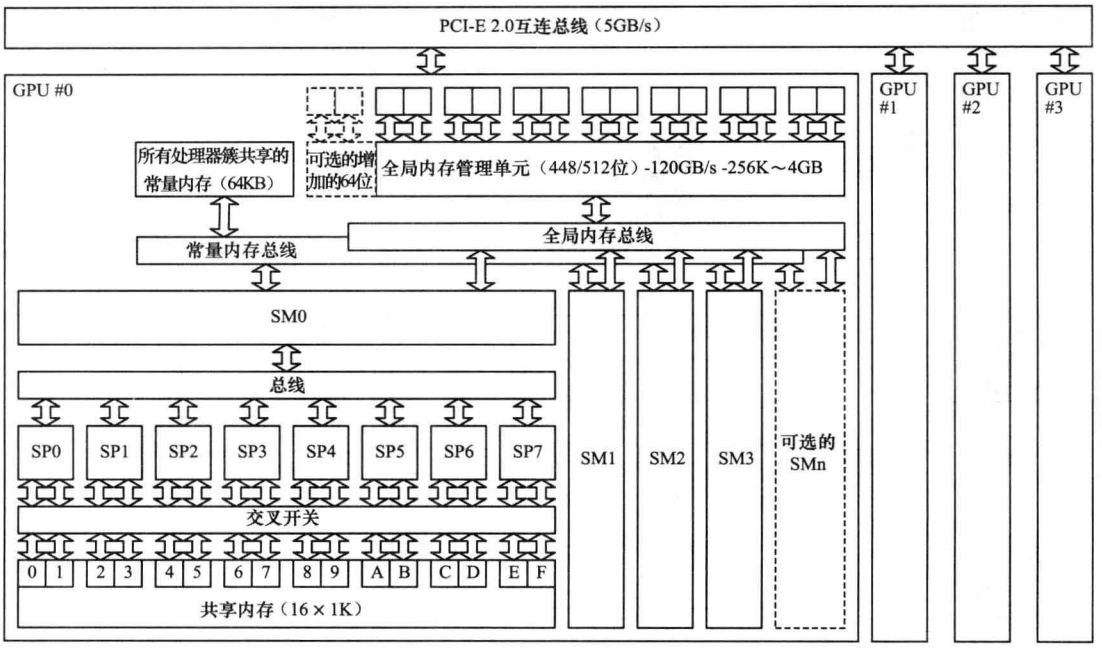


图 3-5 GPU（G80/GT200）卡的组成模块图

这里最值得注意的是，GPU 实际上是一个 SM 的阵列，每个 SM 包含 N 个核（G80 和 GT200 中有 8 个核，费米架构中有 32 ~ 48 个核，开普勒架构中至少再增加 8 个核，如图 3-6 所示）。一个 GPU 设备中包含一个或多个 SM，这是处理器具有可扩展性的关键因素。如果向设备中增加更多的 SM，GPU 就可以在同一时刻处理更多的任务，或者对于同一任务，如果有足够的并行性的话，GPU 可以更快地完成它。

像 CPU 一样，如果程序员编写的代码将处理器使用核的数量限制为 N 个，比如 2 个，那么即便是 CPU 厂商制造出 4 核的设备，用户也不会从中受益。因此，当计算机从双核 CPU 过渡到 4 核 CPU 时，很多软件都必须重写以利用新增加的核。通过不断地增加 SM 的数量以及每个 SM 中的核数，英伟达硬件的性能持续地提高。设计软件时，应该意识到下一代处理器中 SM 的数量或者每个 SM 中的核数可能翻一番。

现在让我们更深入地看看 SM。每个 SM 都是由不同数量的一些关键部件组成，为了简单起见，没有在图中画出。最重要的部分是每个 SM 中有若干个 SP，图中显示的是 8 个 SP，在费米架构中增加至 32 ~ 48 个，在开普勒架构中增加到 192 个。毋庸置疑，下一代产品中每个 SM 中 SP 的数量极有可能继续增加。

每个 SM 都需要访问一个所谓的寄存器文件（register File），这是一组能够以与 SP 相同速度工作的存储单元，所以访问这组存储单元几乎不需要任何等待时间。不同型号 GPU 中，寄存器文件的大小可能是不同的。它用来存储 SP 上运行的线程内部活跃的寄存器。另外，还有一个只供每个 SM 内部访问的共享内存（shared memory），这可以用作“程序可控的”高速缓存。与 CPU 内部的高速缓存不同，它没有自动完成数据替换的硬件逻辑——它完全是由

程序员控制的。

对于纹理内存（texture memory）、常量内存（constant memory）和全局内存（global memory），每一个 SM 都分别设置有独立访问它们的总线。其中，纹理内存是针对全局内存的一个特殊视图，用来存储插值（interpolation）计算所需的数据，例如，显示 2D 或 3D 图像时需要的查找表。它拥有基于硬件进行插值的特性。常量内存用于存储那些只读的数据，所有的 GPU 卡均对其进行缓存。与纹理内存一样，常量内存也是全局内存建立的一个视图。

图形卡通过 GDDR（Graphic Double Data Rate）接口访问全局内存。GDDR 是 DDR（Double Data Rate）内存的一个高速版本，其内存总线宽度最大可达 512 位，提供的带宽是 CPU 对应带宽的 5 ~ 10 倍，在费米架构 GPU 中最高可达 190GB/s。

每个 SM 还有两个甚至更多的专用单元（Special-Purpose Unit，SPU），SPU 专门执行诸如高速的 24 位正弦函数 / 余弦函数 / 指数函数操作等类似的特殊硬件指令。在 GT200 和费米架构 GPU 上还设置有双精度浮点运算单元。

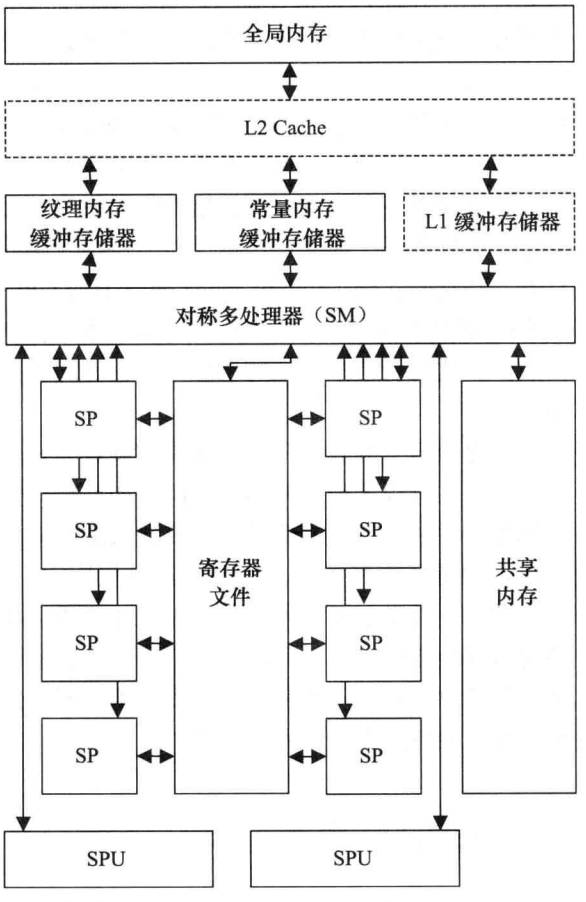


图 3-6 SM 内部组成结构图

3.3 CPU 与 GPU

现在你已经对 GPU 的硬件结构有了初步的了解，你也许会认为它很有意思。但对于程序设计，这意味着什么呢？

参加过大型项目开发的人都知道，项目开发分成不同的阶段，每个阶段的任务由不同的工作组来完成。可能有一个软件规格指定组，一个设计组、一个编码组和一个测试组。让团队中每个人完全了解开发链中在他上游和下游的工作，这对高质量地完成项目开发是很有益处的。

以测试为例，如果设计人员不考虑测试，那么他就不会在程序中设置用于检测因具体软件而引发的硬件故障的测试手段。如果只有发生了硬件失效，测试组才能测试出此硬件故障，那么只能修改硬件让它失效。无疑这是很难做到的。相反，编程人员设计一个反映硬件错误标志的软件标志就容易得多，这样硬件故障就很容易检测出来。如果在测试组里工作，你就会看到如果不这么做，测试工作有多难。除非你狭隘地看待你的岗位职责，你可能才会

说测试不关我的事。

最优秀的工程师都是那些关注在他工作流程之前和之后工作的人。作为软件工程师，知道硬件的工作原理总是有益的。对于串行代码的执行，人们可能会关注到它是怎么工作的，但这并没有达到非如此不可的地步。大多数开发人员可能从来没有学习过计算机体系结构的课程或者读过这方面的书籍，这是很令人惭愧的。这就是我们曾经看到有如此多的低效率软件的主要原因之一。以我自己的经历为例，我在 11 岁时开始学习 BASIC 语言，在 14 岁时就使用 Z80 汇编语言，但只有进入大学后才真正开始理解计算机体系结构。

嵌入式领域的工作经历会培养你的硬件方面的动手能力。由于没有 Windows 操作系统帮助你管理处理器，因此程序设计是一件很底层的工作。嵌入式项目中，上市的产品通常数以百万。糟糕的代码（sloppy code）意味着对 CPU 和现有内存的低效使用，它反过来需要更快的 CPU 或更多的内存。对于一百万件产品，每件附加 50 美分的费用，就是增加了 50 万美元的成本。这也会带来增加的设计与编程时间。显而易见，写更好的代码比买更多的硬件要划算得多。

时至今日，并行程序设计还是与硬件紧密相连的。如果你只埋头编程而不关心程序的性能，那么并行程序设计并不难。但是如果想充分发挥硬件的性能，你就需要知道硬件是如何工作的。举个生活中的例子，大多数人都能够在一档的情况下安全、缓慢地驾驶汽车，但是如果不知道还有其他档，或者不具备使用它们的知识，你就永远不能快速地从 A 点到达 B 点。学习硬件类似于学习汽车驾驶时使用手动换档——刚开始有点复杂，但熟能生巧，一段时间后就会变得自然。用相同的比喻，你也可以买一辆带自动档的汽车，就像使用一个由熟悉底层硬件工作机理的程序员开发的函数库。但是，在不了解硬件工作原理的情况下开发软件，其结果往往不是最优的实现。

3.4 GPU 计算能力

CUDA 支持多个级别的计算。最早的 G80 系列图形卡就是在配有 CUDA 的第一个版本的情况下上市的。硬件的计算能力是固定的。为了升级到一个新的版本，用户必须升级硬件。听起来虽然像是英伟达公司试图强迫用户购买更多的硬件，但是它确实给用户带来了好处。因为当提升一个计算级别时，你就从一个老的平台迁移到了一个新的平台，新的图形卡与原先的图形卡价格相同，计算能力却翻了一番。英伟达公司至少每隔几年就推出一个新的平台，在 CUDA 出现的短短数年内，人们可以获得的计算能力已经有了巨大的提高。

不同计算能力之间的差别列表，作为本书的一部分，可以在附录 G 中找到。因此，我们仅仅在这里介绍不同计算能力之间的主要差别。作为开发者，这是需要知晓的。

3.4.1 计算能力 1.0

计算能力 1.0 出现在早期的图形卡上，例如，最初的 8800 Ultras 和许多 8000 系列卡以及 Tesla C/D/S870s 卡。计算能力 1.0 卡的性能缺陷主要与原子操作有关。原子操作是指那些

必须一次性完成、不会被其他线程中断的操作。要实现这一点，硬件就要在原子函数的入口实现一个栅栏点（barrier point）并确保相应操作（例如，加、减、求最小值、求最大值、逻辑与、逻辑或、逻辑异或等）作为一个整体来完成。计算能力 1.0 现在已经退出市场了。因此，无论为了什么目的和意义，这个限制都可以忽略不计。

3.4.2 计算能力 1.1

计算能力 1.1 出现在许多 9000 系列图形卡后期推出的产品上，例如，曾经红极一时的 9800 GTX 卡。相对于计算能力 1.0 的 G80 硬件，这些卡基于 G92 硬件。

计算能力 1.1 带来的最主要的变化是支持数据传送和内核程序的重叠执行。当然，这个变化出现在大多数，但不是所有的计算能力 1.1 卡上。SDK 调用 cudaGetDeviceProperties() 函数返回 deviceOverlap 属性，该属性定义了这项功能是否可用。实现这项功能需要一个很巧妙的重要改进——双缓冲（double buffering），其工作原理如图 3-7 所示。

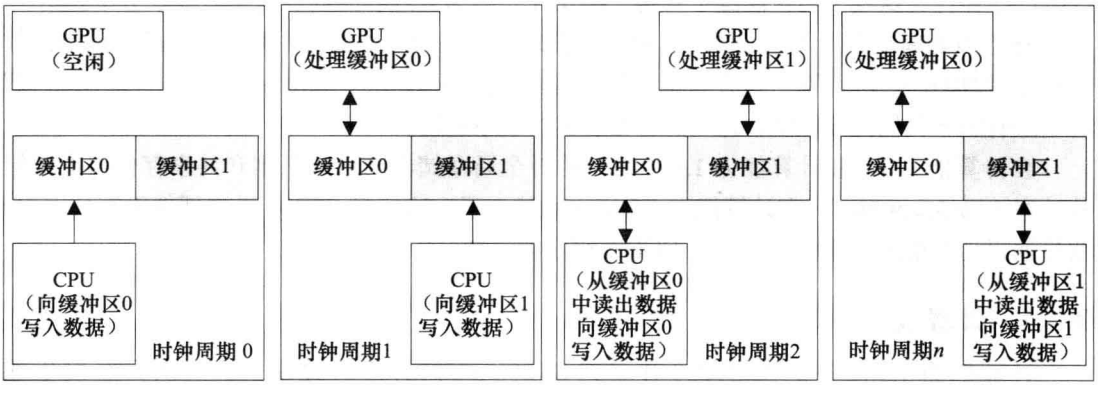


图 3-7 单条 GPU 的双缓冲区技术

要想实现这项功能，我们需要将通常使用的内存空间加倍。如果你的目标市场仅有 512 M 字节显存的卡的话，这一点也许是一个问题。但是，如果使用的是面向科学计算的 Tesla 卡，那么你拥有的 GPU 存储空间最高可达 6GB，实现这项功能就没问题了。让我们看看它的操作步骤，如下所示。

时钟周期 0：在 GPU 的存储空间中划分出两块缓冲区，CPU 将一个数据包写入“缓冲区 0”中。

时钟周期 1：CPU 调用 GPU 上的一个 CUDA 内核程序（即一个 GPU 任务），然后立即返回（这是一个非阻塞调用）。然后，CPU 从硬盘、网络或者其他地方取来一个数据包。与此同时，GPU 正在后台处理交给它的数据包。当数据包取来后，CPU 开始将其往“缓冲区 1”中写。

时钟周期 2：当 CPU 写完后，它又调用一个内核程序来处理“缓冲区 1”。然后检查在时钟周期 1 调用的、处理“缓冲区 0”的内核程序是否完成。如果还没有完成，CPU 则一直

等待直至它完成。完成后，CPU 取走“缓冲区 0”中的计算结果，然后把下一个数据包写入“缓冲区 0”。在这个过程中，本时钟周期一开始就启动的内核程序，则一直在处理 GPU 上“缓冲区 1”中的数据。

时钟周期 N：重复时钟周期 2。在 GPU 处理一个缓冲区的同时，让 CPU 选择另外一个缓冲区进行读或写操作。

GPU-CPU 和 CPU-GPU 的数据传送是在相对较慢的 PCI-E 总线（5GB/s）上进行的这个“双缓冲”技术显著地掩盖了通信延迟，使 CPU 和 GPU 都处于忙碌状态。

3.4.3 计算能力 1.2

计算能力 1.2 设备是与低端的 GT200 系列硬件一起出现的。最早的产品是 GTX260 和 GTX280 卡。随着 GT200 系列硬件的出现，英伟达公司通过将卡上的多处理器数量增倍，使得单个卡上的 CUDA 核（CUDA core）处理器的数量几乎增加了一倍。因此，与上一代产品 G80/G92 相比，这些卡的性能翻了一番。CUDA 核与多处理器的内容将在后续章节中介绍。

在将多处理器数量增倍的同时，英伟达公司还将一个多处理器中并发执行的线程束的数量从 24 增加到了 32。“束”是在一个多处理器内执行的代码块。每个多处理器内可调度“束”的增加有利于我们提高性能，这一点将在后续章节中介绍。

在计算能力 1.0 和计算能力 1.1 中常见的对全局存储器的访问限制和共享存储器中存储片冲突（bank conflict），在计算能力 1.2 中大大减少了。这使得 GT200 系列硬件更容易编程，明显地提高了很多以前艰难编写的 CUDA 程序的性能。

3.4.4 计算能力 1.3

计算能力 1.3 设备是在 GT200 升级到 GT200 a/b 修订版时提出的，这次升级发生在 GT200 系列发布不久。从那时开始，几乎所有的高端卡都兼容计算能力 1.3。

计算能力 1.3 带来的最主要的变化是支持有限的双精度浮点运算。由于 GPU 是针对图形处理的，所以对快速的单精度浮点运算要求很高，但是对双精度浮点运算要求有限。双精度浮点运算性能通常要比单精度浮点运算性能低一个数量级，所以如果程序中只有单精度浮点运算，才能发挥硬件的最大功效。但是在很多情况下，单精度浮点运算和双精度浮点运算会同时出现在程序中，因此硬件中同时设置专用的单精度浮点运算单元和双精度浮点运算单元是最理想的。

3.4.5 计算能力 2.0

计算能力 2.0 设备是伴随费米架构硬件出现的。调整应用程序以适应费米架构的最初指导可参见英伟达公司的网站 http://developer.nvidia.com/cuda/nvidia-gpu-computing-documentation。

计算能力 2.x 硬件的主要改进如下：

- 在每个 SP 上引入了 16K ~ 48K 的一级（L1）缓存。
- 在每个 SM 上引入了一个共享的二级（L2）缓存。

- 在基于 Tesla 的设备上支持基于纠错码（Error Correcting Code，ECC）的内存检查和纠错。
- 在基于 Tesla 的设备上支持双复制（dual-copy）引擎。
- 将每个 SM 的共享内存容量从 16K 扩展到 48K。
- 为了优化数据的合并，数据必须以 128 字节对齐。
- 共享内存的片数从 16 增加到 32。

下面让我们选择几个重要的改进，详细分析一下它们的实现。首先，我们分析一下一级缓存以及引入它意味着什么。一级缓存是设置在芯片内的，它是最快的可用存储器。除了纹理和常量缓存外，计算能力 1.x 的硬件中并没有缓存。引入缓存，使程序员更容易编写出适合在 GPU 硬件上工作的程序，还允许应用程序不必遵循在编译时已知的存储器访问模式。但是，为了利用好缓存，应用程序要么需要具有一个顺序的存储器访问模式，要么需要对某些数据反复使用。

费米型硬件上的二级缓存容量最高可达 768K。重要的是，它是一个统一的缓存。这意味着它是一个共享的缓存，对所有的 SM 提供一个一致的视图。通过二级缓存来实现程序块间通信，要比通过全局原子操作实现快得多。访问 GPU 上的全局存储器，需要越过线程块，比较起来，使用共享缓存要快一个数量级。

对于数据中心（data center）而言，支持 ECC 存储是必须的，因为 ECC 存储器具有自动的检错和纠错功能。电子设备会产生少量的电磁辐射。当与其他设备靠得很近时，这个辐射会改变其他设备中存储单元的内容。虽然这种情况发生的概率很小，但是由于数据中心的设备摆放密度不断增加，出错的概率将会增大到无法接受的水平。因此，就需要引入 ECC 来检测和纠正在一个大型数据中心中可能出现的“单个二进制位反转错误”。当然，引入 ECC 技术会减少可用的 RAM 容量并降低访存带宽。这对图形卡而言是一个严重的缺点，所以目前仅有 Tesla 卡采用了 ECC 技术。

“双复制”引擎是将前面介绍过的“双缓冲技术”扩展应用到多“流”处理技术，“流”的概念将在后续章节中详细介绍。简单地说，“流”就是 N 个独立的内核程序以流水线的方式并行执行，如图 3-8 所示。

图 3-8 流的流水线处理技术

请注意图中内核程序段一个接一个执行的过程。每个复制操作被另一个流中执行的内核程序所隐藏。内核程序与复制引擎是并发执行的，因此相关器件的利用率达到最高。

需要说明的是，“双复制”引擎在绝大多数诸如 GTX480 或 GTX580 这样的高端费米型

GPU中是真实存在的。但是，只有Tesla卡中的双引擎对CUDA驱动程序是可见的，即可以由CUDA驱动程序直接操纵。

共享存储器变化较大，它存在于混合的一级缓存中。一级缓存的容量为64KB。但是为了保证向后兼容性（backward compatibility），必须从中至少划分出16KB的存储空间给共享存储器。这就意味着，一级缓存的实际容量至多为48KB。实际工作中还可以通过一个转换开关，将共享存储区和一级缓存区的功能相互转换，即共享存储器的容量为48KB而一级缓存的容量为16KB。共享存储器的容量从16K提升为48K，某些特定的程序会从中获得巨大的好处。

由于新一代GPU引入了一级和二级缓存，为优化访存而提出的对齐（alignment）要求就更加严格了。两级缓存中，缓存存储块（cache line）的大小均为128B。而每次访问缓存，取来的最少数据量就是一个“存储块”。因此，如果你的程序是顺序访问数据元素，那么这个要求会发挥很好的作用。事实上，绝大多数CUDA程序都是这么工作的，即一组线程读取的都是相邻的存储单元。不过这个改进也带来了一个新的限制，就是数据集应该是128B对齐的。

但是，如果你的程序中每个线程的访存模式是稀疏而分散的，那么你就应该屏蔽掉这个“对齐”要求，并转回到32位的缓存操作模式。

最后，我们再看看“共享存储器的片数从16增加到32”这个改进。新一代GPU会从中获得巨大的好处。它允许当前线程束（包含32个线程）中的每一个线程都可以向共享存储器中的某一片写入数据，而不会引起共享存储片冲突。

3.4.6 计算能力2.1

计算能力2.1出现在专门面向游戏市场的专用卡上，例如，GTX460和GTX560。这些设备在体系结构方面的改进如下：

- 每个SM中的CUDA核心由原先的32个增为48个。
- 每个SM中面向单精度浮点数的专用超越函数计算部件由4个增至8个。
- 双束调度器代替单束调度器。

x60系列卡对中端的游戏市场一直有很强的渗透能力，所以如果你的应用程序面向这类消费市场，那么了解上述改进的内涵是很重要的。

计算能力2.1硬件中一个值得注意的变化就是牺牲掉双精度浮点数的运算器件来换取CUDA核数量的增加。对于单精度浮点数和整数运算占主要地位的内核程序而言，这是一个很好的取舍。绝大多数游戏主要进行单精度浮点数和整数的数学计算，几乎不处理双精度浮点数。

一个线程束就是一组线程，在后续章节中我们将详细介绍。在计算能力2.0的硬件上，单束调度器需要两个时钟周期来从整个束中取出2条指令执行。在计算能力2.1的硬件上，双束调度器每两个时钟周期分发4条指令，而不是2条。在现在的SM硬件中，有3排、每排16个的CUDA核，共计48个CUDA核心，而不是原先的2排、每排16个的CUDA核。

如果英伟达公司还能再挤进 16 个一排的 CUDA 核，那就更理想了，也许在未来我们能看到这样的硬件[⊖]。

计算能力 2.1 硬件的确采用了类似于从最早的奔腾 CPU 开始就一直采用的超标量（superscalar）技术。要想充分利用所有的核，硬件需要在每一个线程中识别出指令级并行性（Instruction-Level Parallelism，ILP）。这与原先推荐的通用的线程级并行性（Thread-Level Parallelism，TLP）有很大的不同。要想体现出 ILP，指令之间应该是相互无关的。借助专用的向量类库是实现 ILP 最早的方法之一，本书的后续章节将详细介绍。

计算能力 2.1 硬件的性能是变化的。某些著名的应用，如 Folding at Home，使用计算能力 2.1 硬件的性能就非常好。其他诸如视频编码压缩这样的应用，由于很难从中挖掘出 ILP 而且存储器带宽是一个关键因素，所以实际使用性能就很糟糕。

截至撰写本书的时候，开普勒型 GPU 和新的计算能力 3.0 平台的最后细节，还没有公开地发布[⊜]。针对已经公布的开普勒型 GPU 特性，本书在第 12 章中将进行深入的讨论。

⊖ 作者的预言已经实现，比如 Tesla 系列的 K10、K20，GTX 系列的 680、TITAN 等，都是采用这一思路。

⊜ 截至 2013 年 6 月，不仅计算能力 3.0，甚至计算能力 3.5 的细节已经公布。

第 4 章

CUDA 环境搭建

4.1 简介

本章面向从未接触过 CUDA 的初学者。我们将依次介绍如何在不同操作系统上安装 CUDA、有哪些可用的 CUDA 工具以及 CUDA 如何编译代码，最后介绍应用程序接口提供的错误处理手段，并帮助读者识别 CUDA 代码和开发过程中必然碰到的应用程序接口报错。

Windows、Mac 和 Linux 三大主流操作系统均支持 CUDA。最易于使用和学习 CUDA 程序开发的操作系统，应该是你最熟悉的那个。对于零基础的初学者，Windows 加上 Microsoft Visual C++ 可能是最好选择。在 Windows 和 Mac 上安装 CUDA 主要是一些点击操作，它们都提供了非常标准的集成开发环境，很适合 CUDA 程序开发。

4.2 在 Windows 下安装软件开发工具包

这里的安装以工具包 4.1 版为例。在基于 Windows 系统的个人计算机上安装 CUDA，需要一些组件，可以到英伟达开发者社区下载，入口在 http://developer.nvidia.com/cuda-toolkit-41。在本书付梓刊印之际，开发包 5.0 版已经处于待发布阶段。请到英伟达官网获取最新版本。

你需要事先安装好 Microsoft Visual Studio 2005、2008 或者 2010。接着首先要下载并安装对应于操作系统的英伟达开发驱动程序，下载地址同上。然后你还要依次下载并安装 32 位或 64 位版本的 CUDA 工具包、GPU 计算软件开发包及其软件开发包的示例程序。要确认你安装的程序版本号与你的操作系统是匹配的。建议的安装次序如下：

1）英伟达开发驱动程序

2）CUDA 工具包

3）CUDA 软件开发工具包

4）GPU 计算软件开发工具包

5）并行 Nsight 调试器

在 Windows 7 系统下，软件开发工具包把全部文件放置在 ProgramData 目录下。这个目录处于 C 盘，是隐藏的。为了查看其中的文件，可以通过桌面上的 CUDA 软件开发工具包图标进行浏览或者进入 Windows Folder Options（文件夹选项）对话框里进行设置，以允许查看隐藏文件，如图 4-1 所示。

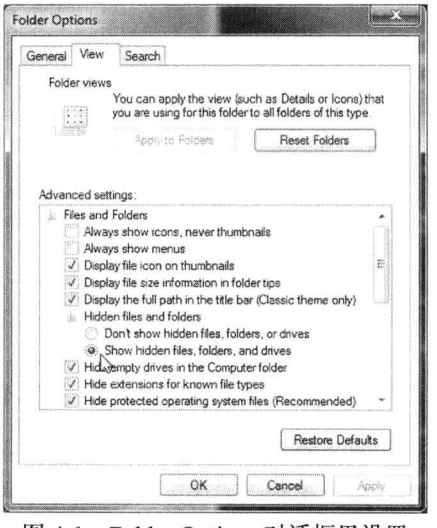


图 4-1 Folder Options 对话框里设置允许查看隐藏文件

4.3 Visual Studio

CUDA 支持 Visual Studio 版本的范围为 2005 ~ 2010，也支持大部分的学习版。学习版可以从 Microsoft 免费得到。专业版也能通过 DreamSpark 计划得到，需要在 https://www.dreamspark.com 网站注册为学生身份免费获得。

注册时，只需提供你的学校信息和身份编号。一旦注册成功，就可以下载 Visual Studio 软件以及很多其他开发工具。这个计划不仅面向美国的学术机构，也涵盖全世界的大学生。

综合来看，Visual Studio 2008 对 CUDA 的支持最好，它的编译速度比 Visual Studio 2010 更快。但 Visual Studio 2010 可以实现源代码的自动语法检查，这一特性非常实用。在使用一项未定义的类型时，它能使用红色下划线指明错误所在，与 Microsoft Word 里提示拼写错误是一样的。对于明显的问题，这一特性极其有用，它将大大节省不必要的编译次数。因此，建议使用 2010 版本进行 CUDA 开发，特别是当你可以从 DreamSpark 获取免费版时。

4.3.1 工程

为了快速新建一个工程，你可以选择一个软件开发工具包示例作蓝本，然后移除其中不需要的工程文件，并插入自己的源文件。你的 CUDA 源代码，应该包含“.cu”扩展名，这样它的编译会采用英伟达编译器而不是 Visual C 编译器。另一种新建工程的方式，可以通过工程模板向导，方便地建立一个基本的工程框架，细节将在后面看到。

4.3.2 64 位用户

当使用 Windows 64 位版本时，要注意一些工程文件默认设置为以 32 位应用程序运行。因此，当尝试生成程序[⊖]时，你可能会收到以下错误消息：“致命错误 LNK1181：无法打开输入文件‘cutil32D.lib’”。

这是因为没有安装该库的缘故。你很有可能只安装了对应 64 位 Windows 的 64 位版本软件开发工具包。要更正此问题，需要把目标平台从 32 位改为 64 位。可以使用 Visual Studio 的 Build（生成）菜单，然后把平台改变为 X64，如图 4-2 所示。

⊖ “生成”这里指通过菜单中的“Build”（生成）菜单条编译并链接程序。——译者注

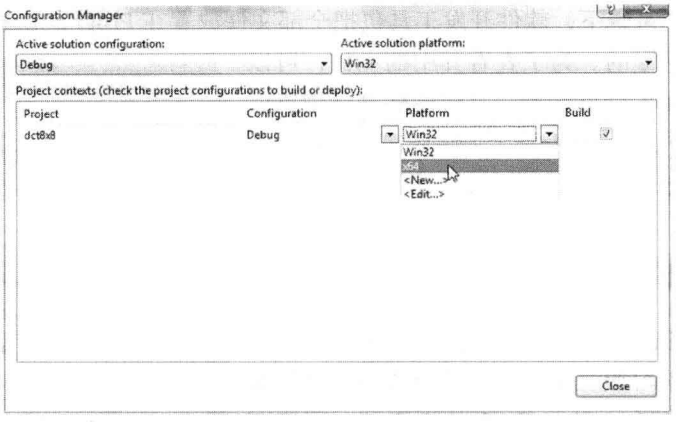


图 4-2 Visual C 目标平台选择

当选择重新生成时，可能会提示你保存该工程。只需添加“_X86”到工程名称之后并保存。该工程将生成为 64 位版本，并链接正确的库文件。

链接时也可能碰到找不到库文件的问题，例如缺少“cutil32.lib”。在安装软件开发工具包的时候，它设置了一个环境变量，`$(CUDA_LIB_PATH)`。这个变量通常设置为：C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v4.1\lib\X64。

有时，可能在默认工程文件的路径设置中没有 `$(CUDA_LIB_PATH)` 这一项。要添加它，可以单击该工程，然后选择 Projed（工程）→ Properties（属性）菜单。这将弹出如图 4-3 所示的对话框。

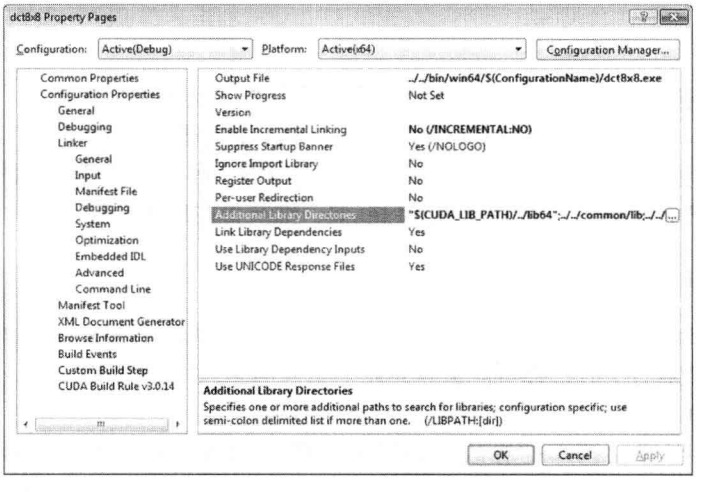


图 4-3 附加库路径

单击在最右侧的“...”按钮，会弹出一个对话框，可以在此处添加库路径（如图 4-4 所示）。只需添加“`$(CUDA_LIB_PATH)`”作为一个新行，该工程即可正常链接。

如果准备构建64位和32位两种CUDA应用程序，则需要提前安装32位和64位的CUDA软件开发工具包。对于软件开发工具包里的样例，要生成32位和64位两个版本的程序，也需要安装两种版本的软件开发工具包。

进入以下目录并生成解决方案文件，就可以建立所需的库文件：

C:\ProgramData\NVIDIA Corporation\NVIDIA GPU Computing SDK 4.1\C\common

C:\ProgramData\NVIDIA Corporation\NVIDIA GPU Computing SDK 4.1\shared

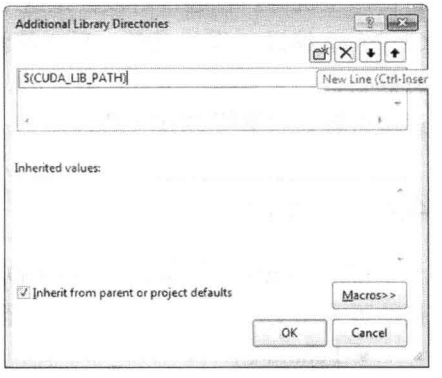


图 4-4 添加库路径

可以在C:\ProgramData\NVIDIA Corporation\NVIDIA GPU Computing SDK 4.1\C\common\lib\x64下找到所需的库文件。也可以手动添加它们到任何需要的工程中。遗憾的是，对于软件开发包里的样例，无法直接生成，所以它们无法自动生成所需的库文件，而二进制的库文件也没有随身附带，所以要生成软件开发包样例程序，会有些小麻烦。

4.3.3 创建工程

要创建支持CUDA的新应用程序，只需通过File（文件）→NEW（新建）→Project Wizard（工程向导）创建CUDA应用程序，如图4-5所示。然后工程向导将创建包含“kernel.cu”文件的单个工程。“kernel.cu”文件里混合了两种不同的代码，一种在CPU端执行，另一种要在GPU端执行。GPU代码包含在addKernel函数里。该函数的入口参数有三个，一个为指向目的数组的指针c，另外一对指针分别指向输入数组a和b。该函数对数组a和b的内容执行加法操作，结果存到目标数组c中。这个简单示例把执行CUDA程序的框架展示了出来。

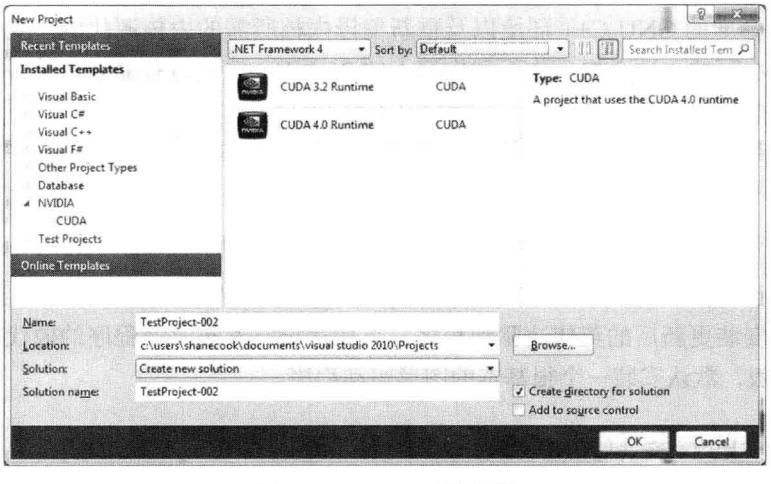


图 4-5 CUDA工程向导

其余的代码是一些基础代码，包括复制数据到GPU设备、调用内核函数、将数据从设备传回主机等。在起步阶段尝试这个简单工程，是大有裨益的，它可以让你实际感受一下在CUDA下编译工程的感觉。后文将涵盖CUDA程序运转所需的标准框架。研究这些代码很有好处，如果可能，尽量理解它们。即便这些代码对你来说难以理解，也没有关系。本书将逐步讲解如何编写CUDA程序。

4.4 Linux

多个Linux发行版都支持CUDA。这些发行版的具体版本号取决于所安装CUDA工具包的版本。支持CUDA的Linux发行版如下：

- Fedora 14
- lRedhat 6.0 和 5.5/CentOS 6.2（Redhat的免费版）
- Ubuntu 11.04
- OpenSUSE 11.2

在Linux平台上安装CUDA的第一步是确保你拥有最新的内核软件。在终端窗口里使用如下的命令达到这一目的：

```
sudo yum update
```

sudo命令允许你以管理员身份登录，yum命令是标准的Linux RPM软件包的安装工具。本条命令行只检查所有已安装的软件包有无可用的更新。这将确保你在安装任何驱动程序之前，系统都是最新的。许多基于图形用户界面的安装程序，不再使用旧版的命令行，而是自带图形用户界面的软件更新程序。

一旦内核更新到最新，运行如下命令：

```
sudo yum install gcc-c++ kernel-devel
```

这将安装标准的GNU C++环境以及重新编译内核所需的内核源代码。要注意，软件包名称是区分大小写的。这会提示你下载内容大约21 MB，安装过程需要数分钟。同样，你也可以通过操作系统的图形用户界面安装程序来安装软件包。

最后，如果你要绘制一些图形输出，就需要一个OpenGL开发环境。这一环境的安装用下面的命令：

```
sudo yum install freeglut-devel libXi-devel libXmu-devel
```

现在，你已经准备好安装CUDA驱动程序。确保你安装的CUDA工具包至少是4.1版。有许多方法来安装更新后的英伟达驱动程序。英伟达没有发布驱动程序的源代码，所以大多数Linux发行版，默认安装一个很基本的图形驱动程序。

内核驱动程序的安装（CentOS、Ubuntu 10.4）

每个CUDA发行版本，应使用特定的一组开发驱动程序。安装驱动程序的方式如果与

这里列出的不一致，可能会导致 CUDA 不能正常工作。对特定版本的 CUDA 工具包，请确认操作系统的版本是否支持它。某些 Linux 最新发行版未必支持；使用旧版本的发行版也极可能无法正常工作。因此，安装的第一步是根据你的 Linux 系统版本采用相应版本的驱动程序，替换现有的驱动程序，如图 4-6 所示。

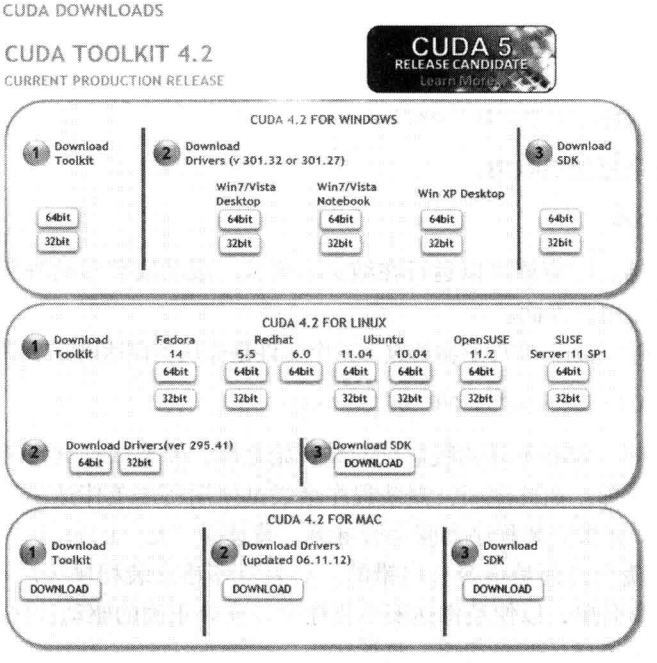


图 4-6　截至 2012 年 9 月，支持的 Linux 下载和支持的驱动程序版本

一旦下载工作完成，你需要引导 Linux 进入纯文本模式。在 Windows 下的安装，始终处于图形用户界面模式，而与此不同，在 Linux 下安装驱动程序需要文本模式。打开终端窗口（通常在图形用户界面的 Systems 菜单中），使用下面的命令，可以让大多数系统在启动时进入文本模式：

```
sudo init 3
```

这将重启 Linux 机器，并把它带入文本模式。如果要还原到图形模式，可以使用 `sudo init 5`。

如果收到“User <user_name> is not in sudoers file”的错误，请使用 su 命令，登录为 root 用户。编辑“/etc/sudoers”文件，添加如下命令：

```
your_user_name ALL=(ALL) ALL
```

注意要把上述命令中的 `your_user_name` 更换为你的登录用户名。

对某些发行版（例如 Ubuntu），init 模式对它失效，仍会引导进入图形用户界面。这里给出一种解决方法。在文本窗口里编辑 GRUB 启动文件：

```
sudo chmod +w /etc/default/grub
sudo nano /etc/default/grub
```

更改以下行：

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX_DEFAULT=""
```

为：

```
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX_DEFAULT="text"
```

请使用如下命令更新 GRUB：

```
sudo update-grub
```

最后，重启机器，它应该可以运行在纯文本模式。成功安装驱动程序之后，使用之前的命令重新启动到图形用户界面。

下面，请定位到“.run”文件所在地点，这个文件是你从英伟达网站下载得到的。然后键入：

```
sudo sh NVIDIA-Linux-x86_64-285.05.33.run
```

下载的驱动程序一定有不同的版本。安装开始之际，将要求你同意英伟达许可证，随后的安装过程需要几分钟。在此期间，安装程序会尝试使用所需英伟达驱动程序，替换默认的 Nouveau 驱动程序。如果安装程序询问是否继续，请选择“是”（Yes）。并不是每个版本的安装都能轻松自如，整个过程是很容易出错的。如果英伟达安装程序无法删除 Nouveau 驱动，那么就要把它列入黑名单，以便英伟达安装程序可以安装正确的驱动程序。

当你正确安装了英伟达驱动程序，请输入：

```
sudo init 5
```

随后的机器重启会恢复到正常的图形模式，请参见前文针对 Ubuntu 的说明。

接下来的任务是安装工具包。根据操作系统发行版是 Fedora、Red Hat、Ubuntu、OpenSUSE 还是 SUSE，选择需要的工具包。与前面类似，只需定位到你安装软件开发包的路径，运行如下命令：

```
sudo sh <sdk_version>.run
```

其中的 <sdk_version> 是你下载的文件。随后，它会安装所需的所有工具，并返回安装成功的信息。接着，它会提醒你必须更新 PATH 和 LD_LIBRARY_PATH 环境变量，这些修改需要你手工来做。为了达到上述目的，需要编辑“/etc/profile”启动文件。添加以下几行：

```
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib:$LD_LIBRARY_PATH
```

请注意，你必须拥有修改该文件的权限。如果需要，请使用“sudo chmod +w /etc/profile”赋予修改权限。你可以使用你喜欢的编辑器，达到类似“sudo nano /etc/profile”命令的效果。

请注销并重新登录，然后键入：

```
env
```

这将列出所有当前的环境变量设置。请检查刚加入的两条新条目。至此，CUDA 已经安装到了“/usr/local/bin”文件夹下。

接着我们需要 GNU C++ 编译器。安装“g++”包到你的系统上，即可完成任务。“g++”包可以系统中在很多正在使用的软件安装程序中找到。

下一步是安装软件开发包样例代码。这些代码是我们生成和测试程序的素材。从英伟达网站下载和运行它们，同样使用 `sh sdk_version.run` 命令（`sdk_version` 改为你实际下载的文件名称）。请不要以 root 身份运行此软件开发包，否则后面对任何样例的生成将要求以 root 身份才能进行。

默认情况下，软件开发包将安装到用户账户区的一个子目录下。它可能会提示找不到 CUDA 安装路径，并将使用默认目录（与之前 CUDA 安装的目录相同）。你可以安全地忽略此消息。

一旦 GPU 计算软件开发包安装完成，你需要去“Common”子目录，运行 make 以创建一组库文件。

这时，你可以生成软件开发包的样例程序。你将在 Linux 下执行你的第一个 CUDA 程序，并测试驱动程序是否工作正常。

4.5 Mac

与其他平台一样，Macintosh 版本的工具也可从 http://developer.nvidia.com/cudatoolkit-41 下载。只需下载并按以下顺序安装软件包：

- 开发驱动程序
- CUDA 工具包
- CUDA 工具软件开发包和代码样例

CUDA 4.1 版本要求 Mac 系统的发行版本为 10.6.8（雪豹）或更高。最新版本（10.7.x）或狮子版本可从苹果商店下载或从苹果单独购买。

软件开发包安装到“GPU Computing”目录下，其高层目录为“Developer”。只需浏览“Developer/GPU Computing/C/bin/darwin/release”目录，你会发现预编译的可执行文件。运行 deviceQuery，这样有助于验证驱动程序安装和运行时环境是否正确。

为了编译样例程序，需要事先安装 XCode。这是 Mac 下等价于 GCC（GNU C 编译器）的工具。XCode 可以从苹果商店下载。这个产品是收费的，但对苹果开发者计划的用户是免费的。苹果开发者计划既包括 Macintosh 的开发，也包括 iPhone/iPad 应用程序的开发。在狮子版操作系统发布不久，XCode 就允许狮子版系统拥有者免费下载。

当 Xcode 安装完成，请打开一个终端窗口。先定位到 Finder，打开 Utilities，然后双击终端窗口，键入以下命令：

```
cd /Developer/'GPU Computing/C/src/project'
make-i
```

替换其中的 project 为需要编译的某个软件开发包应用程序的名称。如果遇到编译错误，可能是你没有下载 XCode 软件包或者目前的版本过旧。

4.6 安装调试器

在 Windows 平台上，CUDA 提供了 Parallel Nsight 调试环境。它支持调试 CPU 和 GPU 代码，并加亮那些运行不够高效的代码段。这也非常利于调试多线程应用程序。

Nsight 是极为有用的免费工具。它只要求你是 CUDA 开发者社区用户，注册过程也是完全免费的。一旦注册成功，你将能够从英伟达网站下载该工具。

请注意，你必须安装了 Visual Studio 2008 或更高版本（不能是学习版），同时要安装 Service Pack 1（SP1）。Nsight 的发行说明中给出了 SP1 的下载链接。

Parallel Nsight 包含两个部分。一个是集成到 Visual Studio 中的应用程序，如图 4-7 所示；另一个是独立的监视程序。监视程序与主应用程序进行协作。监视程序一般与 Visual Studio 环境驻留在同一台机器上，但它们并不是必须在同一台机器上。Parallel Nsight 运行的最佳环境是配备两个支持 CUDA 的 GPU，一个 GPU 专门运行代码，另一个专门执行常规的显示。因此，运行着目标代码的 GPU 无法再用来显示。由于多数 GPU 卡具有双显示器输出，所以你可以通过配置双显示器设置，在显示卡上连接两个显示器。需要指出，在最新的 2.2 版本中，Parallel Nsight 不再需要两个 GPU。

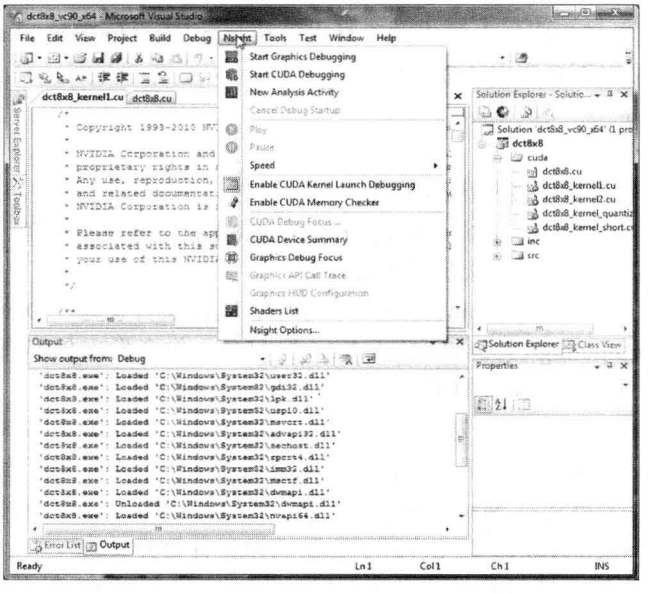


图 4-7 Nsight 集成到 Microsoft Visual Studio 环境

它也可以通过配置调试工具从远程 GPU 获得数据。然而，在大多数情况下，额外购买一个低端 GPU，并安装到个人计算机或者工作站上是更简单的做法。Windows 平台下配置

Parallel Nsight 的第一步是禁用 TDR 功能（参见图 4-8）。超时检测和恢复（Timeout Detection and Recovery，TDR）是 Windows 系统的一种机制，用来检测底层驱动程序代码的异常崩溃情形。如果驱动程序停止响应事件，Windows 则会重置此驱动程序。鉴于在程序断点处，驱动程序将暂停响应，所以为了防止出现重置操作，TDR 功能需要关掉。

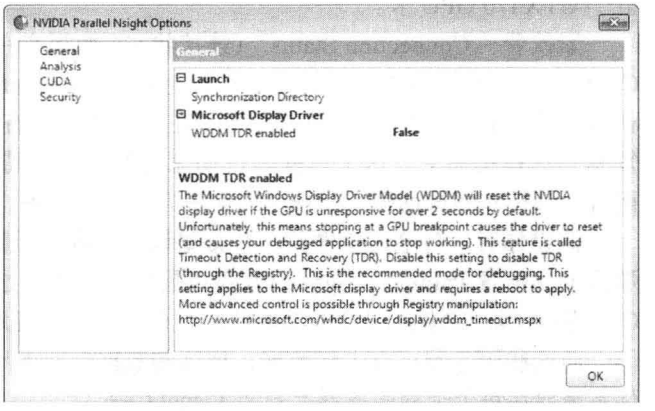


图 4-8 关闭 Windows 内核的超时功能

要设置此值，只需运行监视程序并单击程序对话框右下方的“Nsight Monitor Options”超链接。这将弹出如图 4-8 所示的对话框。设置“WDDM TDR enabled”将通过修改注册表来禁用此功能。重启计算机，Parallel Nsight 将不再给出 TDR 的启用警告。

为了在远程计算机上使用 Parallel Nsight，只需在远程计算机的 Windows 系统安装监视程序。当你第一次运行监视程序时，将收到 Windows 防火墙已阻止“公共网络”（基于互联网的）接入到监视程序的警告。这完全是你所希望的。但是，该工具需要接入局域网，所以你需要把这一例外加入到安装监视器程序的机器中已建立的防火墙规则里。针对局域节点，必须修正 TDR 问题并重启一次。

接下来的步骤是在主机端运行 Visual Studio，并选择一个新的分析活动。你会看到窗口的顶部有一块类似图 4-9 所示的区域。注意，“Connection Name”里填写的是 localhost，意为本地机器。打开 Windows 资源管理器查看局域网络上远程待调试计算机的计算机名称。用 Windows 资源管理器中显示的名称替换 localhost。然后按“Connect”（连接）按钮。你应该看到两个连接已经建立的证据，如图 4-10 所示。

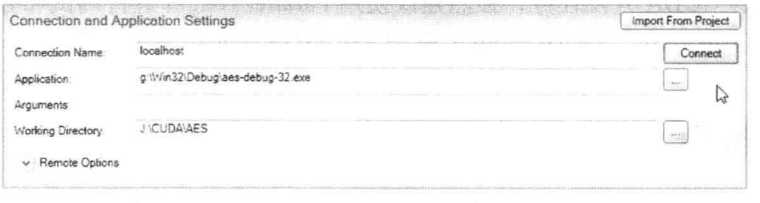


图 4-9 Parallel Nsight 远程连接设置窗口

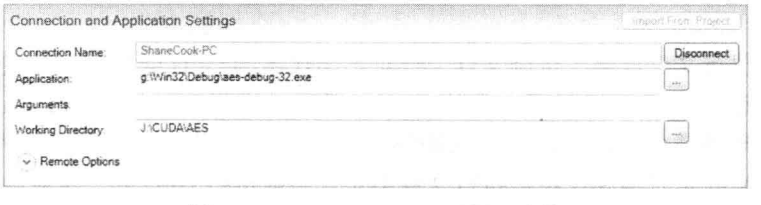


图 4-10 Parallel Nsight 远程已连接

证据一是“Connect”（连接）按钮变为“Disconnect”（未连接）。证据二是“Connection Status”（连接状态）对话框变成绿色，并显示目标机器上的所有 GPU（参见图 4-11）。图中所示情况表明，我们连接到的测试计算机中装有五个 GTX470 GPU 卡。

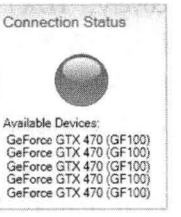


图 4-11 Parallel Nsight 连接状态

紧挨着“Connection Status”（连接状态）面板有一个“Application Control”（应用程序控制）面板，单击上面的“Launch”（启动）按钮，将远程启动目标机器上的应用程序。在此之前，所有必要的文件需要复制到远程机器上。这个过程是自动完成的，可能需要几秒钟。整体而言，分析和调试远程应用程序的这一方式非常简单。

假设你有一台笔记本电脑，并希望调试，或者干脆在 GPU 服务器上远程运行应用程序，那么上述配置 Parallel Nsight 的方式正是你需要的。这种用法包括多人在不同时刻共享使用 GPU 服务器，或者在课堂教学中使用。也可能远程开发者需要在特设的测试服务器上运行代码，而这些服务器存有大量数据，把这些数据传回本地开发机器反而不明智。这同时意味着你不需要在每个远程服务器上安装 Visual C++。

针对 Linux 和 Mac 平台的调试环境是 CUDA-GDB，它提供了一个扩展的 GNU 调试程序包。与 Parallel Nsight 一样，它允许调试主机代码和 CUDA 代码，可以在 CUDA 代码中设置断点、单步跟踪和选择调试线程等。安装软件开发包时，CUDA-GDB 和 Visual Profiler 工具是默认同时安装的，而不像 Parallel Nsight 需要单独下载。截至 2012 年，适用于 Linux 平台下 Eclipse 环境的 Parallel Nsight 也发布了。

Windows 和 Mac/Linux 平台间的主要区别在于对分析工具（profiling tool）的支持。Parallel Nsight 工具在这方面大大优于 Visual Profiler。Visual Profiler 也有 Windows 版本。它针对代码中的问题，提供了比较抽象的概述和建议，比较适合 CUDA 初学者。相比之下，Parallel Nsight 则针对更高级的用户。后续章节会分别介绍 Parallel Nsight 和 Visual Profiler。然而，本书主要使用 Parallel Nsight 作为 GPU 开发的调试 / 分析工具。

对于高级的 CUDA 开发任务，强烈建议使用 Parallel Nsight 进行调试和分析。对于大多数 CUDA 新手来说，结合 Visual Profiler 和 CUDA-GDB 也足以满足开发需要。

4.7 编译模型

英伟达编译器 NVCC 运行在后台，当 CUDA 源文件请求编译时调用。CUDA 源文件或常规源文件的不同类型和含义在表 4-1 给出。文件扩展名决定编译时是使用 NVCC 还是主机编译器。

生成的可执行文件或胖二进制文件，针对不同代次的 GPU 会包含一个或多个二进制可执行映像。它还包含了 PTX 映像，允许 CUDA 运行时进行实时（Just-In-Tine，JIT）编译。这跟 Java 字节码非常相似，它针对一个虚拟的架构，在实际目标硬件上的编译环节在运行程序时进行。PTX 的实时编译仅在可执行文件不包含当前运行 GPU 的二进制映像时发生。因此，所有未来的架构与基本的虚拟架构都是向后兼容的。即使没有在某个 GPU 上执行编译

的程序，通过在运行时编译嵌入在可执行文件中的 PTX 代码，也可以合法运行在该 GPU 上。

跟 Java 类似，CUDA 编译支持代码托管。定义环境变量 CUDA_DEVCODE_CACHE 指向一个目录，该目录将指示运行时把编译的二进制文件保存以备后用。通常情况下，每次编译需要把 PTX 代码根据未知 GPU 的类型进行转换。采用代码托管，这种启动延时就可以避免了。

后面的章节将介绍如何查看真正的目标汇编代码，这是 PTX 翻译为目标代码的结果。

表 4-1 不同的 CUDA 文件类型

文件扩展名	含义	使用的编译器
.cu	混合了主机代码和设备代码的源文件	NVCC
.cup	对 .cu 文件进行预处理后的扩展文件	NVCC
.c, .cc, .cpp	主机 C 或 C++ 源文件	主机编译器
.ptx, .gpu	中间的虚拟汇编文件	NVCC
.cubin	GPU 代码的二进制映像	NVCC

4.8 错误处理

CUDA 的错误处理，乃至 C 语言的错误处理，都是不够好的。它们仅会执行少数运行时检查，对于很多欠考虑的事情，运行时通常也不会阻止。这会导致 GPU 程序奇怪的退出。幸运的时候，你会收到类似编译错误的错误信息。随着这方面经验的增多，你才能学会错误信息的具体含义。

几乎所有的 CUDA 函数调用，都会返回类型为 cudaError_t 的整数值。cudaSuccess 以外的任何值将代表致命错误。这通常是由于你的程序在使用前没有正确设置，或使用了已经销毁的对象。这也可能是源于 Microsoft Windows 中的 GPU 内核超时。如果你安装的 Parallel Nsight 工具没有禁用 TDR，当内核函数运行时间超过数秒钟，就会导致 GPU 内核超时（参见前文介绍）。内存越界访问也可能产生异常，输出各种错误信息到 stderr（标准错误输出）。

每个函数返回一个错误代码时，每次函数调用必须进行错误检查，并编写处理函数。这使得错误处理很麻烦，并导致高度缩进的程序代码。例如：

```
if (cudaMalloc(...) == cudaSuccess)
{
 if (cudaEventCreate(&event) == cudaSucess)
  {
  ...
  }
}
else
{
...
}
```

为了避免这种重复性的编程，本书始终使用下面的宏定义来调用 CUDA 应用程序接口：

```
#define CUDA_CALL(x) {const cudaError_t a = (x); if(a != cudaSuccess) { printf("\nCUDA
Error: %s (err_num=%d) \n", cudaGetErrorString(a), a); cudaDeviceReset(); assert(0);} }
```

这个宏允许你指定 x 为一些函数调用，例如：

```
CUDA_CALL(cudaEventCreate(&kernel_start));
```

接着创建一个临时变量 a 并把 cudaError_t 类型的函数返回值赋予它。然后检查这个值是否等于 cudaSuccess，如果不等于则说明函数调用碰到了错误。如果检测到错误，它会在屏幕上打印返回的错误外加简短的错误解释。它还使用 assert 宏，用来识别发生错误时所在的源文件及代码行，因此有助于你轻松地定位检测到的错误位置。

这种技术适用于除了内核函数调用之外的所有 CUDA 调用。内核函数是在 GPU 设备上运行的程序。它们的执行使用 <<< 和 >>>，如下所示：

```
my_kernel <<<num_blocks, num_threads>>>(param1, param2,...);
```

对于内核的错误检查，我们将使用下面的函数：

```
__host__ void cuda_error_check(const char * prefix, const char * postfix)
{
  if (cudaPeekAtLastError() != cudaSuccess)
  {
    printf("\n%s%s%s", prefix, cudaGetErrorString(cudaGetLastError()), postfix);
    cudaDeviceReset();
    wait_exit();
    exit(1);
  }
}
```

此函数应该在内核函数调用后立即执行。它会检查任何即时错误。一旦发现错误，则输出错误信息、复位 GPU，其中的 wait_exit 函数会等待用户的任一按键操作，然后退出程序。

请注意，内核函数调用与 CPU 代码是异步执行的，所以上述方法并非万无一失。异步执行表示当我们调用 cudaPeekAtLastError 时，GPU 代码正在后台运行。如果此时没有检测到错误，则不会输出错误，函数继续执行下面的代码行。通常情况下，下一条代码行是从 GPU 内存将数据复制到 CPU 内存的操作。内核函数的错误可能会导致随后的应用程序接口调用失败，一般应用程序接口的调用是紧跟内核函数调用的。针对全部的应用程序接口调用，均使用 CUDA_CALL 包裹，这种错误就会被标记出来。

也可以强制在内核函数完成后再进行错误检查，只需在 cudaPeekAtLastError 调用之前加入 cudaDeviceSynchronize 调用即可。然而，这一强制行为只能在调试版程序或者想让 CPU 在 GPU 占满时处于闲置状态时使用。学完本书，你就能明白这种同步操作适合进行调试，但会影响性能。所以，如果这些调用只是为了调试，请务必在产品代码中删去。

4.9 本章小结

现在你成功安装了 CUDA 软件开发包，其中包括 GPU 计算软件开发包样例和调试环境。你学会了生成一个简单的 GPU 计算软件开发包样例，比如 deviceQuery 工程，并在运行时利用它识别系统中的 GPU。

第 5 章

线程网格、线程块以及线程

5.1 简介

英伟达为它的硬件调度方式选择了一种比较有趣的模型，即 SPMD（单程序多数据，Single Program，Multiple Data），属于 SIMD（单指令多数据）的一种变体。从某些方面来说，这种调度方式的选择是基于英伟达自身底层硬件的实现。并行编程的核心是线程的概念，一个线程就是程序中的一个单一的执行流，就像一件衣服上的一块棉，一块块棉交织在一起织成了衣服，同样，一个个线程组合在一起就形成了并行程序。CUDA 的编程模型将线程组合在一起形成了线程束、线程块以及线程网格。本章，就让我们一起来详细了解这些概念。

5.2 线程

线程是并行程序的基本构建块。对大多数做过多核程序设计的 C 程序员而言，这个概念并不陌生。即使一个程序员从来没有在代码中发起过一个线程，起码也执行过一个线程，因为任何序列化的代码都是以单线程的方式执行的。

随着双核、四核、十六核甚至更多核的处理器的出现，我们将更多的注意力放到了程序员如何充分利用这些硬件上。除了近十年，在过去的很多年里，大多数程序员编写的程序都是单线程的，因为当时运行程序的 CPU 也是单核的。当然，你可以利用更多的硬件设备或者成千上万的商用服务器来取代少数的实力强劲的机器，从而通过集群式计算机和超级计算机的方式来尝试进行一个更高层次的并行编程。然而，这些大多仅限于一些大学和大型机构使用，一般无法提供给大众使用。

实现多线程并行很难，但实现一次执行一个任务却要简单得多。在当时，每隔几年，串行处理速度就会提升一倍，因此基本没有什么需要去进行困难的并行编程。串行编程语言 C/C++ 就在这样一个时代应运而生。直至大约十年前，这种情况发生了变化。现在，不管你接不接受，要想提高程序的速度，就必须考虑并行设计。

5.2.1 问题分解

CPU 领域的并行化是向着一个 CPU 上执行不止一个（单一线程）程序的方向发展。但这只是我们之前所提到的任务级的并行。一般而言，程序拥有比较密集的数据集，例如，视频编码。对这种程序我们可以采用数据并行模型，将任务分解成 N 个部分，每个部分单独处理，其中，N 代表可供使用的 CPU 核数。例如，你可以让每个 CPU 核计算一帧的数据，帧与帧之间没有相互的关联。又或者，你可以选择将每一帧分成 *N* 个片段，将每个片段的计算分配到每个独立的核中。

在 GPU 的领域，恰好能看到这些选择方案，当我们尝试加速渲染 3D 游戏中的现实场景时，就会采用多 GPU 的方式。你可以交替发送完整的帧数据到每个 GPU（如图 5-1）。此外，你也可以让一个 GPU 渲染屏幕不同的部分。

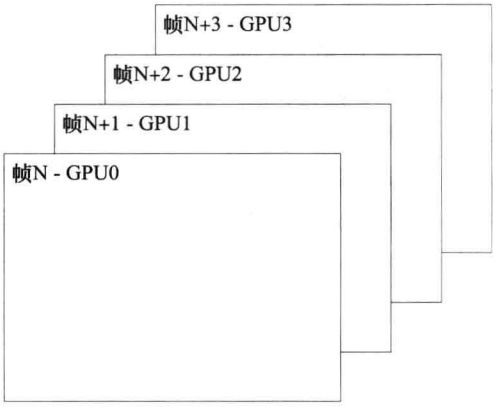


图 5-1 交替帧渲染模式与分割帧渲染模式

图 5-2 粗粒度并行化

然而，这里有一点需要我们来权衡。如果数据集是独立的，通过向 GPU（或者 CPU）提供需要计算的子数据集，我们可以只需要很小的内存来传递少量的数据。如果使用分割帧渲染模式（SFR），那么对于渲染地面的 GPU3 来说，就没有必要去知道渲染天空的 GPU0 里面的数据内容。然而，在实际场景渲染中，有时地面上可能会有一些飞行物的影子，地面的照明度也会随着一天中时间的不同而发生变化。如果出现这种情况，由于共享数据的存在，此时使用交替帧渲染模式（Alternate Frame Rendering，AFR）可能对渲染更有益。

此处所说的 SFR 是根据粗粒度的并行度来划分的，就是以某种方式将大块的数据分配到 N 个强劲的设备中，在数据处理之后又将它们重构成一整块。当我们为一个并行环境设计应用程序时，这一步的选择非常重要，它将严重影响到程序的性能。通常，最好的选择与所使用的设备密切相关。在后文中，你将看到多个贯穿全书的应用程序。

当只有数量较少的强劲设备时，例如在 CPU 上，我们的中心议题是解决平均分配工作量的问题。当然，这个问题很好解决，因为毕竟设备的数量较少。但如果像 GPU 那样拥有大量较小设备时，尽管也能很好的平均工作量，但我们却需要花大量的精力在同步和协调上。

世界经济有宏观（大规模）和微观（小规模）经济，相应地，并行也有粗粒度的和细粒度的并行。然而，只有在那些支持大量线程的设备上才能真正实现细粒度的并行，例如，GPU。相比之下，CPU 同样支持线程，但伴随着大量的开销，因此它只适合解决粗粒度的并行问题。CPU 与 GPU 不同，它遵从多指令多数据（MIMD）模型，即它可以支持多个独立的指令流。这是一种更加灵活的方式，但由于这种方式是获取多个独立的指令流，而不是平摊多个处理器的单指令流，因此它会带来额外的开销。

在此背景下，让我们来考虑用一个图像校正函数来增强数码照片的亮度。如果在 GPU 上，你可能会为照片上的每个像素点分配一个线程。但如果是在一个四核的 CPU 上，你可能会为每个 CPU 核分配照片的 1/4 图像的数据进行处理。

5.2.2 CPU 与 GPU 的不同

GPU 和 CPU 设备的架构是迥异的。CPU 的设计是用来运行少量比较复杂的任务。GPU 的设计则是用来运行大量比较简单的任务。CPU 的设计主要是针对执行大量离散而不相关任务的系统。而 GPU 的设计主要是针对解决那些可以分解成成千上万个小块并可独立运行的问题。因此，CPU 适合运行操作系统和应用程序软件，即便有大量的各种各样的任务，它也能够在任何时刻妥善处理。

CPU 与 GPU 支持线程的方式不同。CPU 的每个核只有少量的寄存器，每个寄存器都将在执行任何已分配的任务中被用到。为了能执行不同的任务，CPU 将在任务与任务之间进行快速的上下文切换。从时间的角度来看，CPU 上下文切换的代价是非常昂贵的，因为每一次上下文切换都要将寄存器组里的数据保存到 RAM 中，等到重新执行这个任务时，又从 RAM 中恢复。相比之下，GPU 同样用到上下文切换这个概念，但它拥有多个寄存器组而不是单个寄存器组。因此，一次上下文切换只需要设置一个寄存器组调度者，用于将当前寄存器组里的内容换进、换出，它的速度比将数据保存到 RAM 中要快好几个数量级。

CPU 和 GPU 都需要处理失速状态。这种现象通常是由 I/O 操作和内存获取引起的。CPU 在上下文切换的时候会出现这种现象。假定此时有足够多的任务，线程的运行时间也较长，那么它将正常地运转。但如果没有足够多的程序使 CPU 处于忙碌状态，它就会闲置。如果此时有很多小任务，每一个都会在一小段时间后阻塞，那么 CPU 将花费大量的时间在上下文切换上，而只有少部分时间在做有用的工作。CPU 的调度策略是基于时间分片，将时间平均分配给每个线程。一旦线程的数量增加，上下文切换的时间百分比就会增加，那么效率就会急剧的下降。

GPU 就是专门设计用来处理这种失速状态，并且预计这种现象会经常发生。GPU 采用的是数据并行的模式，它需要成千上万的线程，从而实现高效的工作。它利用有效的工作池来保证一直有事可做，不会出现闲置状态。因此，当 GPU 遇到内存获取操作或在等待计算结果时，流处理器就会切换到另一个指令流，而在之后再执行之前被阻塞的指令。

CPU 和 GPU 的一个主要差别就是每台设备上处理器数量的巨大差异。CPU 是典型的双核或者四核设备。也就是说它有一定数量的执行核可供程序运行。而目前费米架构的 GPU

拥有 16 个 SM（流多处理器），每个 SM 可看作是 CPU 的一个核。CPU 通常运行的是单线程的程序，即它的每个核的每次迭代仅计算一个数据。然而，GPU 默认就是并行的模式，它的 SM 每次可同时计算 32 个数而不是像 CPU 那样只计算一个数，因此，相对于一个四核的 CPU 来说，GPU 的核数目就是其 4 倍，数据的吞吐量则是其 32 倍。当然，你可能会说 CPU 也可以使用所有的可供使用的计算核，以及像 MMX、SSE 和 AVX 那样的指令扩展集，但问题是又有多少 CPU 程序使用了这种扩展集呢。

GPU 为每个 SM 提供了唯一并且高速的存储器，即共享内存。从某些方面来说，共享内存使用了连接机和 cell 处理器的设计原理，它为设备提供了在标准寄存器文件之外的本地工作区。自此，程序员可以安心地将数据留在内存中，不必担心由于上下文切换操作需要将数据移出去。另外，共享内存也为线程之间的通讯提供了重要机制。

5.2.3 任务执行模式

任务执行的模式主要有两种。一种基于锁步（lock-step）思想，执行 N 个 SP（流处理器）组，每个 SP 都执行数据不同的相同程序。另一种则是利用巨大的寄存器文件，使线程的切换高效并且达到零负载。GPU 能支持大量的线程就是按照这种方式设计的。

所谓的锁步原则到底是什么？指令队列中的每条指令都会分配到 SM 的每个 SP 中。每个 SM 就相当于嵌入了 *N* 个计算核心（SP）的处理器。

传统的 CPU 会将一个单独的指令流分配到每个 CPU 核心中，而 GPU 所用的 SPMD 模式是将同一条指令送到 N 个逻辑执行单元，也就是说 GPU 只需要相对于传统的处理器 1/N 的指令内存带宽。这与许多高端的超级计算机中的向量处理器或单指令多数据处理器很相似。

然而，这样做并不就意味着没有开销。通过后面的学习我们将看到，当 N 个线程执行相同的控制流，如果程序未遵循整齐的执行流，对于每一个分支而言，将会增加额外的执行周期。

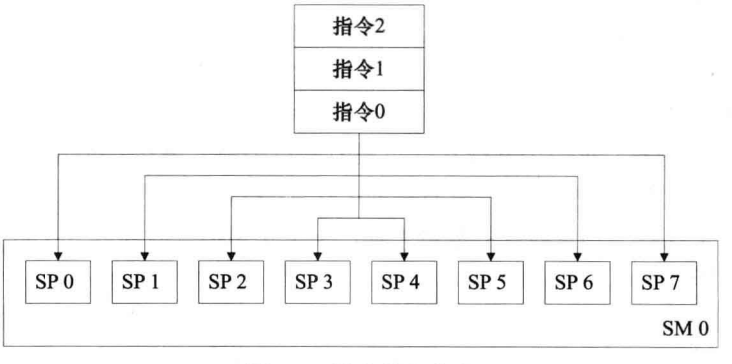


图 5-3 锁步指令分配

5.2.4 GPU 线程

现在，我们再回过头来看看线程。首先，来看一段代码，从编程的角度看看它有什么意义。

```
void some_func(void)
{
  int i;

  for (i=0;i<128;i++)
  {
    a[i] = b[i] * c[i];
  }
}
```

这段代码很简单。它让数组 b 和数组 c 中下标相同的元素进行相乘，然后将所得的结果保存到相同下标的数组 a 中。串行代码需要 128 次 for 循环（从 0 ~ 127）。而在 CUDA 中，我们可以将这段代码直接转换成用 128 个线程，每个线程都执行下面这段代码：

```
a[i] = b[i] * c[i];
```

由于循环中每一轮计算与下一轮计算之间没有依赖，因此将这段代码转换成并行程序非常简单。这种并行转换叫做循环并行化。这种并行化是另一种流行的并行语言扩展——OpenMP 的基础。

在一个四核的 CPU 上，你可以将此计算任务平均分成四部分，让 CPU 的第一个核计算数组下标为 0 ~ 31 的元素，第二个核计算下标为 32 ~ 63 的元素，第三个核计算下标为 64 ~ 95 的元素，第四个核计算下标为 96 ~ 127 的元素。有些编译器自动就可以做这样的并行划分，而有些则需要程序员在程序中指出哪些循环需要并行。其中，Intel 的编译器就非常擅长此道。这种编译器可以按照这种方式，而不是通过增加线程数量的方式，产生嵌入式 SSE 指令以使循环矢量化。GPU 的并行模式与我们所说的这两种并行模式相差并不太多。

在 CUDA 中，你可以通过创建一个内核函数的方式将循环并行化。所谓的内核函数，就是一个只能在 GPU 上执行而不能直接在 CPU 上执行的函数。按照 CUDA 的编程模式，CPU 将主要处理它所擅长的串行代码。当遇到密集计算的代码块时，CPU 则将任务交给 GPU，让 GPU 利用它超强的计算能力来完成密集计算。应该还有人记得 CPU 曾经搭载浮点协处理器的那段时光吧，应用程序在装有浮点协处理器的机器进行大量的浮点计算异常的快。而 GPU 也是如此，它们就是用来加速程序中运算密集的模块的。

从概念上看，GPU 的内核函数和循环体是一样，只不过将循环的结构移除了。下面这段代码就是一个内核函数：

```
__global__ void some_kernel_func(int * const a, const int * const b, const int * const c)
{
  a[i] = b[i] * c[i];
}
```

仔细观察你会发现循环结构没有了，循环控制变量 i 也没有了。除此之外，在 C 的函数前面还多了一个 __global__ 的前缀。__global__ 前缀是告诉编译器在编译这个函数的时候生成的是 GPU 代码而不是 CPU 代码，并且这段 GPU 代码在 CPU 上是全局可见的。

CPU 和 GPU 有各自独立的内存空间，因此在 GPU 代码中，不可以直接访问 CPU 端的

参数，反过来在CPU代码中，也不可以直接访问GPU端的参数。稍后，我们将介绍一种特殊的方法来解决这个问题。现在，我们只需要知道它们是在不同的存储空间。因此，我们之前申明的全局数组a，b，c全是在CPU端的内存中，GPU端的代码是无法直接访问的，所以我们必须在GPU端的内存中也声明这几个数组，然后将数据从CPU端复制到GPU端，以GPU内存指针的方式传递给GPU的内存空间进行读写操作，在计算完毕之后，再将计算的结果复制回CPU端。这些步骤我们会在之后的章节中做一一详解。

下一个问题是，i不再是循环控制变量，而是用来标识当前所运行的线程的一个变量。在此，我们将以线程的形式创建128个该函数的实例，而CUDA则提供了一个特殊的变量，它在每个线程中的值都不一样，使得它可以标识每一个线程。这就是线程的索引，即线程ID。我们可以直接将这个线程标号用作数组的下标对数组进行访问。这和MPI中获取程序优先级很相似。

线程的信息是由一个结构体存储的。在这个例子中，我们只用到了这个结构体中的一个元素，因此，我们将它保存到一个名为thread_idx变量中，以避免每次都访问这个结构体。具体代码如下：

```
__global__ void some_kernel_func(int * const a, const int * const b, const int * const c)
{
  const unsigned int thread_idx = threadIdx.x;
  a[thread_idx] = b[thread_idx] * c[thread_idx];
}
```

注意，有些人可能会使用idx或tid来保存线程的标号，因为这样更加简短方便。

如此一来，线程0中的thread_idx值为0，线程1的为1，依此类推，线程127中的thread_idx值为127。每个线程都进行了两次读内存操作，一次乘法操作，一次存储操作，然后结束。我们注意到，每个线程执行的代码是一样的，但是数据却不相同。这就是CUDA的核心——SPMD模型。

在OpenMP和MPI中，你可能找到与这相似的代码块。对一个给定的循环迭代，将线程标号或线程优先级提取出来并分配给每一个线程，然后在数据集中作为下标使用。

5.2.5 硬件初窥

现在我们知道每个SM中有N个核，那么我们该如何运行128个线程？与CPU很相似，GPU的每个线程组被送到SM中，然后N个SP开始执行代码。在得到每个线程的标号之后的第一件事就是从数组b和数组c中各取一个数然后进行乘法运算。不幸的是，这不是立即发生的。实际上，当从存储子系统取得所需要的数之后，已经过去了400 ~ 600个GPU时钟周期。在这期间，这一组中的N个线程都将挂起。

事实上，线程都是以每32个一组，当所有32个线程都在等待诸如内存读取这样的操作时，它们就会被挂起。术语上，这些线程组叫做线程束（32个线程）或半个线程束（16个线程），这个概念在后面的内容中将会介绍。

因此，我们可以将这 128 个线程分成 4 组，每组 32 个线程。首先让所有的线程提取线程标号，计算得到数组地址，然后发出一条内存获取的指令（如图 5-4 所示）。接着下一条指令是做乘法，但这必须是在从内存读取数据之后。由于读取内存的时间很长，因此线程会挂起。当这组中的 32 个线程全部挂起，硬件就会切换到另一个线程束。

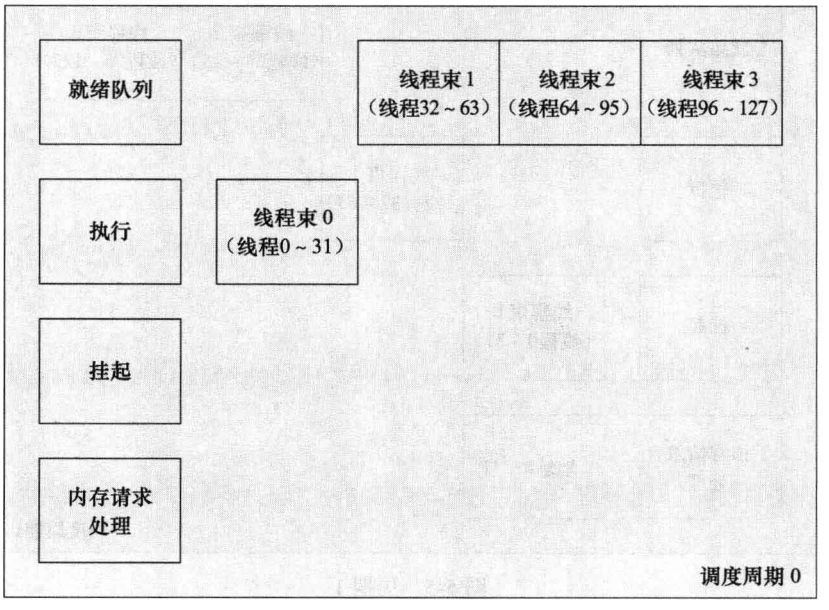


图 5-4 周期 0

在图 5-5 中我们可以看到，当线程束 0 由于内存读取操作而挂起时，线程束 1 就成为了正在执行的线程束。GPU 一直以此种方式运行直到所有的线程束到成为挂起状态（如图 5-6 所示）。

当连续的线程发出读取内存的指令时，读取操作会被合并或组合在一起执行。由于硬件在管理请求时会产生一定的开销，因此这样做将减少延迟（响应请求的时间）。由于合并，内存读取会返回整组线程所需要的数据，一般可以返回整个线程束所需要的数据。

在完成内存读取之后，这些线程将再次置成就绪状态，当再次遇到阻塞操作时，例如另一个线程束进行内存读取，GPU 可能将这个线程束用作另一块内存的读取。

当所有的线程束（每组 32 个线程）都在等待内存读取操作完成时，GPU 将会闲置。但到达某个时间点之后，GPU 将从存储子系统返回一个内存块序列，并且这个序列的顺序通常与发出请求的顺序是一致的。

假设数组下标为 0 ~ 31 的元素在同一时间返回，线程束 0 进入就绪队列。如果当前没有任何线程束正在执行，则线程束 0 将自动进入执行状态（如图 5-4 所示）。渐渐地其他所有挂起的线程束也都完成了内存读取操作，紧接着它们也会返回到就绪队列。

一旦线程束 0 的乘法指令执行完毕，它就只剩下一条指令需要执行，即将计算得到的结

果写入相同下标的数组 a 中。由于再没有依赖该操作的其他指令，线程束 0 全部执行完毕然后消亡。其他的线程束也像这样，最终发出一条写数据的请求，完成之后便消亡。当所有的线程束都消亡之后，整个内核函数也就结束了，最终将控制返回到 CPU 端。

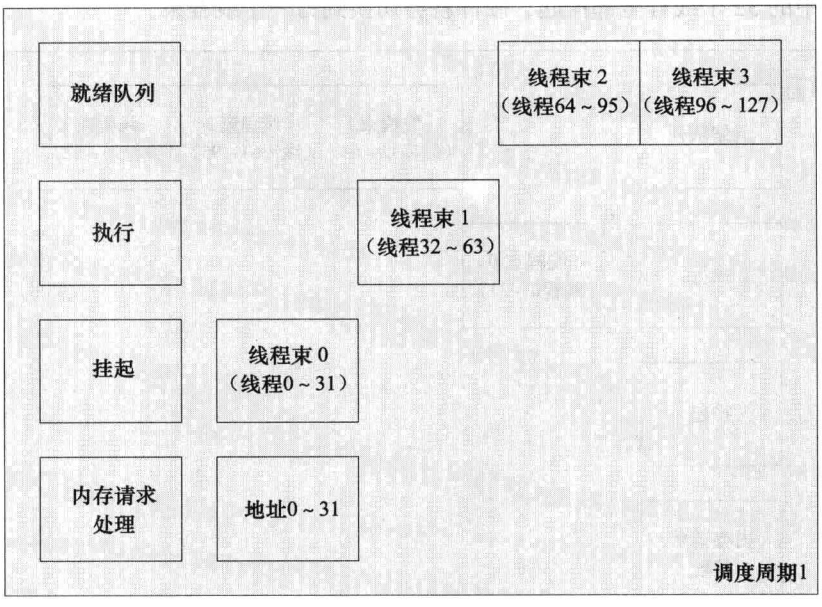


图 5-5　周期 1

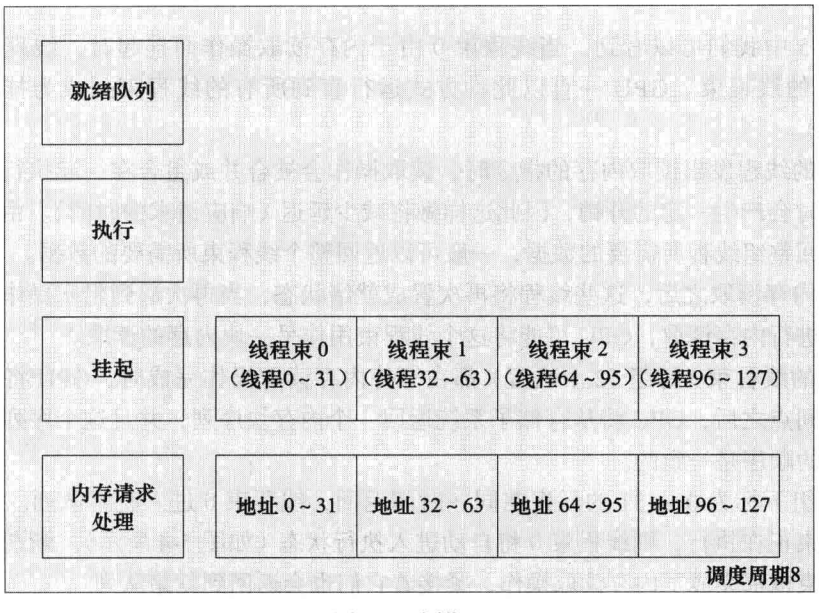


图 5-6　周期 8

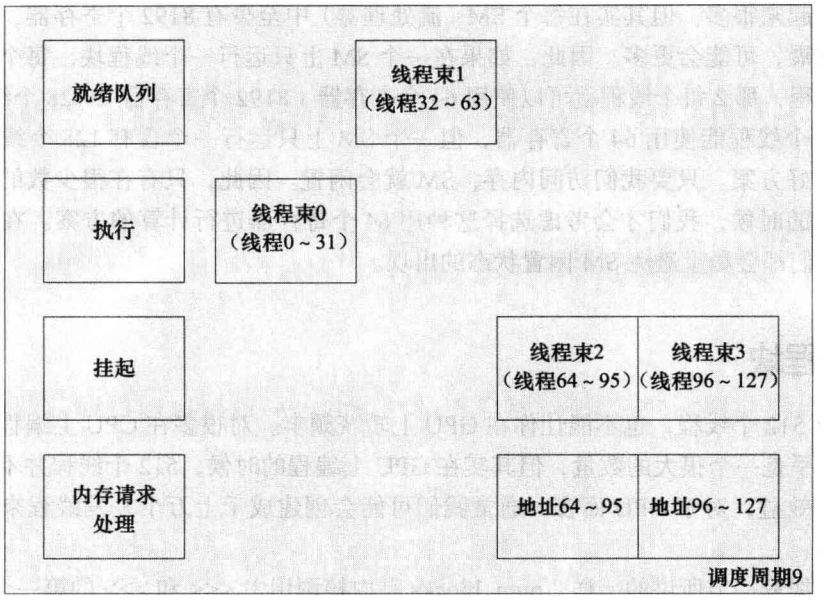


图 5-7 周期 9

5.2.6 CUDA 内核

现在，我们来仔细介绍一下如何调用一个内核。CUDA 专门定义了一个 C 语言的扩展用以调用内核。牢记，一个内核仅仅是一个运行在 GPU 上的函数。调用内核时必须按照以下语法：

```
kernel_function<<<num_blocks, num_threads>>>(param1, param2, ...)
```

一个内核函数中可以传递很多参数，至于如何传递，我们稍后再进行详细介绍。现在，我们来看看另外两个比较重要的参数，num_blocks 与 num_threads。它们可以是实参也可以是形参。在这里，建议使用变量，因为在之后进行性能调优时用起来更加方便。

参数 num_blocks 现在还没涉及，在下一节中我们将进行详细的介绍。现在我们只需保证至少有一个线程块。

参数 num_threads 表示执行内核函数的线程数量。在这个例子中，线程数目即循环迭代的次数。然而，由于受到硬件的限制，早期的一些设备在一个线程块中最多支持 512 个线程，而在后期出现的一些设备中则最多可支持 1024 个线程。本例中，我们无须担心这个问题，但对任何现实的项目而言，这个问题必须注意。在接下来的小节中我们将介绍如何来解决这个问题。

内核调用的下一部分是参数的传递。我们可以通过寄存器或常量内存来进行参数传递，而具体是哪一种方式则视编译器而定。如果使用寄存器传参，每个线程用一个寄存器来传递一个参数。如果现在有 128 个线程，每个线程传递 3 个参数，那么就需要 3 x 128=384 个寄

存器。这听起来很多，但其实在每个 SM（流处理器）中至少有 8192 个寄存器，而且随着后续硬件的发展，可能会更多。因此，如果在一个 SM 上只运行一个线程块，每个线程块中只有 128 个线程，那么每个线程就可以使用 64 个寄存器（8192 个寄存器 ÷ 128 个线程）。

尽管每个线程能使用 64 个寄存器，但一个 SM 上只运行一个含有 128 个线程的线程块并不是一个好方案。只要我们访问内存，SM 就会闲置。因此，只有在很少数的情况下，运算强度很强的时候，我们才会考虑选择这种用 64 个寄存器进行计算的方案。在实际的编程过程中，我们都会尽量避免 SM 闲置状态的出现。

5.3 线程块

即使有 512 个线程，也不能让你在 GPU 上斩获颇丰。对很多在 CPU 上编程的编程人员来说，这似乎是一个很大的数量，但其实在 GPU 上编程的时候，512 个线程并不一定会让你获得很高的效益，对于 GPU 而言，通常我们可能会创建成千上万个并发线程来实现设备上的高吞吐量。

与前面线程一节所讲的一样，num_blocks 是内核调用中 <<< 和 >>> 的第一个参数：

```
kernel_function<<<num_blocks, num_threads>>>(param1, param2,.....)
```

如果我们将这个参数从 1 修改成 2，那么就是告诉 GPU 硬件，我们将启动两倍于之前线程数量的线程，例如：

```
some_kernel_func<<< 2, 128 >>>(a, b, c);
```

这将会调用名为 some_kernel_func 的 GPU 函数共 2 × 128 次，每次都是不同的线程。然而，这样做通常会使 thread_idx 参数的计算变得更加复杂，而 thread_idx 通常又用来表示数组的位置下标。因此，我们之前简单的内核就要稍作调整：

```
__global__ void some_kernel_func(int * const a, const int * const b, const int * const c)
{
  const unsigned int thread_idx = (blockIdx.x * blockDim.x) + threadIdx.x;

  a[thread_idx] = b[thread_idx] * c[thread_idx];
}
```

为了计算 thread_idx 这个参数，我们必须考虑线程块的数量。对第一个线程块而言，blockIdx.x 是 0，因此 thread_idx 直接就等于之前使用过的 threadIdx.x。然而，对于第二个线程块而言，它的 blockIdx.x 的值是 1，blockDim.x 表示本例中所要求的每个线程块启动的线程数量，它的值是 128，那么对第二个线程块而言，在计算 thread_idx 时，要在 threadIdx.x 的基础上加上一个 1 × 128 线程的基地址。

不知道你有没有注意到，在介绍线程块加一个基地址的时候有一个错误？现在我们一共启动了 256 个线程，数组的下标是 0 ~ 255，如果不更改数组的大小，那么第 128 个元素 ~ 256 个元素，将会出现元素访问和写入越界的问题。这种数组越界错误是不会被编译器

发现的，程序代码也会根据数组 a 边界之外的内容来正常执行，因此在调用内核函数的时候要尽量小心，避免这种内存越界访问错误。

对这个例子而言，我们使用一个 128 字节大小的数组，并将启动的两个线程块中的每个线程块的线程数量改成 64：

```
some_kernel_func<<< 2, 64 >>>(a, b, c);
```

你可以从图 5-8 中看到它的表示。

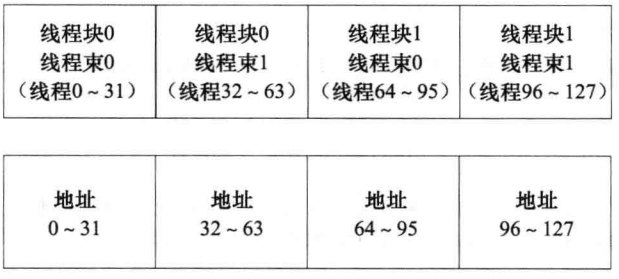


图 5-8 线程块映射的地址空间

注意，尽管我们启动了两个线程块，但 thread_idx 这个参数依然同之前一样等于数组的下标。那么，我们使用线程块的意义究竟在哪？在这个简单的例子中，很明显，没有什么意义。但是在很多现实问题中，我们将不仅仅只处理 512 个元素，很可能更多。事实上，查看线程块的数量限制，你会发现可以使用 65 536 个线程块。

如果使用 65 536 个线程块，每个线程块启动 512 个线程，那么我们一共可以调度 33 554 432（大约 3350 万）个线程。如果每个线程块启动 512 个线程，那么每个 SM 最多可以处理 3 个线程块。事实上，这个限制是基于每个 SM 最多能处理的线程数量。在最新的费米架构的硬件上，每个 SM 每次最多能执行 1536 个线程，而在 G80 的硬件上，只能执行 768 个线程。

如果你打算在费米架构的硬件上每个线程块调度 1024 个线程，那么 65 536 个线程块一共就能调度接近 6400 万个线程，但很不幸的是，如果每个线程块是 1024 个线程，那每个 SM 每次最多运行一个线程块。造成的结果是，除非每个 SM 分配执行的线程块数量是一个以上，否则在单个 GPU 上你将需要 65 536 个 SM 来执行你的程序。目前，在任何 GPU 上 SM 的最大数目都是 30。因此，在线程块需要的 SM 数量超出硬件支持的 SM 数量之前，CUDA 提供了一定的处理机制。这正是 CUDA 的迷人之处——它能扩展为上千个执行单元。并行的极限仅受限于应用程序可以分解的并行程度。

假定现在有 6400 万个线程，每个线程处理数组的一个元素，那么一共可以处理 6400 万个数组元素。假定数组的每个元素都是一个单精度的浮点数，那么每个元素占 4 个字节，总共大约需要 2 亿 5600 万个字节，即约 256MB 数据存储空间。而几乎所有的 GPU 都至少支持这个大小的内存空间。因此，仅使用线程和线程块就可以达到相当大量的并行性和数据覆盖。

很多人会担心大规模数据集问题。它们可能是 GB 级、TB 级，甚至 PB 级的大规模数据。

对于这类问题，这里提供多个解决方案。我们通常会选择一个线程处理多个元素或者使用线程块的其他维度来处理。接下来的小节我们将会进行详细介绍。

线程块的分配

为了确保能够真正地了解线程块的分配，接下来我们写一个简短的内核程序来输出线程块、线程、线程束和线程全局标号到屏幕上。现在，除非你使用的是 3.2 版本以上的 SDK，否则内核中是不支持 printf 的。因此，我们可以将数据传送回 CPU 端然后输出到控制台窗口，内核的代码如下：

```
__global__ void what_is_my_id(unsigned int * const block,
                              unsigned int * const thread,
                              unsigned int * const warp,
                              unsigned int * const calc_thread)
{
  /* Thread id is block index * block size + thread offset into the block */
  const unsigned int thread_idx = (blockIdx.x * blockDim.x) + threadIdx.x;

  block[thread_idx] = blockIdx.x;
  thread[thread_idx] = threadIdx.x;

  /* Calculate warp using built in variable warpSize */
  warp[thread_idx] = threadIdx.x / warpSize;

  calc_thread[thread_idx] = thread_idx;
}
```

在 CPU 端，我们需要执行下面的一部分代码来为数组在 GPU 上分配内存以及将算好的数组数据从 GPU 端复制回来并在 CPU 端显示。

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>

__global__ void what_is_my_id(unsigned int * const block,
              unsigned int * const thread,
              unsigned int * const warp,
              unsigned int * const calc_thread)
{
  /* Thread id is block index * block size + thread offset into the block */
  const unsigned int thread_idx = (blockIdx.x * blockDim.x) + threadIdx.x;

  block[thread_idx] = blockIdx.x;
  thread[thread_idx] = threadIdx.x;

  /* Calculate warp using built in variable warpSize */
  warp[thread_idx] = threadIdx.x / warpSize;

  calc_thread[thread_idx] = thread_idx;
}
```

```
#define ARRAY_SIZE 128
#define ARRAY_SIZE_IN_BYTES (sizeof(unsigned int) * (ARRAY_SIZE))

/* Declare statically four arrays of ARRAY_SIZE each */

unsigned int cpu_block[ARRAY_SIZE];
unsigned int cpu_thread[ARRAY_SIZE];
unsigned int cpu_warp[ARRAY_SIZE];
unsigned int cpu_calc_thread[ARRAY_SIZE];

int main(void)
{
  /* Total thread count = 2 * 64 = 128 */
  const unsigned int num_blocks = 2;
  const unsigned int num_threads = 64;
  char ch;

  /* Declare pointers for GPU based params */
  unsigned int * gpu_block;
  unsigned int * gpu_thread;
  unsigned int * gpu_warp;
  unsigned int * gpu_calc_thread;

  /* Declare loop counter for use later */
  unsigned int i;

  /* Allocate four arrays on the GPU */
  cudaMalloc((void **)&gpu_block, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_thread, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_warp, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_calc_thread, ARRAY_SIZE_IN_BYTES);

  /* Execute our kernel */
  what_is_my_id<<<num_blocks, num_threads>>>(gpu_block, gpu_thread, gpu_warp,
                                             gpu_calc_thread);

 /* Copy back the gpu results to the CPU */
   cudaMemcpy(cpu_block, gpu_block, ARRAY_SIZE_IN_BYTES,
              cudaMemcpyDeviceToHost);
   cudaMemcpy(cpu_thread, gpu_thread, ARRAY_SIZE_IN_BYTES,
              cudaMemcpyDeviceToHost);
   cudaMemcpy(cpu_warp, gpu_warp, ARRAY_SIZE_IN_BYTES,
              cudaMemcpyDeviceToHost);
   cudaMemcpy(cpu_calc_thread, gpu_calc_thread, ARRAY_SIZE_IN_BYTES,
              cudaMemcpyDeviceToHost);

   /* Free the arrays on the GPU as now we're done with them */
   cudaFree(gpu_block);
   cudaFree(gpu_thread);
   cudaFree(gpu_warp);
   cudaFree(gpu_calc_thread);
```

```
    /* Iterate through the arrays and print */
    for (i=0; i < ARRAY_SIZE; i++)
    {
      printf("Calculated Thread: %3u - Block: %2u - Warp %2u - Thread %3u\n",
       cpu_calc_thread[i], cpu_block[i], cpu_warp[i], cpu_thread[i]);
    }
    ch = getch();
}
```

在这个例子中，我们可以看到线程块按照线程块的编号紧密相连。由于处理的是一维数组，所以我们对线程块采用相同的布局便可简单解决问题。以下是此程序的输出结果：

```
Calculated Thread: 0 - Block: 0 - Warp 0 - Thread 0
Calculated Thread: 1 - Block: 0 - Warp 0 - Thread 1
Calculated Thread: 2 - Block: 0 - Warp 0 - Thread 2
Calculated Thread: 3 - Block: 0 - Warp 0 - Thread 3
Calculated Thread: 4 - Block: 0 - Warp 0 - Thread 4
...
Calculated Thread: 30 - Block: 0 - Warp 0 - Thread 30
Calculated Thread: 31 - Block: 0 - Warp 0 - Thread 31
Calculated Thread: 32 - Block: 0 - Warp 1 - Thread 32
Calculated Thread: 33 - Block: 0 - Warp 1 - Thread 33
Calculated Thread: 34 - Block: 0 - Warp 1 - Thread 34
...
Calculated Thread: 62 - Block: 0 - Warp 1 - Thread 62
Calculated Thread: 63 - Block: 0 - Warp 1 - Thread 63
Calculated Thread: 64 - Block: 1 - Warp 0 - Thread 0
Calculated Thread: 65 - Block: 1 - Warp 0 - Thread 1
Calculated Thread: 66 - Block: 1 - Warp 0 - Thread 2
Calculated Thread: 67 - Block: 1 - Warp 0 - Thread 3
...
Calculated Thread: 94 - Block: 1 - Warp 0 - Thread 30
Calculated Thread: 95 - Block: 1 - Warp 0 - Thread 31
Calculated Thread: 96 - Block: 1 - Warp 1 - Thread 32
Calculated Thread: 97 - Block: 1 - Warp 1 - Thread 33
Calculated Thread: 98 - Block: 1 - Warp 1 - Thread 34
Calculated Thread: 99 - Block: 1 - Warp 1 - Thread 35
Calculated Thread: 100 - Block: 1 - Warp 1 - Thread 36
...
Calculated Thread: 126 - Block: 1 - Warp 1 - Thread 62
Calculated Thread: 127 - Block: 1 - Warp 1 - Thread 63
```

正如我们计算的那样，线程索引是 0 ~ 127。一共有两个线程块，每个线程块包含 64 个线程，每个线程块内部线程的索引为 0 ~ 63。一个线程块包含两个线程束。

5.4 线程网格

一个线程网格是由若干线程块组成的，每个线程块是二维的，拥有 X 轴与 Y 轴。此时，每次最多能开启 Y×X×T 个线程。现在，我们用一个实例来进行深入理解。为简单起见，

我们限制 Y 轴方向只有一行线程。

假设我们现在正在看一张标准高清图片，这张图片的分辨率为 1920 × 1080。通常线程块中线程数量最好是一个线程束大小的整数倍，即 32 的整数倍。由于设备是以整个线程束为单位进行调度，如果我们不把线程块上的线程数目设成 32 的整数倍，则最后一个线程束中有一部分线程是没有用的，因此我们必须设置一个限制条件进行限制，防止处理的元素超出 X 轴方向上所规定的范围。在接下来的内容中我们会看到，如果不这样做，程序的性能将会降低。

为了防止不合理的内存合并，我们要尽量做到内存的分布与线程的分布达到一一映射的关系。如果我们没能做到这点，程序的性能可能会降低 5 倍或者更多。有关内存分配的内容，我们将会在下一章进行详细的介绍。

在程序中，要尽量避免使用小的线程块，因为这样做无法充分利用硬件。在本例中，我们将在每个线程块上开启 192 个线程。通常，192 是我们所考虑的最少的线程数目。每个线程块 192 个线程，我们很容易算出处理一行图像需要 10 个线程块（如图 5-9 所示）。在这里，选择 192 这个数是因为 X 轴方向处理的数据大小 1920 是它的整数倍，192 又是线程束大小的整数倍，这使我们的编程变得更加方便。在实际编程中，我们也要尽量做到这一点。

	0	192	384	576	768	960	1152	1344	1536	1728	1920
行0	线程块 0	线程块 1	线程块 2	线程块 3	线程块 4	线程块 5	线程块 6	线程块 7	线程块 8	线程块 9	
行1	线程块 10	线程块 11	线程块 12	线程块 13	线程块 14	线程块 15	线程块 16	线程块 17	线程块 18	线程块 19	
行2	线程块 20	线程块 21	线程块 22	线程块 23	线程块 24	线程块 25	线程块 26	线程块 27	线程块 28	线程块 29	
行…											
行1079	线程块 10 790	线程块 10 791	线程块 10 792	线程块 10 793	线程块 10 794	线程块 10 795	线程块 10 796	线程块 10 797	线程块 10 798	线程块 10 799	

图 5-9 按行分布的线程块

在 X 轴方向的顶部我们可以得到线程的索引，在 Y 轴方向我们可以得到行号。由于每一行只处理了一行像素，每一行有 10 个线程块，因此我们需要 1080 行来处理整张图片，一共 1080 × 10=10 800 个线程块。按照这种一个线程处理一个像素的方式，每个线程块开启 192 个线程，我们一共调度了两百多万个线程。

当我们对单个像素或数据进行单一处理，或者对同一行的数据进行处理时，这种特殊的布局方式是很有用的。在当前费米架构的硬件上，一个 SM 可以处理 8 个线程块，所以上述程序从应用层的角度来说一共需要 1350 个（总共 10 800 个线程块 ÷ 每个 SM 能调度的 8 个线程块）SM 来完全实现并行。但当前费米架构的硬件只有 16 个 SM 可供使用（GT × 580），即每个 SM 将被分配 675 个线程块进行处理。

上述这个例子很简单，数据分布整齐，因此我们很容易就找出了一个好的解决方案，但如果我们的数据不是基于行的，该如何操作呢？由于数组的存在，数据往往可能不是一维的。这时，我们可以使用二维的线程块。例如，在很多的图像算法中使用了 8 × 8 线程块来

处理像素。这里使用像素来进行讲解是因为它们形象直观，更加容易让人理解。而现实中，处理的数据并不一定是基于像素的，也有可能是一个像素的红、绿、蓝三种成分。你可以将其看做一个包含 X、Y 与 Z 轴的空间坐标系下的一个点，也可以用一个二维或者三维的矩阵来存储数据。

5.4.1　跨幅与偏移

为了让 C 语言中的数组也能很好地进行映射，线程块也可以看作是一个二维的结构。然而，对于一个二维的线程块，我们需要提出一些新的概念。就像数组的索引一样，要为一个二维数组的 Y 元素进行索引编号，就必须知道数组的宽度，即 X 元素的数目。如图 5-10 所给的数组。

数组元素0 X = 0 Y = 0	数组元素1 X = 1 Y = 0	数组元素2 X = 2 Y = 0	数组元素3 X = 3 Y = 0	数组元素4 X = 4 Y = 0
数组元素5 X = 0 Y = 1	数组元素6 X = 1 Y = 1	数组元素7 X = 2 Y = 1	数组元素8 X = 3 Y = 1	数组元素9 X = 4 Y = 1
数组元素10 X = 0 Y = 2	数组元素11 X = 1 Y = 2	数组元素12 X = 2 Y = 2	数组元素13 X = 3 Y = 2	数组元素14 X = 0 Y = 2

宽度跨度

偏移

图 5-10　数组元素的映射

数组的宽度也就是存储访问的跨度。偏移即所访问的数据列标号的值，从左开始，第一个总是元素 0。因此，如果要访问数组元素 5 则需要通过索引 [1][5]，或者通过地址计算（行号 × 数组元素大小 × 数组宽度 + 数组元素大小 × 偏移量）。这种计算方式一般是编译器使用的，一般在编写 C 的代码中，在对多维数组进行下标计算时，为了优化，才使用这种方式。

但是，这和 CUDA 中的线程与线程块到底有什么联系？ CUDA 的设计是用来将数据分解到并行的线程与线程块中。它允许我们定义一维、二维或三维的索引（Y × X × T）来方便我们在程序中引用一些并行结构。这样就使得我们程序的结构和内存数据的分布建立一一映射，处理的数据能被分配到单独的 SM 中。不论是在 GPU 上还是在 CPU 上，让数据与处理器保持紧密联系能使性能得到很大的提升。

不过，在对数组进行布局的时候，有一点需要我们特别注意，那就是数组的宽度值最好是线程束大小的整数倍。如果不是，填补数组，使它能充满最后一个线程束。但是这样做会增加数据集的大小。此外，我们还需要注意对填补单元的处理，它和数组中其他单元的处理是不同的。我们可以在程序的执行流中使用分支结构（例如，使用 if 语句），或者也可以在填补单元计算完毕之后再舍弃它们的计算结果。关于如何利用分支来解决这类问题，我们将在之后的小节进行介绍。

5.4.2 X 与 Y 方向的线程索引

在一个线程块上分布一个二维数组也就意味着需要两个线程索引，这样我们才可以用二维的方式访问数组：

```
const unsigned int idx = (blockIdx.x * blockDim.x) + threadIdx.x;
const unsigned int idy = (blockIdx.y * blockDim.y) + threadIdx.y;

some_array[idy][idx] += 1.0;
```

注意 blockDim.x 与 blockDim.y 的使用，这个结构体是由 CUDA 运行时库所提供的，分别表示 X 轴和 Y 轴这两个维度上线程块的数目。现在，我们来修改当前的程序，让它计算一个 32 × 16 维的数组。假设调度四个线程块，我们可以让这四个线程块像条纹布一样布局，然后让数组与线程块上的线程形成一一映射的关系，也可以像方块一样布局，如图 5-11 所示。

线程0～15，线程块0	线程16～31，线程块0
线程32～47，线程块0	线程48～63，线程块0
线程64～79，线程块0	线程80～95，线程块0
线程96～111，线程块0	线程112～127，线程块0
线程0～15，线程块1	线程16～31，线程块1
线程32～47，线程块1	线程48～63，线程块1
线程64～79，线程块1	线程80～95，线程块1
线程96～111，线程块1	线程112～127，线程块1
线程0～15，线程块2	线程16～31，线程块2
线程32～47，线程块2	线程48～63，线程块3
线程64～79，线程块3	线程80～95，线程块3
线程96～111，线程块3	线程112～127，线程块3
线程0～15，线程块4	线程16～31，线程块4
线程32～47，线程块4	线程48～63，线程块4
线程64～79，线程块4	线程80～95，线程块4
线程96～111，线程块4	线程112～127，线程块4

或

线程0～15，线程块0	线程0～15，线程块1
线程16～31，线程块0	线程16～31，线程块1
线程32～47，线程块0	线程32～47，线程块1
线程48～63，线程块0	线程48～63，线程块1
线程64～79，线程块0	线程64～79，线程块1
线程80～95，线程块0	线程80～95，线程块1
线程96～111，线程块0	线程96～111，线程块1
线程112～127，线程块0	线程112～127，线程块1
线程0～15，线程块2	线程0～15，线程块4
线程16～31，线程块2	线程16～31，线程块4
线程32～47，线程块2	线程32～47，线程块4
线程48～63，线程块3	线程48～63，线程块4
线程64～79，线程块3	线程64～79，线程块4
线程80～95，线程块3	线程80～95，线程块4
线程96～111，线程块3	线程96～111，线程块4
线程112～127，线程块3	线程112～127，线程块4

图 5-11 两种不同的线程块布局方式

此外，我们也可以将条纹方式的布局旋转 90 度，使得每个线程块只有一列线程。但是最好不要这样做，因为这样会使得我们的内存访问不连续，造成程序的性能指数级地下降。

因此，当我们在并行化循环的时候要格外的注意，一定要以行的方式进行连续的内存访问，而不是以列的方式。这一点，无论是在 CPU 上还是 GPU 上编码都适用。

那么，我们为什么选择长方形的布局而不是选择正方形的布局？这主要有两个原因。第一个原因是同一个线程块中的线程可以通过共享内存进行通信，这是线程协作中一种比较快的方式。第二个原因是在同一个线程束中的线程存储访问合并在一起了，而在当前费米架构的设备中，高速缓冲存储器的大小是 128 个字节，一次直接访问连续的 128 字节比两次分别访问 64 字节要高效得多。在正方形的布局中，0 ~ 15 号线程映射在一个线程块中，它们访问一块内存数据，但与这块内存相连的数据区则是由另一个线程块访问的，因此，这两块连续的内存数据通过两次存储访问才获得，而在长方形的布局中，这只需要一次存储访问的操作。但如果我们处理的数组更大，例如 64 × 16，那么 32 个线程就能进行连续存储访问，每次读出 128 字节的数据，也就不会出现刚刚所说的那种情况了。

我们通过添加以下代码来选择一种布局方式：

```
dim3 threads_rect(32,4);
dim3 blocks_rect(1,4);
```

或

```
dim3 threads_square(16,8);
dim3 blocks_square(2,2);
```

不论哪种布局方式，线程块中的线程总数都是相同的（32 × 4=128，16 × 8=128），只是线程块中线程的排布方式有所不同。

dim3 是 CUDA 中一个比较特殊的数据结构，我们可以用这个数据结构创建一个二维的线程块与线程网格。例如在长方形布局的方式中，每个线程块的 X 轴方向上开启了 32 个线程，Y 轴方向上开启了 4 个线程。在线程网格上，X 轴方向上有 1 个线程块，Y 轴方向有 4 个线程块。

之后，我们通过以下代码来启动内核：

```
some_kernel_func<<< blocks_rect, threads_rect >>>(a, b, c);
```

或

```
some_kernel_func<<< blocks_square, threads_square >>>(a, b, c);
```

由于在程序中可能不只用到一个维度的线程索引，有可能用到 X 轴与 Y 轴两个维度，因此我们需要修改内核，计算不同维度的索引。除了计算不同维度的相对索引外，有时可能还需要线性计算出相对于整个线程网格的绝对线程索引。为此，我们需要提出一些新的概念以方便线程索引的计算。图 5-12 详细介绍了这些新概念。

以下是对这些新概念的解释：

```
gridDim.——线程网格X维度上线程块的数量
gridDim.——线程网格Y维度上线程块的数量

blockDim.——一个线程块X维度上的线程数量
blockDim.——一个线程块Y维度上的线程数量
```

theadIdx.——线程块X维度上的线程索引
theadIdx.——线程块Y维度上的线程索引

图 5-12 线程网格、线程块及线程的维度

通过找出当前的行索引，然后乘以每一行的线程总数，最后加上在 X 轴方向上的偏移，我们便可以计算出相对于整个线程网格的绝对线程索引。具体代码如下：

```
thread_idx = ((gridDim.x * blockDim.x) * idy) + idx;
```

通过以下代码我们可以获得在 X 轴方向和 Y 轴方向上的线程块索引与线程索引等一些有用信息：

```
__global__ void what_is_my_id_2d_A(
unsigned int * const block_x,
unsigned int * const block_y,
unsigned int * const thread,
unsigned int * const calc_thread,
unsigned int * const x_thread,
unsigned int * const y_thread,
unsigned int * const grid_dimx,
unsigned int * const block_dimx,
unsigned int * const grid_dimy,
unsigned int * const block_dimy)
{
  const unsigned int idx        = (blockIdx.x * blockDim.x) + threadIdx.x;
  const unsigned int idy        = (blockIdx.y * blockDim.y) + threadIdx.y;
  const unsigned int thread_idx = ((gridDim.x * blockDim.x) * idy) + idx;
```

```
  block_x[thread_idx]     = blockIdx.x;
  block_y[thread_idx]     = blockIdx.y;
  thread[thread_idx]      = threadIdx.x;
  calc_thread[thread_idx] = thread_idx;
  x_thread[thread_idx]    = idx;
  y_thread[thread_idx]    = idy;
  grid_dimx[thread_idx]   = gridDim.x;
  block_dimx[thread_idx]  = blockDim.x;
  grid_dimy[thread_idx]   = gridDim.y;
  block_dimy[thread_idx]  = blockDim.y;
}
```

我们可以通过两次调用内核来演示线程块与线程是如何分配布局的。

为了传递一个数据集到GPU端进行计算，我们需要使用cudaMalloc与cudaFree来申请和释放显存，然后再使用cudaMemcpy将数据集从CPU端复制到GPU端，这样，才可以开始计算。由于计算的数组是二维，因此需要注意数组的大小，在申请显存时申请正确大小的显存，传递数据时才能正确地将数据传递到GPU端。

```
#define ARRAY_SIZE_X 32
#define ARRAY_SIZE_Y 16

#define ARRAY_SIZE_IN_BYTES ((ARRAY_SIZE_X) * (ARRAY_SIZE_Y) * (sizeof(unsigned int)))

/* Declare statically six arrays of ARRAY_SIZE each */
unsigned int cpu_block_x[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_block_y[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_thread[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_warp[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_calc_thread[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_xthread[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_ythread[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_grid_dimx[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_block_dimx[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_grid_dimy[ARRAY_SIZE_Y][ARRAY_SIZE_X];
unsigned int cpu_block_dimy[ARRAY_SIZE_Y][ARRAY_SIZE_X];

int main(void)
{
  /* Total thread count = 32 * 4 = 128 */
  const dim3 threads_rect(32, 4); /* 32 * 4 */
  const dim3 blocks_rect(1,4);

  /* Total thread count = 16 * 8 = 128 */
  const dim3 threads_square(16, 8); /* 16 * 8 */
  const dim3 blocks_square(2,2);

  /* Needed to wait for a character at exit */
  char ch;

  /* Declare pointers for GPU based params */
```

```
  unsigned int * gpu_block_x;
  unsigned int * gpu_block_y;
  unsigned int * gpu_thread;
  unsigned int * gpu_warp;
  unsigned int * gpu_calc_thread;
  unsigned int * gpu_xthread;
  unsigned int * gpu_ythread;
  unsigned int * gpu_grid_dimx;
  unsigned int * gpu_block_dimx;
  unsigned int * gpu_grid_dimy;
  unsigned int * gpu_block_dimy;

  /* Allocate four arrays on the GPU */
  cudaMalloc((void **)&gpu_block_x, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_block_y, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_thread, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_calc_thread, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_xthread, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_ythread, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_grid_dimx, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_block_dimx, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_grid_dimy, ARRAY_SIZE_IN_BYTES);
  cudaMalloc((void **)&gpu_block_dimy, ARRAY_SIZE_IN_BYTES);

  for (int kernel=0; kernel < 2; kernel++)
  {
    switch (kernel)
    {
      case 0:
      {
        /* Execute our kernel */
        what_is_my_id_2d_A<<<blocks_rect, threads_rect>>>(gpu_block_x, gpu_block_y,
gpu_thread, gpu_calc_thread, gpu_xthread, gpu_ythread, gpu_grid_dimx, gpu_block_dimx,
gpu_grid_dimy, gpu_block_dimy);
      } break;

      case 1:
      {

       /* Execute our kernel */
       what_is_my_id_2d_A<<<blocks_square,threads_square>>>(gpu_block_x,gpu_block_y,
gpu_thread, gpu_calc_thread,gpu_xthread, gpu_ythread,gpu_grid_dimx, gpu_block_dimx,
gpu_grid_dimy, gpu_block_dimy);
      } break;

      default: exit(1); break;
    }

    /* Copy back the gpu results to the CPU */
    cudaMemcpy(cpu_block_x, gpu_block_x, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
```

```
    cudaMemcpy(cpu_block_y, gpu_block_y, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_thread, gpu_thread, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_calc_thread, gpu_calc_thread, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_xthread, gpu_xthread, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_ythread, gpu_ythread, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_grid_dimx, gpu_grid_dimx, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_block_dimx,gpu_block_dimx, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_grid_dimy, gpu_grid_dimy, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);
    cudaMemcpy(cpu_block_dimy, gpu_block_dimy, ARRAY_SIZE_IN_BYTES,
               cudaMemcpyDeviceToHost);

    printf("\nKernel %d\n", kernel);
    /* Iterate through the arrays and print */
    for (int y=0; y < ARRAY_SIZE_Y; y++)
    {
        for (int x=0; x < ARRAY_SIZE_X; x++)
        {
          printf("CT: %2u BKX: %1u BKY: %1u TID: %2u YTID: %2u XTID: %2u GDX: %1u BDX: %
1u GDY %1u BDY %1u\n", cpu_calc_thread[y][x], cpu_block_x[y][x], cpu_block_y[y][x],
cpu_thread[y][x], cpu_ythread[y][x], cpu_xthread[y][x], cpu_grid_dimx[y][x],
cpu_block_dimx[y][x], cpu_grid_dimy[y][x], cpu_block_dimy[y][x]);

          /* Wait for any key so we can see the console window */
          ch = getch();
          }
    }
    /* Wait for any key so we can see the console window */
    printf("Press any key to continue\n");
    ch = getch();
  }

  /* Free the arrays on the GPU as now we're done with them */
  cudaFree(gpu_block_x);
  cudaFree(gpu_block_y);
  cudaFree(gpu_thread);
  cudaFree(gpu_calc_thread);
  cudaFree(gpu_xthread);
  cudaFree(gpu_ythread);
  cudaFree(gpu_grid_dimx);
  cudaFree(gpu_block_dimx);
  cudaFree(gpu_grid_dimy);
  cudaFree(gpu_block_dimy);
}
```

由于程序的输出内容太多，此处就不一一列举出来。你可以通过下载源代码，然后运行这个程序，在输出中如图 5-12 中那样线程块与线程索引循环输出。

5.5 线程束

在介绍线程的时候就涉及线程束的调度。线程束是 GPU 的基本执行单元。GPU 是一组 SIMD 向量处理器的集合。每一组线程或每个线程束中的线程同时执行。在理想状态下，获取当前指令只需要一次访存，然后将指令广播到这个线程束所占用的所有 SP 中。这种模式比 CPU 模式更加高效，CPU 是通过获取单独的执行流来支持任务级的并行。在 CPU 的模式中，由于每个核运行一个独立的任务，所以理论上，我们可以根据核的数目划分存储带宽，但指令吞吐量的效率会下降。而事实上，如果程序的数据都能放入缓存，CPU 的片上多级缓存可以有效地隐藏由内存读取带来的延迟。

在传统的 CPU 上，我们可以通过 SSE、MME 和 AVX 指令形式看到很多向量类型的指令。这些指令也执行的是单指令多数据这种类型的操作。假设这里有 N 个数，我们让每个数都加 1。如果用 SSE 指令集，那么就有 128 位的寄存器可以利用，每次能同时处理 4 个操作数。而 AVX 指令集则将它的寄存器扩展到 256 位，这使它的处理能力更强大。但直到目前为止，很少有本地支持这种类型的优化，除非你所使用的是英特尔编译器。不过，现在的 GNU gcc 编译器也支持 AVX 指令集，而 Microsoft Visual Studio 2010 则是通过打开“/arch:AVX”编译开关来支持 AVX 指令集。由于缺乏对向量类型指令的支持，向量类型指令并没有被广泛地利用，因此，改变是很重要的。而现在，对向量类型指令的支持也不再只局限于英特尔编译器了。

当使用 GPU 进行编程时，必须使用向量类型指令，因为 GPU 采用的是向量体系结构，只有让代码在成千上万个线程上运行才能充分高效利用 GPU 的资源。当然，你可以写一个单线程的 GPU 程序，只需要用一个 if 语句简单地判断一下线程索引是否为 0。但这相对于 CPU 而言，性能非常的低。然而，当在获取一个连续的 CPU 初始部署工作时这种方法却很有用。这种方法允许我们在将程序并行化之前进行检查，例如，检查复制到 GPU 或从 GPU 复制出的数据是否正常运转。

当前，GPU 上的一个线程束的大小为 32，英伟达公司保留着对这个数修改的权利。因此，他们提供了一个固有变量——wrapSize，我们可以通过这个变量来获取当前硬件所支持的线程束大小。和其他幻数一样，我们不能通过硬编码的方式改变它的大小。然而许多用 SSE 指令集优化的程序就可以用硬编码的方式来假定 SSE 的寄存器大小为 128 位。而到了 AVX 正式发布之后，重新编译代码就不会通过。因此，不要在程序中再犯这样的错误，不要用硬编码的方式去修改它们。

那么我们为什么会如此关注线程束的大小呢？原因有很多，接下来我们就简要地看一看这些原因。

5.5.1 分支

我们之所以如此关注线程束的大小，一个很重要的原因就是分支。一个线程束是一个单

独的执行单元，使用分支（例如，使用 if、else、for、while、do、swith 等语句）可以产生不同的执行流。在 CPU 上使用分支很复杂，因为它需要根据之前的运行情况来预测下一次执行到底要执行哪一块代码。在 CPU 上，指令流通常都会被预提取，然后放入 CPU 的指令管线中。假设预测是准确的，那么 CPU 就避免了一次失速事件。如果预测错误，CPU 则需要重新执行预测指令，然后获取另一个分支的指令，再将其添入管线之中。

相比之下，GPU 对分支的处理就没有这么复杂。GPU 在执行完分支结构的一个分支后会接着执行另一个分支。对不满足分支条件的线程，GPU 在执行这块代码的时候会将它们设置成未激活状态。当这块代码执行完毕之后，GPU 继续执行另一个分支，这时，刚刚不满足分支条件的线程如果满足当前的分支条件，那么它们将被激活，然后执行这一段代码。最后，所有的线程聚合，继续向下执行。具体代码如下：

```
__global__ some_func(void)
{
  if (some_condition)
  {
    action_a();
  }
  else
  {
    action_b();
  }
}
```

代码中，当计算出 some_condition 的值之后，就可进入分支代码块中运行，其中至少会有一个分支会被执行，否则就没有必要在这里演示这个示例程序了。这里我们假设索引为偶数的线程满足条件为真的情况，执行函数 action_a()，索引为奇数的线程满足条件为假的情况，执行函数 action_b()。如图 5-13 所示，我们可以看到这一个线程束中不同线程的执行情况。

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
+	−	+	−	+	−	+	−	+	−	+	−	+	−	+	−	+

图 5-13 不同线程的分支选择

为简单起见，我们只画出了 32 个线程中 16 个线程的执行情况，至于这么做的原因我们马上就可以看到。在图中，所有标记“+”的线程都将执行条件为真的分支代码块，而标记“-”的线程都将执行条件为假的分支代码块。

由于硬件每次只能为一个线程束获取一条指令，线程束中一半的线程要执行条件为真的代码段，一半线程执行条件为假的代码段，因此，这时有一半的线程会被阻塞，而另一半线程会执行满足条件的那个分支。如此，硬件的利用率只达到了 50%。这种情况并不好，这就好像一个双核的 CPU 只有一个核被利用。对待这个问题，很多懒惰的程序员都会选择逃避，不去理它，但这会使我们程序的性能大打折扣。

但这种现象毕竟发生了，因此我们就要想办法来解决它。这里有一个技巧可以用来避免这个问题。事实上，在指令执行层，硬件的调度是基于半个线程束，而不是整个线程束。这

意味着，只要我们能将半个线程束中连续的 16 个线程划分到同一个分支中，那么硬件就能同时执行分支结构的两个不同条件的分支块，例如，示例程序中 if-else 的分支结构。这时硬件的利用率就可以达到 100%。

如果需要让数据进行两种不同类型的处理，那么我们可以将数据以 16 为分界线进行划分，这样可以提升性能。下面这段代码是根据线程的索引划分分支的：

```
if ((thread_idx % 32) < 16)
{
  action_a();
}
else
{
  action_b();
}
```

C 语言中的模运算（%）得到的是操作数整除之后所得的余数，即上述代码中 thread_idx 模 32 后所得的结果总是为 0 ~ 31，循环再次回到 0 次。在理想状态下，如果执行函数 action()_a 的 16 个线程都访问一个浮点数或者一个整数，那么将获取 64 个字节的内存数据，如果另外一半线程束也进行同样的操作，那么就可以只发起一个 128 字节的内存访问命令，同时将这两半线程束所需的数据取得。但这种现象发生的条件很苛刻，只有当缓存大小与线程束需要获取的内存数据大小相同而且数据在内存分布上是连续的才会发生。

5.5.2 GPU 的利用率

我们关注线程束的另一个原因就是防止 GPU 未被充分利用。CUDA 的模式是用成千上万的线程来隐藏内存操作的延迟（从发出存储请求到完成访存操作所花的时间）。比较典型的，如对全局内存访问的延迟一般是 400 ~ 600 个时钟周期。在这段时间里，GPU 会忙于其他任务，而不是空闲地等待访存操作的完成。

当我们为一个 GPU 分配一个内核函数，由于不同的硬件有不同的计算能力，SM 一次最多能容纳的线程数也不尽相同，目前，一个 SM 最多能容纳 768 ~ 2048 个线程。SM 一次最多能容纳的线程数量主要与底层硬件的计算能力有关，随着未来硬件的改变，这个数值也会有所改变。表 5-1 显示了在不同计算能力的设备上，每个线程块上开启不同数量的线程时的设备利用率。

表 5-1 硬件利用率（%）

计算能力 / 每个线程块中的线程数	1.0	1.1	1.2	1.3	2.0	2.1	3.0
64	67	67	50	50	33	33	50
96	100	100	75	75	50	50	75
128	100	100	100	100	67	67	100
192	100	100	94	94	100	100	94

（续）

计算能力 / 每个线程块中的线程数	1.0	1.1	1.2	1.3	2.0	2.1	3.0
256	100	100	100	100	100	100	100
384	100	100	75	75	100	100	94
512	67	67	100	100	100	100	100
768	N/A	N/A	N/A	N/A	100	100	75
1024	N/A	N/A	N/A	N/A	67	67	100

计算能力在 1.0 ~ 1.2 的设备主要是 G80/G92 系列，而 GT200 系列设备的计算能力为 1.3。费米架构的设备计算能力为 2.0 或 2.1，计算能力 3.0 是开普勒架构的设备。

通过表 5-1 我们可以发现，无论计算能力是何级别，当每个线程块开启 256 个线程时，设备地利用率都达到了 100%。为了最大限度地利用设备，使程序性能得到提升，我们应尽量将每个线程块开启的线程数设为 192 或 256。此外，数据集也要和线程布局尽量一一对应，以达到更高的性能优化。例如，当需要处理三维点数据时，我们就可以考虑每个线程块开启 192 个线程的方案。

除了将线程数设置成固定值 256 来获得 100% 的利用率外，通过观察不同计算能力的硬件，选出每个达到 100% 利用率最少所需的线程数，将线程数设为这个值，同样也能高效利用硬件。

讨论了 SM 一次最多能容纳的线程数，我们也需要讨论一下 SM 一次最多能容纳的线程块的数目。SM 容纳线程块的数目会受到内核中是否用到同步的影响。而所谓的同步，就是当程序的线程运行到某个点时，运行到该点的线程需要等待其他还未运行到该点的线程，只有当所有的线程都运行到这个点时，程序才能继续往下执行。例如，我们执行一个读内存的操作，在这个读操作完成之后进行线程同步，所有的线程都要进行读操作，但由于不同线程执行有快有慢，有的线程束很快就完成操作，有的则需要更长的时间，执行完的线程需要等待还未完成读操作的线程，只有当所有的线程都完成读操作，程序才能继续向下执行。

执行一个线程块的时间（或延迟）是不确定的。从负载平衡的角度来看，这并不是什么好消息，因为我们想让更多的线程运行起来。如果现在有 256 个线程，每个线程束包含 32 个线程，那么在计算能力为 2.X 的设备上需要 8 个线程束。在计算能力为 1.X 的设备上，一个 SM 一次能调度处理 24 个线程束（32 × 24=768 个线程），而在计算能力为 2.X 的设备上能一次调度处理 48 个线程束（32 × 48=1536 个线程）。每个线程块只有当它执行完全部指令时才会从 SM 撤走，在计算能力为 2.X 或更高的设备上，每个线程块最多能支持 1024 个线程，因此，有时会出现这种情况，所有的线程束都闲置地等待一个线程束执行完毕，SM 因为这个原因也闲置了，这大大降低了程序的性能。

由此可见，每个线程块开启的线程数越多，就潜在地增加了等待执行较慢的线程束的可能性。因为当所有的线程没有到达同步点时 GPU 是无法继续向下执行的。因此，有时候我们会选择在每个线程块上开启较少的线程，例如，128 个线程，以此来减少之前那种等待的

可能性。但这样做会严重地影响性能，例如，在费米架构上的设备上采取这种方案会使设备的利用率降低到原来的 2/3。从表 5-1 计算能力为 2.0（费米架构）的设备中可以看出，只有当线程块的线程数量至少达到 192 时 SM 才能被充分地利用。

尽管如此，我们也不必过分花费太多精力在线程束的数量上，因为它毕竟只是一种 SM 上线程总数的体现形式，表 5-3 中每个 SM 上总共能容纳的线程数以及表 5-1 中的利用率才是我们最应该关心的部分。

从表 5-3 中我们看到，线程块中的线程数为 128 或者更少时，当设备计算能力从 1.3（GT300 系列）变化到 2.X（费米架构系列），SM 支持执行线程的总数并没有发生变化。这是由于每个 SM 能调度的线程块的数量并没有受到限制。一个 SM 上支持的线程总数增加了，并不意味着支持的线程块的数量也会增加。因此，要获得更好的性能，最好保证你的线程块中最少有 192 个线程。

表 5-2 每个 SM 能容纳的线程块数目

计算能力 / 每个线程块中的线程数	1.0	1.1	1.2	1.3	2.0	2.1	3.0
64	8	8	8	8	8	8	16
96	8	8	8	8	8	8	12
128	6	6	8	8	8	8	16
192	4	4	5	5	8	8	10
256	3	3	4	4	6	6	8
384	2	2	2	2	4	4	5
512	1	1	2	2	3	3	4
768	N/A	N/A	1	1	2	2	2
1024	N/A	N/A	1	1	1	1	2

表 5-3 每个 SM 能容纳的线程数目

计算能力 / 每个线程块中的线程数	1.0	1.1	1.2	1.3	2.0	2.1	3.0
64	512	512	512	512	512	512	1024
96	768	768	768	768	768	768	1536
128	768	768	1024	1024	1024	1024	2048
192	768	768	960	960	1536	1536	1920
256	768	768	1024	1024	1536	1536	2048
384	768	768	768	768	1536	1536	1920
512	512	512	1024	1024	1536	1536	2048
768	N/A	N/A	N/A	N/A	1536	1536	1536
1024	N/A	N/A	N/A	N/A	1024	1024	2048

5.6 线程块的调度

假设现在有 1024 个线程块需要调度处理，但只有 8 个 SM 去调度它们。在费米架构的硬件上，当每个线程块中的线程数量并不多时，每个 SM 一次最多能接受 8 个线程块。当每个线程块中线程的数量比较合理时，一个 SM 一次一般能处理 6 ~ 8 个线程块。

现在，我们将这 1024 个线程块分配到 6 个 SM 中，平均每个 SM 需要处理 170 个线程块，还余 4 个需要处理。接下来我们将详细介绍余下的这 4 个线程块如何处理，因为它们的处理会带来一个比较有趣的问题。

像之前提到的那样，前 1020 个线程块被分配到 SM 中进行处理，那么它们是怎样分配的？硬件也许会分配 6 个线程块分配到第一个 SM，然后紧接着的 6 个线程块分配到第二个 SM，跟着第三个、第四个，依次类推。也有可能是硬件轮流为每个 SM 分配一个线程块，线程块 0 被分配到 0 号 SM 中，线程块 1 被分配到 1 号 SM 中，线程块 2 到 2 号 SM 中，依此类推。英伟达并没有公布他们到底是使用的哪种方式，但很有可能是后面一种，因为这种方式能让 SM 达到合理的负载平衡。

如果现在我们有 19 个线程块和 4 个 SM，将所有的线程块都分配到一个 SM 上进行处理，显然不够合理。如果让前 3 个 SM 每个处理 6 个线程块，最后一个 SM 处理一个线程块，那么最后一个线程块就会很快处理完，然后 SM 会闲置。这样做设备的利用率会很低。

如果我们用轮流交替的方式将线程块分配到 SM 中，每个 SM 处理 4 个线程块（$4 \times 4 = 16$ 个线程块），还剩三个线程块单独用一个 SM 再次处理，假设每个线程块执行所花费的时间是一样的，那么通过让每个 SM 能均衡地处理线程块，而不是像之前那样有的 SM 负载过重，有的 SM 负载过轻，这样我们就可以减少 17% 的执行时间。

而在实际的程序中，我们往往会用到成千上万个线程块。在线程块调度者为每个 SM 初始化分配了线程块之后，就会处于闲置状态，直到有线程块执行完毕。当线程块执行完毕之后就会从 SM 中撤出，并释放其占用的资源。由于线程块都是相同的大小，因此一个线程块从 SM 中撤出后另一个在等待队列中的线程块就会被调度执行。所有的线程块的执行顺序是随机、不确定的，因此，当我们在编写一个程序解决一个问题的时候，不要假定线程块的执行顺序，因为线程块根本就不会按照我们所想的顺序去执行。

当执行到一些相关联的操作时，我们会遇到一个很严重的问题。我们以浮点加法运算作为例子，而事实上，浮点加法运算并没有太多的相关联性。我们对一个浮点数组做加和运算，数组按照不同顺序做加和操作时得到的结果会不同。这主要是由舍入误差所引起的，而浮点运算必定会产生这种误差。但不论是什么顺序，最终的结果都是正确的。因此，这并不是一个并行执行的问题，而是一个顺序的问题。因为即便在 CPU 上用一个单线程的程序来运行，我们也会看到相同的问题。无论是在 CPU 还是 GPU 上，我们对一串随机的浮点数做加和运算，不论是从头加到尾，还是从尾加到头，我们都会得到不同的结果。甚至可能更糟的是，在 GPU 上，由于线程块的不确定调度，多次对相同的数据进行计算，每次得到的结果都不相同，但所有结果都是正确的。关于这种问题的解决办法，本书的后半部分将会有详

细的介绍。到目前为止，我们只需要知道这样做得到的结果会不同，但没有必要认为这个结果是错的。

回到我们之前讨论的剩余线程块的处理的问题上。通常，线程块的数目并不是 SM 数目的整数倍。但由于制造更大、更复杂的处理器很困难，我们往往会看到 CUDA 的设备上搭载着奇数个 SM。在处理器制造工艺中，硅的使用很广泛，但正是由于硅的使用的增加，也降低了其他领域相对再增加的可能性。同其他处理器的生产厂商一样，英伟达禁掉了有瑕疵的 SM 的使用，并将其作为低配置单元出售。这使得其生产量大大增加，从某种角度来说，这些有瑕疵的设备创造了经济价值。然而，对于程序员而言，这只意味着 SM 的总数不总是 2 的整数倍。当前费米 480 系列的显卡以及 Tesla S2050、S2070、C2050、C2070 系列的显卡均搭载 16 个 SM，其中一个 SM 不可用，即只有 15 个 SM 可供使用。目前，580 系列的显卡已经解决了这个问题，但是在未来的 GPU 中，这个问题还可能再度出现。

到目前为止，解决余下线程块的方法只有一个，那就是让我们的内核函数尽量长并且需要足够长的时间完成。例如，我们用一个有限的时间步长来进行模拟。假设在费米 480 系列的显卡上，我们有 16 个线程块需要处理，但现在一共只有 15 个 SM 可用，那么每个 SM 分配一个线程块，余下的一个线程块只在前 15 个线程块中的某一个执行完毕之后再调度处理。如果每次内核执行需要 10 分钟，那么前 15 个线程块几乎会同时完成计算，然后再调度余下的那个线程块，那么其他 14 个 SM 闲置，因此我们还需要等待 10 分钟整个内核函数的调用才会结束。相对于将 16 个线程块划分成几个大块，这种划分成若干小块的解决方案则提供了更好的间隔尺寸。

当运行环境是服务器环境时，此时就不止 15 个 SM 可供我们使用。在用 GPU 搭建的服务器环境中，往往都会有多块 GPU。此时，如果我们的任务仅仅是执行刚刚的内核函数，那么将会有很多 SM 处于闲置状态直至该内核函数结束调用。在这种情况下，我们最好重新设计内核函数，以保证在每个 GPU 中，线程块的数目都是 SM 数目的整数倍，以此提高设备的利用率。

其实从负载平衡的角度来看，这个问题还有待优化。因此，在之后的 CUDA 运行时库中支持重叠的内核以及在同一块 CUDA 设备上可以运行多个单独的内核。通过这种方法，我们就可以维持吞吐量，使 GPU 集群不止有一任务源可以调度。一旦设备出现闲置，它就会从内核流中选择另一个内核进行执行。

5.7 一个实例——统计直方图

在编程中，我们可能经常遇到统计直方图，利用统计直方图，我们可以得到某些类或者某些数据出现的频数，从而得到它们的分布情况。假设统计某些数字出现的次数，我们用一个 bin 数组与这些数字相关联，保存它们出现的次数，如果某个数出现一次，那么相对应的 bin 就加 1。

在这个简单例子中，我们用 256 个 bin 来统计数字 0 ~ 255 出现的次数。我们只需要遍

历一个字节数组，这个数组里面的值只占一个字节，如果数组里面的值为0，那么bin[0]就加1，如果是10，那么bin[10]就加1。我们将这个数组称作输入数组。

用串行算法解决的代码非常简单，如下所示：

```
for (unsigned int i=0; i< max; i++)
{
  bin[array[i]]++;
}
```

这里，我们通过索引i来遍历整个输入数组，用++运算符来对相应的bin进行累加操作。

当将这段串行代码转化成并行的时候，我们会遇到一个问题。如果我们用256个线程来执行，若多个线程同时对同一个bin进行加1操作，将出现竞争的情况。

如果你知道C语言是如何转化成汇编指令的，那么你就会知道这段代码在汇编下会如何执行。细分下来，主要有这样几步操作：

1）从输入数组中读取数据到寄存器；

2）计算出这个数对应的bin的基地址与偏移量；

3）获取当前这个数对应的bin的值；

4）对bin值进行加1；

5）将新的bin值写回内存。

问题主要出现在步骤3、4、5，因为它们没有进行原子操作。所谓的原子操作，就是当某个线程对某项数据进行修改的时候，其他优先级比较低的线程无法打断它的操作，直至该线程完成对该数据的所有操作。由于CUDA采用的是多线程并行模式，如果我们以锁步的方式执行这段伪码就会遇到一个问题。两个或两个以上的线程在步骤3获取了相同的值，接着进行加1操作，然后将新的数据再写回内存。这时，最后做写操作的那个线程写入的数据才是最终的值。这肯定是不对的，因为本来一共加了N，可最后只加了1，数据值错误的少了N-1。

数据的相关性造成了这个问题的产生，而在用顺序执行的代码中我们根本看不到这个问题。bin在线程之间是以共享资源的形式存在，因此，在某个线程读取和修改bin值之前，必须等上一个线程完成对bin的操作才行。

这个问题在日常编程中并不少见，CUDA针对此也提供了一个较为简单的方法：

```
atomicAdd(&value);
```

这个操作保证了对value这个值进行的加法操作在所有线程之间是串行执行的。

既然问题得到了解决，那么我们就来考虑该用什么样的方式将之前的串行代码转化成在线程、线程块及线程网格上运行的并行代码。主要有两种方式，一种是基于任务分解的模型，一种则是基于数据分解的模型。当然，两种方案都需要考虑。

基于任务分解的模型主要是将输入数组中的元素分配到每一个线程中，然后再进行原子加法操作。这种方法对于程序来说是最简单的解决方案，但同时也有一定缺点，因为有些

数据是共享资源。假设现在有 256 个 bin，而输入数组有 1024 个元素，假定这 256 个 bin 所统计的信息是平均分布的，并且每个 bin 最后统计出的结果都为 4，那在计算每个 bin 的时候就会产生 4 次竞争。如果输入数组更大（因为 CUDA 本身就是用来解决大数据问题的），那产生的竞争也就更多，执行的时间也就更长。也就是说，输入数组的大小决定了程序运行的总时间。

如果直方图中所统计的信息都是平均分布的，那么每个 bin 产生竞争的次数就等于输入数组的大小除以所统计的信息种类数，即 bin 的数目。假设输入数组的大小为 512MB（524 288 个元素），一共统计了 4 类信息，那么每个 bin 就会有 131 072 个元素进行竞争。最糟糕的是，如果输入数组中所有元素都对应同一个 bin，由于通过原子操作对这个 bin 进行内存读写，因此我们的并行程序也就完全变成了一个串行程序。

不论是刚刚所说的哪种情况，程序执行的时间都受到硬件处理竞争的能力以及读 / 写内存带宽的影响。

下面是多线程采用任务分解模型的具体实现。以下是 GPU 程序代码：

```
/* Each thread writes to one block of 256 elements of global memory and contends for
write access */

__global__ void myhistogram256Kernel_01(
  const unsigned char const * d_hist_data,
  unsigned int * const d_bin_data)
{
  /* Work out our thread id */
  const unsigned int idx = (blockIdx.x * blockDim.x) + threadIdx.x;

  const unsigned int idy = (blockIdx.y * blockDim.y) + threadIdx.y;

  const unsigned int tid = idx + idy * blockDim.x * gridDim.x;

  /* Fetch the data value */
  const unsigned char value = d_hist_data[tid];

  atomicAdd(&(d_bin_data[value]),1);
}
```

我们用一块 GTX460 显卡对这种方法进行测试，测得其处理速度为 1025MB/s。很有趣的是，该方法的处理速度并不随输入数组的大小改变而加快或降低。无论输入数组的大小是多少，都会得到一个固定的速度值，而且这个速度值非常慢。对于显存为 1GB 的 GTX460 显卡而言，它的存储带宽为 115GB/s，而我们测得的处理速度为 1025MB/s，可想而知，采取这种简单幼稚的方案获取的性能是多么低。

虽然这个处理速度非常低，但它同时也说明这是由于某些因素的限制而造成，因此，我们要找出这些影响性能的因素并且消除它们。在该程序中，其中一个最可能影响性能的因素就是存储带宽。在程序中，一共有两次对内存的读 / 写操作，先是每次从输入数组中获取 N 个数然后又压缩至 N 次写回到一块 1K 的内存区（256 个元素 × 每个数 4 个字节）。

首先我们来讨论一下读操作。每个线程从输入数组中读得一个字节的元素，但由于线程束的读操作合并到了一起，即每半个线程束（16个线程）同时做一次读操作，由于最少的传输大小为32字节，但现在只读了16个字节，因此浪费了50%的存储带宽，性能很低。在最好的情况下，每半个线程束最多能读128个字节，如果是这样，每个线程将从内存中获得4个字节的数据，即可以通过处理4个统计直方图的条目，而不是先前的一个来完成。

现在，我们来看如何每次读4个字节的数据然后进行统计计算。我们可以按照读整型数的方式每次读取一个4字节的数据，然后将这个整型数拆分成4个字节来进行计算，如图5-14所示。理论上这提供了一种更好的合并读的方式并确实能让我们的程序获得比之前更好的性能。以下是修改后的内核代码：

```
/* Each read is 4 bytes, not one, 32 x 4 = 128 byte reads */
__global__ void myhistogram256Kernel_02(
const unsigned int const * d_hist_data,
unsigned int * const d_bin_data)
{
  /* Work out our thread id */
  const unsigned int idx = (blockIdx.x * blockDim.x) + threadIdx.x;

  const unsigned int idy = (blockIdx.y * blockDim.y) + threadIdx.y;

  const unsigned int tid = idx + idy * blockDim.x * gridDim.x;

  /* Fetch the data value as 32 bit */
  const unsigned int value_u32 = d_hist_data[tid];

  atomicAdd(&(d_bin_data[ ((value_u32 & 0x000000FF) ) ]),1);

  atomicAdd(&(d_bin_data[ ((value_u32 & 0x0000FF00) >> 8 ) ]),1);

  atomicAdd(&(d_bin_data[ ((value_u32 & 0x00FF0000) >> 16 ) ]),1);

  atomicAdd(&(d_bin_data[ ((value_u32 & 0xFF000000) >> 24 ) ]),1);
}
```

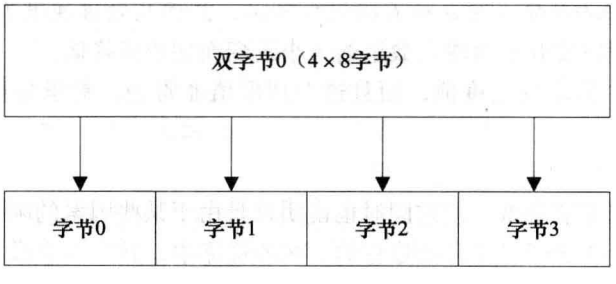


图5-14 字与字节的映射关系

但在运行内核时我们会发现，我们所做的努力只实现了零加速。事实上，这种现象在我们优化程序的时候会经常发生，原因主要是我们还没真正弄清楚引起性能瓶颈的因素。

为什么我们做出了努力可是程序还是没有得到优化呢？原因主要是内核是在计算能力为2.x的硬件上运行的。半个线程束合并读取内存数据的方式对计算能力为2.x的硬件来说并不会产生很大影响，因为它已经将整个线程束的内存读取都合并到一起了。也就是说，在测试设备GTX460上（计算能力为2.1的硬件），来自一个线程束的32次单字节读取被合并为一次32字节的读操作。

很明显，相对于存储带宽带来的微小影响，原子写操作才可能是性能瓶颈的罪魁祸首。为此，我们将采用另外一种方案来编写内核，即基于数据分解的模型。通过观察我们会发现内核中有一些数据会被再次用到，而我们可以将这些被再次利用的数据放入能高效处理共享数据的存储区中，例如二级缓存和共享内存，以此来提高程序的性能。

我们知道最初引起程序性能低下的原因就是对256个bin所产生的竞争。多个SM中多个线程块都要将它们的计算结果写回内存，然后硬件对每个处理器的缓存中的bin数组进行同步。分开来看，就是程序从内存中获取数据，然后做加法操作，最后将计算得到的新值写回内存。而这其中，有好几步操作都可以一直在二级缓存中进行。在费米型架构的硬件上，二级缓存中的数据在SM之间是共享的。相比之下，如果在计算能力为1.x的硬件上，如果将数据都放在全局内存上进行读写操作，程序的性能会降低好几个数量级。

但即使在费米架构的硬件上使用了二级缓存，我们仍需对所有的SM进行同步。另外，由于我们执行的写操作是一种比较分散的模式，它依赖于直方图输入数据的分布特性，有时数据的分布非常分散，造成写操作没法合并，从而严重影响程序的性能。

另一种方案就是让每一个SM都计算出一个统计直方图，最后再将所有的直方图汇总到一块主内存上。无论是CPU编程还是GPU编程，我们都要尽量去实现这种方案。因为利用的资源越接近处理器（例如SM），程序就会运行得越快。

之前我们提到使用共享内存。共享内存是一块比较特殊的内存，因为它存在于芯片上并且它的存取比全局内存更快。我们可以在共享内存上创建一个包含256个bin的局部统计直方图，最后将所有共享内存上计算得到的统计直方图通过原子操作汇总到全局内存。假设每个线程块处理一个统计直方图，而对全局内存读写的操作次数也不会因此而减少，但写回内存的操作却因此可以合并起来。以下是这种方法的内核代码：

```
__shared__ unsigned int d_bin_data_shared[256];

/* Each read is 4 bytes, not one, 32 x 4 = 128 byte reads */
__global__ void myhistogram256Kernel_03(
const unsigned int const * d_hist_data,
unsigned int * const d_bin_data)
{
  /* Work out our thread id */
  const unsigned int idx = (blockIdx.x * blockDim.x) + threadIdx.x;
  const unsigned int idy = (blockIdx.y * blockDim.y) + threadIdx.y;
  const unsigned int tid = idx + idy * blockDim.x * gridDim.x;
```

```
  /* Clear shared memory */
  d_bin_data_shared[threadIdx.x] = 0;

  /* Fetch the data value as 32 bit */
  const unsigned int value_u32 = d_hist_data[tid];

  /* Wait for all threads to update shared memory */
  __syncthreads();

  atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0x000000FF) ) ]),1);
  atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0x0000FF00) >> 8 ) ]),1);
  atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0x00FF0000) >> 16 ) ]),1);
  atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0xFF000000) >> 24 ) ]),1);

  /* Wait for all threads to update shared memory */
  __syncthreads();

  /* The write the accumulated data back to global memory in blocks, not scattered */
  atomicAdd(&(d_bin_data[threadIdx.x]), d_bin_data_shared[threadIdx.x]);
}
```

内核必须针对共享内存做一次额外的清除操作，以免之前执行的内核随机残留一些数据。此外，在将下一个线程块中所有线程的数据更新到共享内存单元之前，必须等待（_Syncthreads）前一个线程块的所有线程完成共享内存单元的清除操作。最后，在将结果写回到全局内存之前，也需要设置一个同步操作，以保证所有的线程都完成了计算。

此时，通过将连续的写操作合并，我们程序的性能立刻就提升了6倍，处理速度达到6800MB/s。但请注意，必须在计算能力为1.2或者更高，并且支持共享内存原子操作的设备上执行这段代码才能获得如此高的处理速度。

将连续的写操作合并起来之后，我们需要考虑一下如何减少全局内存的阻塞。我们已经对读数据的大小进行了优化，每次从源数据中读出一个值，而且每个值只需要读一次，因此，我们只需要考虑减少对全局内存写操作的次数了。假设每个线程块处理的直方图不是一个而是N个，那么我们对全局内存的写操作的带宽就会减少N倍。

表5-4显示了在费米460显卡（包含7个SM）上，对数据总量为512M的统计直方图进行处理，当N值不同时，获得不同的处理速度。其中，当N为64时得到最高的处理速度7886MB/s。以下是内核代码：

```
/* Each read is 4 bytes, not one, 32 x 4 = 128 byte reads */
/* Accumulate into shared memory N times */
__global__ void myhistogram256Kernel_07(const unsigned int const * d_hist_data,
                                        unsigned int * const d_bin_data,
                                        unsigned int N)
{
  /* Work out our thread id */
  const unsigned int idx = (blockIdx.x * (blockDim.x*N) ) + threadIdx.x;
  const unsigned int idy = (blockIdx.y * blockDim.y ) + threadIdx.y;
  const unsigned int tid = idx + idy * (blockDim.x*N) * (gridDim.x);
```

```
    /* Clear shared memory */
    d_bin_data_shared[threadIdx.x] = 0;

    /* Wait for all threads to update shared memory */
    __syncthreads();

    for (unsigned int i=0, tid_offset=0; i< N; i++, tid_offset+=256)
    {
     const unsigned int value_u32 = d_hist_data[tid+tid_offset];

     atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0x000000FF) ) ]),1);
     atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0x0000FF00) >> 8 ) ]),1);
     atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0x00FF0000) >> 16 ) ]),1);
     atomicAdd(&(d_bin_data_shared[ ((value_u32 & 0xFF000000) >> 24 ) ]),1);
    }
    /* Wait for all threads to update shared memory */
    __syncthreads();

    /* The write the accumulated data back to global memory in blocks, not scattered */
    atomicAdd(&(d_bin_data[threadIdx.x]), d_bin_data_shared[threadIdx.x]);
}
```

表 5-4 统计直方图结果

每个线程块计算的直方图数目	MB/s	总线程块数	每个 SM 处理的线程块数	剩余线程块数
1	6766	524 288	74 898	3
2	7304	262 144	37 449	1
4	7614	131 072	18 724	6
8	7769	65 536	9362	3
16	7835	32 768	4681	1
32	7870	16 384	2340	6
64	7886	8192	1170	3
128	7884	4096	585	1
256	7868	2048	292	6
512	7809	1024	146	3
1024	7737	512	73	1
2048	7621	256	36	6
4096	7093	128	18	3
8192	6485	64	9	1
16 384	6435	32	4	6
32 768	5152	16	2	3
65 536	2756	8	1	1

现在，我们来看看这段代码究竟做了什么。首先，我们用一个循环变量 i 进行了 N 次循

环，每次循环我们都处理了在共享内存中的 256 个字节的直方图数据。每个线程块包含 256 个线程，每个线程计算一个 bin。而循环的次数即每个线程块处理的直方图的个数。每执行完一次循环，指向内存的指针向后移动 256 个字节（tid_offset += 256），以指向下一个处理的直方图。

由于自始至终我们都使用了原子操作，因此我们只需要在内核计算的开始与结尾处进行同步操作。不必要的同步会降低程序的性能，但同时也能让内存的访问变得更加整齐统一。

通过观察表，我们可以发现一个很有趣的现象。当每个线程块处理的直方图的数目为 32 或者更多的时候，我们看到吞吐量没有明显的增加。N 值每增加两倍，全局内存的带宽反而在降低。因此，如果全局内存带宽是问题的所在，那么处理速度应该随着 N 值的增长而线性增长。那么，究竟什么才是真正影响性能的因素呢？

其实，真正的主要因素还是原子操作。每个线程都要同其他线程一同对一块共享数据区进行竞争，又由于数据模式设计的并不好，因此，对执行中的时间有了很大的影响。

在之后的章节中，我们将会采用不使用原子操作的方式来实现该算法。

5.8 本章小结

本章中我们讲到了许多关于线程、线程块以及线程网格的知识。现在你应该熟悉了如何利用 CUDA 将任务分解到线程网格、线程块及线程上。此外，本章还涉及了硬件上的线程束的概念以及线程块的调度问题，以及时刻保证硬件上有足够数量的线程的需要。

了解 CUDA 的线程模型是 GPU 编程的基础。我们要清楚 GPU 编程与 CPU 编程的根本不同，同时也要清楚它们之间的相互联系。

在本章中，根据待处理数据来组织线程结构是非常重要的，它对程序性能的影响非常大。另外，尤其是当应用程序需要共享数据时，将一个简单的任务并行化处理并不是一个简单的工作。我们要时刻想寻找一个更好的方案，而不只是寻找一个适合的方案。

此外，本章中我们还谈到了关于原子操作的使用以及原子操作带来的序列化执行问题。另外还有分支结构带来的问题，要牢记保证所有线程遵循相同控制路径的重要性。本书后面的章节中还会对原子操作与分支进行详细的讨论。

在本章 CUDA 内核代码中用到一些 C 语言的扩展语法，本书后面章节中还会出现这样的语法，读者应学会在自己的 CUDA 程序中也采用这样的语法，并明确知道其具体含义。

读完这章相信你已经对 CUDA 有了深刻的了解与认识，希望你不会再对 CUDA 或并行编程感到神秘。

问题

1. 在本章所描述的直方图统计算法中，找出数据分布最好与最坏的模式。普通分布情况中有哪些也是有问题的？你将如何解决？

2. 如果不使用算法，在基于 G80 系列的硬件上进行测试，你认为影响性能的最主要的因素是什么？

3. 如果对一个在 CPU 内存上的数组进行处理，是采用行 - 列顺序执行的方式好还是采用列 - 行顺序执行的方式好？如果是在 GPU 上，会有什么不同吗？

4. 假设现在有一段代码分别使用不同数量的线程块进行执行，一种使用了 4 个线程块，每个线程块包含 256 个线程，另一种使用了 1 个线程块，这个线程块中包含 1024 个线程。请问哪个内核先执行完，为什么？每个线程块在代码中的不同位置调用 syncthreads() 进行了 4 次同步。线程块之间相互不影响。

5. GPU 上基于 SIMD 的实现模式与多核 CPU 上基于 MIMD 的实现模式各有什么优点与缺点？

答案

1. 当统计的数据均匀分布时是最好的情形。因为这种分布使得任务与数据平均分配，对共享内存存储片进行原子操作的次数也是非常平均的。

最坏的情况则是统计的数据全部是同一个值。这使得所有的线程都竞争同一块共享数据，原子操作与共享内存存储冲突使整个程序变成了串行的。

不幸的是，一种常见的用法是使用已排序的数据，这种情况非常糟糕，某几个共享数据可能会遇到连续大量的原子写操作，也使程序成为串行化。

一种解决办法是对数据集进行轮询访问，每次迭代时将数据写入一个新容器内。但这种方法需要我们了解数据的分布。例如，用 32 个容器对 256 个数据点进行一个线性函数的建模。假定数据点 0 ~ 31 进入第一个容器，之后每一个容器都依此类推。每个容器处理一个值，这样就可以将写操作分布到每个容器中，从而避免竞争。在这个例子中，读数据应按这种方式：0，32，64，96，1，33，65，97，2，34，68，98……

2. 由于 G80 设备（计算能力为 1.0 或 1.1）不支持基于共享内存的原子操作，因此代码无法编译通过。假设我们采用全局内存的原子操作对其进行修改，那么相比之前基于共享内存的原子操作方案，这种方案得到的性能将降低 7 倍。

3. 以行 - 列的顺序执行方式更好，因为 CPU 会使用一种“预取”的技术，它可以使后续待读取的数据被同时取到缓存中，最终整个缓存行中会装满从内存获取的数据。也就是说，CPU 在获取数据时会按行获取。

采用列 - 行的方式可能会慢一点。采用先列遍历的方式可能会慢很多。因为 CPU 从内存获取的一个缓存行数据不太可能被后面的循环迭代用到，除非每一行的数据并不多。在 GPU 上，每个线程会取出每一行的一个或多个数据元素。因此，从一个比较高的层面来讲，整个循环应该采用列顺序的方式来执行，因为每一行获取的数据会被其他的线程立即处理掉。同 CPU 的缓存行一样，在计算能力为 2.x 的硬件上也能通过预取的方式将数据取出到缓存行，但有一点不同的是，取出的数据会立即被线程消费掉。

4. 当执行 syncthreads() 进行同步操作时，整个线程块将被阻塞直到线程块内所有的线

程都到达了同步点。当到达同步点之后所有的线程才能再次被调度。让一个线程块拥有非常多的线程意味着当SM在等待单个线程块内的线程到达同步点时可能会出现没有线程束可供调度的情况。由于线程的执行流是未知的，部分线程可能很快地到达同步点，而有的则需要很久才能到达。产生这种现象，主要因为硬件层的设计是针对获取更高吞吐量而不是针对减少延迟。因此，只有当块与块之间不需要通过全局内存进行通信，块内的线程经常通信的时候，一个线程块开启很多线程才能充分发挥性能。

5. 由于每个执行单元的指令流都是相同的，SIMD模式将指令的获取时间均摊到每一个执行单元。但是，当指令流出现分支，指令就会被序列化。而MIMD模式的设计主要是为了处理不同指令流，当指令流出现分支，它不需要对线程进行阻塞。然而它需要更多指令存储以及译码单元，这就意味着硬件需要更多的硅，同时，为了维持多个单独的指令序列，它对指令带宽的需求也非常的高。

一般使用SIMD与MIMD的混合模式才是最好的方案。用MIMD的模式处理控制流，用SIMD的模式处理大数据。在CPU上使用SSE/MME/AVX指令扩展集时就是采用的SIMD与MIMD的混合模式，而在GPU上，当线程束与线程块以高粒度处理分支情况时也是采用的混合模式。

第 6 章

CUDA 内存处理

6.1 简介

在传统的 CPU 模型中，内存是线性内存或平面内存，单个 CPU 核可以无约束地访问任何地址的内存。在 CPU 的硬件实际实现中，有许多一级（L1）、二级（L2）以及三级（L3）缓存。那些善于对 CPU 代码进行优化以及有高性能计算（High-Performance Computing，HPC）背景的人，对这些缓存甚是熟悉。然而对大多数程序员而言，这些概念却显得非常的抽象。

抽象已经在现代程序语言中成为了一种趋势。它使程序员离底层硬件越来越远，以确保程序员不必过多了解底层硬件就可以编写程序。虽然抽象将问题提升到一个更高的层次，使得问题解决更加简单，程序的生产率也提高了一个等级，但它仍需要灵活的编译器将上层的抽象转换成底层硬件能够理解的形式。理论上，抽象是一个非常伟大的概念，但事实上却很难将其毫无瑕疵地实现。相信在未来的几十年里，我们可以看到编译器与程序语言得到巨大改善，使其能自动地利用并行硬件。但目前而言，要想程序在不同平台获得高性能，还必须了解硬件是如何工作的，这是问题的关键所在。

在一个基于 CPU 的系统中，如果要获取好的性能，就必须了解缓存是如何工作的。我们首先来看 CPU 端的缓存，然后将其与 GPU 端的缓存进行对比，看看有哪些相似之处。从指令流的角度来看，大多数程序都是以顺序方式执行，其中包含了各种各样的循环结构。如果程序调用了一个函数，很有可能该程序会很快再次调用这个函数，如果程序对某一块特殊的内存进行了访问，很有可能在很短的时间内程序会再次对这块内存进行访问。当对某块数据已经使用过一次后还可能再次使用，某个函数执行一次之后还可能再次执行，这就是时间局部性（temporal locality）原则。

从一个计算机系统中的主内存 DRAM 中获取数据会非常的慢。相对于处理器的时钟频率，DRAM 的存取速度一直都非常慢。随着处理器时钟频率的不断提高，DRAM 的存取速度就更跟不上处理器的时钟频率了。

目前处理器搭配的 DDR-3 DRAM 一般能达到 1.6GHz。当然，如果配备一些高速的模块以及更好的处理器，其最高能达到 2.6GHz。然而，CPU 的每个核一般都能达到 3.0GHz，如果没有缓存进行快速存取，那么对 CPU 而言，DRAM 的带宽非常不足。由于代码与数据都

存在DRAM中，如果无法快速地从DRAM获取程序与数据，CPU的指令带宽（一个时间帧内指令执行的数量）会明显受到限制。

这里有两个比较重要的概念，一个是存储带宽（memory bandwidth），即在一定时间内从DRAM读出或写入的数据量。另一个是延迟（latency），即响应一个获取内存的请求所花费的时间，通常这个时间会是上百个处理器周期。如果一个程序需要从内存获取四个元素，则将所有请求一起处理然后等待所有数据到达，总要好过处理一个请求然后等待数据到达，处理下一个后再等待。因此，如果没有缓存，处理器的存储带宽将会受到很大限制，延迟也会增大很多。

我们用超市的收银过程作为例子，来形象化地介绍存储带宽与延迟。假设一家超市有N个收银台，并不是每个收银台都有员工在工作。如果只有两个收银台正常工作，顾客将会在这两个收银台后面排起很长的队等待付款。每个收银台在一定时间内（例如，一分钟）处理的顾客的人数就是吞吐量或者带宽。而顾客在队列中需要等待的时间则为延迟，即顾客从进入队列直到付款离开所花费的时间。

由于队伍越来越长，店主可能会增加更多收银台进行工作，顾客就会分流到到新的收银台。如果新开两个收银台工作，收银台的带宽将增加一倍，因为在相同的时间内可以服务两倍于之前的人数。此外，延迟也会减半，因为平均每个队列只有原先的一半长，等待的时间也就减少了一半。

然而，这并不是免费的。更多的收银台意味着需要花更多的钱雇更多的收银员；超市更多的零售空间需要分配给收银台，相应地，货架空间就会减少。从内存总线带宽与内存设备时钟频率的角度来看，处理器的设计也出现了相同的权衡点。设备上硅片占用的空间是有限的，外部内存总线的宽度通常会受到处理器上物理引脚数目的限制。

另一个需要理解的概念为事务开销（transaction overhead）。当收银员为每名顾客处理付款操作时就有明显的开销。有些顾客的购物篮中只有两三件物品，而有些购物车则充满了物品。店主会更喜欢那些购满整个购物车的顾客，因为他们可以更加高效地处理，即收银员会花费更多的时间在清点货物上，而不是处理付款。

在GPU中，我们能看到类似的机理。有些内存事务，相较于处理它们的开销来说，是属于轻量级的。因此，内存单元获取的时间相对于开销而言也就较少，换言之，即达到最高效率的百分比很低。但有些事务比较庞大，需要花大量的时间才能完成，这些事务能高效地处理并能达到接近最高的存储传输速率。它们在一端转化成基于字节的内存事务，而在另一端转换成基于字的存储事务。为了获得最高的存储效率，GPU需要大量的庞大事务和尽可能少的轻量级事务。

6.2 高速缓存

高速缓存是硬件上非常接近处理器核的高速存储器组。高速缓存主要由硅制成，但由

于硅很贵，因此高速缓存的价格也很昂贵。此外，硅还可以用来制成更大的芯片以及低产却价格更贵的处理器。例如，许多服务器上的英特尔至强芯片（the Intel Xeon，常用于服务器）比普通台式机搭载的处理器贵得多，因为它搭载了巨大的三级缓存，而普通处理器搭载的缓存比较小。

高速缓存的最大速度与缓存的大小成反比关系。一级缓存是最快的，但它的大小一般限制在 16KB、32KB 或者 64KB。通常每个 CPU 核会分配一个单独的一级缓存。二级缓存相对而言慢一些，但是它更大，通常有 256KB ~ 512KB。三级缓存可能存在也可能不存在，如果存在，通常是几兆字节的大小。二级缓存或三级缓存一般在处理器的核之间是共享的，或者作为连接于特定处理器核的独立缓存来维护。在传统的 CPU 上，一般而言，至少三级缓存在处理器核之间是共享的。处理器核便可通过设备上这块共享内存快速地进行通信。

G80 与 GT200 系列 GPU 没有与 CPU 中高速缓存等价的存储器，但它们却有两块基于硬件托管的缓存，即常量内存与纹理内存，它们类似 CPU 中的只读缓存。与 CPU 不同，GPU 主要依赖基于程序员托管的缓存或共享内存区。

在费米架构的 GPU 实现中，第一次引入了不基于程序员托管的数据缓存这个概念。这个架构的 GPU 中每个 SM 有一个一级缓存，这个一级缓存既是基于程序员托管的又是基于硬件托管的。在所有的 SM 之间有一个共享的二级缓存。

缓存在处理器的核或 SM 之间共享有什么意义？为什么要这样安排？这主要是为了让设备之间能够通过相同的共享缓存进行通信。共享缓存允许处理器之间不需要每次都通过全局内存进行通信。这在进行原子操作的时候特别有用，由于二级缓存是统一的，所有的 SM 在给出的内存地址处获取一致版本的数据。处理器无须将数据写回缓慢的全局内存中，然后再读出来，它只需要保证处理器核之间的数据一致性。在 G80 与 GT200 系列的硬件设备上，并没有统一的高速缓存，因此与费米以及后续的设备相比，在它们上面做原子操作会非常慢。

对大多数程序而言，高速缓存非常有用。很多程序员既不关心也不了解如何在编写软件时获取更好的性能。缓存的引入使得大多数程序能够更加合理地运转，程序员也不需要过多地了解硬件是如何工作的。但这种简单的编程方式只适合最初的学习，多数情况下，我们需要更加深入地了解。

一名 CUDA 的初学者与一名 CUDA 的专家相比，差距可能非常大。希望读者通过阅读此书，能够迅速学会编写 CUDA 代码，用并行代码替换你原来的串行代码，使你的程序加速好几倍。

数据存储类型

GPU 提供了不同层次的若干区域供程序员存放数据，每块区域根据其能达到的最大带宽以及延迟而定义，如表 6-1 所示。

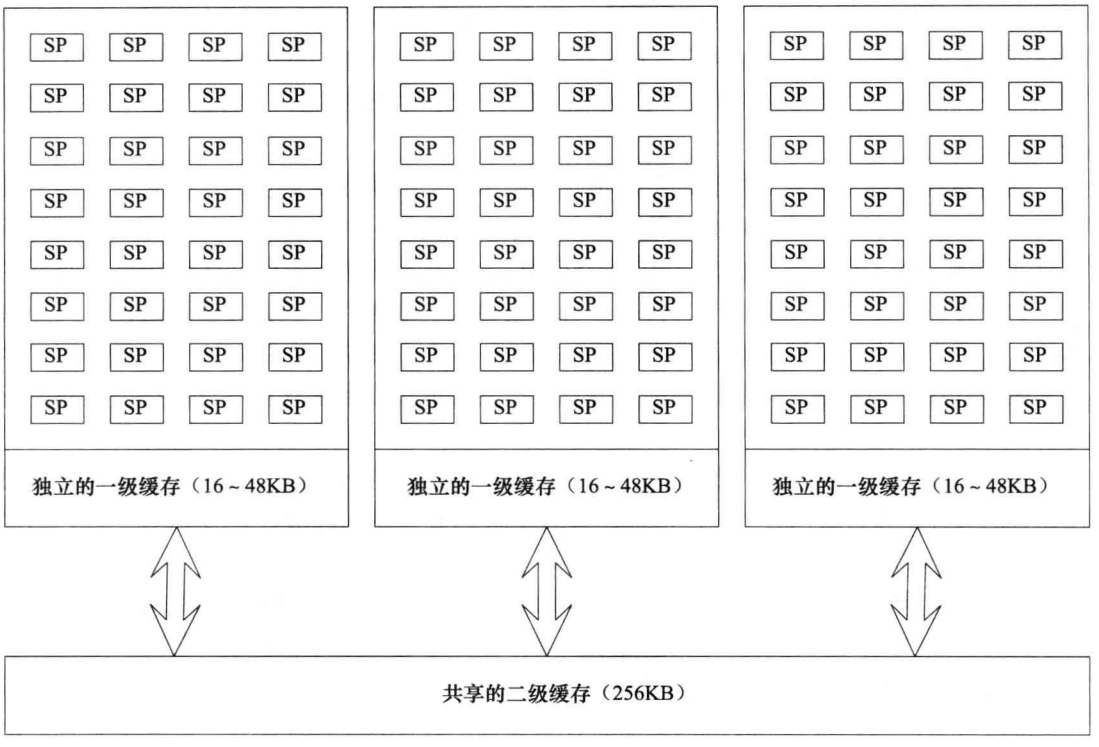


图 6-1 SM 一级缓存与二级缓存数据通路

表 6-1 不同存储类型的访问时间

存储类型	寄存器	共享内存	纹理内存	常量内存	全局内存
带宽	~8TB/s	~1.5TB/s	~200MB/s	~200MB/s	~200MB/s
延迟	1 个周期	1 ~ 32 个周期	400 ~ 600 个周期	400 ~ 600 个周期	400 ~ 600 个周期

最快速也最受偏爱的存储器是设备中的寄存器，接着是共享内存，如基于程序员托管的一级缓存，然后是常量内存、纹理内存、常规设备内存，最后则是主机端内存。注意不同存储器之间的存储速度的数量级的变化规律。本章将依次介绍这几种存储器的使用以及如何最大化地利用它们。

传统上，绝大多数书籍会从全局内存开始介绍，因为它在性能优化中扮演着关键角色。除非获取全局内存的模式正确，否则一旦模式错误，就没有必要再去谈优化了。而本书将采用一种不同的方式进行介绍，本章将首先介绍如何高效地使用最内部的设备，由内而外，最后介绍全局内存以及主机端内存，让读者能够理解不同层次的存储器的效率以及知道如何获得高效。

大多数 CUDA 程序都是逐渐成熟的。一开始使用全局内存初始化，初始化完毕之后再考虑使用其他类型的内存，例如，零复制内存（zero-copy memory）、共享内存、常量内存，最终寄存器也被考虑进来。为了优化一个程序，我们需要在开发过程中考虑这些问题。在程序之初就要考虑使用速度较快的存储器，并且准确知道在何处以及如何提高程序性能，

而不是在程序写完之后才想到用哪些快速的存储器对程序进行优化。另外，不仅要思考如何高效地访问全局内存，也要时刻想办法减少对全局内存的访问次数，尤其在数据会被重复利用的时候。

6.3 寄存器的用法

与 CPU 不同，GPU 的每个 SM（流多处理器）有上千个寄存器。一个 SM 可以看作是一个多线程的 CPU 核。一般 CPU 拥有二、四、六或八个核。但一个 GPU 却有 N 个 SM 核。在高端的费米 GF100 系列的顶级设备上有 16 个 SM，GT200 系列的设备上的 SM 数目多达 32 个，G80 系列的设备 SM 最多也有 16 个。

看起来很奇怪，费米架构设备上的 SM 数目居然比早期的一些设备还要少。这是因为费米架构设备上每个 SM 拥有更多的 SP（流处理器），而所有的工作其实都是 SP 处理的。由于每个核上 SP 数目不同，因此每个核支持的线程数目也有很大的不同。一般地，CPU 每个核会支持一到两个硬件线程。相比之下，GPU 的每个核可能有 8 ~ 192 个 SP，这意味着每个 SM 在任何时刻都能同时运行这些数目的硬件线程。

事实上，GPU 上的应用线程进入流水线、进行上下文切换并分配到多个 SM 中。这意味着在一台 GPU 设备的所有 SM 中活跃的线程数目通常数以万计。

CPU 与 GPU 架构的一个主要区别就是 CPU 与 GPU 映射寄存器的方式。CPU 通过使用寄存器重命名和栈来执行多线程。为了运行一个新任务，CPU 需要进行上下文切换，将当前所有寄存器的状态保存到栈（系统内存）上，然后从栈中恢复当前需要执行的新线程上次的执行状态。这些操作通常需要花费上百个 CPU 时钟周期。如果在 CPU 上开启过多的线程，时间几乎都将花费在上下文切换过程中寄存器内容的换进 / 换出操作上。因此，如果在 CPU 开启过多的线程，有效工作的吞吐量将会快速降低。

然而，GPU 却恰恰相反。GPU 利用多线程隐藏了内存获取与指令执行带来的延迟。因此，在 GPU 上开启过少的线程反而会因为等待内存事务使 GPU 处于闲置状态。此外，GPU 也不使用寄存器重命名的机制，而是致力于为每一个线程都分配真实的寄存器。因此，当需要上下文切换时，所需要的操作就是将指向当前寄存器组的选择器（或指针）更新，以指向下一个执行的线程束的寄存器组，因此几乎是零开销。

注意这里用到了线程束的概念。在第 5 章介绍线程的部分已经详细地介绍了这个概念。一个线程束即同时调度的一组线程。在当前的硬件中，一个线程束包含 32 个线程。因此，在一个 SM 中，每次换进 / 换出、调度都是 32 个线程同时执行。

每个 SM 能调度若干个线程块。在 SM 层，线程块即若干个独立线程束的逻辑组。编译时会计算出每个内核线程需要的寄存器数目。所有的线程块都具有相同的大小，并拥有已知数目的线程，每个线程块需要的寄存器数目也就是已知和固定的。因此，GPU 就能为在硬件上调度的线程块分配固定数目的寄存器组。

然而在线程层，这些细节对程序员是完全透明的。如果一个内核函数中的每个线程需要

的寄存器过多，在每个 SM 中 GPU 能够调度的线程块的数量就会受到限制，因此总的可以执行的线程数量也会受到限制。开启的线程数量过少会造成硬件无法被充分利用，性能急剧下降，但开启过多又意味着资源可能短缺，调度到 SM 上的线程块数量会减少。

这一点要特别注意，因为它可能引起应用程序的性能突然下降。如果一个应用程序先前使用了 4 个线程块，现在改用更多的寄存器，可能导致只有 3 个线程块可供调度，这样 GPU 的吞吐量会降低 1/4。这种类型的问题可以利用许多性能分析的工具来定位，本书的第 7 章将会介绍性能分析的内容。

由于所使用的硬件不同，每个 SM 可供所有线程使用的寄存器空间大小也不同，分别有 8KB、16KB、32KB 以及 64KB。牢记，每线程中每个变量会占用一个寄存器。因此，C 语言中的一个浮点型变量就会占用 N 个寄存器，其中 N 代表调度的线程数量。在费米架构的设备上，每个 SM 拥有 32KB 的寄存器空间。如果每个线程块有 256 个线程，则每个线程可使用 32（32 768/4/256）个寄存器。为了让费米架构的设备上的每个线程可使用的寄存器数目达到最大，即 64 个寄存器（G80 及 GT200 上最多 128 个），每个线程块上的线程数目需要减少一半，即 128。当线程块上的寄存器数目是允许的最大值时，每个 SM 会只处理一个线程块。同样，也可以使用四个线程块，每个线程块 32 个线程（一共 4 × 32=128 个线程），每个线程使用的寄存器数目也能达到最多。

如果能够最大化地利用寄存器，例如，使用寄存器对一个数组的某一块进行计算，会非常高效。由于这一组值通常是数据集中的 N 个元素，元素之间是相互独立的，因此可以在单个线程中实现指令级的并行（Instruction-Level Parallelism, ILP）。这是由硬件将许多独立的指令流水线化实现的。后面将有一个实际例子来具体介绍这种方案。

然而，大多数内核对寄存器的需求量都很低。如果将寄存器的需求量从 128 降到 64，则相同的 SM 上可再调度一个线程块。例如，如果需要 32 个寄存器，则可调度四个线程块。通过这样做，运行的线程总数可以得到提高。在费米型设备上，每个 SM 最多能运行 1536 个线程。一般情况下，占用率越高，程序就运行得越快。当线程级的并行（Thread-Level Parallelism，TLP）足以隐藏存储延迟时将会达到一个临界点，如果想继续提高程序的性能，要么考虑更大块的存储事务，要么引入 ILP，即单个线程处理数据集的多个元素。

但是，每个 SM 能够调度的线程束的数量也是有限制的。因此，将每个线程需要的寄存器数量从 32 降到 16，并不意味着 SM 能调度 8 个线程块。表 6-2 显示了当线程块中的线程数为 192 与 256 以及线程需要的寄存器数目不同时，每个 SM 能够调度的线程块的数目。

表 6-2 费米架构设备每个线程寄存器的可利用率

线程数	最多使用寄存器数目					
192	16	20	24	28	32	64
线程块调度数	8	8	7	6	5	2
线程数	**最多使用寄存器数目**					
256	16	20	24	28	32	64
线程块调度数	6	6	5	4	4	2

表 6-2 中的测试设备为费米架构的设备。至于开普勒架构的设备，只需简单地将表中的寄存器数目与线程块数目加倍即可。此处使用 192 与 256 个线程进行测试是因为它们能保证硬件的充分利用。注意当内核使用的寄存器的数目为 16 与 20 时，SM 上调度的线程块的数目并没有增加，这是因为分配到每个 SM 上的线程束的数量是有限制的。在这种情况下，可以不考虑是否影响每个 SM 运行的线程总数而直接增加寄存器的使用量。

如果想使用寄存器来避免其他更慢的内存类型的使用，需要注意有效地使用它们。例如，一个循环根据一些布尔变量依次设置某个值的每一位。高效的方法是将 32 个布尔值封装到一个 32 位的字中，然后解封装。可以写这样一个循环，每次根据新的布尔值修改内存中的内容，做移位操作，移至字中的正确位置，如下所示：

```
for (i=0; i<31; i++)
{
 packed_result |= (pack_array[i] << i);
}
```

从数组中读出第 i 个元素然后将其封装到一个整型数 packed_result 中。将布尔值向左移位必要的位数，然后与之前的结果做“按位或”操作。

如果变量 packed_result 存于内存中，则需要做 32 次读 / 写内存的操作。但如果将变量 packed_result 设置为局部变量，编译器会将其放入寄存器中，在寄存器中而不是在主内存中做操作，最后再将结果写回主内存中，因此可节省 31 次内存读 / 写的操作。

这时，再回过头来看表 6-1，你会发现每做一次全局内存的读 / 写操作需要花费上百个时钟周期。假定每做一个全局内存的读 / 写操作需要花费 500 个时钟周期。如果每次循环都需要从全局内存读取数据，然后按位或，最后写回全局内存，则一共需要 32 × 读操作时间 +32 × 写操作时间 =64 × 500 个时钟周期 =32 000 个时钟周期。而使用寄存器则可消除 31 次读操作以及 31 次写操作，用一个时钟周期代替之前的 500 个时钟周期，因此一共花费

（1× 读内存时间）+（1× 写内存时间）+（31× 读寄存器时间）+（1× 写寄存器时间）

即

（1×500）+（1×500）+（31×1）+（31×1）=1062 个时钟周期，相比于 32 000 个时钟周期

时钟周期明显减少很多。因此，在相同问题领域里执行较简单的操作能够将性能提高 31 倍。

在 sum、min、max 等普通的归约操作中也会看到类似的关系。所谓的归约操作，即利用函数将某个较大的数据集减少为较小的集合，通常减少到一个单项。例如，max（10，12，1，4，5）会返回数据集中最大的那个值 12。

将结果累积在寄存器中可省去大量的内存写操作。在位包装的这个例子中，写内存的操作次数减少了 31 倍。无论是用 CPU 还是 GPU，这种寄存器的优化方式都会使程序执行速度得到很大提高。

然而，这种优化方式需要程序员思考哪些参数是存于寄存器中，哪些参数是存于内存中，哪些参数需要从寄存器中复制回内存，等等。通常，这对一般的程序员而言显得有些困

难。因此，许多程序员的代码都是直接运行在内存上。绝大多数CPU上的缓存可以通过将累计计算的值保存在一级缓存，从而有效掩饰这类问题。将数据写回缓存，避免将数据写回主内存，程序的性能并不会太差。但需要注意的是一级缓存仍然比寄存器慢，因此这种方案并不是最优化的，它可能比最优方案慢好几倍。

有些编译器可以自动检测出一些不高效的方式，并在优化阶段将数据载入寄存器计算，但有些却不能。这主要由编译器的好坏决定。好的编译器可以利用优化器来弥补编程能力带来的性能不足。如果优化层级提高，程序中的错误也会蔓延滋生。但这并不是编译器的错。C语言的定义比较复杂。由于优化层级的提高，一个诸如缺少volatile限定符的操作就可能引入一些敏感的错误。对于这些错误，自动测试脚本以及连续地对非优化的版本进行测试可以有效地保证程序的正确性。

另外，优化编译器的供应商并不是每次都采纳最好的解决方案。如果编译器的供应商在测试一个优化解决方案时只有1%的程序无法通过，那么这个方案很可能会因为所产生的支持的问题而不被采纳。

GPU的计算能力超过内存带宽容量的好几倍。在费米型架构的硬件上，存储带宽最高能达到190GB/s，而计算能力最快能达到每秒执行超过一万亿次浮点运算，这比五倍的内存带宽还要多。在开普勒GTX680及Tesla K10上，计算能力提高到了3万亿次浮点运算，而内存带宽与GTX580几乎相同。在比特包装这个例子中，如果不用寄存器进行优化，而改用一个没有缓存的设备，每次循环迭代都需要进行一次读操作和一次写操作。每个整型数或浮点数的长度为4个字节，一次读写一共是8个字节，理论上，这个例子能达到的最好性能是1/8的内存带宽。如果内存带宽是190GB/s，则相当于每秒做了250亿次操作。

但实际情况中是不可能达到这么高的性能，因为原始的存储带宽也要考虑循环索引以及迭代。然而，这种快速而无须精细的计算方式可以让我们在编程之前估计出应用程序性能的上限。

理论上，采用这种减少31次内存操作的方式可以使程序的性能提高31倍，每秒执行7750亿次迭代操作。但在实际设备中执行时却会受到限制。尽管这样，相比于使用全局内存，通过寄存器的累加尽可能使用寄存器还是能使程序的性能得到很大的改善。

此处，我们编写了一个程序，分别用全局内存和寄存器进行位包装，从而获得一些真实的数据。以下是测试的结果。

```
ID:0 GeForce GTX 470:Reg. version faster by: 2.22ms (Reg=0.26ms, GMEM=2.48ms)
ID:1 GeForce 9800 GT:Reg. version faster by: 52.87ms (Reg=9.27ms, GMEM=62.14ms)
ID:2 GeForce GTX 260:Reg. version faster by: 5.00ms (Reg=0.62ms, GMEM=5.63ms)
ID:3 GeForce GTX 460:Reg. version faster by: 1.56ms (Reg=0.34ms, GMEM=1.90ms)
```

以下是两个内核函数的代码：

```
__global__ void test_gpu_register(u32 * const data, const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
```

```
 {
  u32 d_tmp = 0;

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d_tmp |= (packed_array[i] << i);
  }

  data[tid] = d_tmp;
 }
}

__device__ static u32 d_tmp = 0;
__global__ void test_gpu_gmem(u32 * const data, const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d_tmp |= (packed_array[i] << i);
  }

  data[tid] = d_tmp;
 }
}
```

这两个内核函数的唯一区别是一个使用全局变量 d_tmp，另一个使用本地寄存器。表 6-3 显示了加速的结果。由表可知，平均加速 7.7 倍。而最让人意外的是，最快的加速比出自 SM 数量最多的那个设备。这说明了一个问题，不知读者是否注意到。在使用全局内存的内核函数中，线程块中的每个线程读 / 写到 d_tmp 中，并没有保证按怎样的顺序执行，因此输出是不确定的。内核函数正常执行，没有检测到任何 CUDA 错误，但是程序的结果通常可能是没有意义的。这种类型的错误在将串行代码转换成并行代码时是非常常见的一种典型错误。

表 6-3 使用寄存器与使用全局内存的加速比

显　卡	寄存器版本	GMEM（全局内存）版本	加　速　比
GTX470	0.26	2.48	9.5
9800GT	9.27	62.14	6.7
GTX260	0.62	5.63	9.1
GTX460	0.34	1.9	5.6
平均			7.7

奇怪的答案告诉人们有些地方可能出错了。但该如何修改才正确呢？在寄存器版本的内核函数中，每个线程将结果写回唯一对应的寄存器。在 GMEM（全局内存）版本的内核函数中，也应做相同操作。因此，只需将原先 d_tmp 的定义

```
d_tmp:
__device__ static u32 d_tmp = 0;
```

修改为

```
__device__ static u32 d_tmp[NUM_ELEM];
```

内核函数应更新为以下代码：

```
__global__ void test_gpu_register(u32 * const data, const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  u32 d_tmp = 0;

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d_tmp |= (packed_array[i] << i);
  }

  data[tid] = d_tmp;
 }
}
```

现在每个线程都是对独立的一块内存区域进行读 / 写。现在加速比又是多少呢？参见表 6-4。

表 6-4 使用寄存器与使用全局内存的实际加速比

显 卡	寄存器版本	GMEM 版本	加 速 比
GTX470	0.26	0.51	2
9800GT	9.27	10.31	1.1
GTX260	0.62	1.1	1.8
GTX460	0.34	0.62	1.8
平均			1.7

由表 6-4 可知，平均加速比降低到 1.7 倍。如果除去 9800GT（计算能力为 1.1 的设备），这段简单代码的平均加速比接近两倍。因此，应尽量用其他方式避免全局内存的写操作。如果操作聚合到同一块内存上，如第一个例子，就会强制使硬件对内存的操作序列化，从而造成严重的性能降低。

现在，让一段代码运行得更快变得非常简单。循环一般非常低效，因为它们会产生分支，造成流水线停滞。更重要的是，它们消耗指令但并不是为了得出最终结果。循环的代码包含了循环计数器的增加、检测是否到了循环的终止状态以及每次迭代的一个分支判断。相比之下，每次迭代有用的指令是将值从 packed_array 中载入，向左移位 N 位，然后与当前的 d_tmp 做“或”操作。所有的操作中，循环的操作占了 50%。我们可以通过查看 PTX（Parallel Thread eXecution，并行线程执行）代码证实这一点。为了方便读者阅读这段虚拟汇编代码，高亮的代码即执行循环的代码。

```
 .entry _Z18test_gpu_register1Pjj (
  .param .u64 __cudaparm__Z18test_gpu_register1Pjj_data,
  .param .u32 __cudaparm__Z18test_gpu_register1Pjj_num_elements)
{
 .reg .u32 %r<27>;
 .reg .u64 %rd<9>;
 .reg .pred %p<5>;
 // __cuda_local_var_108903_15_non_const_tid = 0
 // __cuda_local_var_108906_13_non_const_d_tmp = 4
 // i = 8
 .loc 16 36 0
$LDWbegin__Z18test_gpu_register1Pjj:
$LDWbeginblock_180_1:
 .loc 16 38 0
 mov.u32  %r1, %tid.x;
 mov.u32  %r2, %ctaid.x;
 mov.u32  %r3, %ntid.x;
 mul.lo.u32  %r4, %r2, %r3;
 add.u32  %r5, %r1, %r4;
 mov.s32  %r6, %r5;
 .loc 16 39 0
 ld.param.u32  %r7, [__cudaparm__Z18test_gpu_register1Pjj_num_elements];
 mov.s32  %r8, %r6;
 setp.le.u32  %p1, %r7, %r8;
 @%p1 bra  $L_0_3074;
$LDWbeginblock_180_3:
 .loc 16 41 0
 mov.u32  %r9, 0;
 mov.s32  %r10, %r9;
$LDWbeginblock_180_5:
 .loc 16 43 0
 mov.s32  %r11, 0;
 mov.s32  %r12, %r11;
 mov.s32  %r13, %r12;
 mov.u32  %r14, 31;
 setp.gt.s32  %p2, %r13, %r14;
 @%p2 bra  $L_0_3586;
$L_0_3330:
 .loc 16 45 0
 mov.s32  %r15, %r12;
 cvt.s64.s32  %rd1, %r15;
 cvta.global.u64  %rd2, packed_array;
 add.u64  %rd3, %rd1, %rd2;
 ld.s8  %r16, [%rd3+0];
 mov.s32  %r17, %r12;
 shl.b32  %r18, %r16, %r17;
 mov.s32  %r19, %r10;
 or.b32  %r20, %r18, %r19;
 mov.s32  %r10, %r20;
 .loc 16 43 0
 mov.s32  %r21, %r12;
```

```
 add.s32  %r22, %r21, 1;
 mov.s32  %r12, %r22;
$Lt_0_1794:
 mov.s32  %r23, %r12;
 mov.u32  %r24, 31;
 setp.le.s32  %p3, %r23, %r24;
 @%p3 bra  $L_0_3330;
$L_0_3586:
$LDWendblock_180_5:
 .loc 16 48 0
 mov.s32  %r25, %r10;
 ld.param.u64  %rd4, [__cudaparm__Z18test_gpu_register1Pjj_data];
 cvt.u64.u32  %rd5, %r6;
 mul.wide.u32  %rd6, %r6, 4;
 add.u64  %rd7, %rd4, %rd6;
 st.global.u32  [%rd7+0], %r25;
$LDWendblock_180_3:
$L_0_3074:
$LDWendblock_180_1:
 .loc 16 50 0
 exit;
$LDWend__Z18test_gpu_register1Pjj:
 }
```

这段PTX代码首先判断了for循环是否真正进入了循环。这一部分是在标注$LDWbeginblock_180_5的代码块中执行的。标注$Lt_0_1794的代码块执行循环操作，当执行完32次迭代操作时跳转到标注$L_0_3330的代码块。标注$L_0_3330代码块执行的是下面这段代码：

```
d_tmp |= (packed_array[i] << i);
```

注意，除了循环的开销之外，packed_array用一个变量进行索引，因此每次循环都需要计算内存地址：

```
cvt.s64.s32  %rd1, %r15;
cvta.global.u64  %rd2, packed_array;
add.u64  %rd3, %rd1, %rd2;
```

这非常的低效。相比于将循环展开，我们看到了一些更加有趣的东西：

```
.entry _Z18test_gpu_register2Pjj (
 .param .u64 __cudaparm__Z18test_gpu_register2Pjj_data,
 .param .u32 __cudaparm__Z18test_gpu_register2Pjj_num_elements)
{
.reg .u32 %r<104>;
.reg .u64 %rd<6>;
 .reg .pred %p<3>;
 // __cuda_local_var_108919_15_non_const_tid = 0
 .loc 16 52 0
$LDWbegin__Z18test_gpu_register2Pjj:
$LDWbeginblock_181_1:
```

```
 .loc 16 54 0
 mov.u32  %r1, %tid.x;
 mov.u32  %r2, %ctaid.x;
 mov.u32  %r3, %ntid.x;
 mul.lo.u32  %r4, %r2, %r3;
 add.u32  %r5, %r1, %r4;
 mov.s32  %r6, %r5;
 .loc 16 55 0
 ld.param.u32  %r7, [__cudaparm__Z18test_gpu_register2Pjj_num_elements];
 mov.s32  %r8, %r6;
 setp.le.u32  %p1, %r7, %r8;
 @%p1 bra  $L_1_1282;
 .loc 16 57 0
 ld.global.s8  %r9, [packed_array+0];
 ld.global.s8  %r10, [packed_array+1];
 shl.b32  %r11, %r10, 1;
 or.b32  %r12, %r9, %r11;
 ld.global.s8  %r13, [packed_array+2];
 shl.b32  %r14, %r13, 2;
 or.b32  %r15, %r12, %r14;

[Repeated code for pack_array+3 to packed_array+29 removed for clarity]

 ld.global.s8  %r97, [packed_array+30];
 shl.b32  %r98, %r97, 30;
 or.b32  %r99, %r96, %r98;
 ld.global.s8  %r100, [packed_array+31];
 shl.b32  %r101, %r100, 31;
 or.b32  %r102, %r99, %r101;
 ld.param.u64  %rd1, [__cudaparm__Z18test_gpu_register2Pjj_data];
 cvt.u64.u32  %rd2, %r6;
 mul.wide.u32  %rd3, %r6, 4;
 add.u64  %rd4, %rd1, %rd3;
 st.global.u32  [%rd4+0], %r102;
$L_1_1282:
$LDWendblock_181_1:
 .loc 16 90 0
 exit;
$LDWend__Z18test_gpu_register2Pjj:
 }
```

现在，几乎所有的指令都是为了计算得到结果，循环的开销没有了，无用指令减少了很多。packed_array 的地址计算也精简到一个编译时就分配的基地址加上偏移地址。所有计算都变简单了，但同时也变长了，无论是 C 代码还是虚拟 PTX 汇编代码。

这里，我们无须花费过多的时间来理解 PTX 代码，只需要知道 C 代码的一点小小的改动就会为虚拟汇编代码的生成代码带来很大的改变。了解诸如将循环展开的这种技巧会对今后编程有很大的帮助。在第 9 章中，本书将详细介绍 PTX 以及如何将其转换成机器能执行的实际代码。

表 6-5 显示了采用这种方式之后得到的加速比。由表可知，该方法在 9800GT 或 GTX260 上运行时，没有产生任何效果，但是，在计算能力为 2.x 以上的硬件上运行时，例如 GTX460 与 GTX470，加速比分别为 2.4 与 3.4。与纯 GMEM 实现方案相比，GTX470 执行得到的加速比为 6.4。直观点说，如果原程序需要花 6.5 小时来运行，优化之后只需要一个小时。

表 6-5 循环展开的影响

显　卡	寄存器版本	循环展开版本	加　速　比
GTX470	0.27	0.08	3.4
9800GT	9.28	9.27	1
GTX260	0.62	0.62	1
GTX460	0.34	0.14	2.4
平均			2

基于寄存器的优化能够为代码的执行时间带来巨大影响。读者可以利用节省的这段时间认真理解程序的内循环 PTX 代码是如何生成的，学会将循环展开成单个的或一组表达式。如果学会这样去思考程序代码，程序的性能将会有质的飞跃。使用寄存器可以有效消除内存访问，或提供额外的 ILP，以此实现 GPU 内核函数的加速，这是最为有效的方法之一。

6.4 共享内存

共享内存实际上是可受用户控制的一级缓存。每个 SM 中的一级缓存与共享内存共享一个 64KB 的内存段。在开普勒架构的设备中，根据应用程序的需要，每个线程块可以配置为 16KB 的一级缓存或共享内存。而在费米架构的设备中，可以根据喜好选择 16KB 或 48KB 的一级缓存或共享内存。早期的费米架构的硬件（计算能力为 1.x）中只有固定的 16KB 共享内存而没有一级缓存。共享内存的延迟极低，大约有 1.5TB/s 的带宽，远远高于全局内存的 190GB/s，但是它的速度只有寄存器的 1/10。

在实际情况中，低端显卡全局内存的速度只有高端显卡的 1/10，但是共享内存的速度几乎在所有（大约有 20% 不同）的 GPU 中都一致，因为共享内存的速度受核时钟频率驱动。因此，在任何显卡中，无论是否为高端显卡，除了使用寄存器，还要更有效地使用共享内存。

事实上，仅看带宽数据，共享内存的带宽为 1.5TB/s，全局内存的带宽最高为 190GB/s，比率为 7∶1。换言之，有效使用共享内存有可能获得 7 倍的加速比。毫无疑问，共享内存是所有关心性能的 CUDA 程序员应该认真掌握的一个概念。

然而，GPU 执行的是一种内存的加载 / 存储模型（load-store model），即所有的操作都要在指令载入寄存器之后才能执行。因此，加载数据到共享内存与加载数据到寄存器中不同，只有当数据重复利用、全局内存合并，或线程之间有共享数据时使用共享内存才更合适。否

则，将数据直接从全局内存加载到寄存器性能会更好。

共享内存是基于存储体切换的架构（bank-switched architecture）。在费米架构的设备上有 32 个存储体，而在 G200 与 G80 的硬件上只有 16 个存储体。每个存储体可以存 4 个字节大小的数据，足以用来存储一个单精度的浮点型数据，或者一个标准的 32 位的整型数。开普勒架构的设备还引入了 64 位宽的存储体，使双精度的数据无须再跨越两个存储体。无论有多少线程发起操作，每个存储体每个周期只执行一次操作。因此，如果线程束中的每个线程访问一个存储体，那么所有线程的操作都可以在一个周期内同时执行。此时无须顺序地访问，因为每个线程访问的存储体在共享内存中都是独立的，互不影响。实际上，在每个存储体与线程之间有一个交叉开关将它们连接，这在字的交换中很有用，例如即将介绍的一个排序算法。

此外，当线程束中的所有线程同时访问相同地址的存储体时，使用共享内存会有很大帮助。同常量内存一样，当所有线程访问同一地址的存储单元时，会触发一个广播机制到线程束的每个线程中。通常 0 号线程会写一个值然后与线程束中的其他线程进行通信，如图 6-2 所示。

然而，如果有其他的方式，存储体冲突将不同程度地得到解决。这意味着，线程访问共享内存需要排队等候，当一个线程访问共享内存时，线程束中的其他线程将阻塞闲置。这方面一个很重要的问题就是延迟并没有因为切换到另一个线程束而得到隐藏，事实上我们将整个 SM 都阻塞了。由于 SM 会处于闲置状态直到存储体请求被满足，因此，存储体冲突应尽可能地避免。

然而，这并不是很实用。例如，第 5 章中的统计直方图的例子。由于数据是未知的，因此存储体的划分是完全依赖于数据形式的。

最糟糕的情形就是所有的线程都对同一个存储体执行写操作，这将导致对同一存储体顺序进行 32 次访问操作。典型地，当跨幅超过 32 时，就会发生多个线程访问同一存储体。当跨幅以二的幂次倍减少时（如并行归约），也会发生这种情况，连续的轮询将导致更多的存储体冲突。

6.4.1 使用共享内存排序

此处，我们将介绍一个使用排序的实例。排序算法会对一个随机的数据集进行处理，生成一个有序的数据集。因此，我们需要 N 个输入数据项和 N 个输出数据项。排序的关键是保证最小化读 / 写内存的次数。很多排序算法是多通道的，这意味着我们将对 N 个元素中的每个元素读 M 次，显然这是不够好的。

快速排序是串行编程中一个比较受青睐的排序算法。作为一个分治算法，快速排序看起来也可以用并行实现。然而，快速排序默认用到递归调用，而在 CUDA 中，只有计算能力为 2.x 的设备才支持递归。一般地并行实现是通过为每个分离的数据产生一个新的线程。当前的 CUDA 模型（另见第 12 章中对开普勒动态并行性的讨论）需要在启动内核时对总的线程数进行说明，或者每层进行一连串的内核调用。数据会导致产生分支，这对 GPU 而言是不利的。

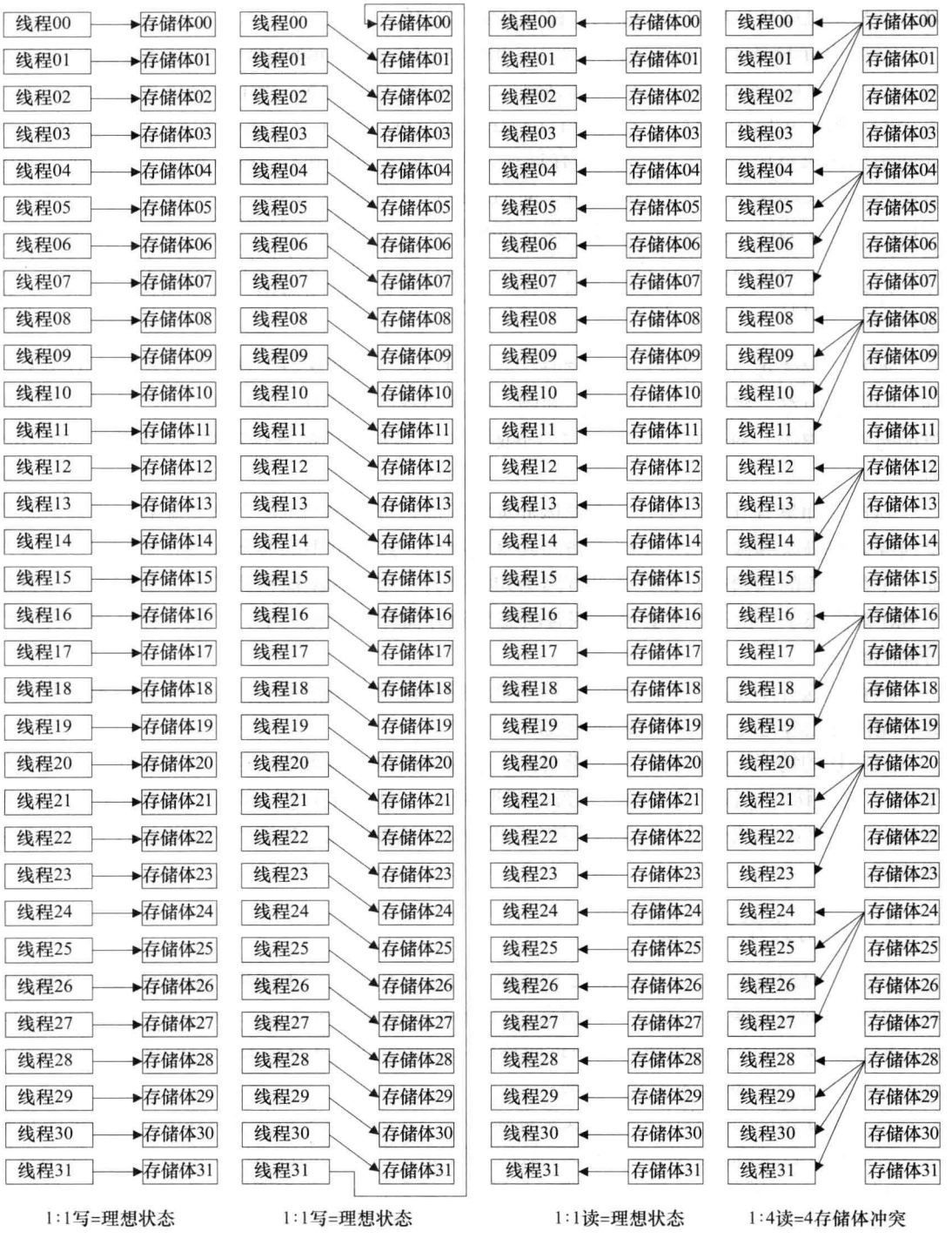

线程00 存储体00
线程01 存储体01
线程02 存储体02
线程03 存储体03
线程04 存储体04
线程05 存储体05
线程06 存储体06
线程07 存储体07
线程08 存储体08
线程09 存储体09
线程10 存储体10
线程11 存储体11
线程12 存储体12
线程13 存储体13
线程14 存储体14
线程15 存储体15
线程16 存储体16
线程17 存储体17
线程18 存储体18
线程19 存储体19
线程20 存储体20
线程21 存储体21
线程22 存储体22
线程23 存储体23
线程24 存储体24
线程25 存储体25
线程26 存储体26
线程27 存储体27
线程28 存储体28
线程29 存储体29
线程30 存储体30
线程31 存储体31
1:1写=理想状态
1:1写=理想状态
1:1读=理想状态
1:4读=4存储体冲突

图 6-2 共享内存模式

有很多方法可以解决这个问题，但是，这些问题使得快速排序在先于开普勒 GK110 或 Tesla K20 之前的设备上并不是最优的算法。事实上，你会发现很多最优的串行算法转化成并行算法之后并不是最优的并行算法，因此，我们最好重新挖掘该如何做才是最优。

归并排序是并行世界中一个比较常见的算法（如图 6-3 所示）。它递归地将数据划分成越来越小的数据包，直到只剩两个数值需要排序。然后再将排好序的列表合并起来，以产生一个整体的、排好序的列表。

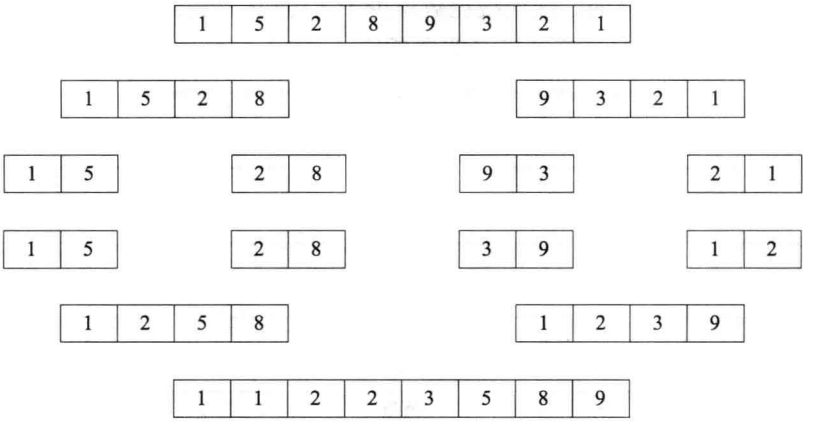


图 6-3 简单的归并排序例子

递归在早于计算能力为 2.x 的设备上并不支持，因此，这样的算法该如何执行呢？任何递归算法在某一时刻都有一个大小为 N 的数据集。在 GPU 上，线程块的大小或线程束的大小可以理想地映射为数据集的大小。因此，要实现一个递归算法，只需要将数据划分成大小为 32 的块，或者采用最小 *N* 值作为更大的块。

如果对一个数据集进行归并排序，例如 {1，5，2，8，9，3，2，1}，我们从第 4 个元素开始进行划分，得到两个更小的数据集，{1，5，2，8} 和 {9，3，2，1}，然后使用两个线程对这两个数据集分别执行一个排序算法。并行执行路径的数目即刻从 1 变化为 2 了。

将之前划分得到的两个数据集再次划分可得到四个数据集，{1，5}、{2，8}、{9，3} 以及 {2，1}。此时，再用 4 个线程进行处理显得并不是很重要，因为每个数据集只对两个数据进行比较并在需要时进行数据交换。最终，我们可以得到 4 个排好序的数据集：{1，5}、{2，8}、{3，9} 以及 {1，2}。到此，排序阶段就结束了。在此阶段，最大并行度可达到 N/2 个独立的线程。因此，当处理一个大小为 512KB 的数据集时，共 128K 个 32 位的元素，我们最多可使用 64K 个线程（N=128K，N/2=64K）。由于 GTX580 的 GPU 上有 16 个 SM，每个 SM 最多支持 1536 个线程，因此，在每个 GPU 上最多可支持 24K 个线程。按照如此划分，64K 的数据对只需 2.5 次迭代即可完成排序操作。

现在，我们将进入归并排序中一个比较经典的阶段，合并阶段。在此阶段中，我们将从每个列表中选出最小的元素然后将它加入到整个输出列表中。对每个列表重复执行该操作，直至输入列表中的所有元素都被添加入输出列表中。在刚刚的例子中，排好序的列表

为 {1，5}、{2，8}、{3，9} 以及 {1，2}。传统的归并排序中，首先是合并成 {1，2，5，8} 和 {1，2，3，9}，然后再将这两个列表按之前的方式合并成最终的、排好序的列表，{1，1，2，2，3，5，8，9}。

因此，每完成一次合并，并行度将减半。另外还有一种方案，即当 N 很小时，我们可以直接划分成 N 个列表，然后对每个值进行扫描并将其加入到输出列表的正确位置中，跳过图 6-4 中显示的合并阶段。如图 6-4 所示，主要问题在于排序过程中对应图中高亮区域的消除操作是由两个线程完成的。任何线程数量少于 32 意味着我们只需要用到少于一个线程束的线程数量，这对于 GPU 而言效率是非常低的。

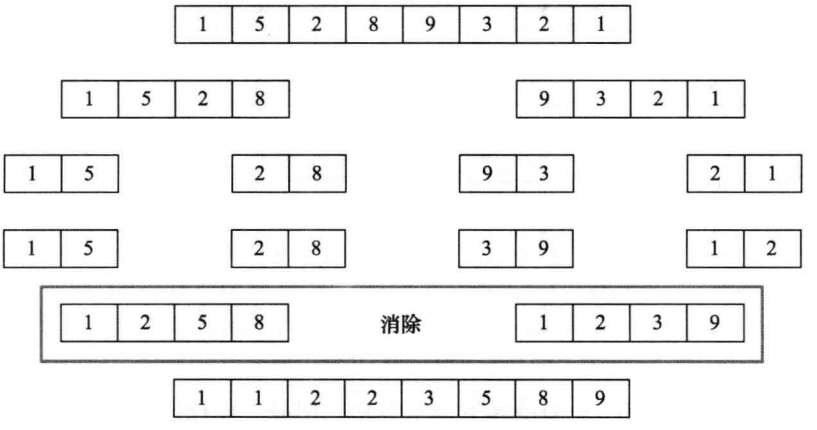


图 6-4 同时合并 N 个列表

这个方法的缺点就是我们需要从每个排好序的列表集中读出第一个元素。对于一个 64K 的集合，需要 64K 次读操作，即从内存中获取 256MB 的数据。显然，当数据集很大时这个方案并不合适。

因此，我们的方法就是通过限制对原始问题迭代的次数以获得一个更好的合并解决方案，当需要的线程数量减少为一个线程束的大小 32 时，停止迭代，而不是像传统归并排序中当每个集合只剩两个元素时才停止。对之前的例子实施这种方案，则集合数目将从原先的 64K 减少到 4K。同样，当从 N/2 到 N/32，最大并行数量增加了。在先前的 128K 个元素的例子中，这意味着需要处理 4K 个元素。在 GTX580 上，这将为每个 SM 分配 256 个处理元素（线程束）。由于费米架构设备上的每个 SM 最多能执行 48 个线程束，多个块将被循环访问，这使得问题能以更小的规模得到处理，而且在未来的硬件上能得到更好的加速。如图 6-5 所示。

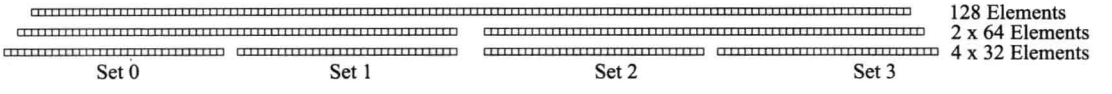


图 6-5 基于共享内存的分解

通过将数据集以每行 32 个元素的方式在共享内存中进行分布，然后线程以列的方式进行访问，我们可以获得零存储体冲突的内存访问（如图 6-6 所示）。

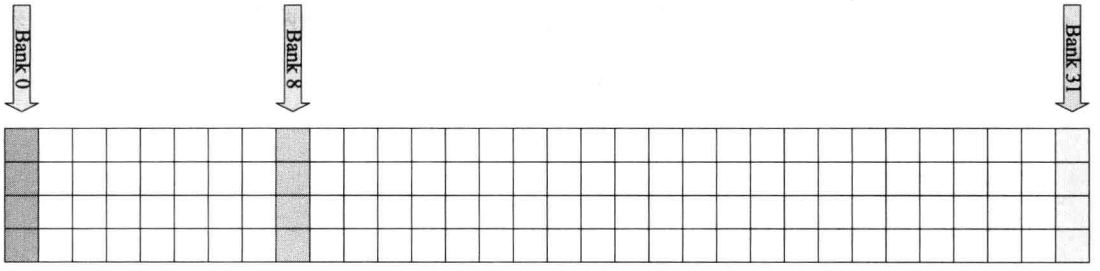


图 6-6 共享内存存储体访问

为了将对全局内存的访问合并起来，我们需要以每行 32 个元素的方式从全局内存中获取数据，这一点在下一节中我们具体介绍。然后对每一列实施排序算法，而无须担心共享内存冲突。唯一需要关心的就是分支。我们需要确保每个线程都遵循相同的执行流，尽管每个线程处理不同的数据。

这种策略的一个副作用就是我们需要做一个折中。假设每个 SM 只处理一个线程束，则不会出现共享内存体冲突。然而，每个 SM 处理一个线程束将无法隐藏全局内存的读 / 写延迟。至少在内存获取和写回的阶段，我们需要很多线程。但是，在排序阶段，多个线程束可能与其他线程束产生冲突。单个线程束不会存在任何存储体冲突，但又无法隐藏指令执行的延迟。因此，在实际中，在整个排序的阶段中我们需要多个线程束。

6.4.2 基数排序

基数排序是一个具有固定迭代次数和一致执行流的算法。它通过从最低有效位到最高有效位一一进行比较，对数值排序。对于一个 32 位的整型数，使用一个基数位，无论数据集有多大，整个排序需要迭代 32 次。我们将以下面这个数据集介绍我们的例子。

```
{ 122, 10, 2, 1, 2, 22, 12, 9 }
```

它们的二进制表示分别为

```
122 = 01111010
 10 = 00001010
  2 = 00000010
 22 = 00010010
 12 = 00001100
  9 = 00001001
```

在第一轮比较中，最低有效位（最右边）为 0 的元素形成一个列表，最低有效位为 1 的元素形成第二个列表，得到如下结果

```
0 = { 122, 10, 2, 22, 12 }
1 = { 9 }
```

然后将第二个列表附加到第一个列表中，得到

```
{ 122, 10, 2, 22, 12, 9 }
```

然后重复以上操作，对每个比特位都进行比较，下一轮生成的两个列表都是根据上一轮的结果进行排序的：

```
0 = { 12, 9 }
1 = { 122, 10, 2, 22 }
```

合并之后得到

```
{ 12, 9, 122, 10, 2, 22 }
```

扫描第三个比特位，得到

```
0 = { 9, 122, 10, 2, 22 }
1 = { 12 }
= { 9, 122, 10, 2, 22, 12 }
```

接着程序重复执行上述操作，直至32位全部进行完比较。为了建立列表，我们需要$N+2N$个内存单元，一个用来存放源数据，一个用来存放比特位为0的列表，一个用来存放比特位为1的列表。事实上，我们并不一定严格地需要$2N$个额外的内存单元，因为我们可以将比特位为0的数从列表头开始存放，比特位为1的数从列表尾部开始存放。然而，为了简便起见，我们将使用两个单独的列表。

基数排序的串行代码如下所示：

```
__host__ void cpu_sort(u32 * const data,
                       const u32 num_elements)
{
 static u32 cpu_tmp_0[NUM_ELEM];
 static u32 cpu_tmp_1[NUM_ELEM];

 for (u32 bit=0;bit<32;bit++)
 {
  u32 base_cnt_0 = 0;
  u32 base_cnt_1 = 0;

  for (u32 i=0; i<num_elements; i++)
  {
   const u32 d = data[i];
   const u32 bit_mask = (1 << bit);

   if ( (d & bit_mask) > 0 )
   {
    cpu_tmp_1[base_cnt_1] = d;
    base_cnt_1++;
   }
   else
   {
    cpu_tmp_0[base_cnt_0] = d;
```

```
      base_cnt_0++;
     }
    }

    // Copy data back to source - first the zero list
    for (u32 i=0; i<base_cnt_0; i++)
    {
     data[i] = cpu_tmp_0[i];
    }

    // Copy data back to source - then the one list
    for (u32 i=0; i<base_cnt_1; i++)
    {
     data[base_cnt_0+i] = cpu_tmp_1[i];
    }
   }
  }
```

这段代码传入了两个参数，一个是指向需要排序的数据的指针，另一个是数据集的元素数量。通过对未排序的数据进行重写以获得排好序的数据集。外层循环是对 32 位整型数的 32 个比特位进行循环迭代，内层循环是对列表的所有元素进行循环。因此，算法一共需要 32N 次迭代，整个数据集的读 / 写次数达到 32 次。

当数据的大小少于 32 位时（如 16 位或 8 位的整型数），排序算法会快两倍或四倍，因为相对的计算任务减少为原先的一半或 1/4。在 CUDA SDK4.0 之前的版本中，可以在 Thrust 库中找到基数排序的实现，因此可以不用实现我们自己的基数排序（参见图 6-7）。

数据	数据 & 0x01	0列表	1列表	合并列表	数据 & 0x10	0列表	1列表	合并列表（已排序）
0xFF000003	1	0xFF000002	0xFF000003	0xFF000002	1	0xFF000000	0xFF000002	0xFF000000
0xFF000002	0	0xFF000000	0xFF000001	0xFF000000	0	0xFF000001	0xFF000003	0xFF000001
0xFF000001	1			0xFF000003	1			0xFF000002
0xFF000000	0			0xFF000001	0			0xFF000003

图 6-7 简单的基数排序

在内层循环中，数据被分为两个列表，根据当前字被处理的比特位将数据划分到 0 列表以及 1 列表中。然后将这两个列表重组，通常我们将 0 列表中的值写在 1 列表之前。

GPU 版本的代码可能有一点复杂，因为我们需要考虑多线程。

```
__device__ void radix_sort(u32 * const sort_tmp,
                           const u32 num_lists,
                           const u32 num_elements,
                           const u32 tid,
                           u32 * const sort_tmp_0,
                           u32 * const sort_tmp_1)
```

```
{
 // Sort into num_list, lists
 // Apply radix sort on 32 bits of data
 for (u32 bit=0;bit<32;bit++)
 {
  u32 base_cnt_0 = 0;
  u32 base_cnt_1 = 0;

  for (u32 i=0; i<num_elements; i+=num_lists)
  {
   const u32 elem = sort_tmp[i+tid];
   const u32 bit_mask = (1 << bit);

   if ( (elem & bit_mask) > 0 )
   {
    sort_tmp_1[base_cnt_1+tid] = elem;
    base_cnt_1+=num_lists;
   }
   else
   {
    sort_tmp_0[base_cnt_0+tid] = elem;
    base_cnt_0+=num_lists;
   }
  }

  // Copy data back to source - first the zero list
  for (u32 i=0; i<base_cnt_0; i+=num_lists)
  {
   sort_tmp[i+tid] = sort_tmp_0[i+tid];
  }

  // Copy data back to source - then the one list
  for (u32 i=0; i<base_cnt_1; i+=num_lists)
  {
   sort_tmp[base_cnt_0+i+tid] = sort_tmp_1[i+tid];
  }
 }
 __syncthreads();
}
```

此处 GPU 内核是以一个设备函数的形式编写的。设备函数即只能被 GPU 内核调用的函数。它相当于 C 语言函数声明之前添加一个“static”，或 C++ 中的“private”。

注意内循环中有所变化，在串行代码中索引每次循环加一，而在并行代码中，每次循环增加参数传入的 num_lists。这个数表示基数排序所产生的独立列表的数目，它应该等于内核函数每个线程块启动的线程数目。为避免存储体冲突，它的理想值应该是线程束的大小 32。然而，从隐藏存储延迟的角度来看，这个值还小于真正的理想值。

该 GPU 版本的基数排序将通过 num_lists 个线程产生 num_lists 个独立的排好序的列表。由于 GPU 上的 SM 有 32 个共享内存存储体，SM 可以与运行一个线程相同的速度运行 32 个

线程，你可能想象 num_lists 最合适的值应该是 32。见表 6-6 与图 6-8。

表 6-6 并行基数排序结果（ms）

设备 \ 线程数	1	2	4	8	16	32	64	128	256
GTX470	39.4	20.8	10.9	5.74	2.91	1.55	0.83	0.48	0.3
9800GT	67	35.5	18.6	9.53	4.88	2.66	1.44	0.82	0.56
GTX260	82.4	43.5	22.7	11.7	5.99	3.24	1.77	1.02	0.66
GTX460	31.9	16.9	8.83	4.56	2.38	1.27	0.69	0.4	0.26

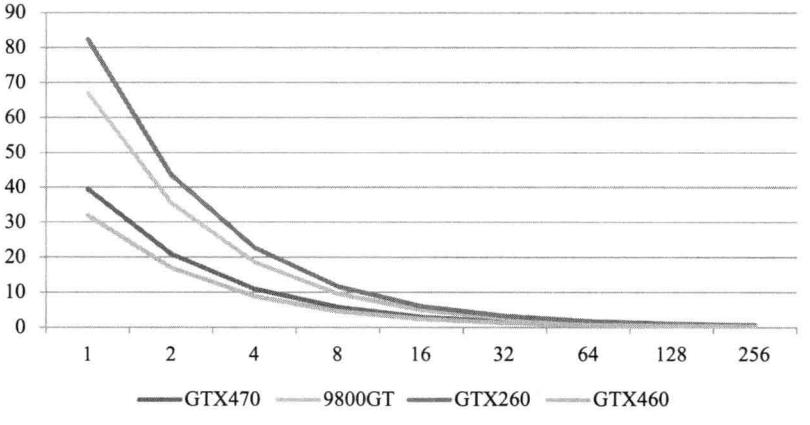


图 6-8 并行基数排序结果图

从图表中我们可以看到，基数排序确实非常高效。我们可以看到直到 128 个线程，程序速度几乎都是线性增长。这并不让人感到惊讶，因为每增加一倍的线程数量，每个线程处理的数据将减少为原来的一半。让人感兴趣的应该是线性关系增长停止的那个点，因为那说明程序达到了硬件中某方面的限制。当线程数增加到 256 时，程序的执行速度只增长了 2/3，因此我们知道最佳情况应该是 128 个线程。然而，我们也需考虑使用 128 个线程时可能限制了 SM 的使用，特别是在计算能力为 2.x 的硬件上。所以，我们根据多个线程块之间的相互作用选择 256 个线程。恰好，共享内存又是限制每个 SM 处理线程块数目的主要因素。

如果你查看初始的基数排序函数，它可能不是非常高效。那我们该如何优化这个函数呢？最明显的改变就是不需要将 0 列表与 1 列表分开。0 列表可以通过重复利用原始列表空间进行创建。这不仅允许我们舍弃 1 列表，还省去了将数据复制回原始列表的操作，节省了很多不必要的工作。

最后，你是否已注意到位掩码事实上是与 bit 循环的单次迭代有关的常量？它是伴随循环索引 i 的一个常量，因此可以将它移到 bit 循环的外面。这是一种标准的编译器优化方式，叫做不变量分析。大多数编译器会将它移到循环的外面。众所周知，编译器优化在文档记录方面一直做得很差，不同编译器的优化方式可能不同，即便是相同的编译器，不同版本的优

化方式也可能不同。依赖编译器的优化步骤是比较差的编程习惯，最好避免。因此，我们将明确地把计算移到正确位置以保证它能执行。参见第 9 章中涉及的典型编译器优化。

以下是稍作优化的代码：

```
__device__ void radix_sort2(u32 * const sort_tmp,
                            const u32 num_lists,
                            const u32 num_elements,
       const u32 tid,
       u32 * const sort_tmp_1)
{
 // Sort into num_list, lists
 // Apply radix sort on 32 bits of data
 for (u32 bit=0;bit<32;bit++)
 {
  const u32 bit_mask = (1 << bit);
  u32 base_cnt_0 = 0;
  u32 base_cnt_1 = 0;

  for (u32 i=0; i<num_elements; i+=num_lists)
  {
   const u32 elem = sort_tmp[i+tid];

   if ( (elem & bit_mask) > 0 )
   {
    sort_tmp_1[base_cnt_1+tid] = elem;
    base_cnt_1+=num_lists;
   }
   else
   {
    sort_tmp[base_cnt_0+tid] = elem;
    base_cnt_0+=num_lists;
   }
  }

  // Copy data back to source from the one's list
  for (u32 i=0; i<base_cnt_1; i+=num_lists)
  {
   sort_tmp[base_cnt_0+i+tid] = sort_tmp_1[i+tid];
  }
 }
 __syncthreads();
}
```

我们还可以进行更多的优化，此处的关键问题是只用了一个暂存区，这也使允许我们处理更多的元素。这一点很重要，因为稍后我们会看到，列表的数量将是一个重要的因素。这些改变到底怎样影响了基数排序的性能呢？

由表 6-7 我们可以看到，最坏情况是使用一个线程的执行时间从之前的 82ms 减少到 52ms，最好的情况是从之前的 0.26ms 减少到了 0.21ms，执行速度提高了 20%。

表 6-7 优化后的基数排序结果（ms）

设备 \ 线程数	1	2	4	8	16	32	64	128	256
GTX470	26.51	14.35	7.65	3.96	2.05	1.09	0.61	0.36	0.24
9800GT	42.8	23.22	12.37	6.41	3.3	1.78	0.98	0.63	0.4
GTX260	52.54	28.46	15.14	7.81	4.01	2.17	1.2	0.7	0.46
GTX460	21.62	11.81	6.34	3.24	1.69	0.91	0.51	0.31	0.21

6.4.3 合并列表

合并排好序的列表是并行编程中另一个比较常用的算法。然而，我们将从串行合并任意数目的有序列表的代码看起，因为这是最简单的一种情况。

```
void merge_array(const u32 * const src_array,
                 u32 * const dest_array,
                 const u32 num_lists,
                 const u32 num_elements)
{
 const u32 num_elements_per_list = (num_elements / num_lists);

 u32 list_indexes[MAX_NUM_LISTS];

 for (u32 list=0; list < num_lists; list++)
 {
  list_indexes[list] = 0;
 }

 for (u32 i=0; i<num_elements;i++)
 {
  dest_array[i] = find_min(src_array,
                           list_indexes,
                           num_lists,
                           num_elements_per_list);
 }
}
```

假定需要从 num_lists 个列表中选取数据，则我们需要跟踪当前所处列表中的位置。程序中的 list_indexes 数组就是用来做这件事的。由于列表的数量可能很少，因此我们可以使用栈，将数组声明为本地变量。但对于 GPU 内核而言，这可能是一个坏主意，因为根据 GPU 的不同，栈可能分配到缓慢的全局内存上。共享内存可能是 GPU 上的最优选择，但这依赖于需要的列表的数目。

首先，将索引值全部设为 0。然后程序对所有的元素进行迭代，将 find_min 函数得到的结果值划分到结果集（即已排序的数组）中。find_min 函数将从 num_lists 个数值中找到最小的那一个数。

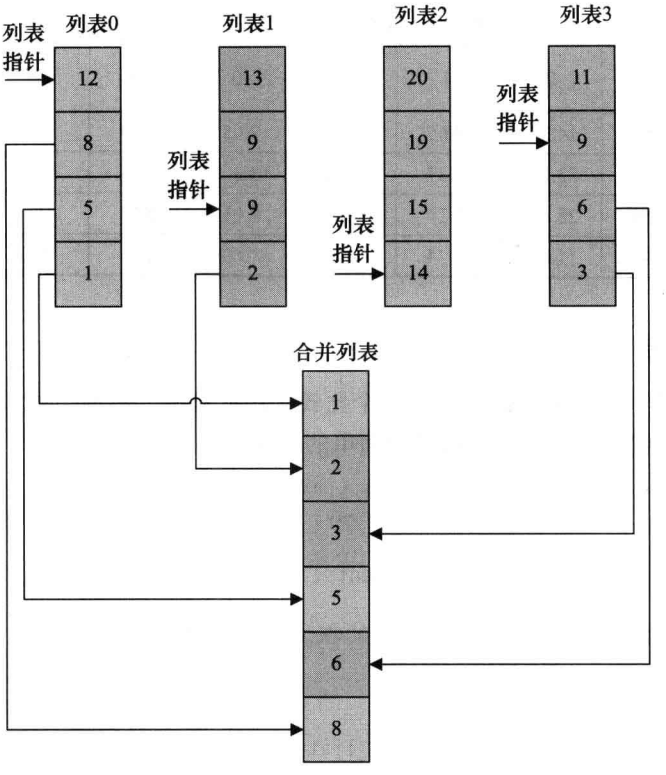


图 6-9 多个列表部分合并

```
u32 find_min(const u32 * const src_array,
             u32 * const list_indexes,
             const u32 num_lists,
             const u32 num_elements_per_list)
{
 u32 min_val = 0xFFFFFFFF;
 u32 min_idx = 0;

 // Iterate over each of the lists
 for (u32 i=0; i<num_lists; i++)
 {
  // If the current list has already been emptied
  // then ignore it
  if (list_indexes[i] < num_elements_per_list)
  {
   const u32 src_idx = i + (list_indexes[i] * num_lists);

   const u32 data = src_array[src_idx];

   if (data <= min_val)
   {
     min_val = data;
```

```
      min_idx = i;
    }
   }
  }

  list_indexes[min_idx]++;
  return min_val;
}
```

该函数对排好序的列表进行迭代访问，并对每个列表都用到了一个索引进行维护。如果函数找到一个值比当前的 min_val 还要小，就简单地将 min_val 更新为当前这个新值。当扫描完所有的列表，对最小值对应的列表索引进行加一操作，然后返回得到的最小值。

现在我们就来看一个该算法用 GPU 实现的代码。首先是最顶层的函数：

```
__global__ void gpu_sort_array_array(
 u32 * const data,
 const u32 num_lists,
 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 __shared__ u32 sort_tmp[NUM_ELEM];
 __shared__ u32 sort_tmp_1[NUM_ELEM];

 copy_data_to_shared(data, sort_tmp, num_lists,
                     num_elements, tid);

 radix_sort2(sort_tmp, num_lists, num_elements,
             tid, sort_tmp_1);

 merge_array6(sort_tmp, data, num_lists,
              num_elements, tid);
}
```

到目前为止，程序还比较简单。函数仅仅是调用了具有 N 个线程的单个线程块。我们将在此基础上写一个例子来具体介绍如何使用共享内存。我们来看第一个函数，以下是它的代码：

```
__device__ void copy_data_to_shared(const u32 * const data,
                                    u32 * const sort_tmp,
                                    const u32 num_lists,
                                    const u32 num_elements,
                                    const u32 tid)
{
 // Copy data into temp store
 for (u32 i=0; i<num_elements; i+=num_lists)
 {
  sort_tmp[i+tid] = data[i+tid];
 }
 __syncthreads();
}
```

该函数中，程序以行的方式而不是以列的方式将数据从全局内存读入到共享内存。这一

步很重要，原因有两个。首先，程序会对内存进行反复的读 / 写操作，所以，要想使程序尽可能快，就要使用共享内存代替全局内存。第二，当以行的方式访问全局内存时性能最好。以列的方式访问将产生离散的内存模式，导致硬件无法合并访问操作，除非每个线程都访问同一列的值并且所有的地址都是相邻的。因此，在大多数情况下 GPU 会发出远多于实际所需次数的内存获取操作，程序的速度会以指数级下降。

当编译程序时，在 nvcc 编译器选项中添加 -v 标志，编译器将打印出一条看起来不相关的信息，用于说明创建了一个栈帧，例如

```
1>ptxas info : Function properties for _Z12merge_arrayPKjPjjjj
1> 40 bytes stack frame, 40 bytes spill stores, 40 bytes spill loads
```

当函数调用一个子函数并传入参数时，这些参数必须以某种方式提供给被调用的函数。例如程序执行以下的函数调用：

```
dest_array[i] = find_min(src_array,
                         list_indexes,
                         num_lists,
                         num_elements_per_list);
```

此处有两种方式可以采用，一种是通过寄存器传递需要的值，另一种方式就是创建一个叫栈帧的内存区。大多数现代处理器有一个很大的寄存器组（32 个寄存器或者更多）。对一层调用而言，通常这已足够了。老式架构的处理器会用到栈帧，将参数值压入到栈中，然后被调用的函数将值从栈中弹出。由于我们要求内存使用这种方式，对 GPU 而言这可能意味着需要使用“本地”内存。这里所谓的“本地”只是指哪个线程能访问这块内存。事实上，“本地”内存存储在全局内存上，这样非常不高效，特别是在没有高速缓存的老式架构（计算能力为 1.x）的设备上。由于这个原因，我们需要重写合并的程序，以避免函数调用。新的程序如下：

```
// Uses a single thread for merge
__device__ void merge_array1(const u32 * const src_array,
                             u32 * const dest_array,
                             const u32 num_lists,
                             const u32 num_elements,
                             const u32 tid)
{
 __shared__ u32 list_indexes[MAX_NUM_LISTS];

 // Multiple threads
 list_indexes[tid] = 0;
 __syncthreads();

 // Single threaded
 if (tid == 0)
 {
  const u32 num_elements_per_list = (num_elements / num_lists);

  for (u32 i=0; i<num_elements;i++)
  {
   u32 min_val = 0xFFFFFFFF;
```

```
    u32 min_idx = 0;

    // Iterate over each of the lists
    for (u32 list=0; list<num_lists; list++)
    {
     // If the current list has already been
     // emptied then ignore it
     if (list_indexes[list] < num_elements_per_list)
     {
      const u32 src_idx = list + (list_indexes[list] * num_lists);

      const u32 data = src_array[src_idx];
      if (data <= min_val)
      {
       min_val = data;
       min_idx = list;
      }
     }
    }
    list_indexes[min_idx]++;
    dest_array[i] = min_val;
   }
  }
}
```

该函数将原先的 merge_array 函数与 find_min 函数合并起来。重新编译，将不再产生栈帧。运行代码得到结果如表 6-8 所示，图 6-10 更加清晰地展示了程序的性能。

表 6-8 初始单线程归并排序性能结果

设备 \ 线程数	1	2	4	8	16	32	64	128	256
GTX470	27.9	16.91	12.19	12.31	17.82	31.46	59.42	113.3	212.7
9800GT	44.83	27.21	19.55	19.53	28.07	51.08	96.32	183.08	342.16
GTX260	52.03	33.38	24.05	24.15	34.88	62.9	118.71	225.73	422.55
GTX460	22.76	13.85	10.11	10.41	15.29	27.18	51.46	90.26	184.54

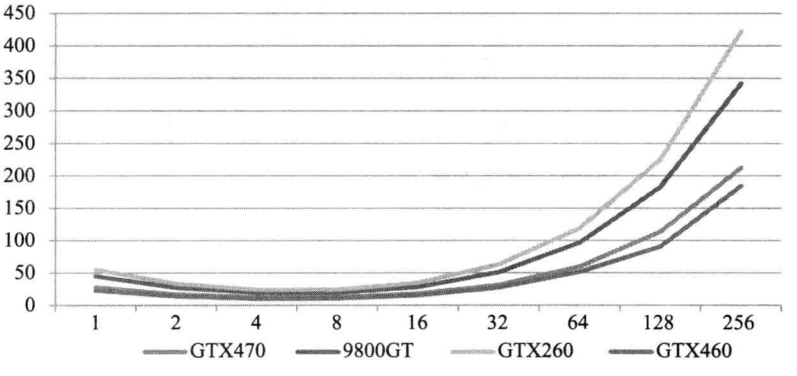


图 6-10 初始单线程归并排序性能结果图

从图中我们会惊讶地发现性能最差的居然是 GTX260，比早先的 9800GT 还要慢。同样让人感到奇怪的是，在这个实验中 GTX460 比 GTX470 还要快。为了弄清楚这一点，我们需要来看看这些特殊设备的参数，如表 6-9 所示。

表 6-9 设备时钟频率以及带宽

设　备	9800GT	GTX260	GTX460	GTX470
核心时钟	650Hz	625MHz	726MHz	608MHz
存储带宽	61GB/s	123GB/s	115GB/s	134GB/s

由图 6-10 我们会发现 9800GT 的时钟频率比 GTX260 高，同样，GTX460 的时钟频率比 GTX470 的时钟频率高。由于程序只用到了一个 SM，显然，内存访问主要以共享内存获取的时间为主。

然而，图 6-10 中最让人感兴趣的一点则是当线程数量增加超过某个固定值之后程序的计算速度开始减缓。在没理清关系之前，乍一看这有违常理。但这种类型的结果表明，程序可能存在资源冲突，或者当增加线程数量时，问题的处理速度并不会以线性方式增长。

当前问题属于后者。合并操作是针对单个线程并对 N 个列表的每个元素进行访问。当列表数量增加，问题空间会伴随着线程数量线性增加为 2N、4N、8N 等。从时间的角度来看，该算法最优情况是在 4 个与 8 个列表数据之间。但这并不是非常好，因为它潜在地限制了并行度。

6.4.4 并行合并

为了获取更好的性能，显然只用一个线程进行合并是不够的。然而，使用多个线程会引入一个问题，因为我们是将数据写到单一列表中的。为了实现这一点，线程必须以某种方式进行合作，这使得合并变得更加复杂了。

```
// Uses multiple threads for merge
// Deals with multiple identical entries in the data
__device__ void merge_array6(const u32 * const src_array,
                             u32 * const dest_array,
                             const u32 num_lists,
                             const u32 num_elements,
                             const u32 tid)
{
 const u32 num_elements_per_list = (num_elements / num_lists);

 __shared__ u32 list_indexes[MAX_NUM_LISTS];
 list_indexes[tid] = 0;

 // Wait for list_indexes[tid] to be cleared
__syncthreads();

// Iterate over all elements
for (u32 i=0; i<num_elements;i++)
```

```
{
 // Create a value shared with the other threads
 __shared__ u32 min_val;
 __shared__ u32 min_tid;

 // Use a temp register for work purposes
 u32 data;

 // If the current list has not already been
 // emptied then read from it, else ignore it
 if (list_indexes[tid] < num_elements_per_list)
 {
  // Work out from the list_index, the index into
  // the linear array
  const u32 src_idx = tid + (list_indexes[tid] * num_lists);

  // Read the data from the list for the given
  // thread
  data = src_array[src_idx];
 }
 else
 {
  data = 0xFFFFFFFF;
 }

 // Have thread zero clear the min values
 if (tid == 0)
 {
  // Write a very large value so the first
  // thread thread wins the min
  min_val = 0xFFFFFFFF;
  min_tid = 0xFFFFFFFF;
 }

 // Wait for all threads
 __syncthreads();

 // Have every thread try to store it's value into
 // min_val. Only the thread with the lowest value
 // will win
 atomicMin(&min_val, data);

 // Make sure all threads have taken their turn.
 __syncthreads();
  // If this thread was the one with the minimum
  if (min_val == data)
  {
   // Check for equal values
   // Lowest tid wins and does the write
   atomicMin(&min_tid, tid);
  }
```

```
    // Make sure all threads have taken their turn.
    __syncthreads();

    // If this thread has the lowest tid
    if (tid == min_tid)
    {
     // Incremene the list pointer for this thread
     list_indexes[tid]++;

     // Store the winning value
     dest_array[i] = data;
    }
   }
}
```

这个版本的函数中使用了 num_lists 个线程进行合并操作，但只用了一个线程一次将结果写到输出数据列表中，这样保证了单个输出列表的正确性。

在该函数中使用到了 atomicMin 函数。每个线程以从列表中获取的数据作为入参调用 atomicMin 函数，取代了原先的单个线程访问列表中所有的元素并找出最小值。当每个线程调用 atomicMin 函数时，线程读取最小值并与当前线程中的值进行比较，然后把结果写回到最小值对应内存中。如果当前线程获取的值与当前最小值相等，则以当前线程的值为准，最小值对应的线程号也变为当前线程。然而，这样做存在一个问题，由于列表中的数据可能会重复，因此可能出现多个线程获取的值都是最小值。所以，对于那些拥有相同数据值的线程还需要第二个消除步骤，但大多数情况下，第二个步骤是不必要的。然而，排序最坏的情况就是列表中所有的值都是相同的，这将导致所有的线程都执行两次消除步骤。

表 6-10 与图 6-11 显示了这个版本函数的性能。大规模并行（128 个线程以及 256 个线程）的总运行时间减少了近 10 倍。然而，单个线程的执行时间却没有改变。更重要的是，最好的情况从原先的 8 线程变成了 16 线程，最优执行时间也减少了一半。

表 6-10 使用 atomicMin 的并行归约排序结果（ms）

设备 \ 线程数	1	2	4	8	16	32	64	128	256
GTX470	29.15	17.38	10.96	7.77	6.74	7.43	9.15	13.55	22.99
GTX260	55.97	32.67	19.87	13.22	10.51	10.86	13.96	19.97	36.68
GTX460	23.78	14.23	9.06	6.54	5.86	6.67	8.41	12.59	21.58

有一点不得不提的就是基于共享内存的 atomicMin 操作只能在计算能力为 1.2 及以上的设备执行。9800GT 的计算能力为 1.1，所以此处并没有显示内核在该设备上的运行结果。

如果稍微了解一下硬件并用 Parallel Nsight 这类的分析工具进行分析，我们会发现，当超过 32 个线程时分支的数量以及共享内存访问的数量开始迅速增长。当前，我们获得了一个比较好的解决方案，但有没有其他选择使程序运行得更快呢？

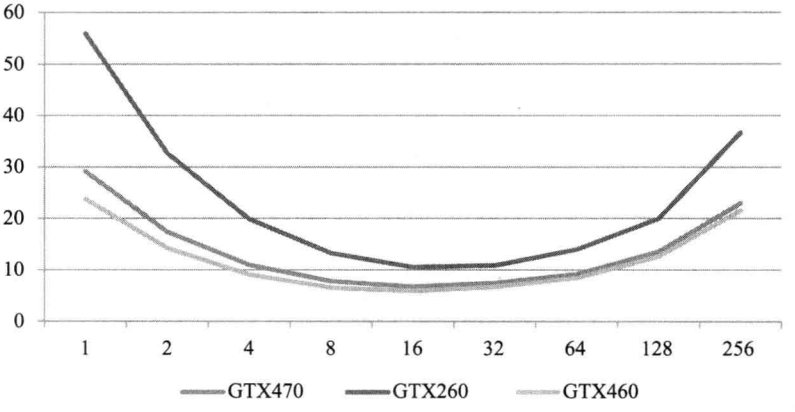


图 6-11 使用 atomicMin 的并行归约排序结果图

6.4.5 并行归约

对这个问题，一个很常用的方法就是并行归约。并行归约适用于许多问题，寻找最小值操作只是其中的一种。它使用数据集元素数量一半的线程，每个线程将当前线程对应的元素与另一个元素进行比较，计算两者之间的最小值，并将得到的最小值移到前面。每进行一次比较，线程数就减少一半，如此反复，直到只剩一个元素为止，这个元素就是需要的结果。

使用 CUDA 我们必须牢记 SM 的执行单元是线程束。因此，任何线程数少于线程束的大小都意味着未充分利用硬件。另外，分支线程必须全部被执行，但分支线程束就未必了。

当选择给定线程的其他元素供线程计算比较时，你可能选择同一个线程束中的元素进行归约，但这会明显地导致线程束内产生分支，每个分支将使 SM 做双倍的工作，继而影响程序的性能。一个很好的选择元素的方式就是选择另一半数据集的元素。

图 6-12 显示了一个元素与另一半数据集中的元素进行比较。阴影单元表示当前活跃的线程。

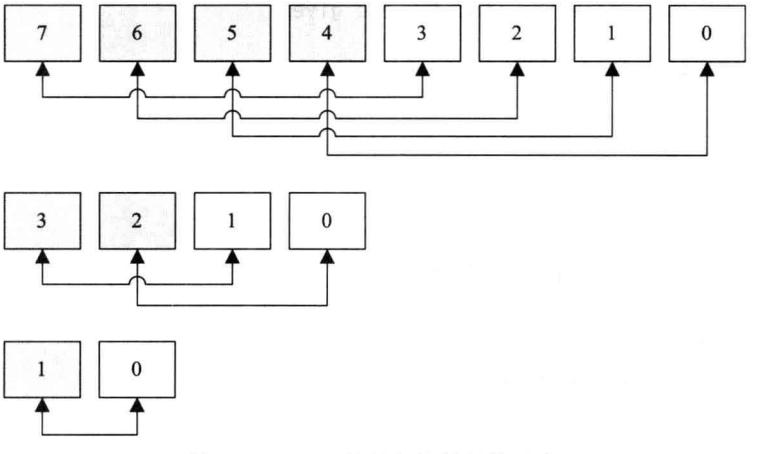


图 6-12 GPU 并行归约的最终阶段

```
// Uses multiple threads for reduction type merge
__device__ void merge_array5(const u32 * const src_array,
                             u32 * const dest_array,
                             const u32 num_lists,
                             const u32 num_elements,
                             const u32 tid)
{
 const u32 num_elements_per_list = (num_elements / num_lists);

 __shared__ u32 list_indexes[MAX_NUM_LISTS];
 __shared__ u32 reduction_val[MAX_NUM_LISTS];
 __shared__ u32 reduction_idx[MAX_NUM_LISTS];

 // Clear the working sets
 list_indexes[tid] = 0;
 reduction_val[tid] = 0;
 reduction_idx[tid] = 0;
 __syncthreads();

 for (u32 i=0; i<num_elements;i++)
 {
  // We need (num_lists / 2) active threads
  u32 tid_max = num_lists >> 1;

  u32 data;

  // If the current list has already been
  // emptied then ignore it
 if (list_indexes[tid] < num_elements_per_list)
 {
  // Work out from the list_index, the index into
  // the linear array
  const u32 src_idx = tid + (list_indexes[tid] * num_lists);

  // Read the data from the list for the given
  // thread
  data = src_array[src_idx];
 }
 else
 {
  data = 0xFFFFFFFF;
 }

 // Store the current data value and index
 reduction_val[tid] = data;
 reduction_idx[tid] = tid;

 // Wait for all threads to copy
 __syncthreads();

 // Reduce from num_lists to one thread zero
```

```
while (tid_max != 0)
{
 // Gradually reduce tid_max from
 // num_lists to zero
 if (tid < tid_max)
 {
  // Calculate the index of the other half
  const u32 val2_idx = tid + tid_max;

  // Read in the other half
  const u32 val2 = reduction_val[val2_idx];

  // If this half is bigger
  if (reduction_val[tid] > val2)
  {
   // The store the smaller value
   reduction_val[tid] = val2;
   reduction_idx[tid] = reduction_idx[val2_idx];
  }
 }

 // Divide tid_max by two
 tid_max >>= 1;

 __syncthreads();
 }

 if (tid == 0)
 {
  // Incremenet the list pointer for this thread
  list_indexes[reduction_idx[0]]++;

  // Store the winning value
  dest_array[i] = reduction_val[0];
 }

 // Wait for tid zero
 __syncthreads();
}
```

这段代码在共享内存中创建了一个临时的列表，用来保存每次循环中从 num_list 个数据集列表中选取出来进行比较的数据。如果进行合并的列表已经为空，则将临时列表中对应该列表的数据区赋值为 0xFFFFFFFF，这将把该值排除在列表之外。每轮 while 循环之后，活跃线程数将减少一半，直到只剩一个线程活跃为止，即 0 号线程。然后将结果复制到输出列表中并将最小值对应的列表索引做加一操作以保证元素不会被处理两次。

注意在循环中和结尾处用到的 _syncthreads 指令。程序需要在已使用线程数大于 32（即一个线程束）时对线程束进行同步操作。

表 6-11 与图 6-13 显示了这个版本的内核的执行性能。我们发现该版本比 atomicMin 版本的内核要慢，归约版本的内核最快需要 8.4ms，而 atomicMin 版本最快则需 5.86ms（GTX460，16 线程）。这比 atomicMin 版本几乎慢了 50%。然而，值得注意的是，使用 256 个线程的执行时间几乎比 atomicMin 版本快了两倍（12.27ms，相比于 21.58ms）。但比 16 线程版本的执行时间却慢了近一半。

表 6-11 并行归约结果（ms）

线程数 / 设备	1	2	4	8	16	32	64	128	256
GTX470	28.4	17.67	12.44	10.32	9.98	10.59	11.62	12.94	14.61
9800GT	45.66	28.35	19.82	16.25	15.61	17.03	19.03	21.45	25.33
GTX260	56.07	34.71	24.22	19.84	19.04	20.6	23.2	26.28	31.01
GTX460	23.22	14.52	10.3	8.63	8.4	8.94	9.82	10.96	12.27

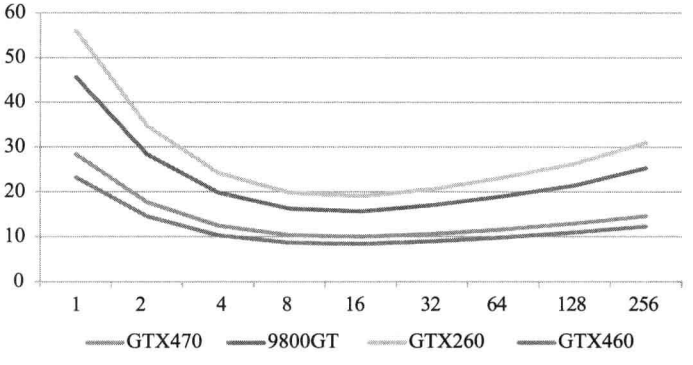


图 6-13 并行归约结果图

尽管这种版本的内核比较慢，但它也有一定的优势，那就是不需要使用 atomicMin 函数。该函数只可用在计算能力为 1.2 以上的设备上，但这个问题只要考虑一下消费者市场或者找一个真正支持旧式 Tesla 系统即可解决。真正的问题是 atomicMin 函数只支持整型数的计算。但现实世界的问题通常是基于浮点数的。此种情况下，这两种算法我们都需要。

不管怎样，我们可以从 atomicMin 和并行归约这两种方案中得出一个结论，那就是传统的利用两个列表进行归并排序在 GPU 上并不是最理想的方案。我们可以通过增加列表数量从而增加基数排序的并行度来提高性能，然而却在合并阶段因为增加并行度并超出 16 个列表而导致程序性能降低。

6.4.6 混合算法

我们可以利用这两种算法的优势，创造一种混合的方案。将合并归并排序的代码重写，

如下所示。

```
#define REDUCTION_SIZE 8
#define REDUCTION_SIZE_BIT_SHIFT 3
#define MAX_ACTIVE_REDUCTIONS ( (MAX_NUM_LISTS) / REDUCTION_SIZE )

// Uses multiple threads for merge
// Does reduction into a warp and then into a single value
__device__ void merge_array9(const u32 * const src_array,
                             u32 * const dest_array,
                             const u32 num_lists,
                             const u32 num_elements,
                             const u32 tid)
{
 // Read initial value from the list
 u32 data = src_array[tid];

 // Shared memory index
 const u32 s_idx = tid >> REDUCTION_SIZE_BIT_SHIFT;

 // Calcuate number of 1st stage reductions
 const u32 num_reductions = num_lists >> REDUCTION_SIZE_BIT_SHIFT;
const u32 num_elements_per_list = (num_elements / num_lists);

// Declare a number of list pointers and
// set to the start of the list
__shared__ u32 list_indexes[MAX_NUM_LISTS];
list_indexes[tid] = 0;

// Iterate over all elements
for (u32 i=0; i<num_elements;i++)
{
 // Create a value shared with the other threads
 __shared__ u32 min_val[MAX_ACTIVE_REDUCTIONS];
 __shared__ u32 min_tid;

 // Have one thread from warp zero clear the
 // min value
 if (tid < num_lists)
 {
  // Write a very large value so the first
  // thread thread wins the min
  min_val[s_idx] = 0xFFFFFFFF;
  min_tid = 0xFFFFFFFF;
 }

 // Wait for warp zero to clear min vals
 __syncthreads();

 // Have every thread try to store it's value into
```

```
// min_val for it's own reduction elements. Only
// the thread with the lowest value will win.
atomicMin(&min_val[s_idx], data);

// If we have more than one reduction then
// do an additional reduction step
if (num_reductions > 0)
{
 // Wait for all threads
 __syncthreads();

 // Have each thread in warp zero do an
 // additional min over all the partial
 // mins to date
 if ( (tid < num_reductions) )
 {
  atomicMin(&min_val[0], min_val[tid]);
 }
 // Make sure all threads have taken their turn.
 __syncthreads();
}

// If this thread was the one with the minimum
if (min_val[0] == data)
{
 // Check for equal values
 // Lowest tid wins and does the write
 atomicMin(&min_tid, tid);
}

// Make sure all threads have taken their turn.
__syncthreads();

// If this thread has the lowest tid
if (tid == min_tid)
{
 // Incremenet the list pointer for this thread
 list_indexes[tid]++;

 // Store the winning value
 dest_array[i] = data;

 // If the current list has not already been
 // emptied then read from it, else ignore it
 if (list_indexes[tid] < num_elements_per_list)
  data = src_array[tid + (list_indexes[tid] * num_lists)];
 else
  data = 0xFFFFFFFF;
}
```

```
    // Wait for min_tid thread
    __syncthreads();
  }
}
```

简单的 1 ~ N 归约的一个主要问题就是当 N 值增大时，程序的速度变得缓慢。从先前的测试中我们可以看到，N 的理想值大约在 16 个元素左右。该内核函数将数据集划分成 N 块分别找寻每块中的最小值，然后再将这 N 块每块得到的结果最终归约到一个值中。这种方式与归约的例子非常相似，但同时又省略了多次迭代。

注意，原先的 min_val 由单一数据扩展成为一个共享数据的数组。这样做是有必要的，因为每个独立的线程需要从它对应的数据集中获取得到最小值。每个最小值是一个 32 位宽的数值，因此，每个最小值可以存储在独立的共享内存存储体中，假如第一轮归约得到的最大元素为 32 或少于 32 个，这将意味着不会出现存储体冲突。

内核中 REDUCTION_SIZE 的值设为了 8，这表示程序将对包含 8 个数值的组分布找出每个组的最小值，然后再在这些组获得的最小值中找寻最终的最小值。对于一个最大包含 256 个元素的数据集，我们可获得 32 个独立的存储体供归约使用。在 256 个元素的数据集中，归约比为 256 : 32 : 1。对于一个包含 128 个元素的数据集，归约比为 128 : 16 : 1。

该内核函数中另一个重要的变化是只有最终最小值对应的那个线程的 data 会更新，其他线程的 data 值不会更新。data 值存储在寄存器中，每个线程对应一个 data 值。在之前的内核中，所有的线程会从对应的列表中重新读入 data 值，因为在之前的内核中，每一轮结束之后，只有最终最小值对应的线程的索引数组指针会变化。因此，随着 N 值的增大，这将变得越来越不高效。但乍一看，即便是修改之后的方式，也并不会带来很大的性能提升。

那么，究竟这个版本的内核性能如何呢？从表 6-12 可以看到，最短时间由原先 atomicMin 版本的 5.86ms 减少到 5.67ms，变化并不明显，但值得注意的是结果曲线图的形状（如图 6-15 所示）。图 6-15 的形状不再是像原先那样呈“U”字型的形状。不论是 32 个线程还是 64 个线程版本，执行速度都超过了原先 atomicMin 版本中 16 线程的执行速度。通过采用这种新的合并方式，表 6-12 与图 6-15 所示的向上增加的趋势逐渐开始变得平滑。

表 6-12 原子操作与并行归约混合策略的结果（ms）

线程数 / 设备	1	2	4	8	16	32	64	128	256
GTX470	29.41	17.62	11.24	8.98	7.2	6.49	6.46	7.01	8.57
GTX260	56.85	33.54	20.83	15.29	11.87	10.5	10.36	11.34	14.65
GTX460	24.12	14.54	9.36	7.64	6.22	5.67	5.68	6.27	7.81

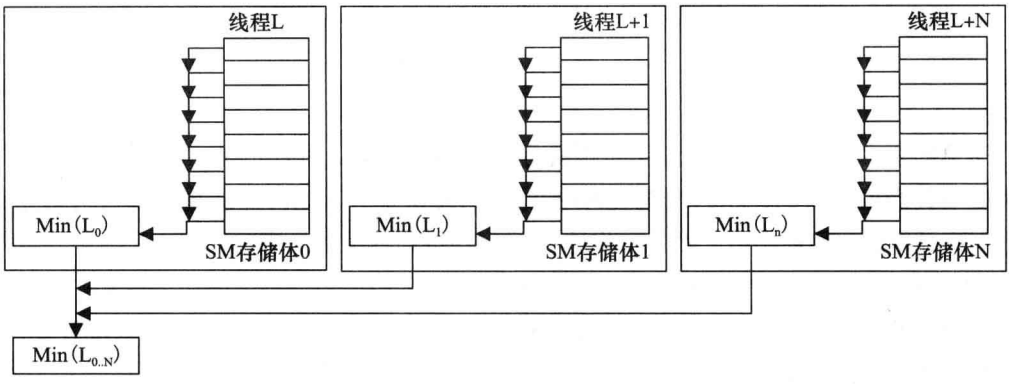


图 6-14 混合并行归约

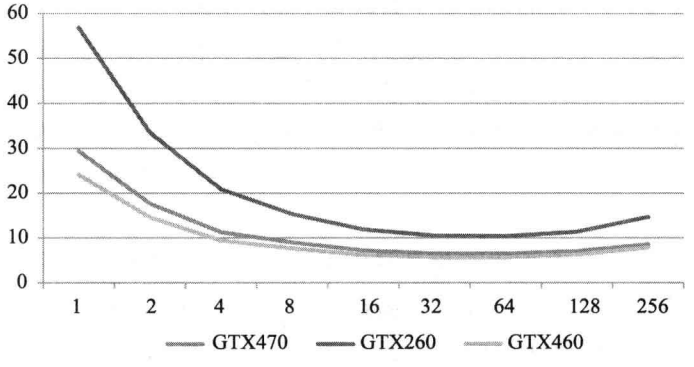


图 6-15 原子操作与并行归约混合策略结果图

6.4.7 不同 GPU 上的共享内存

并不是所有的 GPU 都是按相同方式制造的。在计算能力为 2.x 的设备上，共享内存的大小变得可配置。计算能力为 2.x（费米型）的设备上，共享内存的默认大小为 48K，而原先计算能力为 1.x 的设备共享内存大小为 16K。

共享内存的大小随着不同硬件的发布也在不断变化。在编写性能得到提高的新 GPU 程序时，我们必须书写可移植的代码。为了支持这一点，CUDA 允许我们通过下面一段代码查询当前设备可提供的共享内存大小。

```
struct cudaDeviceProp device_prop;
CUDA_CALL(cudaGetDeviceProperties(&device_prop, device_num));
printf("\nSharedMemory: %u", device_prop.sharedMemPerBlock);
```

拥有更多的共享内存意味着我们可以选择两个策略中的一个。我们可以将共享内存从 16K 扩展到 48K，或者可以简单地调度更多的线程块到单个 SM 上。最优的选择取决于我们手头上的应用程序。以当前排序的程序为例，48K 的共享内存允许每个 SM 以三的倍数降低列表数量。与我们之前看到的一样，需要合并的列表的数目对整个程序的运行时间有重要的

影响。

6.4.8 共享内存小结

到目前为止我们仍在单个 SM 中进行排序，事实上是在单个线程块上。将单线程块版本扩展为多线程块的版本需引进其他需要排序的数据集。每个线程块产生一个独立的排好序的列表，然后再将这些列表合并起来，不过这次是合并到全局内存上。列表的大小可以超过共享内存可容纳大小。对于多 GPU 程序，也可以用相同的方式处理，每个 GPU 产生至少一个排好序的列表，然后再将这至少 N 个列表合并，其中 N 代表系统中 GPU 的数目。

本节首先介绍了块内线程如何通过共享内存进行协作。此处选择合并的例子主要是为了演示这种方式并不是特别复杂，理解相对简单。研究并行排序的人非常多，从内存使用及 SM 优化的角度来看，只有更加复杂的算法才会更加高效。这里我们只是用了一个简单易学的例子进行介绍，然后处理了一些并不是简单进行归约为单一数值的数据。

在接下来的小节中，我们将继续使用排序的例子，除了线程级通信之外，我们还将介绍线程块之间如何通信与协作。

关于共享内存的问题

1. 在 radix_sort 算法中，如何减少共享内存的使用，并解释这样做有什么好处？

2. 内核函数中的同步点都是必要的吗？讨论在每个实例中为什么要使用同步语句，哪些情况下不需要使用它们？

3. 使用 C++ 模板会对执行时间有怎样的影响？

4. 如何进一步优化该排序算法？

关于共享内存的答案

1. 对于这个问题有许多解决方案。其中一种就是仅使用用于排序而分配的内存。利用最高有效位基数排序，将列表中第一个元素与最后一个元素进行交换。0 号列表内的元素从前向后计算，1 号列表中的元素从后向前计算。当它们相遇之后，直到最低有效位排好序，下一个数才会进行排序。减少内存使用是非常有帮助的，因为这样做可以允许共享内存中保存更大的列表，减少了所需列表的数量，而这一点对程序执行时间会产生很大影响。

2. 使用同步点最主要的原因是线程束的数目超过一个。同一线程束中的指令都是同步执行的。分支会导致不满足分支条件的线程阻塞。在分支汇集处，我们必须保证所有指令都同步执行，尽管线程束可能立即再次进行分支。注意，如果要保证线程之间写操作的可见性，内存必须声明为 volatile，或者在线程束内进行线程同步操作。参见第 12 章对于 volatile 修饰符常见问题的讨论。

3. 模板可以允许我们在编译时用字面值代替运行时对 num_lists 参数的计算。参数通常为 2 的指数倍，并且在实际中其最大值被限制为 256。因此，可以创建许多模板然后在运行

时调用适当的函数。考虑到迭代的次数在编译时就已是已知的，而不用在运行时计算，所以编译器可以高效地将循环展开并且使用字面值将读取次数变量替换。此外，模板可以用来支持不同数据类型的多重实现，例如，使用 atomicMin 针对整型值的版本的内核来对浮点数进行并行归约。

4. 这是一个开放性问题，所以问题的答案并不唯一。随着需要合并的列表数量的增加，问题规模将变得越来越大。消除合并操作是一个比较好的解决方案。我们可以通过值的大小将原始列表划分排序到 N 个子列表中，然后将排好序的子列表串联起来而不是合并起来。这种方法属于另一种基本的排序算法，叫做样本排序，在本章接下来的小节中我们将介绍该算法。

依然以案例程序中数据集的大小为例，处理 1 024 个元素，使用 256 个线程，相当于 256 个列表，每个列表 4 个元素。使用单个比特位的基数排序是非常不高效的，因为鉴于元素的个数，每个线程需要进行 128 次迭代。对于 N 值比较小的情况，使用基于比较的排序会更快。

在本例中，我们使用了单个比特位进行基数排序。使用多个比特位，则可以减少传递数据集所产生的中间寄存器开销。当前，我们可以用迭代的方法将元素排序到序列化的列表中。通过对基数的位进行计数，使用前缀和的函数计算出数据应该移动到的内存的索引，最后将数据写到该索引对应的内存区中。稍后本章将介绍前缀和。

6.5 常量内存

常量内存其实只是全局内存的一种虚拟地址形式，并没有特殊保留的常量内存块。常量内存有两个特性，一个是高速缓存，另一个是它支持将单个值广播到线程束中的每个线程。

常量内存，通过名字我们就能猜到它是只读内存。这种类型的内存要么是在编译时声明为只读内存，要么是在运行时通过主机端定义为只读内存。常量只是从 GPU 内存的角度而言。常量内存的大小被限制为 64K。

在编译时声明一块常量内存，需要用到 __constant__ 关键字，例如：

```
__constant__ float my_array[1024] = { 0.0F, 1.0F, 1.34F, ... };
```

如果要在运行时改变常量内存区中的内容，只需在调用 GPU 内核之前简单地调用 cudaCopyToSymbol 函数。如果在编译阶段或主机端运行阶段都没有定义常量内存，那么常量内存区将未定义。

6.5.1 常量内存高速缓存

1. 计算能力为 1.x 的设备

在计算能力为 1.x 的设备上（费米架构之前的设备），常量内存有一个特性，就是将数据缓存到一块 8KB 的一级缓存上，使随后的访问变得更加快速。但前提是内存中的数据可能在程序中重复利用。此外，广播访问也被高度优化，使得访问相同内存地址的线程能够在单个周期内完成。

64KB 的常量内存与 8KB 的高速缓存意味着可缓冲的内存大小比为 8：1，这已经非常高了。如果能够将访问的数据包含或局部化到常量内存中的一个 8KB 大小的块中，那么程序将获得很高的性能。在某些设备上将数据局部化到更小的块中可以获得更高的性能。

对于非均匀的常量内存访问，如果缓存中没有命中所需的数据，将导致 N 次对全局内存的访问，而不单是从常量缓存上获取数据。因此，对于那些数据不太集中或数据重用率不高的内存访问，尽量不要使用常量内存。另外，由于需要等待常量内存，内存获取中的分支可能导致序列化。所以，在同一线程束中，32 个线程对常量内存进行独立访问的时间比访问单个数据项的时间长至少 32 倍。如果这中间还出现缓存未命中，执行时间将增加得更明显。

相对于全局内存访问的上百个周期延迟，单个周期的访问带来的性能提升有质的飞跃。然而，只要有足够多的线程束供 SM 调度，通过线程束的任务交换，对于全局内存的几百个周期的访问延迟就能被很好地隐藏。因此使用常量内存的高速缓存这一特性带来的好处取决于从全局内存获取数据的时间以及算法中数据重复利用的程度。与共享内存一样，低端设备上的全局内存带宽更低，因此在低端设备上使用这种技术比在高端设备上使用获得的性能提升会明显。

大多数算法都会将它们的数据从一个很大的问题中分解成一些很小的块（例如，更小的数据集）。事实上，只要问题无法在物理上通过一台机器解决，就需要进行数据划分。同样对于多核 CPU 也可以进行数据划分，将任务分给 N 个核，每个核处理 1/N 的数据。而 GPU 上的每个 SM 就可以看作是一个支持上千线程的 CPU 核。

想象一下将需要处理的数据覆盖到一个网格上，然后对数据进行划分，其中数据划分到网格上的块或单元数等于所期望划分到的核（SM）的数目。对这些基于 SM 的块再进一步划分，将它们至少再划分成八块。现在，我们就将数据划分到了 N 个 SM 中，每个 SM 分配了 M 个块进行处理。

实际上，这种划分得到的块通常都太大，并且不适用于之后发布的一些 SM 数量与线程块数量已增长的 GPU，因此，这种划分并不会带来很多益处。此外，当 SM 的数量未知时这种划分也不适用，例如，写一个需要在用户机器上运行的商业程序。到目前为止，设备上 SM 的数目最多为 32 个（GT200 系列）。针对企业的开普勒和费米架构的设备，SM 的数目最多分别为 15 和 16。至于为游戏设计的设备，SM 通常最多有 8 个。

另一点需要重点考虑的是线程间通信的方式。只有使用多个线程时才可能需要线程通信。在费米和开普勒架构的设备上，每个线程块上的线程数目限制为 1024 个，在早期的设备上更少。当然，你也可以让一个线程处理多个数据项，这样就不会像起初所表现出的那么强的限制。

最后需要考虑的是负载平衡。在许多早期发布的 GPU 设备上，SM 的数目并不是 2 的指数倍，例如，GTX460 有 7 个 SM，而 GTX260 有 30 个 SM，等。因此，使用过少的线程块将导致粒度过小，这样会导致 SM 在最终的计算阶段出现闲置状态。

针对常量内存进行分块，意味着划分的每块大小不能超过 64KB。块的大小最好为 8KB 或者更小。有时分块需要对边界出现的重影区（halo region）或假想单元（ghost cell）进行处理，使值能够在块之间共享。如果出现重影区，大块通常比小块的效果更好，因为大块之间

需要通信的区域变小了。

实际中进行分块时需要考虑更多。通常最好的解决方案是通过对线程数量、每个线程处理的元素个数、线程块的数目以及块的宽度进行组合、遍历，找出针对当前问题的最优方案。第 9 章中将对此进行具体介绍。

2. 计算能力为 2.x 的设备

在费米架构（计算能力为 2.x）的设备以及后续发布的设备上出现了二级缓存。费米架构设备上的二级缓存可以在 SM 之间进行共享。所有的内存访问将自动地缓存到二级缓存中。此外，通过牺牲 SM 上 32K 的共享内存，一级缓存的大小可以从 16K 增加到 48K。由于费米架构上的所有内存访问都进行了缓存，因此如何使用常量内存需要进行一些额外的考虑。

费米架构的设备与计算能力为 1.x 的设备不同，它允许任何常量区域的数据都以常量内存对待，即使该数据并没有明确声明为常量内存数据。在计算能力为 1.x 的设备上，常量内存需要通过一些如 cudaMemcpyToSymbol 的特殊功能的函数进行管理，或者在编译时就进行声明。而对于费米架构的设备，对任何声明为常量的内存区进行基于非线程的访问都将通过常量缓存。所谓的基于非线程的访问，即对所访问的数组的索引进行计算时不需要用到 threadIdx.x。

如果对常量数据需要进行基于线程的访问，在计算能力为 1.x 的设备上，只需要在编译时对变量使用 __constant__ 前缀进行声明或者在运行时调用 cudaMemcpyToSymbol 即可。

然而，需要意识到的一点是二级缓存仍然存在，而且它比常量内存更大。如果实现一个分块算法中块之间有重影区或假想单元，那么解决方案通常需要包含将重影单元复制到常量内存或共享内存的操作。由于费米架构的设备拥有二级缓存，这种方法通常比将整个块单元复制到共享内存或常量内存然后从全局内存读取重影单元要慢。二级缓存会从之前访问内存得到的块中收集重影单元。因此，对于计算能力为 1.x 的设备，全局内存访问是真正在全局内存上访问数据，而在拥有二级缓存之后，与计算能力为 1.x 的设备相比重影单元能更快地从二级缓存获取数据。

6.5.2 常量内存广播机制

常量内存有一个非常有用的特性，该特性主要将数据分配或广播到线程束的每个线程中。广播能在单个周期内发生，因此，该特性非常有用。相比之下，在计算能力为 1.x 的硬件上，完成一次对全局内存的合并访问通常会有几百个周期的延迟。从存储子系统获取到数据之后，硬件再以相同的方式将数据分配到所有线程中，但在内存子系统提供数据之前必须经过漫长等待。不幸的是，由于存储速度无法与处理器时钟同步，这种问题非常常见。

想象一下我们可能以与从全局内存获取数据相同的方式从磁盘获取数据。我们不会写一个需要多次从磁盘获取数据的程序，因为那样实在是太慢了。我们必须考虑什么样的数据需要获取，一旦获取之后，在后台程序触发从磁盘引入下一个数据块的同时，尽可能多地重复

使用这些数据。

在基于二级缓存访问机制的费米结构设备上，同样可以使用广播机制，通过广播机制，我们可以快速地将数据分配到线程束的多个线程中。这在利用多线程执行一些常见的变换时特别有用。每个线程从常量内存读取元素 N，这样会触发广播机制，在读取到数据之后广播到线程束中的每个线程。通常，部分通过常量内存获取值的处理可能伴随着对全局内存的读 / 写操作。接着从常量内存中获取元素 N+1，同样通过广播机制进行数据分配，依此类推。由于常量内存区几乎可以达到与一级缓存相同的速度，因此这类算法使用常量内存性能会非常好。

然而，需要注意的一点是，如果一个常量只是字面值，那么最好用 #define 对字面值进行定义，因为这样可以减少常量内存的使用。所以，尽量不要把诸如 PI 这样的字面值放到常量内存中，而是用 #define 进行宏定义。事实上，这两种方案只是使用了不同的内存，但在速度上会产生一定影响。让我们一起来看下面这个程序案例：

```
#include "const_common.h"
#include "stdio.h"
#include "conio.h"
#include "assert.h"

#define CUDA_CALL(x) {const cudaError_t a=(x);if(a != cudaSuccess){ printf("\nCUDA
Error: %s (err_num=%d) \n", cudaGetErrorString(a),a);cudaDeviceReset();assert(0);}}
#define KERNEL_LOOP 65536

__constant__ static const u32 const_data_01 = 0x55555555;
__constant__ static const u32 const_data_02 = 0x77777777;
__constant__ static const u32 const_data_03 = 0x33333333;
__constant__ static const u32 const_data_04 = 0x11111111;

__global__ void const_test_gpu_literal(u32 * const data, const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  u32 d = 0x55555555;

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d ^= 0x55555555;
   d |= 0x77777777;
   d &= 0x33333333;
   d |= 0x11111111;
  }

  data[tid] = d;
 }
}

__global__ void const_test_gpu_const(u32 * const data, const u32 num_elements)
```

```
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  u32 d = const_data_01;

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d ^= const_data_01;
   d |= const_data_02;
   d &= const_data_03;
   d |= const_data_04;
  }
  data[tid] = d;
 }
}

__host__ void wait_exit(void)
{
 char ch;

 printf("\nPress any key to exit");
 ch = getch();
}

__host__ void cuda_error_check(
 const char * prefix,
 const char * postfix)
{
 if (cudaPeekAtLastError() != cudaSuccess)
 {
  printf("\n%s%s%s", prefix, cudaGetErrorString(cudaGetLastError()), postfix);
  cudaDeviceReset();
  wait_exit();
  exit(1);
 }
}

__host__ void gpu_kernel(void)
{
 const u32 num_elements = (128*1024);
 const u32 num_threads = 256;
 const u32 num_blocks = (num_elements+(num_threads-1)) / num_threads;
 const u32 num_bytes = num_elements * sizeof(u32);
 int max_device_num;
 const int max_runs = 6;

 CUDA_CALL(cudaGetDeviceCount(&max_device_num));

 for (int device_num=0; device_num < max_device_num; device_num++)
 {
```

```
{
 CUDA_CALL(cudaSetDevice(device_num));

 for (int num_test=0;num_test < max_runs; num_test++)
 {
  u32 * data_gpu;
  cudaEvent_t kernel_start1, kernel_stop1;
  cudaEvent_t kernel_start2, kernel_stop2;
  float delta_time1 = 0.0F, delta_time2=0.0F;
  struct cudaDeviceProp device_prop;
  char device_prefix[261];
CUDA_CALL(cudaMalloc(&data_gpu, num_bytes));
CUDA_CALL(cudaEventCreate(&kernel_start1));
CUDA_CALL(cudaEventCreate(&kernel_start2));
CUDA_CALL(cudaEventCreateWithFlags(&kernel_stop1, cudaEventBlockingSync));
CUDA_CALL(cudaEventCreateWithFlags(&kernel_stop2, cudaEventBlockingSync));

// printf("\nLaunching %u blocks, %u threads", num_blocks, num_threads);
CUDA_CALL(cudaGetDeviceProperties(&device_prop, device_num));
sprintf(device_prefix, "ID:%d %s:", device_num, device_prop.name);

// Warm up run
// printf("\nLaunching literal kernel warm-up");
const_test_gpu_literal <<<num_blocks, num_threads>>>(data_gpu, num_elements);

cuda_error_check("Error ", " returned from literal startup kernel");

// Do the literal kernel
// printf("\nLaunching literal kernel");
CUDA_CALL(cudaEventRecord(kernel_start1,0));
const_test_gpu_literal <<<num_blocks, num_threads>>>(data_gpu, num_elements);

cuda_error_check("Error ", " returned from literal runtime kernel");

CUDA_CALL(cudaEventRecord(kernel_stop1,0));
CUDA_CALL(cudaEventSynchronize(kernel_stop1));
CUDA_CALL(cudaEventElapsedTime(&delta_time1, kernel_start1, kernel_stop1));
//  printf("\nLiteral Elapsed time: %.3fms", delta_time1);

// Warm up run
// printf("\nLaunching constant kernel warm-up");
const_test_gpu_const <<<num_blocks, num_threads>>>(data_gpu, num_elements);

cuda_error_check("Error ", " returned from constant startup kernel");

// Do the constant kernel
// printf("\nLaunching constant kernel");
CUDA_CALL(cudaEventRecord(kernel_start2,0));

const_test_gpu_const <<<num_blocks, num_threads>>>(data_gpu, num_elements);
```

```
    cuda_error_check("Error ", " returned from constant runtime kernel");

    CUDA_CALL(cudaEventRecord(kernel_stop2,0));
    CUDA_CALL(cudaEventSynchronize(kernel_stop2));
    CUDA_CALL(cudaEventElapsedTime(&delta_time2, kernel_start2, kernel_stop2));
    // printf("\nConst Elapsed time: %.3fms", delta_time2);
       if (delta_time1 > delta_time2)
        printf("\n%sConstant version is faster by: %.2fms (Const=%.2fms vs. Literal=
    %.2fms)", device_prefix, delta_time1-delta_time2, delta_time1, delta_time2);
       else
        printf("\n%sLiteral version is faster by: %.2fms (Const=%.2fms vs. Literal=
    %.2fms)", device_prefix, delta_time2-delta_time1, delta_time1, delta_time2);

       CUDA_CALL(cudaEventDestroy(kernel_start1));
       CUDA_CALL(cudaEventDestroy(kernel_start2));
       CUDA_CALL(cudaEventDestroy(kernel_stop1));
       CUDA_CALL(cudaEventDestroy(kernel_stop2));
       CUDA_CALL(cudaFree(data_gpu));
      }

      CUDA_CALL(cudaDeviceReset());
      printf("\n");
     }

     wait_exit();
    }
```

该程序包含了两个 GPU 内核，const_test_gpu_literal 与 const_test_gpu_const。注意这两个内核函数的声明都使用的是 __global__ 前缀，表明这两个函数的范围都是全局范围的。在这两个内核函数的 for 循环中，一个是从常量内存中取值然后对局部变量 d 进行修改，另一个则是通过字面值对局部变量 d 进行赋值计算。完成所有操作之后，再将该值写回到全局内存中。这一点对避免编译器优化掉这段代码是有必要的。

接下来的一段代码是获取得到当前 CUDA 设备的数目，然后通过调用 cudaSetDevice 函数对每个设备进行迭代。注意，在循环的末尾，主机端代码会通过调用 cudaDeviceReset 清除当前上下文。

设置设备之后，程序分配了全局内存，创建了两个事件，一个开始定时器事件，一个停止定时器事件。伴随着内核的调用，这两个事件将被注入到执行流中。因此，执行流将包含一个开始事件、内核调用以及一个停止事件。这两个事件将与 CPU 异步发生，即这些事件不会阻塞 CPU 端代码的执行，并且它们将以并行的方式执行。由于 CPU 定时器无法看到运行时间，这使计时变得有些困难。程序通过调用 cudaEventSynchronize 以等待内核的最后一个事件即停止事件执行完毕。然后通过开始事件和停止事件计算出时间差，这样就计算出了内核的执行时间。

接着对常量内核与字面值内核进行重复调用，为避免初始缓存被填充的影响，这中间还

包含了一个热身函数的调用执行。以下是执行结果：

```
ID:0 GeForce GTX 470:Constant version is faster by: 0.00ms (C=345.23ms, L=345.23ms)
ID:0 GeForce GTX 470:Constant version is faster by: 0.01ms (C=330.95ms, L=330.94ms)
ID:0 GeForce GTX 470:Literal version is faster by: 0.01ms (C=336.60ms, L=336.60ms)
ID:0 GeForce GTX 470:Constant version is faster by: 5.67ms (C=336.60ms, L=330.93ms)
ID:0 GeForce GTX 470:Constant version is faster by: 5.59ms (C=336.60ms, L=331.01ms)
ID:0 GeForce GTX 470:Constant version is faster by: 14.30ms (C=345.23ms, L=330.94ms)

ID:1 GeForce 9800 GT:Literal version is faster by: 4.04ms (C=574.85ms, L=578.89ms)
ID:1 GeForce 9800 GT:Literal version is faster by: 3.55ms (C=578.18ms, L=581.73ms)
ID:1 GeForce 9800 GT:Literal version is faster by: 4.68ms (C=575.85ms, L=580.53ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 5.25ms (C=581.06ms, L=575.81ms)
ID:1 GeForce 9800 GT:Literal version is faster by: 4.01ms (C=572.08ms, L=576.10ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 8.47ms (C=578.40ms, L=569.93ms)

ID:2 GeForce GTX 260:Literal version is faster by: 0.27ms (C=348.74ms, L=349.00ms)
ID:2 GeForce GTX 260:Literal version is faster by: 0.26ms (C=348.72ms, L=348.98ms)
ID:2 GeForce GTX 260:Literal version is faster by: 0.26ms (C=348.74ms, L=349.00ms)
ID:2 GeForce GTX 260:Literal version is faster by: 0.26ms (C=348.74ms, L=349.00ms)
ID:2 GeForce GTX 260:Literal version is faster by: 0.13ms (C=348.83ms, L=348.97ms)
ID:2 GeForce GTX 260:Literal version is faster by: 0.27ms (C=348.73ms, L=348.99ms)

ID:3 GeForce GTX 460:Literal version is faster by: 0.59ms (C=541.43ms, L=542.02ms)
ID:3 GeForce GTX 460:Literal version is faster by: 0.17ms (C=541.20ms, L=541.37ms)
ID:3 GeForce GTX 460:Constant version is faster by: 0.45ms (C=542.29ms, L=541.83ms)
ID:3 GeForce GTX 460:Constant version is faster by: 0.27ms (C=542.17ms, L=541.89ms)
ID:3 GeForce GTX 460:Constant version is faster by: 1.17ms (C=543.55ms, L=542.38ms)
ID:3 GeForce GTX 460:Constant version is faster by: 0.24ms (C=542.92ms, L=542.68ms)
```

通过执行结果我们不难发现，从总的执行时间百分比来看，两个内核的执行时间几乎相同。至于是常量版本的内核更快还是字面值版本的内核更快，从结果中我们看到分布很平均，几乎不相上下。如果与全局内存进行比较又会怎样呢？为了测试这一点，我们将字面值内核中的字面值改为了全局内存数据，代码如下所示：

```
__device__ static u32 data_01 = 0x55555555;
__device__ static u32 data_02 = 0x77777777;
__device__ static u32 data_03 = 0x33333333;
__device__ static u32 data_04 = 0x11111111;

__global__ void const_test_gpu_gmem(u32 * const data, const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  u32 d = 0x55555555;

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d ^= data_01;
```

```
        d |= data_02;
        d &= data_03;
        d |= data_04;
      }

      data[tid] = d;
    }
  }
```

注意，在 GPU 内存空间上声明全局内存变量只需在变量之前加一个 __device__ 说明符即可。内核还是与之前一样，只是循环是对 4 个全局内存值读取 N 次并进行操作。然而，在这个例子中我们将 KERNEL_LOOP 从 64K 改为了 4K，否则内核的执行时间将非常长。因此，在进行时间比较时，需要记住，现在内核只做了之前内核 1/16 的工作。以下是程序得到的结果，非常有趣。

```
ID:0 GeForce GTX 470:Constant version is faster by: 16.68ms (G=37.38ms, C=20.70ms)
ID:0 GeForce GTX 470:Constant version is faster by: 16.45ms (G=37.50ms, C=21.06ms)
ID:0 GeForce GTX 470:Constant version is faster by: 15.71ms (G=37.30ms, C=21.59ms)
ID:0 GeForce GTX 470:Constant version is faster by: 16.66ms (G=37.36ms, C=20.70ms)
ID:0 GeForce GTX 470:Constant version is faster by: 15.84ms (G=36.55ms, C=20.71ms)
ID:0 GeForce GTX 470:Constant version is faster by: 16.33ms (G=37.39ms, C=21.06ms)

ID:1 GeForce 9800 GT:Constant version is faster by: 1427.19ms (G=1463.58ms, C=36.39ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1425.98ms (G=1462.05ms, C=36.07ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1426.95ms (G=1463.15ms, C=36.20ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1426.13ms (G=1462.56ms, C=36.44ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1427.25ms (G=1463.65ms, C=36.40ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1427.53ms (G=1463.70ms, C=36.17ms)

ID:2 GeForce GTX 260:Constant version is faster by: 54.33ms (G=76.13ms, C=21.81ms)
ID:2 GeForce GTX 260:Constant version is faster by: 54.31ms (G=76.11ms, C=21.80ms)
ID:2 GeForce GTX 260:Constant version is faster by: 54.30ms (G=76.10ms, C=21.80ms)
ID:2 GeForce GTX 260:Constant version is faster by: 54.29ms (G=76.12ms, C=21.83ms)
ID:2 GeForce GTX 260:Constant version is faster by: 54.31ms (G=76.12ms, C=21.81ms)
ID:2 GeForce GTX 260:Constant version is faster by: 54.32ms (G=76.13ms, C=21.80ms)

ID:3 GeForce GTX 460:Constant version is faster by: 20.87ms (G=54.85ms, C=33.98ms)
ID:3 GeForce GTX 460:Constant version is faster by: 19.64ms (G=53.57ms, C=33.93ms)
ID:3 GeForce GTX 460:Constant version is faster by: 20.87ms (G=54.86ms, C=33.99ms)
ID:3 GeForce GTX 460:Constant version is faster by: 20.81ms (G=54.77ms, C=33.95ms)
ID:3 GeForce GTX 460:Constant version is faster by: 20.99ms (G=54.87ms, C=33.89ms)
ID:3 GeForce GTX 460:Constant version is faster by: 21.02ms (G=54.93ms, C=33.91ms)
```

通过执行结果我们注意到，所有硬件上常量内存版本的内核都比全局内存版本的内核访问快。在计算能力为 1.1 的硬件（9800GT）上加速比为 40：1，计算能力为 1.3 的硬件（GTX260）上加速比为 3：1。在计算能力为 2.0 的设备（GTX470）上加速比为 1.8：1，计算能力为 2.1 的设备（GTX460）上加速比为 1.6：1。

最有趣的可能是在费米架构的设备（GTX460 与 GTX470）上，通过使用常量内存而不是全局内存所使用的一级或二级缓存，将获得比较显著的加速比。因此，即使是费米架构的设备，通过使用常量内存似乎也能显著地提高吞吐量。然而，真的是这样吗？

为了进一步验证，我们将查看生成的 PTX（虚拟汇编）代码。为了看到这些信息，我们必须使用 -keep 编译器选项。以下是常量内存版本内核的 PTX 代码：

```
.const .u32 const_data_01 = 1431655765;
.const .u32 const_data_02 = 2004318071;
.const .u32 const_data_03 = 858993459;
.const .u32 const_data_04 = 286331153;

.entry _Z20const_test_gpu_constPjj (
 .param .u64 __cudaparm__Z20const_test_gpu_constPjj_data,
 .param .u32 __cudaparm__Z20const_test_gpu_constPjj_num_elements)
{
.reg .u32 %r<29>;
.reg .u64 %rd<6>;
.reg .pred %p<5>;
// __cuda_local_var_108907_15_non_const_tid = 0
// __cuda_local_var_108910_13_non_const_d = 4
// i = 8
.loc 16 40 0
$LDWbegin__Z20const_test_gpu_constPjj:
$LDWbeginblock_181_1:
.loc 16 42 0
mov.u32  %r1, %tid.x;
mov.u32  %r2, %ctaid.x;
mov.u32  %r3, %ntid.x;
mul.lo.u32  %r4, %r2, %r3;
add.u32  %r5, %r1, %r4;
mov.s32  %r6, %r5;
.loc 16 43 0
ld.param.u32  %r7, [__cudaparm__Z20const_test_gpu_constPjj_num_elements];
mov.s32  %r8, %r6;
setp.le.u32  %p1, %r7, %r8;
@%p1 bra  $L_1_3074;
$LDWbeginblock_181_3:
.loc 16 45 0
mov.u32  %r9, 1431655765;
mov.s32  %r10, %r9;
$LDWbeginblock_181_5:
.loc 16 47 0
mov.s32  %r11, 0;
mov.s32  %r12, %r11;
mov.s32  %r13, %r12;
mov.u32  %r14, 4095;
setp.gt.s32  %p2, %r13, %r14;
@%p2 bra  $L_1_3586;
$L_1_3330:
.loc 16 49 0
mov.s32  %r15, %r10;
xor.b32  %r16, %r15, 1431655765;
mov.s32  %r10, %r16;
.loc 16 50 0
```

```
    mov.s32  %r17, %r10;
    or.b32  %r18, %r17, 2004318071;
    mov.s32  %r10, %r18;
    .loc 16 51 0
    mov.s32  %r19, %r10;
    and.b32  %r20, %r19, 858993459;
    mov.s32  %r10, %r20;
    .loc 16 52 0
    mov.s32  %r21, %r10;
    or.b32  %r22, %r21, 286331153;
    mov.s32  %r10, %r22;
    .loc 16 47 0
    mov.s32  %r23, %r12;
    add.s32  %r24, %r23, 1;
    mov.s32  %r12, %r24;
$Lt_1_1794:
    mov.s32  %r25, %r12;
    mov.u32  %r26, 4095;
    setp.le.s32  %p3, %r25, %r26;
    @%p3 bra  $L_1_3330;
$L_1_3586:
$LDWendblock_181_5:
    .loc 16 55 0
    mov.s32  %r27, %r10;
    ld.param.u64  %rd1, [__cudaparm__Z20const_test_gpu_constPjj_data];
    cvt.u64.u32  %rd2, %r6;
    mul.wide.u32  %rd3, %r6, 4;
    add.u64  %rd4, %rd1, %rd3;
    st.global.u32  [%rd4+0], %r27;
$LDWendblock_181_3:
$L_1_3074:
$LDWendblock_181_1:
    .loc 16 57 0
    exit;
$LDWend__Z20const_test_gpu_constPjj:
    } // _Z20const_test_gpu_constPjj
```

我们没有必要准确理解这段汇编代码。此处展示该函数的完整 PTX 代码只是为了让读者对一小段 C 代码如何展开成汇编代码有些认知。PTX 代码按以下这种格式进行书写：

<操作符 >< 目标寄存器 >< 源寄存器 A>< 源寄存器 B>

因此，

```
xor.b32  %r16, %r15, 1431655765;
```

将 15 号寄存器中的值与字面值 1431655765 进行 32 位的按位异或操作，然后将结果值保存到 16 号寄存器中。注意先前 PTX 代码中高亮加粗的数字。编译器将内核中用到的常量值替换为字面值。这就是为什么每当结果与我们预期的不一样时更值得我们深究的原因。为了进行对比，以下是从全局内存获取数值的代码：

```
ld.global.u32  %r16, [data_01];
xor.b32  %r17, %r15, %r16;
```

此时程序是从全局内存中加载数值，而常量内存版本的内核则不需要任何读操作。当编译器将 C 代码转换成 PTX 汇编代码时会将常量值替换成字面值。这一点可以通过将常量以数组的形式声明而不是一些标量变量来解决。因此，新的函数如下所示：

```
__constant__ static const u32 const_data[4] = { 0x55555555, 0x77777777, 0x33333333,
0x11111111 };

__global__ void const_test_gpu_const(u32 * const data, const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  u32 d = const_data[0];

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d ^= const_data[0];
   d |= const_data[1];
   d &= const_data[2];
   d |= const_data[3];
  }

  data[tid] = d;
 }
}
```

对应生成的 PTX 代码如下所示，

```
ld.const.u32  %r15, [const_data+0];
mov.s32  %r16, %r10;
xor.b32  %r17, %r15, %r16;
mov.s32  %r10, %r17;
.loc 16 47 0
ld.const.u32  %r18, [const_data+4];
mov.s32  %r19, %r10;
or.b32  %r20, %r18, %r19;
mov.s32  %r10, %r20;
```

现在，通过来自常量数组首元素的地址进行访问，这一点是我们希望看到的。这样做得到的结果又如何？

```
ID:0 GeForce GTX 470:Constant version is faster by: 0.34ms (G=36.67ms, C=36.32ms)
ID:0 GeForce GTX 470:Constant version is faster by: 1.11ms (G=37.36ms, C=36.25ms)
ID:0 GeForce GTX 470:GMEM  version is faster by: 0.45ms (G=36.62ms, C=37.07ms)
ID:0 GeForce GTX 470:GMEM  version is faster by: 1.21ms (G=35.86ms, C=37.06ms)
ID:0 GeForce GTX 470:GMEM  version is faster by: 0.63ms (G=36.48ms, C=37.11ms)
ID:0 GeForce GTX 470:Constant version is faster by: 0.23ms (G=37.39ms, C=37.16ms)

ID:1 GeForce 9800 GT:Constant version is faster by: 1496.41ms (G=1565.96ms, C=69.55ms)
```

```
ID:1 GeForce 9800 GT:Constant version is faster by: 1496.72ms (G=1566.42ms, C=69.71ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1498.14ms (G=1567.78ms, C=69.64ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1496.12ms (G=1565.81ms, C=69.69ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1496.91ms (G=1566.61ms, C=69.70ms)
ID:1 GeForce 9800 GT:Constant version is faster by: 1495.76ms (G=1565.49ms, C=69.73ms)

ID:2 GeForce GTX 260:Constant version is faster by: 34.21ms (G=76.12ms, C=41.91ms)
ID:2 GeForce GTX 260:Constant version is faster by: 34.22ms (G=76.13ms, C=41.91ms)
ID:2 GeForce GTX 260:Constant version is faster by: 34.19ms (G=76.10ms, C=41.91ms)
ID:2 GeForce GTX 260:Constant version is faster by: 34.20ms (G=76.11ms, C=41.91ms)
ID:2 GeForce GTX 260:Constant version is faster by: 34.21ms (G=76.12ms, C=41.91ms)
ID:2 GeForce GTX 260:Constant version is faster by: 34.20ms (G=76.12ms, C=41.92ms)

ID:3 GeForce GTX 460:GMEM  version is faster by: 0.20ms (G=54.18ms, C=54.38ms)
ID:3 GeForce GTX 460:GMEM  version is faster by: 0.17ms (G=54.86ms, C=55.03ms)
ID:3 GeForce GTX 460:GMEM  version is faster by: 0.25ms (G=54.83ms, C=55.07ms)
ID:3 GeForce GTX 460:GMEM  version is faster by: 0.81ms (G=54.24ms, C=55.05ms)
ID:3 GeForce GTX 460:GMEM  version is faster by: 1.51ms (G=53.54ms, C=55.05ms)
ID:3 GeForce GTX 460:Constant version is faster by: 1.14ms (G=54.83ms, C=53.69ms)
```

此时得到的结果才与我们预期的一样：在费米（计算能力为 2.x 的硬件）架构的设备上，全局内存借助一级缓存也能达到与常量内存相同的访问速度。只有在计算能力为 1.x 的设备上，由于全局内存没有用到缓存技术，此时使用常量内存才会获得明显的性能提升。

6.5.3 运行时进行常量内存更新

GPU 上的常量内存并不是真正意义上的常量内存，因为 GPU 上并没有专门常量内存预留的特殊内存区。由于常量内存是通过 16 位的地址进行访问的，而 16 位地址能够快速进行访问，因此常量内存最大限制为 64KB。这样做会带来一定好处但也会带来一些问题。首先，通过调用 cudaMemcpyToSymbol 函数，常量内存可以按块或片的形式进行更新，一次最多能更新 64K。重新修改我们的常量内存程序，看看它是如何执行的。

```
#include "stdio.h"
#include "conio.h"
#include "assert.h"

typedef unsigned short int u16;
typedef unsigned int u32;

#define CUDA_CALL(x) {const cudaError_t a = (x); if (a != cudaSuccess) { printf
("\nCUDA Error: %s (err_num=%d) \n", cudaGetErrorString(a), a); cudaDeviceReset()
; assert(0);} }
#define KERNEL_LOOP 4096

__constant__ static const u32 const_data_gpu[KERNEL_LOOP];
__device__ static  u32 gmem_data_gpu[KERNEL_LOOP];
static u32 const_data_host[KERNEL_LOOP];

__global__ void const_test_gpu_gmem(u32 * const data, const u32 num_elements)
```

```
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  u32 d = gmem_data_gpu[0];

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d ^= gmem_data_gpu[i];
   d |= gmem_data_gpu[i];
   d &= gmem_data_gpu[i];
   d |= gmem_data_gpu[i];
  }

  data[tid] = d;
 }
}

__global__ void const_test_gpu_const(u32 * const data, const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
 {
  u32 d = const_data_gpu[0];

  for (int i=0;i<KERNEL_LOOP;i++)
  {
   d ^= const_data_gpu[i];
   d |= const_data_gpu[i];
   d &= const_data_gpu[i];
   d |= const_data_gpu[i];
  }

  data[tid] = d;
 }
}

__host__ void wait_exit(void)
{
 char ch;

 printf("\nPress any key to exit");
 ch = getch();
}

__host__ void cuda_error_check(const char * prefix, const char * postfix)
{
 if (cudaPeekAtLastError() != cudaSuccess)
 {
  printf("\n%s%s%s", prefix, cudaGetErrorString(cudaGetLastError()), postfix);
  cudaDeviceReset();
```

```
  wait_exit();
  exit(1);
 }
}

__host__ void generate_rand_data(u32 * host_data_ptr)
{
 for (u32 i=0; i < KERNEL_LOOP; i++)
 {
  host_data_ptr[i] = (u32) rand();
 }
}

__host__ void gpu_kernel(void)
{
 const u32 num_elements = (128*1024);
 const u32 num_threads = 256;
 const u32 num_blocks = (num_elements+(num_threads-1)) / num_threads;
 const u32 num_bytes = num_elements * sizeof(u32);
 int max_device_num;
 const int max_runs = 6;

 CUDA_CALL(cudaGetDeviceCount(&max_device_num));

 for (int device_num=0; device_num < max_device_num; device_num++)
 {
  CUDA_CALL(cudaSetDevice(device_num));

  u32 * data_gpu;
  cudaEvent_t kernel_start1, kernel_stop1;
  cudaEvent_t kernel_start2, kernel_stop2;
  float delta_time1 = 0.0F, delta_time2=0.0F;
  struct cudaDeviceProp device_prop;
  char device_prefix[261];

  CUDA_CALL(cudaMalloc(&data_gpu, num_bytes));
  CUDA_CALL(cudaEventCreate(&kernel_start1));
  CUDA_CALL(cudaEventCreate(&kernel_start2));
  CUDA_CALL(cudaEventCreateWithFlags(&kernel_stop1, cudaEventBlockingSync));
  CUDA_CALL(cudaEventCreateWithFlags(&kernel_stop2, cudaEventBlockingSync));

  // printf("\nLaunching %u blocks, %u threads", num_blocks, num_threads);
  CUDA_CALL(cudaGetDeviceProperties(&device_prop, device_num));
  sprintf(device_prefix, "ID:%d %s:", device_num, device_prop.name);

  for (int num_test=0;num_test < max_runs; num_test++)
  {
   // Generate some random data on the host side
   // Replace with function to obtain data block from disk, network or other
   // data source
```

```
generate_rand_data(const_data_host);

// Copy host memory to constant memory section in GPU
CUDA_CALL(cudaMemcpyToSymbol(const_data_gpu, const_data_host,
        KERNEL_LOOP * sizeof(u32)));
// Warm up run
// printf("\nLaunching gmem kernel warm-up");
const_test_gpu_gmem <<<num_blocks, num_threads>>>(data_gpu, num_elements);
cuda_error_check("Error ", " returned from gmem startup kernel");

// Do the gmem kernel
// printf("\nLaunching gmem kernel");
CUDA_CALL(cudaEventRecord(kernel_start1,0));

const_test_gpu_gmem <<<num_blocks, num_threads>>>(data_gpu, num_elements);

cuda_error_check("Error ", " returned from gmem runtime kernel");

CUDA_CALL(cudaEventRecord(kernel_stop1,0));
CUDA_CALL(cudaEventSynchronize(kernel_stop1));
CUDA_CALL(cudaEventElapsedTime(&delta_time1, kernel_start1, kernel_stop1));
// printf("\nGMEM Elapsed time: %.3fms", delta_time1);

// Copy host memory to global memory section in GPU
CUDA_CALL(cudaMemcpyToSymbol(gmem_data_gpu, const_data_host,
        KERNEL_LOOP * sizeof(u32)));
// Warm up run
// printf("\nLaunching constant kernel warm-up");
const_test_gpu_const <<<num_blocks, num_threads>>>(data_gpu, num_elements);

cuda_error_check("Error ", " returned from constant startup kernel");

// Do the constant kernel
// printf("\nLaunching constant kernel");
CUDA_CALL(cudaEventRecord(kernel_start2,0));

  const_test_gpu_const <<<num_blocks, num_threads>>>(data_gpu, num_elements);

  cuda_error_check("Error ", " returned from constant runtime kernel");

  CUDA_CALL(cudaEventRecord(kernel_stop2,0));
  CUDA_CALL(cudaEventSynchronize(kernel_stop2));
  CUDA_CALL(cudaEventElapsedTime(&delta_time2, kernel_start2, kernel_stop2));
  // printf("\nConst Elapsed time: %.3fms", delta_time2);

  if (delta_time1 > delta_time2)
   printf("\n%sConstant version is faster by: %.2fms (G=%.2fms, C=%.2fms)",
device_prefix, delta_time1-delta_time2, delta_time1, delta_time2);
  else
   printf("\n%sGMEM  version is faster by: %.2fms (G=%.2fms, C=%.2fms)",
device_prefix, delta_time2-delta_time1, delta_time1, delta_time2);
```

```
    else
     printf("\n%sGMEM  version is faster by: %.2fms (G=%.2fms, C=%.2fms)",
device_prefix, delta_time2-delta_time1, delta_time1, delta_time2);
   }

   CUDA_CALL(cudaEventDestroy(kernel_start1));
   CUDA_CALL(cudaEventDestroy(kernel_start2));
   CUDA_CALL(cudaEventDestroy(kernel_stop1));
   CUDA_CALL(cudaEventDestroy(kernel_stop2));
   CUDA_CALL(cudaFree(data_gpu));

   CUDA_CALL(cudaDeviceReset());
   printf("\n");
  }

 wait_exit();
}
```

注意 cudaMemcpyToSymbol 函数的工作原理。该函数可以将数据复制到 GPU 上任何以全局符号命名的内存区域，无论该符号是全局内存还是常量内存。因此，我们可以将一块 64K 大小的数据块复制一个 64K 大小的内存区上，从而通过常量内存缓存进行访问。当所有线程访问同一数据元素时，这种访问方式非常有用，因为我们可以借助缓存技术从常量内存区获取数据然后再广播到每个线程中。

此外，也要注意内存的分配、事件的创建、事件的销毁以及在主循环之外的设备内存释放。从 CPU 的角度来看，这种类型的 CUDA 函数调用通常都非常耗费 CPU 的运行时间。通过简单地改动，该程序 CPU 的负载明显减少了许多。记住总是在最初进行分配操作，在最后进行销毁或释放操作。绝不可以在循环体内部做这些工作，否则应用程序的性能将明显下降。

常量内存问题

有一个大小为 16K 的数据结构，每个线程块以随机的方式对其进行访问，但每个线程束都以统一的方式对其进行访问。该数据结构最好用寄存器、常量内存还是共享内存进行保存？为什么？

常量内存答案

尽管将一个比较大的数组放入寄存器中有些麻烦，但将每个线程的寄存器按块进行划分将获得最快的访问速度，无论访问方式是怎样的。然而，每个 SM 的寄存器大小被限制为 32KB(计算能力小于 1.2 的设备)、64KB(计算能力为 1.2 和 1.3 的设备)、128KB(计算能力为 2.x 的设备）以及 256KB（计算能力为 3.x 的设备)。我们必须将这些寄存器分配给每个线程进行计算。在费米架构的设备上，每个线程最多可分配 64 个寄存器，32 个用来保存数据，另外 32 个作为工作集进行计算，此时我们可以拥有 128 个活跃的线程，即 4 个线程束。只要程序访问片外存储器（如全局内存)，延迟就会阻塞整个 SM。因此，如果要使该方案成为一个好方案，则需要绝大多数的操作都是基于寄存器的。

将该数据结构存入共享内存看起来是最好的解决方案，尽管实际访问时可能产生共享内存存储体冲突。统一的线程束访问允许将从共享内存获取的数据广播到单个线程束中的每个线程。只有当来自两个不同线程块的线程束访问同一个存储体时才会产生共享内存冲突。

然而，16KB 共享内存几乎消耗了计算能力为 1.x 的设备上一个 SM 所有的共享内存。而在计算能力为 2.x 以及 3.x 的硬件上，这会限制每个 SM 最多处理三个线程块。

在计算能力为 1.x 的设备上，常量内存也是一个比较合理的选择。常量内存的广播机制对于线程也有很大帮助。然而，常量内存的缓存无法存下 16KB 数据。另外，更重要的是常量内存的缓存是针对线性访问优化过的，也就是说，每次访问都是基于缓存行获取的。因此，靠近初始访问的访存数据将被缓存。非基于缓存行的访问将导致缓存未命中，造成的损失比直接访问全局内存而未发生缓存命中的损失还要大。

在计算能力为 2.x 和 3.x 的设备上，使用全局内存可能更快一些，因为编译器会将每个线程束的统一访问翻译为线程束级的统一全局内存访问。它提供了本应该在计算能力为 1.x 的设备上支持的常量内存广播访问机制。

6.6 全局内存

全局内存可能是所有类型的内存中最有意思的一种，因为它是我们必须理解的一种存储类型。GPU 的全局内存之所以是全局的，主要是因为 GPU 与 CPU 都可以对它进行写操作。任何设备都可以通过 PCI-E 总线对其进行访问。GPU 之间不通过 CPU，直接将数据从一块 GPU 卡上的数据传输到另一个 GPU 卡上。这种点对点的特性是在 CUDA 4.x SDK 中引入的，并不是所有的平台都支持。目前，只有特斯拉硬件通过 TCC 驱动模型能够支持 Windows 7 和 Windows Vista 平台。而对于 Linux 或 Windows XP 平台，消费级 GPU 卡和特斯拉卡都支持该特性。

CPU 主机端处理器可以通过以下三种方式对 GPU 上的内存进行访问：

- 显式地阻塞传输；
- 显式地非阻塞传输；
- 隐式地使用零内存复制。

GPU 的内存位于 PCI-E 总线的另一端。PCI-E 总线是双向总线，理论上，每个方向的带宽最高可达 8GB/s（PCI-E 2.0），但实际上通常只有 4GB/s ~ 5GB/s。此外，依据所使用的硬件的不同，并不是所有的设备都支持非阻塞传输和隐式内存传输。本书第 9 章将详细介绍这些问题。

通常的执行模型是 CPU 将一个数据块传输到 GPU，GPU 内核对其进行处理，然后再由 CPU 将数据块传输回主机端内存中。比较高级的模型是使用流（稍后将进行介绍）将数据传输和内核执行部分重叠，以保证 GPU 一直在工作，如图 6-16 所示。

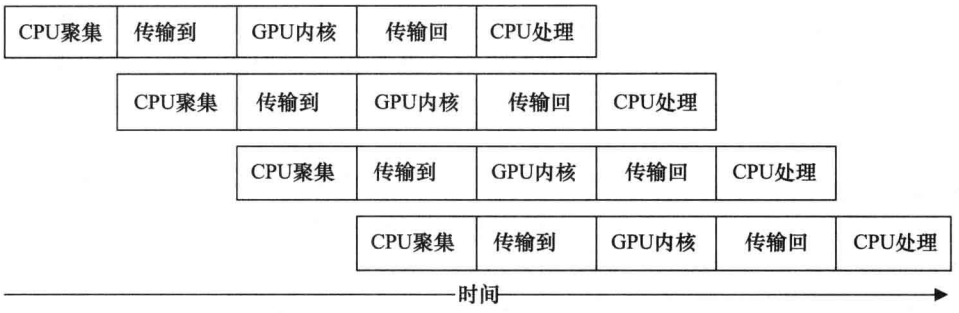


图 6-16 内核执行与内存传输重叠进行

一旦数据进入 GPU，主要问题就成为了如何在 GPU 中对它们高效访问。记住，从计算能力的角度来看，GPU 每秒能执行超过 3 万亿次浮点运算，但主内存的带宽最高只能达到 190GB/s，最低只有 25GB/s。相比之下，英特尔 I7 处理器或 AMD 羿龙 CPU 只能达到 25GB/s ~ 30GB/s，并且根据特殊的设备速度以及使用的存储总线宽度不同还会有一定波动。

图形卡使用的是高速绘图记忆体（GDDR）或者图形动态存储器，具有高度稳定的带宽，但同所有的内存一样，存在严重的延迟。所谓延迟，即从访问开始到返回访问数据的第一个字节所花的时间。因此，我们可以按图 6-16 所示一样将内核流水线化，这种存储访问也就流水线化了。通过创建一个每十次计算只需一次访存的模式，内存延迟能明显地被隐藏，但前提是对全局内存的访问必须是以合并的方式。

究竟什么是合并的访问方式呢？所谓合并访问就是所有线程访问连续的对齐的内存块，如图 6-17 所示。假定以基于字节的方式对数据进行访问，此处显示的 Addr 即从基地址算起的逻辑偏移地址。TID 表示线程标号。如果我们对内存进行一对一连续对齐访问，则每个线程的访问地址可合并起来，只需一次存储事务即可解决问题。假设我们访问一个单精度值或整型值，每个线程将访问一个 4 字节的内存块。内存会基于线程束的方式进行合并（在老式的 G80 硬件上使用半个线程束），也就是说访问一次内存将得到 32 × 4 = 128 个字节的数据。

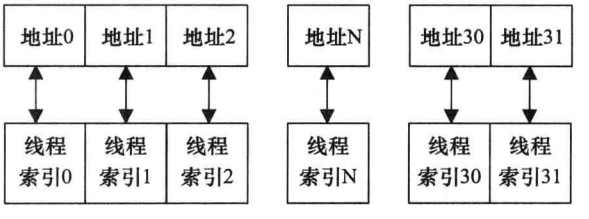


图 6-17 通过线程索引访问地址

合并大小支持 32 字节、64 字节以及 128 字节，分别表示线程束中每个线程以一个字节、16 位以及 32 位为单位读取数据，但前提是访问必须连续，并且是以 32 字节为基准对齐的。

将标准的 cudaMalloc 替换成 cudaMallocPitch，使用这种特殊的分配内存指令可以得到对齐的内存块。其语法如下：

```
extern __host__ cudaError_t CUDARTAPI cudaMallocPitch(void **devPtr, size_t *pitch,
size_t width, size_t height);
```

cudaMallocPitch 的第一个参数表示指向设备内存指针的指针，第二个参数表示指向对齐

之后每行真实字节数的指针，第三个参数为需要开辟的数组的宽度，单位为字节，最后一个参数为数组的高度。

因此，如果有一个数组，行数为 100，每一行有 60 个浮点元素，若使用传统的 cudaMalloc 分配内存，则分得的内存大小为 100 × 60 × sizeof(float) 个字节，即 100 × 60 × 4 = 24 000 个字节。访问数组索引为 [1][0] 的元素（即第一行第零个元素）将导致非合并的访问。这是因为每一行有 60 个元素，其长度为 240 字节，显然不是二的指数倍。

这些线程访问的地址中，第一个地址都无法满足合并对齐的要求。使用 cudaMallocPitch 函数，其将根据当前设备的对齐要求对每一行进行必要的填充（如图 6-18 所示）。在这个例子中，大多数情况下每一行将被填充至 64 个元素，即 256 个字节。设备填充之后每一行的大小由 cudaMallocPitch 中的 pitch 参数返回得到。

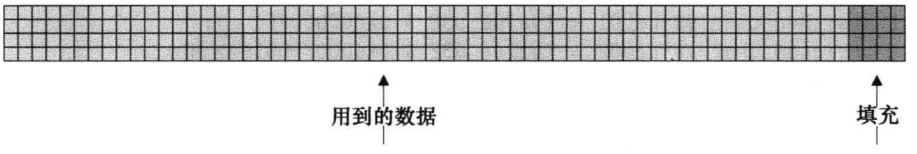


图 6-18　通过 cudaMallocPitch 进行填充

让我们来看其实际是如何执行的。非对齐的访问将导致多次内存获取。当等待内存获取时，线程束中的所有线程将阻塞直到所有的内存获取都从硬件中返回。因此，为了获得最好的吞吐量，我们需通过对齐并且连续合并的访问方式，将大量的内存获取请求合并，达到减少内存获取次数的效果。

如果数据以某种方式交错分布，例如，一个结构体，将会怎样？

```
typedef struct
{
 unsigned int a;
 unsigned int b;
 unsigned int c;
 unsigned int d;
} MY_TYPE_T;

MY_TYPE_T some_array[1024]; /* 1024 * 4 bytes = 4K */
```

图 6-19 显示了 C 语言结构体在内存中的分布情况。

索引0 元素A	索引0 元素B	索引0 元素C	索引0 元素D	索引1 元素A	索引1 元素B	索引1 元素C	索引1 元素D	索引*N* 元素A,B,C,D

图 6-19　内存中的数组元素

结构体中定义的元素以连续的方式分布在内存中。以图 6-20 所示的方式对该结构体进行访问。由图 6-20 可知，被访问的结构体元素在内存中的地址并非连续。这意味着我们无法合并访问，存储带宽将以指数级下降。根据数据元素的大小，每个线程可能读得一个比较大的值，然后在线程内部将必要的位进行屏蔽。例如，如果我们需要基于字节的数据，可以按

以下方式进行操作：

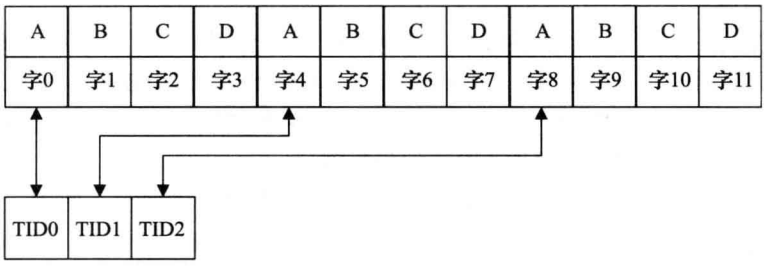


图 6-20 每个线程访问一个元素（非合并访问）

```
const unsigned int value_u32 = some_data[tid];
const unsigned char value_01 = (value_u32 & 0x000000FF)  );
const unsigned char value_02 = (value_u32 & 0x0000FF00) >> 8 );
const unsigned char value_03 = (value_u32 & 0x00FF0000) >> 16 );
const unsigned char value_04 = (value_u32 & 0xFF000000) >> 24 );
```

同样，我们也可以简单地将结构体数组看做一个字的数组，以保持一个线程映射一个数据元素，为结构体的每个元素分配一个线程。然而，这种类型的解决方案并不适合于当需要的数据为结构体中的好几个元素时，例如，1 号线程需要结构体中的 x、y 和 z 坐标。当出现这种情况时，最好对数据重新排列，可能在 CPU 端加载或传输数据阶段就需要将数据划分到 N 个分离的数组。这样，数组就能独立并存于内存中。我们可以直接对数组 a、b、c 或 d 进行访问，而不是像之前那样通过 strcut->a 对结构体元素间接引用访问。相比于交错、非合并的访问方式，现在我们得到了 4 个合并的访问，保持全局内存带宽的使用最优。

让我们一起来看一个读全局内存的例子。在这个例子中，我们会用两种方式将结构体中的每个元素加起来得到总和。首先，我们将对结构体数组的元素进行加和运算，然后对结构体中的数组进行加和运算。

```
// Define the number of elements we'll use
#define NUM_ELEMENTS 4096

// Define an interleaved type
// 16 bytes, 4 bytes per member
typedef struct
{
 u32 a;
 u32 b;
 u32 c;
 u32 d;
} INTERLEAVED_T;

// Define an array type based on the interleaved structure
typedef INTERLEAVED_T INTERLEAVED_ARRAY_T[NUM_ELEMENTS];

// Alternative - structure of arrays
```

```
typedef u32 ARRAY_MEMBER_T[NUM_ELEMENTS];

typedef struct
{
 ARRAY_MEMBER_T a;
 ARRAY_MEMBER_T b;
 ARRAY_MEMBER_T c;
 ARRAY_MEMBER_T d;
} NON_INTERLEAVED_T;
```

在这段代码中，我们声明了两种类型：首先是 INTERLEAVED_T，它表示一个结构体数组，结构体中的成员为 a、b、c 和 d。然后我们声明了 NON_INTERLEAVED_T 作为一个结构体，结构体中的每个元素是一个数组，分别包含了 4 个数组，a、b、c 和 d。如类型名所示，第一种类型的数据我们希望在内存中以交错的方式分布，而第二种类型的数据则是以连续的方式在内存中分布。

首先来看 CPU 版的代码。

```
__host__ float add_test_non_interleaved_cpu(
 NON_INTERLEAVED_T * const host_dest_ptr,
 const NON_INTERLEAVED_T * const host_src_ptr,
 const u32 iter,
 const u32 num_elements)
{
 float start_time = get_time();

 for (u32 tid = 0; tid < num_elements; tid++)
 {
  for (u32 i=0; i<iter; i++)
  {
   host_dest_ptr->a[tid] += host_src_ptr->a[tid];
   host_dest_ptr->b[tid] += host_src_ptr->b[tid];
   host_dest_ptr->c[tid] += host_src_ptr->c[tid];
   host_dest_ptr->d[tid] += host_src_ptr->d[tid];
  }
 }

 const float delta = get_time() - start_time;

 return delta;
}

__host__ float add_test_interleaved_cpu(
 INTERLEAVED_T * const host_dest_ptr,
 const INTERLEAVED_T * const host_src_ptr,
 const u32 iter,
 const u32 num_elements)
{
 float start_time = get_time();

 for (u32 tid = 0; tid < num_elements; tid++)
```

```
 {
  for (u32 i=0; i<iter; i++)
  {
   host_dest_ptr[tid].a += host_src_ptr[tid].a;
   host_dest_ptr[tid].b += host_src_ptr[tid].b;
   host_dest_ptr[tid].c += host_src_ptr[tid].c;
   host_dest_ptr[tid].d += host_src_ptr[tid].d;
  }
 }

 const float delta = get_time() - start_time;
 return delta;
}
```

这两个加和函数明显很相似。每个函数都对列表中所有元素迭代 iter 次，从源数据结构中读取一个值，然后加和到目标数据结构中。每个函数的返回值均为其执行时间。由于这段代码是在 CPU 上执行的，因此我们将用 CPU 系统时间进行计算。

GPU 版本的代码与 CPU 版本的类似，只是内核调用 N 个线程的内核调用，通过 tid 代替了 CPU 版本中的外层循环。

```
__global__ void add_kernel_interleaved(
 INTERLEAVED_T * const dest_ptr,
 const INTERLEAVED_T * const src_ptr,
 const u32 iter,
 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < num_elements)
 {
  for (u32 i=0; i<iter; i++)
  {
   dest_ptr[tid].a += src_ptr[tid].a;
   dest_ptr[tid].b += src_ptr[tid].b;
   dest_ptr[tid].c += src_ptr[tid].c;
   dest_ptr[tid].d += src_ptr[tid].d;
  }
 }
}
__global__ void add_kernel_non_interleaved(
 NON_INTERLEAVED_T * const dest_ptr,
 const NON_INTERLEAVED_T * const src_ptr,
 const u32 iter,
 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < num_elements)
 {
  for (u32 i=0; i<iter; i++)
```

```
    dest_ptr->b[tid] += src_ptr->b[tid];
    dest_ptr->c[tid] += src_ptr->c[tid];
    dest_ptr->d[tid] += src_ptr->d[tid];

  }
 }
}
```

在 GPU 函数的调用者中，数据通过标准的内存复制函数复制到设备中，并在该调用函数中进行时间统计。由于两个函数基本相同，此处只列出了交错版本的调用者函数。

```
__host__ float add_test_interleaved(
 INTERLEAVED_T * const host_dest_ptr,
 const INTERLEAVED_T * const host_src_ptr,
 const u32 iter,
 const u32 num_elements)
{
 // Set launch params
 const u32 num_threads = 256;
 const u32 num_blocks = (num_elements + (num_threads-1)) / num_threads;

 // Allocate memory on the device
 const size_t num_bytes = (sizeof(INTERLEAVED_T) * num_elements);
 INTERLEAVED_T * device_dest_ptr;
 INTERLEAVED_T * device_src_ptr;

 CUDA_CALL(cudaMalloc((void **) &device_src_ptr, num_bytes));

 CUDA_CALL(cudaMalloc((void **) &device_dest_ptr, num_bytes));

 // Create a stop and stop event for timing
 cudaEvent_t kernel_start, kernel_stop;
 cudaEventCreate(&kernel_start, 0);
 cudaEventCreate(&kernel_stop, 0);

 // Create a non zero stream
 cudaStream_t test_stream;
 CUDA_CALL(cudaStreamCreate(&test_stream));

 // Copy src data to GPU
 CUDA_CALL(cudaMemcpy(device_src_ptr, host_src_ptr, num_bytes,
cudaMemcpyHostToDevice));

 // Push start event ahread of kernel call
 CUDA_CALL(cudaEventRecord(kernel_start, 0));

 // Call the GPU kernel
 add_kernel_interleaved<<<num_blocks, num_threads>>>(device_dest_ptr, device_src_ptr,
iter, num_elements);

 // Push stop event after of kernel call
 CUDA_CALL(cudaEventRecord(kernel_stop, 0));
```

```
    // Wait for stop event
    CUDA_CALL(cudaEventSynchronize(kernel_stop));

    // Get delta between start and stop,
    // i.e. the kernel execution time
    float delta = 0.0F;
    CUDA_CALL(cudaEventElapsedTime(&delta, kernel_start, kernel_stop));

    // Clean up
    CUDA_CALL(cudaFree(device_src_ptr));
    CUDA_CALL(cudaFree(device_dest_ptr));
    CUDA_CALL(cudaEventDestroy(kernel_start));
    CUDA_CALL(cudaEventDestroy(kernel_stop));
    CUDA_CALL(cudaStreamDestroy(test_stream));

    return delta;
}
```

执行代码之后得到如下结果：

```
Running Interleaved / Non Interleaved memory test using 65536 bytes (4096 elements)
ID:0 GeForce GTX 470: Interleaved time: 181.83ms
ID:0 GeForce GTX 470: Non Interleaved time: 45.13ms

ID:1 GeForce 9800 GT: Interleaved time: 2689.15ms
ID:1 GeForce 9800 GT: Non Interleaved time: 234.98ms

ID:2 GeForce GTX 260: Interleaved time: 444.16ms
ID:2 GeForce GTX 260: Non Interleaved time: 139.35ms

ID:3 GeForce GTX 460: Interleaved time: 199.15ms
ID:3 GeForce GTX 460: Non Interleaved time: 63.49ms

        CPU (serial): Interleaved time: 1216.00ms
        CPU (serial): Non Interleaved time: 13640.00ms
```

得到的结果很有趣，但同时也是意料之中。在计算能力为 2.x 的硬件上，交错内存访问方式的执行时间比非交错访问方式的时间多出 3 ~ 4 倍。在计算能力为 1.3 的 GTX260 上，交错内存访问方式的执行时间是非交错的三倍，而在计算能力为 1.1 的 9800GT 上则更慢，交错的访问方式慢了 11 倍，这主要是因为在老式的设备上合并的要求更加严格。

通过 Parallel Nsight 这样的分析工具我们可以更深入地理解为什么交错的访问方式如此慢而非交错的访问方式却要快得多。在 Parallel Nsight 中我们可以看到，非交错版本中存储事务（CUDA 内存统计实验）的数量大约为交错版本的 1/4，这意味着非交错版本中内存数据读 / 写操作次数只为交错版本的 1/4。

另一个比较有趣的现象是 CPU 版本的执行时间恰好相反。这看起来很奇怪，但如果了解了访问方式以及缓存重用，你就不会这么觉得了。在交错访问的例子中，CPU 访问元素 a 的同时会将结构体中元素 b、c 以及 d 读入缓存中，使它们在相同的缓存行中。然而，非交

错版本则需要对 4 个独立的物理内存区进行访问。这意味着存储事务的数目为交错版本的 4 倍，并且 CPU 使用的任何预读策略都不会起作用。

因此，如果存在一个 CPU 应用程序需要以交错的方式对结构体中的元素进行处理，可简单地将其复制到 GPU 中执行，但由于低效的内存合并会导致相当大的开销。如果在声明阶段就对数据重新排布，改变访问机制，如本例中所做的那样，通过小小的付出我们将获得显著的加速比。

6.6.1 记分牌

全局内存还有另一个比较有趣的特性，就是其可以使用记分牌。如果我们需要从全局内存中加载一个数据（例如，a=some_array[0]），内存获取指令会被初始化，本地变量 a 被列入即将执行存储事务的列表中。与传统 CPU 代码不同，只有在表达式中用到变量 a 时 GPU 才会真正到内存中取值，因此，在对 a 进行赋值操作时我们根本看不到阻塞或者上下文切换到另一个线程束。GPU 遵循了一种惰性计算模型，只有在真正需要变量内容进行计算时才会到内存中取值。

你可能认为这有点像预订一辆出租车然后再准备离开。准备好离开可能需要五分钟，但出租车到达则需要 15 分钟。在真正需要之前就进行预订，这样我们就可以在出租车到来的这段时间里做准备离开的工作。如果在完成准备离开的工作之后再预订出租车，则使准备离开与等待出租车这两件事串行化了。

上述情况同样也适用于内存事务。存储事务就像缓慢的出租车，需要花费多个 GPU 周期才能完成。在存储事务完成的这段时间内，GPU 可以忙于计算算法的其他方面。为了做到这一点，我们只需简单地将内存获取操作放到内核的开始处，然后再在内核中进行调用。事实上，我们只是将内存获取延迟与其他有用的 GPU 计算指令时间重叠，从而降低了内核中存储延迟带来的影响。

6.6.2 全局内存排序

此处，我们将沿用 6.4 节中用到的排序例子。相同的算法在基于全局内存上执行的效果会如何？我们还需要考虑什么？首先，我们需要考虑存储合并。之前的排序算法是针对共享内存的 32 个存储体特别开发的，其是以列的方式访问共享内存。再次看图 6-8，我们会发现如果所有线程同时进行读操作，同样可以对全局内存合并访问。

合并访问主要出现在基数排序阶段，每个线程对其对应的列表进行遍历。线程的访问被硬件合并（联合）到一起。由于 1 列表大小可能不同，因此写操作无法合并。然而，0 列表的读取和写入都是对相同范围的地址空间进行操作，因此访问可以合并。

合并阶段中，程序最初从全局内存中对每个列表读取一个值到共享内存中。在合并的每轮迭代中，有一个值会被写回全局内存，然后再从全局内存读入一个值到共享内存中以替换之前写回到全局内存的那个值。每次存储访问都执行了很多操作。因此，如果将访问合并，存储延迟将明显得到隐藏。让我们一起来看实际的运行效果如何。

从表 6-13 与图 6-21 中我们可以看到，当线程数为 32 时执行的效果非常好，但在所有的测试设备上，线程数为 64 才是最优的情况，执行的时间都稍少于线程数为 32 的执行时间。这似乎意味着增加一个线程束可以隐藏一定的延迟，少量提升存储带宽的利用率。

表 6-13 单个 SM 全局内存排序（1K 元素）

线 程	GTX470	GTX260	GTX460
1	33.27	66.32	27.47
2	19.21	37.53	15.87
4	11.82	22.29	9.83
8	9.31	16.24	7.88
16	7.41	12.52	6.36
32	6.63	10.95	5.75
64	6.52	10.72	5.71
128	7.06	11.63	6.29
258	8.61	14.88	7.82

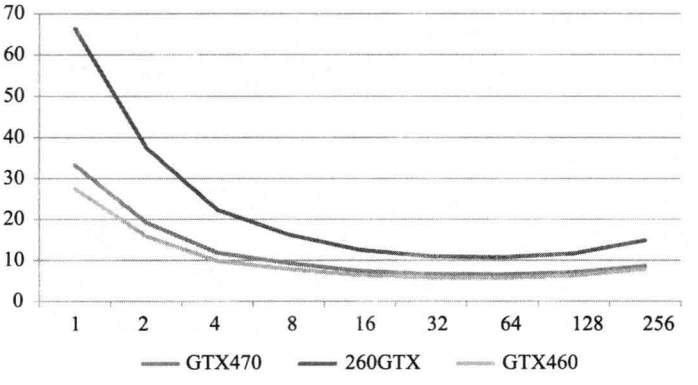


图 6-21 单个 SM 全局内存排序结果图（1K 元素）

当线程数超过 64 时程序性能就开始降低了。因此，如果我们将线程数固定为 64，然后增加数据集的大小，结果又会怎样？表 6-14 以及图 6-22 显示了其结果。事实上，按照当前的做法，使用一个 SM，不同设备得到的结果几乎呈线性关系。

表 6-14 不同大小的数据集全局内存排序

	绝对时间（ms）			每 KB 所花时间（ms）		
大小（kb）	GTX470	GTX260	GTX460	GTX470	GTX260	GTX460
1	1.67	2.69	1.47	1.67	2.69	1.47
2	3.28	5.36	2.89	1.64	2.68	1.45
4	6.51	10.73	5.73	1.63	2.68	1.43
8	12.99	21.43	11.4	1.62	2.68	1.43
16	25.92	42.89	22.75	1.62	2.68	1.42
32	51.81	85.82	45.47	1.62	2.68	1.42

（续）

	绝对时间（ms）			每 KB 所花时间（ms）		
大小（kb）	GTX470	GTX260	GTX460	GTX470	GTX260	GTX460
64	103.6	171.78	90.94	1.62	2.68	1.42
128	207.24	343.74	181.89	1.62	2.69	1.42
256	414.74	688.04	364.09	1.62	2.69	1.42
512	838.25	1377.23	737.85	1.64	2.69	1.44
1024	1692.07	2756.87	1485.94	1.65	2.69	1.45

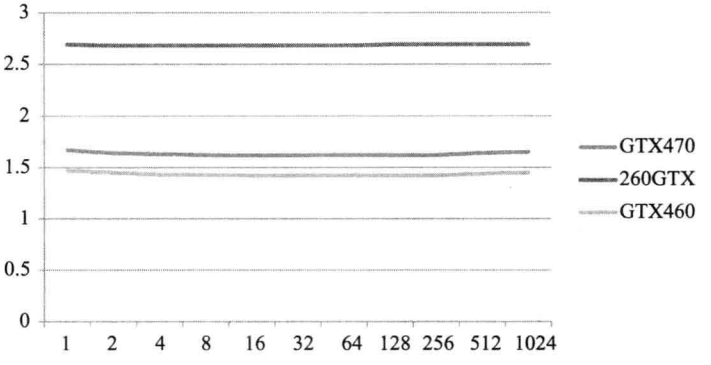


图 6-22 不同大小的数据集全局内存排序结果图

如表 6-14 所示，在 GTX460 上，1024KB（1MB）大小的数据集进行排序花了 1486ms。这意味着对 1MB 的数据排序需要大约 1.5 秒（准确为 1521ms），如果不考虑每个数据的大小，每分钟大约能处理 40MB 数据。

对 1GB 的数据进行排序大概需要 25 ~ 26 分钟，效率并不高。因此该如何解决呢？当前我们只用了一个线程块，SM 的数量也就限制为一个。在进行测试的 GPU 中，GTX470 由 14 个 SM 组成，GTX260 有 27 个 SM，GTX460 有 7 个 SM。显然，每张卡只有少部分被用到了，利用率非常低。这主要是由于内核过于简化。现在，让我们一起来看使用多个线程块会有什么样的效果。

某个 SM 的输出集是单个线性排序列表，因此，两个 SM 的输出集应该是两个线性排序列表，但这并不是我们想要的。考虑如下来自配置 2 个线程块的排序输出记录。原始数值介于 0x01 ~ 0x100，降序排列。以下显示的这些数据中第一个值表示数组索引，第二个为此数组索引所表示的值。

```
000:00000041 001:00000042 002:00000043 003:00000044 004:00000045 005:00000046
006:00000047 007:00000048
008:00000049 009:0000004a 010:0000004b 011:0000004c 012:0000004d 013:0000004e
014:0000004f 015:00000050
016:00000051 017:00000052 018:00000053 019:00000054 020:00000055 021:00000056
022:00000057 023:00000058
024:00000059 025:0000005a 026:0000005b 027:0000005c 028:0000005d 029:0000005e
```

```
030:0000005f 031:00000060
032:00000061 033:00000062 034:00000063 035:00000064 036:00000065 037:00000066
038:00000067 039:00000068
040:00000069 041:0000006a 042:0000006b 043:0000006c 044:0000006d 045:0000006e
046:0000006f 047:00000070
048:00000071 049:00000072 050:00000073 051:00000074 052:00000075 053:00000076
054:00000077 055:00000078
056:00000079 057:0000007a 058:0000007b 059:0000007c 060:0000007d 061:0000007e
062:0000007f 063:00000080

064:00000001 065:00000002 066:00000003 067:00000004 068:00000005 069:00000006
070:00000007 071:00000008
072:00000009 073:0000000a 074:0000000b 075:0000000c 076:0000000d 077:0000000e
078:0000000f 079:00000010
080:00000011 081:00000012 082:00000013 083:00000014 084:00000015 085:00000016
086:00000017 087:00000018
088:00000019 089:0000001a 090:0000001b 091:0000001c 092:0000001d 093:0000001e
094:0000001f 095:00000020
096:00000021 097:00000022 098:00000023 099:00000024 100:00000025 101:00000026
102:00000027 103:00000028
104:00000029 105:0000002a 106:0000002b 107:0000002c 108:0000002d 109:0000002e
110:0000002f 111:00000030
112:00000031 113:00000032 114:00000033 115:00000034 116:00000035 117:00000036
118:00000037 119:00000038
120:00000039 121:0000003a 122:0000003b 123:0000003c 124:0000003d 125:0000003e
126:0000003f 127:00000040
```

这两个排好序的列表中的值分别为 0x41 ~ 0x80 以及 0x01 ~ 0x40。你可能会说这并不是什么大问题，只需要进行将这两个列表再合并一次就好了。但是，这将使我们遇到第二个问题：基于一个线程的内存访问。

假设我们仅使用两个线程，一个线程处理一个列表。0 号线程访问第 0 号元素，1 号线程访问第 64 号元素。对于硬件来说是无法将这两个访问合并的，因此，硬件将执行两次独立的内存获取。

即使我们可以做到合并几乎不消耗时间，假定最多有 16 个 SM，并且每个 SM 都被使用，设备带宽没有溢出，最好的情况也只是每分钟处理 16 × 40MB=640MB 的数据，即每秒处理 10.5MB 的数据。这并不理想，因此，可能我们需要另一种解决方案。

6.6.3 样本排序

样本排序尝试避免了合并排序中的问题，即合并操作。样本排序的工作原理是将数据划分成 N 个数据块，对每个数据块单独排序，并保证第 N 个数据块中的元素比第 N+1 个数据块中的元素数量少，比第 N–1 个数据块中的元素数量大。

首先我们将介绍一个用三个处理器对 24 个数据项进行排序的例子。排序的第一阶段是从数据集中选出 S 个等距的样本。设整个数据集的元素个数为 N，则这 S 个元素是 N 个元素的子集。选出的 S 个元素要具有代表性，这一点很重要。当整个数据集的数据元素分布比较

均匀时，以等距的方式选择样本比较适合。如果数据集是多峰分布，而且这些峰的宽度不太大，这时要么需要选择更多的样本，要么要在已知峰值附近选样本。这里，我们假设样本点均匀分布，所以我们等距地选择样本。

然后对样本数据进行排序，假定按升序排序，使数值最小的元素位于列表的第一位。接着根据可供使用的处理器数目将样本数据划分到不同的桶中。对数据进行扫描然后确定每个桶中有多少个样本。然后将每个桶中的样本数目加起来形成一个前缀和，以用作另一个数组的索引。

前缀和表示当前元素之前所有元素的数目。在本例中，我们看到有九个元素被分配到 0 号桶中。因此，第二个数据集的开始位置是在第 9 个元素。第二个桶同样也被分配了 9 个元素，加上第一个桶中的 9 个元素，总和为 18，这样我们就计算出了下一个数据集的起始索引。之后每个数据集的索引都按这种方法进行计算。

接着数据被打乱，0 号桶中的所有元素作为前缀和的第一个索引（0）写入内存，1 号桶中的元素从下一个前缀和索引对应的地址开始写入，依此类推。这样我们可以对每个桶中的元素单独进行排序，并且第 N–1 号桶中的所有样本值都比第 N 号桶中的值要小，第 N 号桶中的值比第 N+1 号桶中的值小。接下来需要做的就是将这些桶分配给 P 个处理器并行排序。如果使用了原地排序，则只要最后一个数据块排好序，整个数据集就排好序了，其中不需要任何合并操作。图 6-24 显示了本例使用 6 个处理器进行排序时的过程。

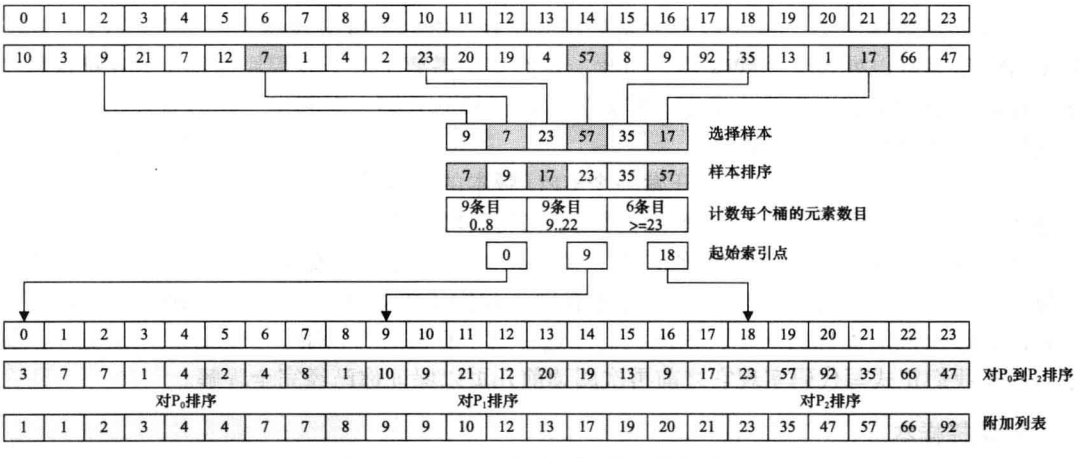


图 6-23 使用 3 个处理器进行样本排序

注意，当基于 6 个样本使用 3 个处理器进行排序时，每个桶的大小分别为 9、9、6。而使用 6 个处理器时，每个桶的大小分别为 6、3、5、4、1、5。事实上，让我们最感兴趣的只有桶大小中的最大值，因为在 P 个处理器上最大块将决定着整个排序所需时间。在本例中，桶元素的最大值由 9 个减少到 6 个，因此增加一倍的处理器数目使得桶大小的最大值减少了 1/3。

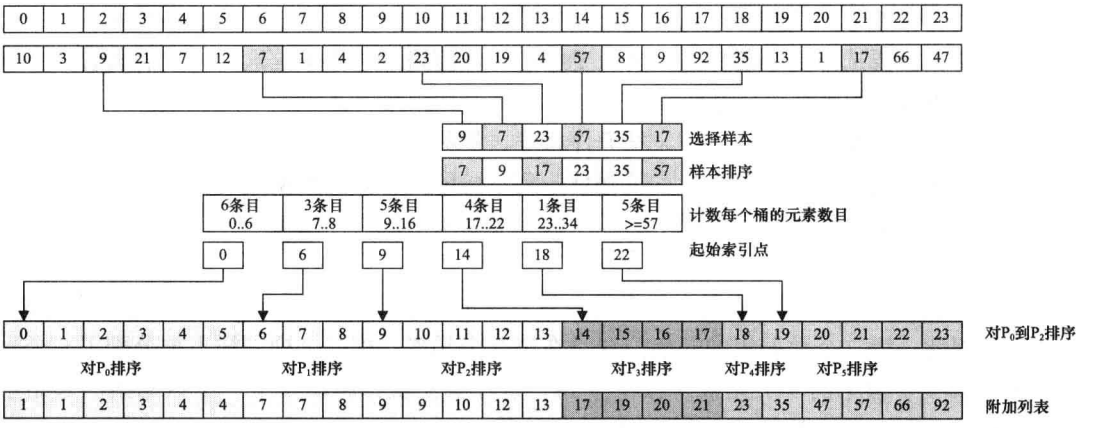


图 6-24 使用 6 个处理器进行样本排序

事实上，该分布很大程度上取决于数据集的分布。最常见的数据集通常是一个几乎排好序的列表或者是一个已经排好序的列表加上几个必须加入的新数据元素。因此，对于大多数数据集而言，分配给每个处理器处理的元素数目几乎是平均分配的。而对于少数数据集，则需要适当调整取样策略。

对于 GPU 而言，我们不仅仅只有 6 个处理器，而是有 N 个 SM，每个 SM 可以处理若干个线程块。如果只是为了最有效地隐藏存储延迟，则每个线程块开启 256 个线程最优，尽管在之前的基数排序中，64 个线程才是最好的选择。在 GTX470 设备上，一共有 14 个 SM，每个 SM 最多可以处理 8 个线程块。因此，我们至少需要 112 个线程块使每个 SM 都保持工作。稍后，我们需要在实践中确定怎样配置才是最好的方案。看起来，我们需要比现在多得多的线程块才能维持负载平衡。

然而，我们的第一个任务是开发一个 CPU 版本的样本排序算法，以便于更好地理解整个过程。我们将依次介绍每一步操作并讨论如何将其转化成并行的解决方案。

为了便于理解后续部分的代码讲解，读者最好已经完全熟悉了我们刚刚介绍的样本排序算法。正是由于该算法比较复杂，它才有助于我们了解如何在 GPU 上解决更困难的实际问题以及涉及性能优化的问题，因此我们才选择这个算法进行讲解。如果你只是浏览了该算法，请在我们正式写代码实现它之前再次阅读前几页以保证你已经完全理解。

1. 选择样本

样本排序的第一部分是从源数据中选择 N 个样本。该部分 CPU 版本使用了一个标准的循环对源数据进行访问，每经过一次循环，索引值增加 sample_interval。而样本索引的计数值每次循环只增加一。

```
__host__ TIMER_T select_samples_cpu(
 u32 * const sample_data,
 const u32 sample_interval,
 const u32 num_elements,
 const u32 * const src_data)
{
```

```
 const TIMER_T start_time = get_time();
 u32 sample_idx = 0;

 for (u32 src_idx=0; src_idx<num_elements; src_idx+=sample_interval)
 {
  sample_data[sample_idx] = src_data[src_idx];
  sample_idx++;
 }

 const TIMER_T end_time = get_time();
 return end_time - start_time;
}
```

在 GPU 版本中，我们使用了一个经典的循环消除方法，即每个线程创建一个样本点，然后扩散到所有的线程块上。因此，首先需要进行如下声明：

```
const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
```

将每个线程块的索引值乘以每个线程块的线程数目，然后加上当前线程的索引值，得到当前线程在所有线程块上的绝对索引值。

```
__global__ void select_samples_gpu_kernel(u32 * const sample_data,
 const u32 sample_interval, const u32 * const src_data)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 sample_data[tid] = src_data[tid*sample_interval];
}

__host__ TIMER_T select_samples_gpu(
 u32 * const sample_data,
 const u32 sample_interval,
 const u32 num_elements,
 const u32 num_samples,
 const u32 * const src_data,
 const u32 num_threads_per_block,
 const char * prefix)
{
 // Invoke one block of N threads per sample
 const u32 num_blocks = num_samples / num_threads_per_block;

 // Check for non equal block size
 assert((num_blocks * num_threads_per_block) == num_samples);

 start_device_timer();

 select_samples_gpu_kernel<<<num_blocks, num_threads_per_block>>>(sample_data,
sample_interval, src_data);
 cuda_error_check(prefix, "Error invoking select_samples_gpu_kernel");
 const TIMER_T func_time = stop_device_timer();

 return func_time;
}
```

最后，为了计算出所选样本数据在源数据中的索引值，我们只简单地将线程绝对索引值乘以样本间隔大小。为了简单起见，我们将只讨论数据集大小正好整除线程块内线程数目的情况。

注意，无论是 CPU 版本还是 GPU 版本，返回值均为操作所需时间，并且在接下来排序的各个阶段中我们将计算各种操作的时间开销。

2. 样本排序

下一步我们需要对选择出的样本进行排序。在 CPU 上，我们可以简单地从标准 C 函数库中调用 qsort（快速排序）方法。

```
__host__ TIMER_T sort_samples_cpu(
 u32 * const sample_data,
 const u32 num_samples)
{
 const TIMER_T start_time = get_time();

 qsort(sample_data, num_samples, sizeof(u32),
   &compare_func);

 const TIMER_T end_time = get_time();
 return end_time - start_time;
}
```

然而，在 GPU 上，标准库中并没有提供该函数，因此我们可以利用之前开发的基数排序。注意，Thrust 库提供了基数排序方法，此处我们不必再将该方法重写一遍，我也不会重复介绍该代码，因为在之前 6.4 节中我们已经仔细地介绍了该方法。

另外需要我们注意的一点是，之前我们开发的基数排序算法是基于单个 SM 上的共享内存的，并且合并阶段在共享内存中进行了归约操作。这并不是最优的方案，但至少在初始测试阶段，我们将仍然使用它。

3. 样本桶计数

接下来我们需要知道每个样本桶中有多少个元素。CPU 版代码如下所示。

```
__host__ TIMER_T count_bins_cpu(const u32 num_samples,
 const u32 num_elements,
 const u32 * const src_data,
 const u32 * const sample_data,
 u32 * const bin_count)
{
 const TIMER_T start_time = get_time();
 for (u32 src_idx=0; src_idx<num_elements; src_idx++)
 {
  const u32 data_to_find = src_data[src_idx];
  const u32 idx = bin_search3(sample_data,
                              data_to_find,
                              num_samples);
  bin_count[idx]++;
```

```
    }
    const TIMER_T end_time = get_time();
    return end_time - start_time;
}
```

计算每个桶的元素个数只需简单地对源数据集迭代，对每个元素调用一个查找函数，找出其应该属于那个桶，然后根据该桶的索引对该桶的计数值做累加操作。

对于查找，我们有两种选择：二分查找或顺序查找。二分查找主要用到了样本列表已经由先前的步骤排好序的特点。它通过将原列表划分成两部分，然后计算出当前查找值应该在数据集的前半部分还是后半部分中。通过反复划分迭代查找，直到最终找到这个值。

二分查找的最坏情形需要的查找次数为 $\log_2 N$。然而在许多情况下我们都将遇到最坏的情形，因为大多数数据都不在样本列表中。因此，当对这两种查找方案进行比较时，我们假定所有情况下我们都遇到最坏的情形。

顺序查找的最坏情形需要查找 N 次，即从列表头开始查找，但每个元素都不等于查找的元素，横向地从列表头查到列表尾。然而，对于一个排好序且数据均匀分布的列表，通常只需做 $N/2$ 次查找。因此，对于一个包含 1024 个元素的样本集，二分查找需要 10 次迭代，而顺序查找则需 512 次。很明显，对于我们涉及的查找空间而言，二分查找才是最优的选择方案。

然而，从合并内存访问以及分支的角度来看，二分查找并不适合 GPU。只要线程束中某个线程产生分支，硬件将产生两个控制路径。我们可能都遇到过这种情况，线程束产生分支以致于每个线程形成完全单独的控制路径。在这种情况下，总体执行时间的计算只需将单个路径执行时间乘以分支线程的数量即可。通常这种情况是在最大的迭代次数，即 $\log_2 N$ 时出现。因此，样本数量要尽量大以抵消线程束中所有线程都产生分支的开销。

每个线程访问的样本在内存中可能处于不同的存储区，因此不存在内存访问合并，在全局内存带宽方面，程序的性能将以指数级下降。事实上，在计算能力为 2.x 的设备上，一级缓存和二级缓存可以有效地隐藏这类影响，但这依赖于样本空间的大小。此外，我们也可以将样本空间保存在共享内存中，同样也可以缓解合并访问问题。

标准的 C 语言库提供了 bsearch 函数，其返回数组中找到的具体值。然而，我们并不需要距离最近的值，而是数组的索引。因此，我们重写了一个二分查找的函数，同时可在 CPU 和 GPU 上执行。注意函数前面指定的 __host__ 与 __device__ 这两个说明符，它们保证了在 CPU 和 GPU 上使用同样的源代码，但非二进制代码是不同的。以下是该函数的源代码。

```
__host__ __device__ u32 bin_search3(
 const u32 * const src_data,
 const u32 search_value,
 const u32 num_elements)
{
 // Take the middle of the two sections
 u32 size = (num_elements >> 1);

 u32 start_idx = 0;
 bool found = false;
```

```
 do
 {
  const u32 src_idx = (start_idx+size);
  const u32 test_value = src_data[src_idx];

  if (test_value == search_value)
   found = true;
  else
   if (search_value > test_value)
    start_idx = (start_idx+size);

  if (found == false)
   size >>= 1;

 } while ( (found == false) && (size != 0) );

 return (start_idx + size);
}
```

二分查找一直查找到直至参数 size 减少为零，其返回值为索引，即查找的元素应该放入哪个桶。

```
// Single data point, atomic add to gmem
__global__ void count_bins_gpu_kernel5(
 const u32 num_samples,
 const u32 * const src_data,
 const u32 * const sample_data,
 u32 * const bin_count)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Read the sample point
 const u32 data_to_find = src_data[tid];

 // Obtain the index of the element in the search list
 const u32 idx = bin_search3(sample_data, data_to_find, num_samples);

 atomicAdd(&bin_count[idx],1);
}

__host__ TIMER_T count_bins_gpu(
 const u32 num_samples,
 const u32 num_elements,
 const u32 * const src_data,
 const u32 * const sample_data,
 u32 * const bin_count,
 const u32 num_threads,
 const char * prefix)
{
 const u32 num_blocks = num_elements / num_threads;
```

```
 start_device_timer();

 count_bins_gpu_kernel5<<<num_blocks, num_threads>>>(num_samples, src_data,
sample_data, bin_count);
 cuda_error_check(prefix, "Error invoking count_bins_gpu_kernel");

 const TIMER_T func_time = stop_device_timer();
 return func_time;
}
```

与选择样本的函数不同，样本选择函数中最大线程数通过样本数进行限制，而此处，我们通过源数组中元素数目对最大线程数进行限制。因此，主机端函数启动内核时，每个线程处理一个元素。

内核函数对每个元素进行计算，并以整齐合并访问的方式将数据从源数据集中读取出来。增加每个线程块的线程数目可提高全局内存的读取带宽。

线程束中每个线程进入二分查找，经过若干轮迭代即可得到结果。对于随机分布的元素列表，有些线程可能产生分支。然而，对于大多数几乎排好序的列表而言，所有的线程基本遵循相同的执行路径，因此实际上并不会产生太多的线程的分支。

当线程束中所有线程通过二分查找返回结果值后，则对全局内存中该值对应的桶计数器进行原子加一操作。无论发出原子指令操作的线程属于哪个 SM，全局内存原子操作均可以彻底地、不被打断地执行完成。因此，我们可以放心地让多个线程对相同地址的存储区进行写操作。很明显，当写操作产生冲突时，这些请求都是顺序执行的，每次只允许一个线程执行写操作。

不幸的是，对于一个几乎排好序的列表而言，由于块是依次分配的，大多数有效块几乎在相似的内存区域中。这意味着在执行写操作时，所有的线程将命中相同的内存区域。稍后我们将看到，对于一个排好序的列表而言，该方案的执行速度会有一定退化，但并不明显。

4. 前缀和

前缀和可以用来创建一个表，保存具有变长记录的数组的索引值。在本例中，每个桶的大小是可变的，每个桶在内存中都是顺序地保存。因此，我们可以计算一个前缀和数组，并保存每个桶的起始位置，例如，数组的 0 号元素保存的是 0 号桶起始位置，等。

前缀和的 CPU 代码比较简单：

```
__host__ TIMER_T calc_bin_idx_cpu(const u32 num_samples,
      const u32 * const bin_count,
      u32 * const dest_bin_idx)
{
 const TIMER_T start_time = get_time();
 u32 prefix_sum = 0;

 for (u32 i=0; i<num_samples; i++)
 {
  dest_bin_idx[i] = prefix_sum;
  prefix_sum += bin_count[i];
 }
```

```
    const TIMER_T end_time = get_time();
    return end_time - start_time;
}
```

此处，我们只需简单地对保存每个桶中元素个数的 bin_count 数组进行迭代。前缀和初始化为零，变量 prefix_sum 保存了当前已计算的所有桶的元素个数总和，每进行一轮迭代，首先将 prefix_sum 保存到前缀和数组中，然后将 prefix_sum 加上当前桶元素的个数。

然而，这段关于前缀和计算的代码有一个严重的问题，即其本质是一个串行问题。我们无法在前面的值还没计算出时计算最后一个前缀和的值。对单个处理器系统而言，通过循环遍历所有元素计算前缀和是一个非常高效的方法。但对于使用多个 SM 的并行方式而言，我们该如何高效地计算前缀和呢？

事实证明，简单的前缀和实现只针对于小规模的元素集高效快速。对于样本元素比较大的集合而言（4 096 以上），如果要更快速计算前缀和，则需要更复杂的解决方案。

我们可以将所有桶划分成一些的块，对每个块进行前缀和计算，以此达到并行的效果。将每个块终结点的前缀和值保存到另一个数组中，然后再对该数组求前缀和，最后将该数组中每个前缀和的结果对应累加到之前每个块中的前缀和中。这样我们就可以利用 GPU 简单地进行前缀和计算了（如图 6-25 所示）。

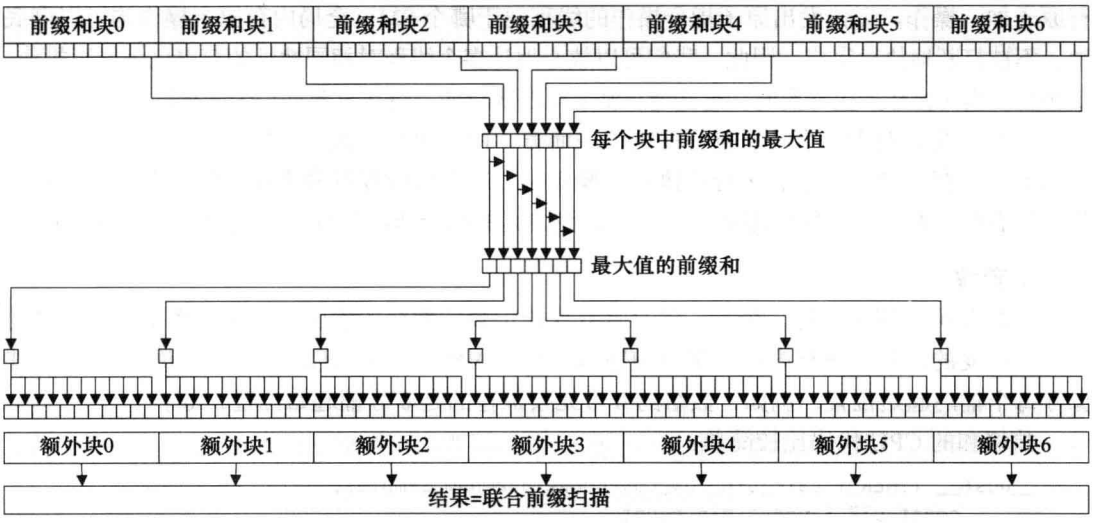


图 6-25 并行前缀和计算

由于每个块由一个线程进行前缀和计算，每个线程处理的元素数目是相同的，因此简单地对块中所有元素进行迭代，不会有线程分支。然而，读取内存访问却无法很好地合并，因为 0 号线程访问的地址偏移从零开始，而 1 号线程访问的地址偏移从（NUM_SAMPLES/NUM_BLOCKS）开始。

如果要在多个 SM 上进行前缀和计算，就意味着需要创建多个块。在这中间需要一次同步操作，只有所有的块完成每个块的前缀和计算之后才能继续计算。因此，我们需要启动一

个内核初始化每个块的前缀和计算，然后用另一个内核对每个块的前缀和最后一个值再做前缀和计算，最后一个内核做加法操作。

这样做对我们非常有帮助，因为它允许我们更改线程块和使用的线程数量。在一个内核中我们可能用一个线程处理一个前缀和块，而在另一个内核中，附加的内核并行度却会受到样本数量的限制。因此，我们可以使用 N 个线程块，每个线程块包含 M 个线程，对 N×M 个样本进行处理，最大化 GPU 的利用率。

与大多数复杂算法一样，选择复杂算法还是简单算法都有一个折中点。对于串行前缀和算法与基于块的前缀和算法而言，这个折中点是 4096 个样本点。我们还可以更近一步，在第一阶段使用更加复杂的前缀和计算方法。但除非我们的数据集足够大，否则前缀和的计算并不会成为整个排序时间的关键因素。

以下是 GPU 版的代码，让我们一起来仔细阅读这段代码。

```
__global__ void calc_prefix_sum_kernel(
 const u32 num_samples_per_thread,
 const u32 * const bin_count,
 u32 * const prefix_idx,
 u32 * const block_sum)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 const u32 tid_offset = tid * num_samples_per_thread;
 u32 prefix_sum;

 if (tid == 0)
  prefix_sum = 0;
 else
  prefix_sum = bin_count[tid_offset-1];

 for (u32 i=0; i<num_samples_per_thread; i++)
 {
  prefix_idx[i+tid_offset] = prefix_sum;
  prefix_sum += bin_count[i+tid_offset];
 }

 // Store the block prefix sum as the value from the last element
 block_sum[tid] = prefix_idx[(num_samples_per_thread-1uL)+tid_offset];
}
```

首先，我们基于线程块索引与线程索引计算得到绝对索引 tid。然后根据每个线程需要进行前缀和计算的样本数目计算得到 tid_offset。

接着我们对第一块的前缀和初始化为零，而其他块的前缀和初始化为上一个块的最后一个元素值。之后的操作与先前 CPU 版本实现相同，只不过索引增加了 tid_offset 的偏移，使得每个线程能读 / 写对应的元素。

```
__global__ void add_prefix_sum_total_kernel(
 u32 * const prefix_idx,
 const u32 * const total_count)
{
```

```
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 prefix_idx[tid] += total_count[blockIdx.x];
}
```

加法操作的内核非常简单。内核函数只需要计算出每个线程对应的绝对索引 tid，利用该值作为目标数组的索引。增加的值为当前块之前所有块中桶的总数。在该实现中，我们私下假定调用者将启动 N 个线程，其中 N 表示之前的内核中每个线程处理的样本数目。这样做我们就可以使用 blockIdx.x（线程块索引）而不是线程索引进行访问，它可以使获取的数据存放到标准常量内存缓存中，然后触发广播机制，将数据广播到线程块中所有线程中。

此外，我们还需要一个简单的前缀和内核对少量的元素进行处理。由于并行版本的前缀和计算需要先对每个块进行部分前缀和计算，然后再做加法操作，中间还需要同步，因此消耗的时间比较长。只有在块的大小很大时，为了充分利用硬件以得到明显的加速比，我们才会选择更加复杂的计算方式。

```
__global__ void calc_prefix_sum_kernel_single(
 const u32 num_samples,
 const u32 * const bin_count,
 u32 * const dest_bin_idx)
{
 u32 prefix_sum = 0;

 for (u32 i=0; i<num_samples; i++)
 {
  dest_bin_idx[i] = prefix_sum;
  prefix_sum += bin_count[i];
 }
}
```

最后是主机端串行化调用内核函数代码：

```
__host__ TIMER_T calc_bin_idx_gpu(
 const u32 num_elements,
 const u32 * const bin_count,
 u32 * const dest_bin_idx,
 const u32 num_threads_per_block,
 u32 num_blocks,
 const char * prefix,
 u32 * const block_sum,
 u32 * const block_sum_prefix)
{
 start_device_timer();

 if (num_elements >= 4096)
 {
  const u32 num_threads_total = num_threads_per_block
                                * num_blocks;

  const u32 num_elements_per_thread = num_elements / num_threads_total;
```

```
  // Make sure the caller passed arguments which correctly divide the elements to blocks
and threads
  assert( (num_elements_per_thread *
           num_threads_total) == num_elements );

  // First calculate the prefix sum over a block
  calc_prefix_sum_kernel<<<num_blocks,
num_threads_per_block>>>(num_elements_per_thread,bin_count, dest_bin_idx, block_sum);

  cuda_error_check(prefix, "Error invoking calc_prefix_sum_kernel");

  // Calculate prefix for the block sums
  // Single threaded
  calc_prefix_sum_kernel_single<<<1,1>>>(num_threads_total, block_sum,
block_sum_prefix);
  cuda_error_check(prefix, "Error invoking calc_prefix_sum_kernel_single");

  // Add the prefix sums totals back into the original prefix blocks
  // Switch to N threads per block
  num_blocks = num_elements /
               num_elements_per_thread;
  add_prefix_sum_total_kernel<<<num_blocks, num_elements_per_thread>>>(dest_bin_idx,
block_sum_prefix);

  cuda_error_check(prefix, "add_prefix_sum_total_kernel");
 }
 else
 {
  // Calculate prefix for the block sums
  // Single threaded
  calc_prefix_sum_kernel_single<<<1,1>>>(num_elements, bin_count, dest_bin_idx);

  cuda_error_check(prefix, "Error invoking calc_prefix_sum_kernel_single");
 }
 const TIMER_T func_time = stop_device_timer();
 return func_time;
}
```

在该函数中我们首先验证了简单的前缀和算法与复杂的前缀和算法哪个更适合用来计算。对于复杂的解决方案，首先计算出每个线程初始需要处理的元素数目，然后顺序地调用三个内核。我们可以根据需要对参数 num_threads_per_block 与 num_blocks 进行调整。

样本元素的数目为 4K 到达了选择这两种算法的过渡点，此时简单版本的前缀和算法与复杂版本的前缀和算法的速度基本相同，但当元素数目继续增加时，复杂算法的执行速度更快。当样本数目增加到 16K 时，复杂版本的算法比简单版本的快 4 倍。

5. 挑选数据到每个桶

为了避免合并操作，所有样本必须预先分拣到 N 个桶中。这至少需要对整个数组遍历一遍，将数据分配到正确的桶中。以下是 CPU 版本的代码：

```
__host__ TIMER_T sort_to_bins_cpu(
 const u32 num_samples,
 const u32 num_elements,
 const u32 * const src_data,
 const u32 * const sample_data,
 const u32 * const bin_count,
 const u32 * const dest_bin_idx,
 u32 * const dest_data)
{
 const TIMER_T start_time = get_time();
 u32 dest_bin_idx_tmp[NUM_SAMPLES];

 // Copy the dest_bin_idx array to temp storage
 for (u32 bin=0;bin<NUM_SAMPLES;bin++)
 {
  dest_bin_idx_tmp[bin] = dest_bin_idx[bin];
 }

 // Iterate over all source data points
 for (u32 src_idx=0; src_idx<num_elements; src_idx++)
 {
  // Read the source data
  const u32 data = src_data[src_idx];

  // Identify the bin in which the source data
  // should reside
  const u32 bin = bin_search3(sample_data,
                              data,
                              num_samples);

  // Fetch the current index for that bin
  const u32 dest_idx = dest_bin_idx_tmp[bin];

  // Write the data using the current index
  // of the correct bin
  dest_data[dest_idx] = data;

  // Increment the bin index
  dest_bin_idx_tmp[bin]++;
 }

 const TIMER_T end_time = get_time();
 return end_time - start_time;
}
```

源数组中的每个数据点都需要放入在内存中线性排布的 N 个桶中。每个桶在整个数组上的起始和终止偏移已由之前的操作求出。我们需要保留这一数据，并同时创建一个大小为 N 的数组跟踪每个桶当前的索引指针。索引数据的初始化只需复制保存有前缀和索引的 dest_bin_idx 数组即可。

然后我们需要对整个源数据进行迭代。对每个源数据进行二分查找，确定其应放入哪个桶中，最后将该数据复制至相应桶中并对该桶的索引指针进行加一操作。

接下来需要做的是将该算法转化成并行的解决方案，转化的过程中我们仍然会遇到多个线程尝试对同一数据项进行写操作的问题。对该问题可以用两种方法进行解决。第一种方法是将数据划分到N个独立的块中，每个块单独处理，最后合并到输出集中。在之前内核的前缀和的并行解决方案中我们就用到了这种方法。此外，还有另一种方法可供我们使用，此处我们将使用这第二种方法。

```
__global__ void sort_to_bins_gpu_kernel(
 const u32 num_samples,
 const u32 * const src_data,
 const u32 * const sample_data,
 u32 * const dest_bin_idx_tmp,
 u32 * const dest_data)
{
 // Calculate the thread we're using
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Read the sample point
 const u32 data = src_data[tid];

 // Identify the bin in which the
 // source data should reside
 const u32 bin = bin_search3(sample_data,
                             data,
                             num_samples);

 // Increment the current index for that bin
 const u32 dest_idx = atomicAdd(&dest_bin_idx_tmp[bin],1);

 // Write the data using the
 // current index of the correct bin
 dest_data[dest_idx] = data;
}
```

该方法使用了原子操作，使得在大多数情况下实现更加简单，但必须以性能作为代价。当然，未来我们也可以用数据划分再合并数据的算法代替原子操作。至于选择哪种复杂性的算法编程实现，这是一个折中的问题，复杂性越高可能意味着需要更长的开发时间以及更多的错误，但性能却仅能得到少量提升。如果你有充足的时间，可以两种方案都尝试一下。无论如何至少需要提供一种解决方案，使其能够在旧式的、不支持原子操作的设备上运行。

基于原子操作的sort_to_bins_gpu_kernel函数将CPU版中的对源数据的循环访问展开到N个并行的线程中。每个线程处理一个元素，通过线程索引与线程块索引的结合计算得到对应的元素索引。

每个线程从源数据中读取得到相应的元素，然后在样本数据空间上进行二分查找，找到该元素对应的桶号。然而，我们需要单个线程来增加该元素对应桶的索引指针。我们不能像

CPU 版代码那样直接做加一操作。

```
// Increment the bin index
dest_bin_idx_tmp[bin]++;
```

我们应该用 atomicAdd 原子调用进行替代：

```
// Increment the current index for that bin
const u32 dest_idx = atomicAdd(&dest_bin_idx_tmp[bin],1);
```

当对全局内存上的数据使用 atomicAdd 函数时，则需要第二个形参，本例中为 1，将该值加到第一个参数所指地址的值中。如果多个线程同时调用该函数，我们也能保证加法操作能正确的完成。atomicAdd 函数的返回值为做加法操作之前保存的值。因此，我们可以用该返回值作为数组的索引将新数据写入对应桶的相应位置中。

然而，该算法会改变每个桶中元素的位置，因为所有的块是无序的。因此，这并不是简单的内存复制操作，因为多个线程可能同时对相同的桶执行写操作。此外，还需要注意的一点是对于一个几乎排好序的列表，绝大多数线程可能同时访问相同的原子地址空间。正如我们所料，与均匀分布的列表相比，这可能导致执行更加缓慢。

6. 对每个桶排序

将数据挑选到每个桶之后，我们需要以并行的方式对每个桶单独进行排序。在 CPU 上，我们可以对每个桶使用 qsort（快速排序），而在 GPU 上，我们将用基数排序。

```
__host__ TIMER_T sort_bins_gpu(
 const u32 num_samples,
 const u32 num_elements,
 u32 * const data,
 const u32 * const sample_data,
 const u32 * const bin_count,
 const u32 * const dest_bin_idx,
 u32 * const sort_tmp,
 const u32 num_threads,
 const char * prefix)
{
 start_device_timer();

 const u32 num_blocks = num_samples / num_threads;

 sort_bins_gpu_kernel3<<<num_blocks, num_threads>>>(num_samples, num_elements, data,
sample_data, bin_count, dest_bin_idx, sort_tmp);

 cuda_error_check(prefix, "Error invoking sort_bins_gpu_kernel");

 const TIMER_T func_time = stop_device_timer();
 return func_time;
}
```

我们使用一个主机端函数启动 num_samples 个线程，根据每个块需要的线程数将这些线

程划分若干块。

```
__global__ void sort_bins_gpu_kernel3(
 const u32 num_samples,
 const u32 num_elements,
 u32 * const data,
 const u32 * const sample_data,
 const u32 * const bin_count,
 const u32 * const dest_bin_idx,
 u32 * const sort_tmp)
{
 // Calculate the thread we're using
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid != (num_samples-1))
  radix_sort(data, dest_bin_idx[tid], dest_bin_idx[tid+1], sort_tmp);
 else
  radix_sort(data, dest_bin_idx[tid], num_elements, sort_tmp);
}
```

由于 dest_bin_idx 数组只保存了起始索引，因此该内核分两层。对于最后一个元素，通过 [tid+1] 作为索引进行访问会造成溢出。因此最后一个线程的处理稍有不同。

我们将对第 5 章开发的基数排序进行修改，利用该修改版本对多个块进行排序。

```
__device__ void radix_sort(
 u32 * const data,
 const u32 start_idx,
 const u32 end_idx,
 u32 * const sort_tmp_1)
{
 // Sort into num_list, lists
 // Apply radix sort on 32 bits of data
 for (u32 bit=0;bit<32;bit++)
 {
  // Mask off all but the bit we're interested in
  const u32 bit_mask = (1u << bit);

  // Set up the zero and one counter
  u32 base_cnt_0 = start_idx;
  u32 base_cnt_1 = start_idx;

  for (u32 i=start_idx; i<end_idx; i++)
  {
   // Fetch the test data element
   const u32 elem = data[i];

   // If the element is in the one list
   if ( (elem & bit_mask) > 0u )
   {
    // Copy to the one list
    sort_tmp_1[base_cnt_1++] = elem;
```

```
        }
        else
        {
         // Copy to the zero list (inplace)
         data[base_cnt_0++] = elem;
        }
       }

       // Copy data back to source from the one's list
       for (u32 i=start_idx; i<base_cnt_1; i++)
       {
        data[base_cnt_0++] = sort_tmp_1[i];
       }
      }
     }
```

基数排序只是简单地对给定块的数据集进行迭代，根据每一位将数据划分到 0 列表或 1 列表中。调用者给定了数组需要排序部分的起始索引和终止索引。

7. 结果分析

将样本大小设为 16K，对一个基本排好序且大小为 1MB 的源数据集进行测试，得到的结果如下所示。

```
ID:3 GeForce GTX 460: Test 32 - Selecting 16384 from 1048576 elements using 512 blocks
of 32 threads
Select Sample Time - CPU: 0.19 GPU:0.03
Sort Sample Time   - CPU: 2.13 GPU:125.57
Count Bins Time    - CPU: 157.59 GPU:17.00
Calc. Bin Idx Time - CPU: 0.03 GPU:0.58
Sort to Bins Time  - CPU: 163.81 GPU:16.94
Sort Bins Time     - CPU: 72.06 GPU:64.46
Total Time         - CPU: 395.81 GPU:224.59
Qsort Time         - CPU: 185.41 GPU:N/A

ID:3 GeForce GTX 460: Test 32 - Selecting 16384 from 1048576 elements using 256 blocks
of 64 threads
Select Sample Time - CPU: 0.53 GPU:0.03
Sort Sample Time   - CPU: 2.06 GPU:125.57
Count Bins Time    - CPU: 157.75 GPU:19.07
Calc. Bin Idx Time - CPU: 0.13 GPU:0.26
Sort to Bins Time  - CPU: 164.09 GPU:19.09
Sort Bins Time     - CPU: 72.31 GPU:62.11
Total Time         - CPU: 396.88 GPU:226.13
Qsort Time         - CPU: 184.50 GPU:N/A

ID:3 GeForce GTX 460: Test 32 - Selecting 16384 from 1048576 elements using 128 blocks
of 128 threads
Select Sample Time - CPU: 0.28 GPU:0.03
Sort Sample Time   - CPU: 2.09 GPU:125.57
```

```
Count Bins Time     - CPU: 157.91 GPU:13.96
Calc. Bin Idx Time - CPU: 0.09 GPU:0.26
Sort to Bins Time  - CPU: 164.22 GPU:14.00
Sort Bins Time      - CPU: 71.19 GPU:91.33
Total Time          - CPU: 395.78 GPU:245.16
Qsort Time          - CPU: 185.19 GPU:N/A

ID:3 GeForce GTX 460:Test 32 - Selecting 16384 from 1048576 elements using 64 blocks of
256 threads
Select Sample Time - CPU: 0.22 GPU:0.03
Sort Sample Time    - CPU: 2.00 GPU:125.57
Count Bins Time     - CPU: 158.78 GPU:12.43
Calc. Bin Idx Time - CPU: 0.13 GPU:0.49
Sort to Bins Time  - CPU: 164.38 GPU:12.39
Sort Bins Time      - CPU: 71.16 GPU:84.89
Total Time          - CPU: 396.66 GPU:235.80
Qsort Time          - CPU: 186.13 GPU:N/A
```

注意，GPU 上的整个排序过程（224 ~ 245ms）中，绝大部分时间在执行样本数据集的排序（~125ms）。随着样本数据集越来越大，此阶段使用归并排序策略的执行效率将越来越低。

针对这个问题，我们可以通过对样本数据使用样本排序来解决。当样本集很大时，使用该方案非常高效。然而，对于一个 16K 的样本集，GPU 样本排序需要 9ms，而 CPU 使用快速排序只需要 2ms。

当给定一个解决方案，充分合理利用所有设备能使程序更加高效快速。对于小规模样本集，CPU 往往比 GPU 运行得更快。但当数据集的大小增加超过某个值后，GPU 的执行效率将比 CPU 高。因此，最优的解决方案应该是在 CPU 上对样本集进行快速排序，然后再将数据传输到 GPU 进行剩余的大规模并行排序工作。

当使用这种解决方案时，执行时间下降非常明显。

```
ID:3 GeForce GTX 460: Test 32 - Selecting 16384 from 1048576 elements using 512 blocks
of 32 threads
Select Sample Time - CPU: 0.09 GPU:0.09
Sort Sample Time    - CPU: 2.09 GPU:2.09
Count Bins Time     - CPU: 157.69 GPU:17.02
Calc. Bin Idx Time - CPU: 0.09 GPU:0.58
Sort to Bins Time  - CPU: 163.78 GPU:16.94
Sort Bins Time      - CPU: 71.97 GPU:64.47
Total Time          - CPU: 395.72 GPU:101.19
Qsort Time          - CPU: 184.78 GPU:N/A
```

我们看到，当样本大小为 16K 时，GPU 上整个样本排序的时间大约为 CPU 上快速排序的 55%(GPU：101ms,CPU：185ms)。如果改变样本集大小，增加并行度，得到的结果表 6-15 与图 6-26 所示。

表 6-15 样本排序结果（ms）

样本数 / 设备	256	512	1024	2048	4096	8K	16K	32K	64K	128K	256K
快速排序	184	184	184	184	184	184	184	184	184	184	184
GTX460	506	273	158	151	115	105	101	101	69	64	85
GTX470	546	290	161	94	91	72	62	60	43	46	68
GTX260	1082	768	635	485	370	286	215	190	179	111	88

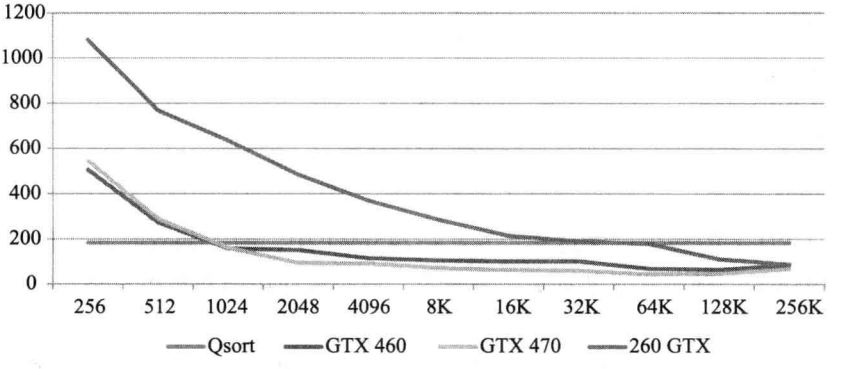


图 6-26 样本排序结果图

由表 6-15 与图 6-26 可知，当样本集大小增加时，执行时间明显下降。在 GTX460 上，当样本集大小为 128K 时，即排序数据集大小的 1/8 时，执行时间最短。由于 GTX470 拥有更多的 SM，当样本集大小超过 2048 时，GTX470 的执行速度明显超过 GTX460。相比之下，对于前一代的设备 GTX260 而言，只有处理更大的样本集，其性能才能赶上费米架构设备的性能。

当样本集大小为 128K 时样本排序的时间再次成为整个排序过程中最耗时的阶段（见表 6-16），我们所使用的在 CPU 上进行快速排序的策略成为了性能提升的瓶颈。仔细观察 GTX470 的测试结果，我们发现当样本集大小为 256K 时，相对于 128K 的样本集，样本排序阶段的执行时间增加了 50%。此时，通过样本排序对样本数据进行处理才是更好的解决方案（参见表 6-16）。

表 6-16 GTX470 样本排序结果（几乎有序的数据集）

样本数 / 操作	256	512	1024	2048	4096	8K	16K	32K	64K	128K	256K
选择样本	0	0.03	0	0.09	0.06	0.28	0.5	0.5	0.84	0.97	1.78
样本排序	0	0.06	0.6	0.28	0.38	0.97	2.03	4.34	9.38	19.72	41.72
桶计数	14.6	14.2	13.62	12.38	10.66	9.34	9.26	8.38	6.03	5.35	5.38
前缀和	0.13	0.27	0.53	1.05	0.19	0.33	0.62	1.2	0.57	0.87	1.46
选择元素到桶	14.6	14.2	13.65	12.4	10.7	9.37	9.33	8.45	6.09	5.41	5.46
桶排序	517	261	133.5	68.29	69.86	52.5	40.9	37.2	20.2	13.94	12.15
总时间	546	290	162	94	92	73	63	60	43	46	68

为了对几乎排好序的数据集与完全随机的数据集进行对比，我们同样在一个随机的数据

集上进行了测试（参见表 6-17）。测试表明当每个线程块启动 128 个线程时获得的性能最优。

表 6-17 针对随机数据的样本排序

样本数 / 设备	256	512	1024	2048	4096	8K	16K	32K	64K	128K	256K
快速排序	337	337	337	337	337	337	337	337	337	337	337
GTX460	735	470	235	139	92	91	75	73	81	108	160
GTX470	831	535	263	156	97	70	77	68	67	90	155
GTX260	1311	919	463	255	170	124	106	100	106	123	178

由表 6-17 我们可以看到，在 GTX470 上最快的执行时间为 67ms。这比 CPU 端执行串行的快速排序要快 5 倍。针对 1MB 的数据集，样本集大小选择 32K，每个线程块开启 128 个线程得到性能最优，参见图 6-27。

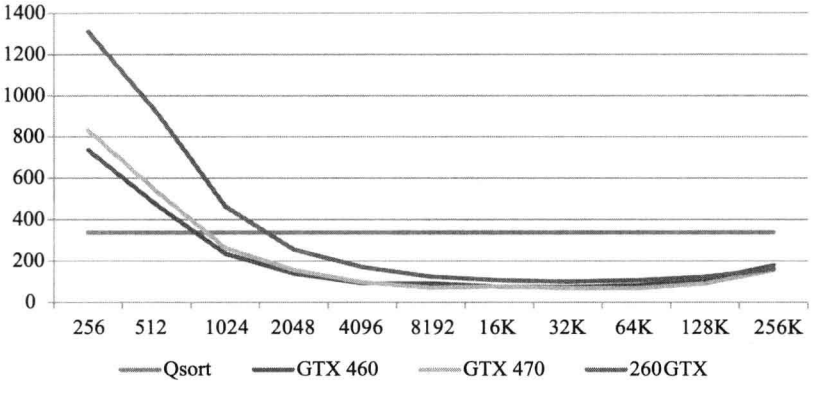


图 6-27 针对随机数据的样本排序结果图

关于全局内存的问题

1. 讨论为什么对几乎有序的数据集使用快速排序更快？

2. 针对当前的样本排序算法你将如何改进？

3. 当处理更大的数据集时，你能预测到哪些问题？为了处理更大的数据集，哪些部分必须改动？

关于全局内存的答案

1. 当列表几乎有序时样本排序更快，这主要是因为线程分支明显减少了。每个桶中的元素数量基本相同。只要数据集足够大，能够选出足够多的样本，就能合理用到更多线程块，每个 SM 的任务分配就接近最优。

2. 该算法的一个关键问题是在基数排序阶段，全局内存的访问是非合并的方式。这主要是前缀和计算在每个块的上下界之间进行原地排序造成的结果。如果不对样本集进行划分，

就会同共享内存基数排序的例子那样，大量线程交错访问每个样本，这样大部分排序就能合并访问了。唯一的缺点就是由于有的列表条目多，而有的少，因此会潜在地浪费空间。

另一个显著的解决方案就是完善样本排序阶段。当样本数目为 128K 时，样本排序阶段的时间占总排序时间的 43%。然而，实际中，我们并不会用到如此多的样本，一般实际中 32K 的样本数据更加理想。此时，样本排序时间只占了总时间的 7%（参见表 6-16）。对每个桶排序的时间占的比重最大，为总时间的 62%，选择数据到每个桶的时间为 14%，对桶计数的排序时间为 14%。基数排序显然是应该最先考虑的。

3. 随着数据规模的逐渐增大，很快我们就会达到单个维度上允许的最大线程块的数量限制（计算能力为 2.x 以及更低的平台上为 65 535）。这种情况下，我们需要将各种不同的内核调用中的 num_block 计算转换成 dim3 类型，以使线程块的布局包含 x 与 y 两个维度，如果数据集足够大，可能还会用到多个线程网格维度。当然，线程块包含 x、y 两个维度之后我们需要对内核进行修改，从而根据线程块维度与线程网格大小计算得到正确的线程块索引。

6.7 纹理内存

本书中我们不会对纹理内存进行详细的介绍。然而，我们将介绍关于纹理内存的一些特殊用法以便读者可以在自己的应用程序中用到。使用纹理内存主要基于以下两个目的：

- 计算能力为 1.x 与 3.x 硬件上的缓存机制
- 基于硬件的内存读取操作

6.7.1 纹理缓存

由于在计算能力为 1.x 的硬件上根本没有高速缓存，因此，每个 SM 上的 6 ~ 8KB 的纹理内存为此设备提供了唯一真正缓存数据的方法。然而，随着费米架构的硬件出现，一级缓存最大可达 48KB，共享二级缓存最大可达 768KB，使得纹理缓存的这项属性基本被淘汰。然而，费米架构的设备依然保留着纹理缓存以保证对旧式硬件上的代码兼容。

纹理缓存通常用来做局部优化，即它希望数据提供给连续的线程进行操作。这与费米架构设备上一级缓存的缓存机制基本相同。除非你是为了使用纹理内存其他方面的特性，否则在费米架构设备上，对纹理内存大量编程并不会获得很高的效益。然而，在开普勒架构的设备上，纹理内存使用了一种特殊的计算方式，使得对其编程不再那么复杂，详见第 12 章中对开普勒架构设备的介绍。注意，在计算能力为 1.x 的硬件上，若所有线程访问同一内存地址，常量内存的缓存是所有缓存中唯一一种可广播访问的。

然而，在计算能力为 1.x 的硬件设备上，纹理缓存被广泛使用。如果一次内存读取具有一定局部性，那么使用纹理缓存则可以省去大量存储访问。假设我们需要对内存进行聚集操作（gather operation），即将一系列非连续内存地址中的数据读取到 N 个线程中。除非线程的访问方式符合内存对齐要求并且访问连续的内存单元，否则硬件合并访问机制将导致多次内存读取。如果通过纹理内存加载数据，多数访问将在纹理缓存中多次命中，这样会使性能得

到明显的提升。

此外，为达到这个目的，我们同样可以使用共享内存。首先以合并的访问方式从全局内存中读取数据，然后在对共享内存上的数据执行读取操作。由于在计算能力为 1.x 的设备上共享内存的大小被限制为 16K，因此，为达到一个特殊的目的，我们可以分配一块共享内存，或者在内存访问方式并不确定的情况下使用纹理内存。

6.7.2 基于硬件的内存获取操作

纹理内存的第二个、几乎也是最有用的一个特性就是当访问存储单元时，其允许 GPU 某些硬件方面的自动实现。

一个比较有用的方面就是基于硬件的低分辨率线性插值。通常，当函数的输出不太满足要求或数学计算代价昂贵时会使用线性插值。有时，我们可能需要对从感应器传入的输入数据在其最大值与最小值之间进行校正。与其对感应器的输入进行建模，我们可以简单地将大量的点存入一个表中，代表其区间范围内的一些离散值。通过将这些点线性地落入真实的数据点中来模拟近似值。

假设插值表如下所示：

```
P = 10, 20, 40, 50, 20
X =  0,  2,  4,  6,  8
```

对于 X 中的一个新值 3，应该用什么值插入到 P 中？由于 3 落在 2 与 4 之间，2 对应 P 中的 20，4 对应 P 中的 40，因此，可以简单地计算得到 3 应该对应 30，参见图 6-28。

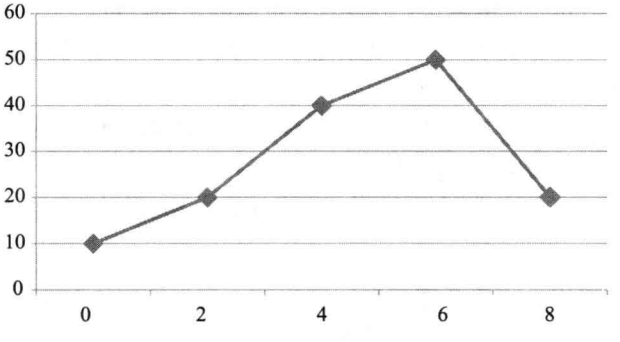


图 6-28 插值

如果使用纹理内存，我们可以通过将 P 定义成归一化到 0 ~ 1 或 -1 ~ +1 的数组来解决这个问题。获取到的数据可以通过硬件用来自动插值。将此特性与缓存特性联合使用，可以快速处理数据，而这对于纯计算操作来说很难做到。此外，对于二维数组与三维数组，分别可以支持硬件级双线性插值与三线性插值。

纹理的另一个比较实用的特性是其可以根据数组索引自动处理边界条件。我们可以在数组边界按照环绕方式或夹取方式来对纹理数组进行处理。这一点非常有用，因为通常情况下它不需要通过嵌入特殊的边缘处理代码就可以对所有元素进行处理。特殊情况的处理代码通

常会导致线程分支的产生，并且对于具有缓存特性的费米架构设备而言，这样做根本没有必要（参见第 9 章的相关优化议题）。

6.7.3 使用纹理的限制

纹理一词来源于 GPU 图形世界，相对于其他标准 CUDA 类型，纹理的灵活性更差一些。纹理必须声明为固定类型，例如，在编译时声明为各种对齐向量类型中的一种（u8、u16、u32、s8、s16、s32），其值在运行时解释执行。对于 GPU 内核而言，纹理内存是只读内存，并且只有通过特殊的纹理 API（例如，texiDfetch() 等）与纹理数组边界才能对其访问。

通常，纹理在计算能力为 1.x 的设备上使用的比较多。由于纹理的使用非常特殊，有时花费一定时间去了解其 API，然后用其编程并不值得。因此，本节中我们并没有在 API 方面进行介绍，而只是简单地对纹理的基本使用进行了描述。对于纹理内存，只有在应用程序真正需要的时候对其进行了解，我们的主要精力应该放在精通掌握全局内存、共享内存以及寄存器的使用上。

若想对纹理进一步了解，请阅读《CUDA C 编程指南》。

6.8 本章小结

本章我们对 GPU 上各种存储系统的使用进行了详细的介绍。无论是在 CPU 上还是在 GPU 上，内存吞吐量对程序的性能起着决定性影响。读者应该正确理解局部性原理（例如，越靠近设备的数据，访问的速度越快）以及片外资源的访问代价。

深入理解三种主要的存储类型——寄存器、共享内存以及全局内存，这样我们就能在编写程序中正确高效使用各种存储类型。

对于全局内存，我们需要思考出一种合理的方式以便能对数据合并访问，尽量减少设备对内存子系统再次发出访问操作的次数。

当需要将相同数据分配到多个线程中，或者将相同数据分配到多个线程块的线程中时，考虑使用常量内存。

对于共享内存的使用，需要考虑数据的重用性。如果数据不被重用，则直接将数据从常量内存或全局内存读入寄存器即可。如果数据存在一定重用性或者需要更多的寄存器空间，此时可考虑使用共享内存。

尽量使用寄存器，尽量将数据声明为局部变量。当数据需要多次访问或重复利用时可考虑使用寄存器。通过将数据先写回寄存器最后写回全局内存可避免多次全局内存写操作。寄存器是唯一一种可获得全部设备吞吐量的存储方式，但同样也是具有重要价值又极度缺乏的资源。注意避免过度使用寄存器以致寄存器的数据溢出到本地内存中，从而导致程序的性能下降。

了解了各种内存类型的原理之后，在接下来的章节中，我们将进一步解释优化以及这些内存类型在实际中如何使用。

第 7 章

CUDA 实践之道

7.1 简介

本章将通过介绍一些不常见的 GPU 使用案例，使读者深入理解如何解决不同类型的计算问题。并研究一些将使用 GPU 进行解决的计算问题。

7.2 串行编码与并行编码

7.2.1 CPU 与 GPU 的设计目标

CPU 与 GPU 虽然都是用来执行程序的，但是它们的设计目标却大不相同。CPU 使用 MIMD（多指令、多数据）的解决方案，而 GPU 采用 SIMT（单指令线程、多线程）指令模型。

CPU 并行的方法是执行多个独立的指令流，并在这些指令流中尝试提取出指令级并行。因此，CPU 的指令流水线非常长，CPU 从指令流水线中找出可以送到独立执行单元的指令。这些执行单元通常包括至少一个浮点单元、至少一个整数单元、一个分支预测单元和至少一个加载 / 存储单元。

计算机架构师从事分支预测（branch prediction）已超过十年。分支的主要问题是单指令流分成两个指令流，其中一个执行，另一个不执行。例如 for、while 这样的循环程序结构，每当一次循环结束，分支就会重新回到循环的开始，如此重复，直到结束。由此可见，在很多情况下，分支是可以静态预测的。一些编译器通过对分支指令设置一个比特位来判断分支条件是否满足。因此，循环语句中的跳转通常预测为“采取”（taken）（即跳回起始点），而其中的条件判断通常预测为“不采取”（not taken）。这样做可以完全避免分支，同时也增加了优势，即后续指令已预提取到了缓存。

分支预测从简单但高效的静态模型演化成为记录先前分支历史的动态模型。由于错误预测分支会导致重新填充指令流水线，代价高昂，因此，现代处理器都采用了多层的复杂分支预测机制。

同分支预测技术一样，一种叫预测执行的技术也被采纳。考虑到 CPU 可能正确预测分

支，那么在此分支地址开始执行指令流就是正确的。然而，这增加了错误预测分支的成本，一旦分支预测错误，当前已经执行的指令流必须回退或丢弃。

对于分支预测与预测执行，可以通过简单地将每个分支都执行，然后在已知真实的分支之后再提交计算结果来进行优化。由于分支通常是嵌套的，使用这种方法需要多层硬件，因此该方法很少用到。

直到目前为止，CPU 与 GPU 的一个主要区别就是缓存等级、数量的不同。CPU 程序模型工作在良好的抽象原则下，程序员不需要关心内存在哪，因为硬件会解决这个问题。除了那些需要运行很快的程序，对于大部分程序而言，这种方法已足够使用。过去，指令周期很长，但随着芯片密度的不断增长，指令周期缩短了。现在，内存访问成了现代处理器设计的主要瓶颈，目前一般通过多级缓存应对解决。

然而，GPU 选择了另一种设计方式，以费米架构的 GPU 为例。费米架构的设计者相信程序员会充分利用用高速内存，将数据放在距离处理器近的地方，因此，他们在每个 SM 上都设计了一个共享内存。共享内存类似于传统处理器上的一级缓存，是一小块低延时、高带宽的内存。如果考虑大部分程序，这样做会很有意义。程序员比其他人更了解程序，因此应该清楚哪些片外内存访问可以通过使用共享内存避免。

费米架构将其片上存储空间扩展为 64K，其中 16K 必须分配给一级缓存，还有一部分总是作为共享内存的形式存在。它不允许将整个空间都分配给一级缓存或者共享内存。费米架构默认分配 48K 为缓存，16K 为共享内存，但也可以反过来分配 48K 为共享内存，16K 为缓存。开普勒额外引进了 32K/32K 的分法。在没有利用共享内存的程序中，用一级缓存替代共享内存，这样的设置可以减少内核函数受内存限制而导致的性能下降。这种设置需要调用：

```
cudaFuncSetCacheConfig(cudaFuncCachePreferL1);
```

在本章后面将要讨论优化的样本排序程序中，这个简单的改变会减少 15% 的整体执行时间。由于启用了默认关闭的特性，我们收到了巨大的回馈。

从内存获取数据的角度来看，通过使用一级缓存，GPU 和 CPU 彼此变得更相近了。对于早先的 GPU，内存获取会进行合并以获得一定性能的提升。考虑在基于 G80 和 GT200 硬件上的不合并内存获取。如果线程 0 从内存地址 0x1000 读取数据，线程 1 从 0x2000 读取数据，线程 2 从 0x3000 读取数据等。这样每次内存访问，每个线程会获取 32 字节的数据。注意，不是 32 位而是 32 字节，这是最小内存存取单位。接着访问（0x1004、0x2004、0x3004 等）也完全相同。

同 CPU 一样，费米架构的 GPU 将每次内存访问获取的 128 字节的数据放入一个缓存行（cache line）中，随后相邻线程的访问通常会命中缓存，即在缓存中找到需要的数据，避免了再次访问全局内存。这使编程模型更加灵活，也更加接近大多数程序员所熟悉的 CPU 编程模型。

GPU 设计与 CPU 设计的一个显著不同就是 SIMT 执行模型。在 MIMD 模型中，每个线程都有独立的硬件来操作整个独立的指令流。如果线程执行相同的指令流但不同的数据，那

么这种方法非常浪费硬件资源。而 GPU 提供了一组硬件来运行这 N 个线程，N 正好为当前线程束的大小 32。

这种模型对 GPU 程序设计有重大影响。GPU 中的 SIMT 模型实现类似于原先的矢量架构 SIMD 模型。但该模型在 70 年代初期被废弃，由于串行 CPU 持续增速，使得基于 SIMD 机器上的艰难设计不再具有吸引力。SIMT 模型解决了一个关键的问题，就是程序员不必再为每个执行相同路径的线程写代码。线程可以产生分支然后在之后的某一点汇集。但由于只有一组硬件设备执行不同的程序路径，因此程序的灵活性有所下降，不同程序路径在控制流汇集之前顺序或轮流执行。作为一个程序员，必须意识到这点，并在设计内核函数时考虑到这一点。

最后，我们来看 CPU 和 GPU 之间另一个显著的不同。CPU 采用的是顺序控制流执行方式。如果执行一个需要很多个周期的指令，将会阻塞当前的线程。因此，Intel 采用了超线程技术。当正在运行的线程受阻时，硬件会内部切换到另外一个线程。但 GPU 不止还有 1 个线程，而是设计有成千上万个线程可以切换。单线程情形，当遇到这种由指令延迟与存储延迟引起的阻塞时，CPU 处理器会闲置等待操作完成。而线程模型的设计就是用来隐藏这两种延迟的。

此外，由于 GPU 使用了惰性计算模型（lazy evaluation model），这使它拥有了另一个优势，即它不用阻塞当前线程，除非需要访问一个有依赖性的寄存器。因此，我们可以在内核中较早地请求读入一个值到寄存器，只有该寄存器被用到时线程才会被阻塞。CPU 模型在遇到内存加载或长延迟的指令时会阻塞，进行读取。考虑下面的程序段：

程序段 1：

```
int sum=0;
for (int i=0; i< 128; i++)
{
 sum += src_array[i];
}
```

在第一段代码中程序必须先计算 src_array[i] 的地址，然后载入数据，最后加在 sum 的现有值上。每个操作都独立于先前的操作。

程序段 2：

```
int sum=0;
int sum1=0, sum2=0, sum3=0, sum4=0;
for (int i=0; i< 128; i+=4)
{
 sum1 += src_array[i];
 sum2 += src_array[i+1];
 sum3 += src_array[i+2];
 sum4 += src_array[i+3];
}
sum = sum1 + sum2 + sum3 + sum4;
```

但在第二段代码中，每次迭代执行 4 次操作。4 个独立的 sum 值被用到，这需要硬件计

算 4 个独立的加法。操作的并行度取决于处理器上的可执行单元的数量。这些处理单元可以是多核处理器执行单元（以使用线程的处理器核的形式），或者是大规模处理器设计中的执行单元。

程序段 3：

```
int sum=0;
int sum1=0, sum2=0, sum3=0, sum4=0;
for (int i=0; i< 128; i+=4)
{
 const int a1 = src_array[i];
 const int a2 = src_array[i+1];
 const int a3 = src_array[i+2];
 const int a4 = src_array[i+3];

 sum1 += a1;
 sum2 += a2;
 sum3 += a3;
 sum4 += a4;
}
sum = sum1 + sum2 + sum3 + sum4;
```

最后，在代码段 3 中，我们将由内存中读取的数据从计算步骤中分离。因此，在 a1 的读取操作之后还有 3 个加载操作，还有数组下标的增加计算，都是在其参与 sum1 计算之前执行的。

在 CPU 的热情计算模型（eager evaluation model）中，当第一次向 a1 读入值时就会产生阻塞，后面的读取操作也是。而在 GPU 的惰性计算模型中，如果数据当前无法使用，只有数据当前不可用时才会产生阻塞，随后的三段代码也是如此。大多数 CPU 和 GPU 设计都是超标量处理器，使用流水线指令，这使得它们在使用单线程时获益。

7.2.2 CPU 与 GPU 上的最佳算法对比

数十年来，计算机科学开发出了成百上千种算法，其中有许多还针对串行 CPU 进行了优化。但并不是所有的算法都适合用来实施并解决并行问题。然而，绝大多数问题都可以以一种或多种方式体现并行。许多问题可以分解成在数据集上的操作。如果从数据或任务并行的角度看，多数情况下，这些操作是内在可并行的。

并行工作中一个很重要的算法就是扫描，或者叫前缀求和。在串行的计算中这是不需要的，因而也不存在。假如一些函数输出的元素数量不相同，我们可以对每个输出分配固定大小的存储空间，例如数组，但这也就意味着在内存中会有缝隙。0 号输出可能产生 10 个实体，1 号输出 5 个，3 号输出 9 个。分配的数组的大小需要至少能存 10 个实体，因此有 6 个空间的浪费。

在每个单独的数组中，前缀求和将存入每个输出涉及的元素数目。实际数据被压缩（空白空间被移除）以形成一个单一的线性数组。现在的问题是把线程 2 输出的值写在哪？为计算每个输出的索引，可以简单地预先累加所有的输出。这样，2 号输出必须写到数组下标为

10 的位置，因为 1 号输出写了 10 个元素（0 ~ 9）。2 号输出写了 5 个元素（10 ~ 14），因此 3 号输出会在第 15 个元素的位置开始写，依此类推。

在第 6 章中涉及了一个使用前缀求和的样本排序的例子，此处，我们将不再重新介绍它是如何并行计算的，而是要着重理解使用前缀和可以将很多算法转化成 N 个独立的输出。不受原子操作的限制、独立地写每个输出，即避免资源竞争，是非常重要的。一旦存在限制，将严重影响内核执行的速度，具体结果取决于超负载的程度。

并不是所有并行架构都是等同的。许多并行程序和并行语言是针对 MIMD 模型的，即线程是独立的，不需要像在 GPU 上那样成组地执行。因此，不是所有的并行程序都能在 GPU 上不做改变就工作。实际上，到目前为止这一直都是并行程序的一个问题，对具体架构的优化通常需要将应用程序与特定的硬件相绑定。

诸如 MPI 和 OpenMP 的标准十分不适合 GPU 模型。OpenMP 可能更相近些，因为它需要共享的内存。在 OpenMP 中，编译器负责产生共享一块公共数据区的线程。程序员通过使用不同的预编译指令指定哪些循环需要使用并行化，编译器会处理所有复杂的并行计算。而对于 MPI，所有的程序都是相同的，更适合集群式计算而不适合单个节点的机器计算。

也许你会为每一个 MPI 进程分配一个 GPU 线程或一个线程块。除非可以确定所有 MPI 的程序组都执行相同的执行流并且可以在 GPU 上将 MPI 程序组合并成线程束，否则这两种方案都无法在 GPU 上很好地运行。通常，MPI 实现成 CPU 与 GPU 混用的模式，这里使用 CPU 处理网络通信以及硬盘的输入输出（I/O）。通过一块共享内存，可以使用 GPU Direct 实现页面直接将数据传输到 InfiniBand 网卡上。应该采用直接通过 PCI-E 总线点对点（P2P）地进行数据传送，无须使用 CPU 的主内存。RDMA(远程 DMA）是新开普勒架构中的一个特性，它使得 GPU 在网络中成了一个更加独立的节点。

随着 GPU 越来越多地被配置到数据中心和超级计算机，OpenMP 和 MPI 将会不可避免地顺应针对加速计算而设计的硬件。在第 10 章中，我们将讨论 OpenACC 的使用，这是一种基于指令的 GPU 计算方法。OpenMP4ACC（OpenMP 加速器）标准可以很好地将这样的指令移入主流 OpenMP 标准中。

在使用 GPU 解决一个问题时需要考虑使用的线程数量是有限制的。一般来说，费米与开普勒架构的设备中一个线程块最多能开启 1024 个线程，早期的设备能开启的线程数更少。事实上，由于受寄存器使用的限制，一般合理且复杂的内核函数会将线程数限制为 256 或 512。线程间通信的问题是分解问题的关键。线程间的通信通常通过使用高速共享内存实现，同一块内的线程可以快速通信，并且延迟很小。相比之下，线程块之间的通信只能通过重新调度单独的内核和使用全局内存来实现。但使用全局内存速度会慢一个数量级。开普勒对此模型进行了扩展，不使用共享内存就可以实现线程束之间的通信。

使用 GPU 计算时另一个需要考虑的主要因素是设备可提供的内存。单个 GPU 能提供的最大内存是 TeslaM 2090 显卡上的 6GB 内存大小。这与在一般主机上 16 ~ 64GB 的内存相比，显然太小。然而，这个问题可以通过使用多个 GPU 解决，利用可以容纳 4 个 PCI-E 显卡的高端主板，可以使每个节点最多能提供 24GB 字节的 GPU 内存。

另外，GPU 上的递归也有它的盲点。只有计算能力为 2.x 的 GPU 才支持递归运算，并且只有带 __device__ 前缀的函数可以使用，带 __global__ 前缀的函数无法使用。未来的开普勒 K20 上的动态并行特性将会在很多方面对递归算法产生帮助。

许多 CPU 算法利用了递归。递归可以很容易地将问题分解成越来越小的问题直到成为一个非常简单的问题。二分查找就是一个典型的利用递归的例子。二分查找将一些排好序的数字分成两组，并判断需要查找的数是在左边那组还是在右边那组，一直这样重复划分直到找到需要的数或者划分的数据集只剩两个元素，这样就很容易地解决了问题。

然而，任何递归算法都可以描述成迭代算法。刚提到的二分查找在第 6 章样本排序的例子中就采取了迭代解决方案。快速排序也是一个常见的算法例子，它通常也采用递归方式实现。该算法选择了一个基准点，把小于或等于这个基准点的数放在左侧，大于的放在右侧。然后得到两个独立的数据集可以通过两个单独的线程进行排序。下一次迭代就可以使用 4 个线程，然后 8 个，16 个等。

GPU 内核函数调用时需要的线程数量是固定的，尽管这在开普勒 K20 正式发布之后可能有所改变，但现在仍不能实现动态并行。动态并行的并行度随时间而改变。在快速排序问题中，它以 2 的指数级增长。而在寻找路径的问题中，发现一个新节点可能会引入 30 000 多个其他路径。

然而，在 GPU 上该如何实现这些算法呢？有许多方法，最简单的就是当知道并行度扩展的方式时，例如快速排序，可以简单地调用一个内核，这个内核执行算法迭代中的第一个层级或第 N 个层级，将这些内核一个接着一个送入一个执行流。当某一层级结束时，将其状态信息写回全局内存，然后选择下一层级的内核执行。由于所有的内核都压入流中，随时准备执行，中间不需要 CPU 的干预就能启动下一个流，如图 7-1 所示。

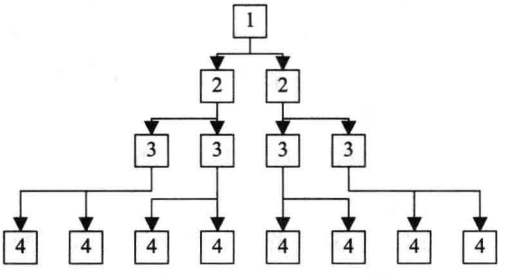


图 7-1 递归算法的内核调用

当每次迭代并行度增加的数量不确定时，依然可以将该状态存储在全局内存。但此时需要与主机端进行通信以确定下次迭代需要开启的线程数目。我们可以使用原子写操作将信息写到块内的共享内存中，然后在线程块执行完毕之前做原子加法操作将信息写回全局内存。接着使用 memcpy 将数据复制回主机端以调整启动下个内核需要的参数。

以快速排序的第一层迭代为例，刚开始，每个线程块中的数据可以使用一个线程进行计算。随后不断启动只含单个线程块的内核函数，直到达到 GPU 可用 SM 数量的若干倍。期间，当一个 SM 上的线程块数量饱和后，即每个 SM 容纳了 16 个线程块，可以尝试扩大每个线程块上的线程数。当每个线程块启动 256 个线程时，可以再次尝试增加线程块的数目。

尽管这种方法很容易实现，但它有一些缺点。首先，没有充分地利用 GPU。在第一层迭代只有一个线程，内核函数闲置很明显。即使是在第四层迭代，也仅仅用了 8 个线程块，

只充满了 GTX580 设备上 16 个 SM 的一半。直到第五层迭代每个 SM 才分配了一个线程块。如果使用 16 个 SM，每个 SM 分配 8 个线程块，每个线程块 256 个线程，则当所有 SM 高效运转时需要 32K 个数据点，并包含 16 次内核函数调用。

使用计算能力为 2.x 的设备可以解决这个问题。最开始的几层可以用递归调用来计算，直到到达一定深度的层数，以保证有足够的内核函数可供调度。另外一种办法就是先在 CPU 上进行一些初始计算，当需要大规模并行时再将数据复制到 GPU 进行计算。不要认为所有的计算都必须在 GPU 上执行。CPU 是一个很好的伙伴，尤其是在执行这种并行度不高的任务时。

这类问题的另一种解决方案就是使用一种叫分段扫描的特殊扫描操作。利用分段扫描，可以对数据集进行有规律的扫描操作（最大值、最小值、总和等等），并附带一个额外的数组，将原来的数组分成不同大小的块。每块分配一个或多个线程进行计算。由于附加的数组可以在运行时进行更新，因此如果分段扫描能保持在一个单独线程块内执行，就可以减少调用多内核的需要。否则，则需要采用一种更简单的解决办法，该方法能够在多数情况下正常工作，并且线程与线程块的数量能随着并行度增加或缩减灵活改变。

所有这些方法都试图解决本不是 GPU 设计初衷应该解决的问题。作为一名程序员，必须清楚一个算法是否能很好地适应硬件设计模型。当今 GPU 的递归问题通常都使用迭代问题的框架。选择一个适合硬件的算法并以正确的布局方式获取数据是提高 GPU 运行性能的关键。

7.3 数据集处理

一般的数据集采集都会获得感兴趣的数据、不感兴趣的数据以及信号的噪声。去除噪声的一种简单方法就是过滤掉高于或低于阈值的数据。如图 7-2 中的数据集，白线表示设置的阈值。当提高阈值时，则过滤掉了低级别的噪声。为了正确地获取数据，我们希望能够将低级别噪声全部移除，因为我们只对数据的峰值感兴趣。

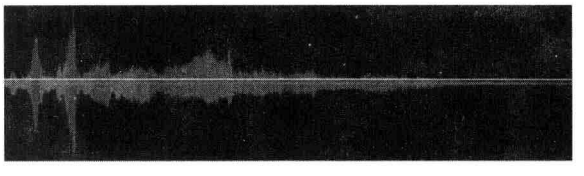


图 7-2 示例数据和阈值级别

如果数据集需要过滤的数据非常少，则可以简单地将数据添加到同一张数据表中，因为添加操作的频率很低。然而，随着数据过滤操作的频率变高，对单个表的竞争将成为性能的瓶颈。虽然这种方法适合小规模的并行操作，例如在一个 4 核 CPU 上使用 4 个线程，利用上锁的方法将数据写到单个表中，不具备扩展性。

一个更好的方法是将其分成若干个表，然后在完成相应操作之后将它们合并。事实上，大多数并行处理算法都以不同的方式采用这个方法以避免更新公共数据结构而带来的序列化瓶颈。此外，这一方式能够很好地与分片模式建立映射，后者是 CUDA 分解问题的标准模型

之一。

过滤操作实际上是一种常见的并行模式，一种分割操作。分割操作基于一些关键因素将已知数据集划分成 N 个部分。在过滤的例子中，我们将是否超过阈值的条件作为关键因素，根据它提取出超过阈值的数据。低于阈值的数据对我们而言并不重要。分割操作会产生两个表，一个满足标准，另一个不满足。

当并行执行这种操作时，有许多问题需要我们考虑。第一个问题就是，满足标准的数据的数量以及不满足标准的数据的数量都是未知的。第二个问题就是需要处理的元素的数量是未知的，而它决定着输出集的建立。最后，原始数据集的排序必须保留。

原始的扫描方法非常有用并且能够用在许多数据处理的问题中。例如，从数据库中一个不按特定顺序排列的学生列表中提取出属于班级 CS-192 的所有学生。最终会得到两个数据集，一个是满足标准的，另一个则不满足。

又如，在赤道附近有一个气象站，进行温度采集的工作。每分钟采集一次温度，持续了很多年。我们想获知样本中温度超过 40 摄氏度的样本数目或计算出采集的分钟数。

此外，需要查看的数据还可能是财务数据，例如交易金额。有时，我们需要查看是否一些数据交易值超出了某一数值，具体有多少数据。一些法规还严格要求某些高价交易必须提交交易报告或记录，以此防止洗钱。有些公司的政策要求员工具体描述交易并对其严格地审查。当遇到这些情况时，我们总希望能够从大量的数据中简单快速地提取中我们感兴趣的那部分数据。

如果数据是从科学仪器中获取的数据，我们可能希望筛选出感兴趣的异常信息包。这些包含异常的信息包对深入地开展研究以及分析非常有意义，而其他普通的信息包则不太重要，它们将被发送到其他地方或者丢弃。不同应用程序感兴趣的数据是不同的，但扫描和过滤数据的基本操作在很多领域都是需要的。

CPU 上扫描一百万个数据元素会很耗时，因为这是标准的数据集循环问题。然而利用 GPU 则可以并行扫描数据集。如果数据集很大，那么唯一的限制就是能够分配给特定问题利用的 GPU 数量。目前容量最大的 GPU 为 TeslaM 2090，能够容纳 6GB 字节的数据。然而，如果不使用主机内存或磁盘存储，每个节点能够处理的数据集大小将限制在 18 ~ 24GB。[⊖]

接下来，我们将介绍一些鲜为人知的 CUDA 特性来进行数据处理。当然，这种方法适用于任何形式的数据，任何以不同方式处理输入数据的问题都可采用这种方法。

使用 ballot 和其他内置方法

对于计算能力为 2.0 的设备，英伟达引进了一个十分有用的函数：

```
unsigned int __ballot(int predicate);
```

这个函数主要判断由给定线程传入的一个判定值。此处，该判定值可以简单地认为是一个真、假布尔值。如果该值非零，则返回一个只有第 N 个比特位为 1 的值，其中 N 表示线

⊖ 某些服务器主板，配置有 7 条 PCI-E 插槽，一般能容纳 3-4 个 GPU。——译者注

程索引（threadIdx.x）。采用原子操作的 C 语言代码如下：

```
__device__ unsigned int __ballot_non_atom(int predicate)
{
 if (predicate != 0)
  return (1 << (threadIdx.x % 32));
 else
  return 0;
}
```

无原子操作的版本和 CUDA 内置的版本速度相近，但无原子操作的版本可以工作在所有不同计算能力的硬件上。之后我们将用到这个函数来向旧设备提供后向兼容性。

除非与另一种原子或操作（atomicOr）联合使用，否则 ballot 的用处不会立刻显现。该操作的原型是：

```
int atomicOr(int * address, int val);
```

该函数根据指针变量 address 读取值，然后与 val 值做“位或”运算（C 语言中的“|”操作），最后将得到的结果存回指针变量并返回原来的值。该函数可以根据如下代码与 __ballot 函数协同使用：

```
volatile __shared__ u32 warp_shared_ballot[MAX_WARPS_PER_BLOCK];

// Current warp number - divide by 32
const u32 warp_num = threadIdx.x >> 5;

atomicOr( &warp_shared_ballot[warp_num],
                              __ballot(data[tid] > threshold) );
```

在此调用中，使用的数组可以存储在共享内存，也可以存储在全局内存，但明显共享内存更适合，因为其速度更快。根据线程束的数量确定数组的索引，此处假定为 32。因此，线程束中的每个线程都会作为该线程束结果值的一个比特位。

对于判定条件，只需要判断源数据中的 data[tid] 是否大于设定的阈值。每个线程从数据集中读取一个元素。每个线程的结果通过按位的或操作合并起来，0 号线程设置第 0 个比特位为 0 或 1，1 号线程设置第 1 个比特位为 0 或 1，以此类推。

此外，还可使用另一个内置函数 __proc。该函数的输入参数为一个 32 位的值，返回值为该 32 位值中比特位被置为 1 的数目。通过以下代码可以累加得到一个线程块中所有线程束中比特位为 1 的总数：

```
atomicAdd(&block_shared_accumulate,
          __popc(warp_shared_ballot[warp_num]));
```

通过这段代码可以得到给定 CUDA 线程块中每个线程束中来符合判定条件的线程数，并累加获得总数。在这个例子中，判定条件是数据值大于某个阈值。基于线程块的累加在许多算法中都非常有用，但是一个 CUDA 内核函数会包含很多线程块，一般会有上千个。如果需要知道整个数据集中满足判定条件数据项的数目，则需将每个线程块得到的符合条件的数目相加。

有许多方法可用于求加和。如果线程块数目很少，可以将每块的结果传回 CPU，用 CPU 来计算总和。这样可以避免 CPU 闲置，同时 GPU 也可以执行其他的任务流（第 8 章将详细介绍如何实现）。

另一种方法就是把所有线程块计算得到的部分和写回到 GPU 的全局内存中。然而，要完成对所有单独线程块的求和，需要等待所有 SM 中所有的线程块都完成了判定。唯一的解决办法就是结束当前的内核函数然后调用另一个。然后之前写入全局内存的值通过诸如并行归约的方式读入，并计算得到最终的总和。尽管这与传统 CPU 并行编程方法有些类似，但从 GPU 性能的角度来看，这并不是最好的方法。

在费米架构的设备上，每个 SM 最多能容纳 8 个线程块，但通常只会容纳 6 个。假设现在每个线程块容纳 8 个线程块。费米架构的设备上最多有 16 个 SM。因此，设备每次最多可以容纳 128（$8 \times 16 = 128$）个线程块。由于每个线程块对总和只更新一次，所以可以使用 atomicAdd 函数对存于全局内存的总和值进行简单的累加操作。

从统计学的角度来看，多个线程块同时到达执行原子加法指令的可能性非常小。考虑到读取源数据的内存操作会按顺序到达，实际上 SM 中的执行流就实现了很好的序列化，并且能够保证进行原子加法操作时不会与其他 SM 产生竞争。

使用这项技术可以在 5ms 内扫描一百万个元素，其中不包括将数据拷贝到 GPU 的时间。排除将数据拷贝到 GPU 的时间主要是因为全部数据将始终保存在 GPU 上。因此，GPU 每秒可以对该数据集进行大约两亿次查询。但实际上，判定的过程可能更复杂，稍后我们将具体介绍它是如何影响性能的。

以下是该函数的全部代码，以具体实现这一技术。

```
__device__ int predicate_gt(const u32 a, const u32 b)
{
 return (a > b);
}

__global__ void kernel_gt_u32(const u32 * const data,
                              u32 * const block_results,
                              u32 * const acum,
                              const u32 num_elements,
                              const u32 threshold)
{
 kernel_ballot_u32_acum(data, block_results, acum,
                        num_elements, threshold,
                        &predicate_gt);
}
```

在上述代码中，我们声明了两个函数：一个是设备函数，用来计算判定条件，另一个是全局函数，封装对 ballot 函数的调用，该函数传递了 6 个参数，分别为需要扫描的数据集、存放每个线程块结果的内存空间、存放累加结果的内存空间、需要处理的元素个数、阈值以及一个函数指针。

通过编写一个新的判定函数以及将这种方式进行封装可以很简单地实现其他操作，例如

判定小于、等于等，以下是具体代码：

```
// Pad the SM array by 16 elements to ensure alignment
// on 32 element boundary to avoid bank conflicts
#define SM_PADDING 16

// Max threads is 1024 so therefore max warps
// is 1024 / 32 = 48
#define MAX_WARPS_PER_BLOCK (48 + (SM_PADDING))

#define WARP_SIZE 32

// SM output per warp
volatile __shared__ u32 warp_shared_ballot[MAX_WARPS_PER_BLOCK];

// SM output per block
volatile __shared__ u32 block_shared_accumulate;

// Ballot and accumulate if predicate function is non zero
__device__ void kernel_ballot_u32_acum(
 const u32 * const data,
 u32 * const block_results,
 u32 * const gmem_acum,
 const u32 num_elements,
 const u32 threshold,
 int (*predicate_func)(const u32 a, const u32 b) )
{
 // Calculate absolute thread number
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Current warp number - divide by 32
 const u32 warp_num = threadIdx.x >> 5;

 // Total number of warp number - divide by 32
 const u32 number_of_warps = blockDim.x >> 5;

 // If not off the end of the array then contribute
 if (tid < num_elements)
 {
  // Have the first thread of every warp
  // clear the shared memory entry
  if ((threadIdx.x % WARP_SIZE) == 0)
  {
    warp_shared_ballot[warp_num] = 0;
  }

  // Call __ballot to set the N'th bit in the word
  // with a warp if the predicate is true

  // OR the bits from all threads in the warp into
```

```
  // one value per warp held in shared memory

  atomicOr( &warp_shared_ballot[warp_num],
            __ballot_non_atom( predicate_func(data[tid], threshold)) );
 }

 // Wait for all warps to complete
 __syncthreads();

 // From the first warp, activate up to 32 threads
 // Actual number of threads needed is the number
 // warps in the block
 // All other warps drop out at this point
 if (threadIdx.x < number_of_warps)
 {
  // Have thread zero, zero the accumulator
  if (threadIdx.x == 0)
  {
   block_shared_accumulate = 0;
  }

  // Add to the single accumulator the number
  // of bits set from each warp.
  // Max threads equals number of warps
  // which is typically 8 (256 threads), but
  // max 32 (1024 threads)
  atomicAdd(&block_shared_accumulate,
            __popc(warp_shared_ballot[threadIdx.x]));

  // No sync is required as only warp zero
  // accumulates

  // Have thread zero write out the result
  if (threadIdx.x == 0)
  {
   // Read from SMEM the result for the block
   const u32 block_result = block_shared_accumulate;

   // Store the value for the block
   block_results[blockIdx.x] = block_result;
   // Add the value into GMEM total for all blocks
   atomicAdd( gmem_acum, block_result );
  }
 }
}
```

函数的第一部分计算了线程的绝对索引：

```
// Calculate absolute thread number
const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
```

该函数只适合计算一维线程。当处理大数据集时（多于一千六百万个元素），则需用到另

外一个维度，否则开启的线程块数目将大于 64K。

```
// Current warp number - divide by 32
const u32 warp_num = threadIdx.x >> 5;

// Total number of warp number - divide by 32
const u32 number_of_warps = blockDim.x >> 5;
```

然后通过将线程索引除以 32（向右移位），计算得到当前线程所属的线程束的索引。同样的方法，根据线程块的维度可计算得到当前线程块包含的线程束的数量。

```
// If not off the end of the array then contribute
if (tid < num_elements)
{
```

接着我们对线程的绝对索引 tid 进行检查，判断该绝对索引是否在数据集的范围内。由于数据集元素的数目可能不是二的整数倍，因此，最后一个线程块计算得到的 tid 可能超过了数据集的范围，为了避免内存读 / 写访问越界，因此检查是很有必要的。

注意，这也暗示在 if 代码块中不能使用 __syncthreads 进行同步操作，因为所有的线程都需执行同步操作，无论它是否超出数据集的范围。

```
// Have the first thread of every warp
// clear the shared memory entry
if ((threadIdx.x % WARP_SIZE) == 0)
{
 warp_shared_ballot[warp_num] = 0;
}
```

接下来清空将要使用的共享内存。共享内存可以存储上个内核函数的结果，并且不会隐式地初始化为 0，因此使用前应进行清零操作。由于每个线程束只对共享内存的一个值进行写操作，因此只需每个线程束的第一个线程将对应的共享内存清零。注意，此处不需要任何同步，因为只有每一个线程束中的第一个线程执行写操作。当线程束遇到分支时，线程束中不满足条件的线程将隐式地在 if 语句块的末尾等待满足条件的线程执行完毕。

```
// Call __ballot to set the N'th bit in the word
// with a warp if the predicate is true

// OR the bits from all threads in the warp into
// one value per warp held in shared memory

atomicOr( &warp_shared_ballot[warp_num],
         __ballot_non_atom( predicate_func(data[tid], threshold)) );
```

现在，我们可以对每个活跃的线程束中的每个线程调用 atomicOr 函数执行原子或操作。该函数传递两个参数，一个是当前线程束对应的共享内存元素的地址，另一个是做或操作的值，该值由对 __ballot 函数的调用返回。而 __ballot 的输入参数又是函数指针 predicate_func 对应函数的返回值，后者的入参为需要进行判定的两个值。随后将跳到之前定义的 predicate_gt 函数里执行。

```
// Wait for all warps to complete
```

```
__syncthreads();
```

在执行第二部分块级累加之前，程序需要等待线程块中的所有线程束都完成计算。

```
// From the first warp, activate up to 32 threads
// Actual number of threads needed is the number
// warps in the block
// All other warps drop out at this point
if (threadIdx.x < number_of_warps)
{
```

由于每个线程块能容纳的最大线程数为 1024，每个线程块最多包含的线程束的数量为 32（1024 ÷ 32 = 32）。因此，使用一个线程束就可以完成累加计算。我们可以如先前那样用每个线程束的 0 号线程进行计算，但此时，我们要等待其他线程束的结束，而不是让它们中每一个执行一个线程。

```
// Have thread zero, zero the accumulator
if (threadIdx.x == 0)
{
 block_shared_accumulate = 0;
}
```

由于不清楚共享内存元素是否保留有上次内核执行的结果，因此，在使用它之前仍需对其进行一次清零操作。注意，由于只有一个线程束在运行，因此不需要同步操作。0 号线程会进入条件内执行，其他线程隐式地在 if 语句块的末尾等待，当 0 号线程执行完毕之后与其汇集。

```
// Add to the single accumulator the number
// of bits set from each warp.

// Max threads equals number of warps
// which is typically 8 (256 threads), but
// max 32 (1024 threads)

atomicAdd(&block_shared_accumulate,
          __popc(warp_shared_ballot[threadIdx.x]));
```

接着，我们需要把线程块其他线程束产生的结果中设置的比特位数量累加到基于线程块的共享内存累加值中。线程束计算的结果在共享内存中是连续存放的，每个线程束对应一个元素。因此不会产生读取共享内存存储体冲突问题（bank conflict）。然而，所有线程需要串行化写操作，将得到的数目累加到累加值，以保证结果的正确性。一般每个线程块会开启 256 个线程，共 8 个线程束。这时串行并不会真正确保进行并行归约。但当线程束的数量比较多时，使用并行归约的方式可能更快一些。

```
// No sync is required as only warp zero
// accumulates

// Have thread zero write out the result
if (threadIdx.x == 0)
{
```

由于每个线程块最终得到一个结果，所以只需要一次写操作将结果写回全局内存，此处，选择 0 号线程来执行下一次写操作。

```
// Read from SMEM the result for the block
const u32 block_result = block_shared_accumulate;

// Store the value for the block
block_results[blockIdx.x] = block_result;

// Add the value into GMEM total for all blocks
atomicAdd( gmem_acum, block_result );
```

最后，由于共享内存中每个线程块保存的累加值会在之后使用两次，因此我们可以先将它读到寄存器中。接着把线程块的结果写到全局内存中，因为除了整体结果外，每个线程块的累加值也是我们很感兴趣的。

然后调用 atomicAdd 函数将每个线程块的累加结果累加到一个全局的累加值中。注意，不能在任何线程块中对保存最终累加结果的变量进行赋值为 0 的操作。它必须由调用该函数的主机端来完成。这样做的原因很简单。线程块以及线程块中的线程束可能以任何顺序执行。因此，不能用诸如 if（threadIdx.x == 0）&&（blockIdx.x == 0）这样的条件判断对保存最终累加结果的累加值进行初始化为 0 的操作。尽管如果 0 号线程块中的 0 号线程束恰巧第一个被执行，这样做可能有效，但实用性很低。CUDA 的执行模型中，线程块的执行顺序是无序的，我们不能假设线程块按某种顺序执行。

通过一点小改动，解决了 GTX 260（计算能力为 1.3 设备）不支持 __ballot 函数的问题，使得内核函数在许多设备上都能运行。但注意，不能使用 9800GT，因为它是计算能力为 1.1 的设备，不支持基于共享内存的原子操作。

```
Processing 48 MB of data, 12M elements
ID:0 GeForce GTX 470: GPU Reduction Passed. Time 8.34 ms
ID:2 GeForce GTX 260: GPU Reduction Passed. Time 12.49 ms
ID:3 GeForce GTX 460: GPU Reduction Passed. Time 17.35 ms
```

以上是该内核函数在不同硬件设备上的测试结果，让人感到很奇怪的是 GTX260 比更先进的 GTX460 还要快 50%。这是因为 GTX260 的 SM 数几乎是 GTX460 的 4 倍。每一个 SM 都有各自内置的共享内存，因此 GTX260 共享内存的带宽比 GTX460 的更宽。

此外，由于使用了 atomicOr 函数，而 __ballot 根本无须额外的原子操作，因此，我们也可以做一个小改动，得到一个非原子操作的版本。这样时间消耗会稍稍改观。

```
Processing 48 MB of data, 12M elements
ID:0 GeForce GTX 470: GPU Reduction Passed. Time 7.35 ms
ID:2 GeForce GTX 260: GPU Reduction Passed. Time 12.53 ms
ID:3 GeForce GTX 460: GPU Reduction Passed. Time 14.11 ms
Result: 8742545
```

结果显示在费米架构的设备上速度提升很明显，而因为 GTX260 已经采用非原子操作的版本，因此速度并没有提升。经过修改后，GTX470 的运行时间减少了 15%，相应 GTX460 的运行时间减少了 21%。运行时间的小提升使得单个 GTX470 设备每秒能扫描 1 632 000 000 个元素。但

随着判定方式更加复杂或每个线程块上数据集需要的线程维度大于 1，这种优势将随之减小。

想象一下，如果将判定增加一个上限而不是简单地认为大于阈值，时间将会发生怎样的变化呢？为此，我们将判定条件修改如下：

```
__device__ int predicate_within(const u32 a,
                                const u32 b,
                                const u32 c)
{
 return ( (a > b) && (a < c) );
}
```

我们引入了另外一个判定条件，理论上将会增加整体运行时间。但事实上会产生怎样的影响呢？

```
Processing 48 MB of data, 12M elements
ID:0 GeForce GTX 470: GPU Reduction Passed. Time 7.49 ms
ID:2 GeForce GTX 260: GPU Reduction Passed. Time 12.62 ms
ID:3 GeForce GTX 460: GPU Reduction Passed. Time 14.23 ms
Result: 7679870
```

由结果可知，增加的条件最多只有微弱的影响，执行时间上有 0.1ms 的差别。这意味着适当增加判定复杂度并不会明显地降低性能。

通过使用更加复杂的判定条件，我们可以在 GPU 上实现更加复杂的操作。即使数据需要以某种方式聚集也可以通过这些基本操作实现。我们需要做的就是调整判定条件以处理更多数据。

7.4 性能分析

本节，我们再次使用第 6 章中样本排序的例子，通过这个例子深入理解如何使用性能分析工具来找出给定算法实现中的问题。

样本排序的例子中已经包含了一些计时元素，通过它们可以调整各种参数。如果你对样本排序的原理不熟，请重新阅读第 6 章中样本排序的例子。

样本排序的主要参数是样本的数量和线程的数量。为了让程序探索可能的搜索空间，每次迭代都将样本的数量翻倍，并分别使用 32、64、128 或 256 个线程，找到如下几种较好的配置：

ID：0 GeForce GTX 470：Test 16- 用每个线程块含有 256 个线程的 64 个线程块从 1 048 576 个元素中选出 16 384 个。

线程数：				32	64	128	256
选样本的时间 -	CPU：	0.56	GPU：	0.56	0.19	0.06	0.38
样本排序时间 -	CPU：	5.06	GPU：	5.06	5.06	5.06	5.06
计数桶时间 -	CPU：	196.88	GPU：	7.28	4.80	4.59	4.47
计算桶线程号时间 -	CPU：	0.13	GPU：	1.05	0.71	0.70	0.98
分类桶时间 -	CPU：	227.56	GPU：	7.63	4.85	4.62	4.49
排序桶时间 -	CPU：	58.06	GPU：	64.77	47.88	60.58	54.51

总时间 - CPU： 488.25 GPU： 86.34 63.49 75.61 69.88
快速排序时间 - CPU： 340.44

ID ： 0 GeForce GTX 470 ： Test 16-Test 16- 用每个线程块含有 256 个线程的 64 个线程块从 1 048 576 个元素中选出 32 768 个。

线程数：				32	64	128	256
选样本的时间 -	CPU：	0.63	GPU：	0.63	0.63	0.75	0.38
样本排序时间 -	CPU：	10.88	GPU：	10.88	11.06	10.63	10.69
计数桶时间 -	CPU：	222.19	GPU：	7.85	5.51	5.39	5.22
计算桶线程号时间 -	CPU：	0.19	GPU：	1.76	0.99	0.98	1.16
分类桶时间 -	CPU：	266.06	GPU：	8.19	5.53	5.40	5.24
排序桶时间 -	CPU：	37.38	GPU：	57.57	39.40	44.81	41.66
总时间 -	CPU：	537.31	GPU：	86.88	63.13	67.96	64.35
快速排序时间 -	CPU：	340.44					

ID ： 0 GeForce GTX 470 ： Test 16- 用每个线程块含有 256 个线程的 64 个线程块从 1 048 576 个元素中选出 65 536 个。

线程数：				32	64	128	256
选样本的时间 -	CPU：	1.00	GPU：	1.00	0.88	0.81	0.94
样本排序时间 -	CPU：	22.69	GPU：	22.69	22.50	22.44	23.00
计数桶时间 -	CPU：	239.75	GPU：	8.32	5.90	5.79	5.62
计算桶线程号时间 -	CPU：	0.25	GPU：	1.49	1.98	1.60	1.65
分类桶时间 -	CPU：	300.88	GPU：	8.69	5.97	5.82	5.67
排序桶时间 -	CPU：	24.38	GPU：	52.32	33.55	30.85	32.21
总时间 -	CPU：	588.94	GPU：	94.50	70.78	67.32	69.09
快速排序时间 -	CPU：	340.44					

通过分析其中一种情况的饼状图，我们可以清晰地看出不同阶段的时间开销（如图 7-3 所示）。

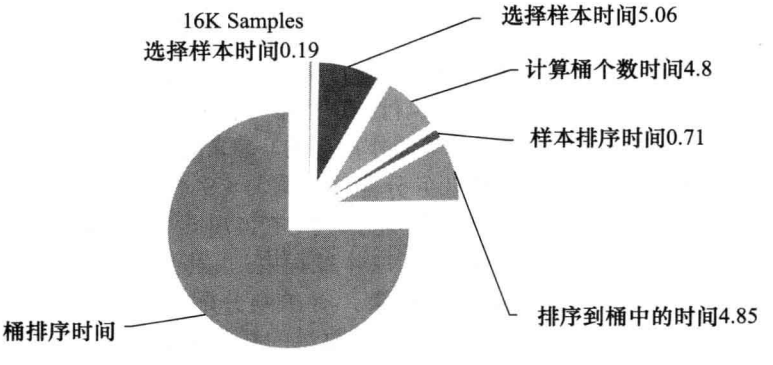


图 7-3 样本数为 16K 时样本排序时间分布

从图中可以清晰地看到有 3/4 的时间被用来排序，1/4 用来建立样本排序。然而，当增加样本数时，时间发生了变化（参见图 7-4）。

如图 7-4 所示，样本排序突然减少到总时间的 1/3。随着使用的样本数与线程数的变化，各阶段使用的时间也发生了很大变化。此处，我们选择了一种中间情形进行集中优化，即每个线程块用 64 个线程处理 32K 样本。

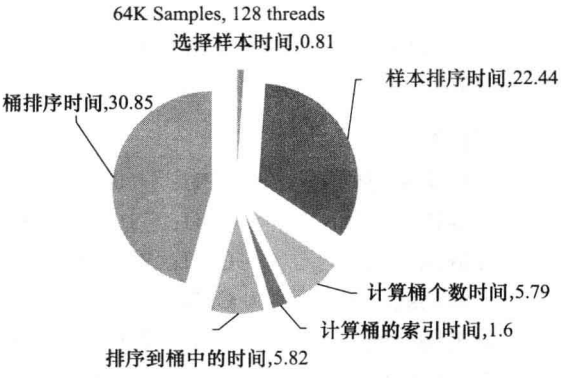


图 7-4 样本数为 64K 时样本排序时间分布

Parallel Nsight 在“New Analysis Activity”（新建分析活动）菜单下提供了一个非常有用的特性。Parallel Nsight 是一个免费的调试和分析工具，它对判断性能瓶颈非常有帮助。

Parallel Nsight 中的第一个选项首选“Profile”活动类型（参见图 7-5）。默认将会执行数次“Achieved Occupancy”（已实现的占用）和“Instruction Statistics”（指令统计）。对样本排序的例子进行测试，然后会产生一个摘要。在摘要页的顶端有一个下拉列表框。选择“CUDA Launches”（启动 UDA）显示一些有用的信息，如图 7-6 所示。

第一个视图“Occupancy View”（占用视图，在图 7-6 的左下角）。这个视图显示了启动内核的一些参数，红色区域为限制使用率的一些因素。在本例中，每个设备的线程块被限制为 8，这就限制了设备上活跃线程束的最大数目。记住，线程束是可供调度器调度的一组线程。调度器在调度线程束间切换，以隐藏内存延迟以及指令延迟。如果没有足够多的线程束供 GPU 调度，程序的性能将因此受到限制。

由视图可知，该内核函数开启了大约 16 个线程束，而每个设备可开启的最大线程束的数目为 48，利用率只达到整个设备的 1/3。这暗示我们应该通过增加每个设备的线程束的数目来提高设备利用率，归根结底，就是增加线程的数量。然而测试的结果显示这产生了相反的效果，性能反而降低了。

第二个视图显示了“Instruction Stats”（如图 7-7 所示）。在这值得注意的是（IPC 部分）有许多发布的指令没有被执行。在视图左下方的条形图中，粉色区域代表已执行的指令。蓝色部分表示由于串行化的原因正在重新发出指令。串行化产生的原因主要是没有办法以一个完整线程束（包含 32 个线程的组）的形式执行。这通常与控制流产生分支、内存访问没有合并或者由于冲突（如共享内存、原子操作等操作产生的）带来的吞吐量受限等因素有关。

此外，还需要注意一点，SM 上的任务分配并不均匀（SM 活动块，参见图 7-7）。如果开启 512 个线程块，每个线程块 64 个线程，考虑到 GTX470 设备有 14 个 SM 可供使用，我们希望每个 SM 处理的线程块能超过 36 个（72 个线程束）。但事实上，每个 SM 处理的线程束的数量并不相同，有的分配了 68 个线程束，有的则分配了 78 个（线程束启动区，如图 7-7 所示）。另外，尽管一些 SM 分配了相同数量的线程束，但它们的执行时间也并不相同，这意味着所有线程束分配的工作量并不相同。

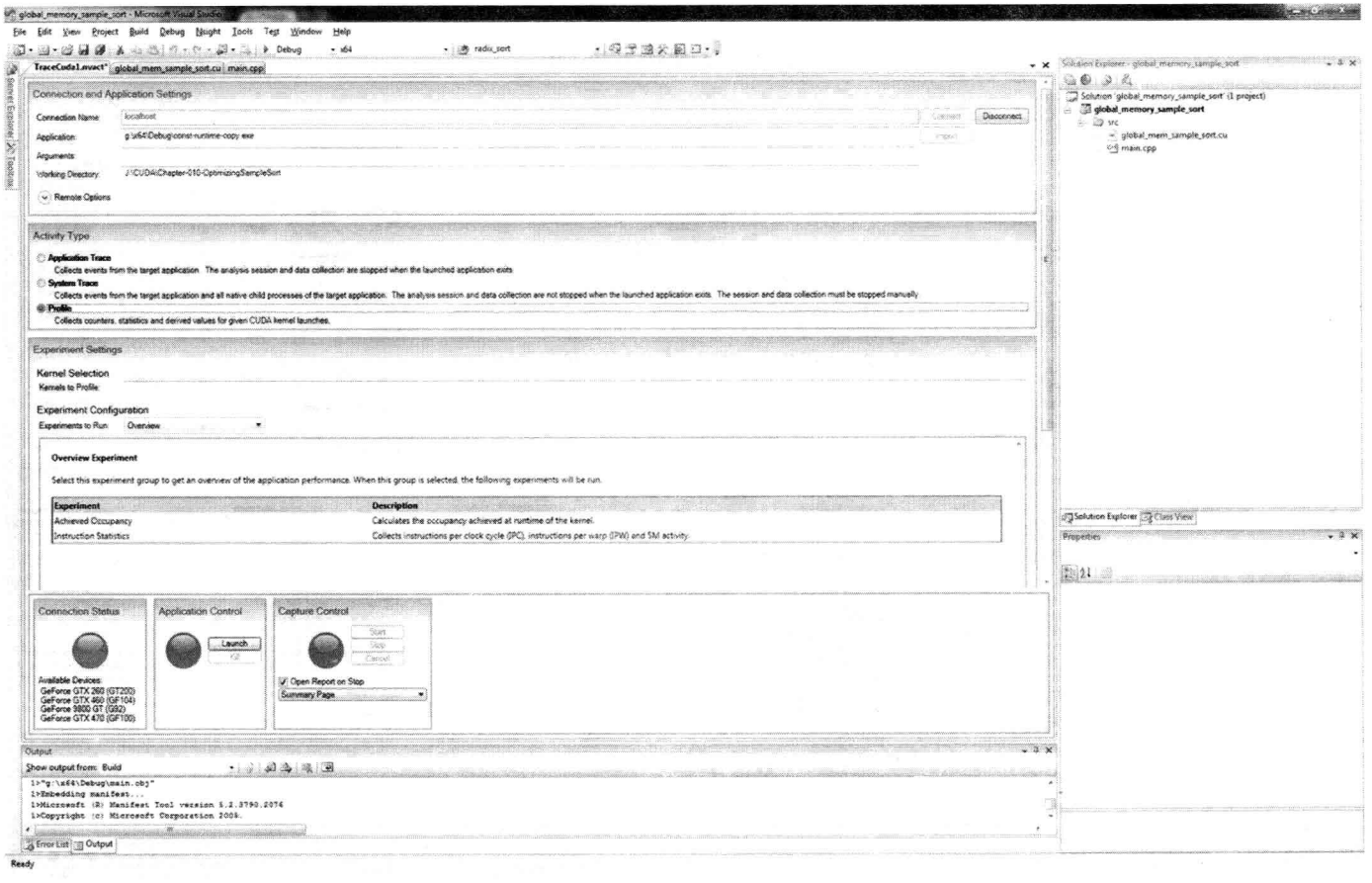


图 7-5　Parallel Nsight 启动选项

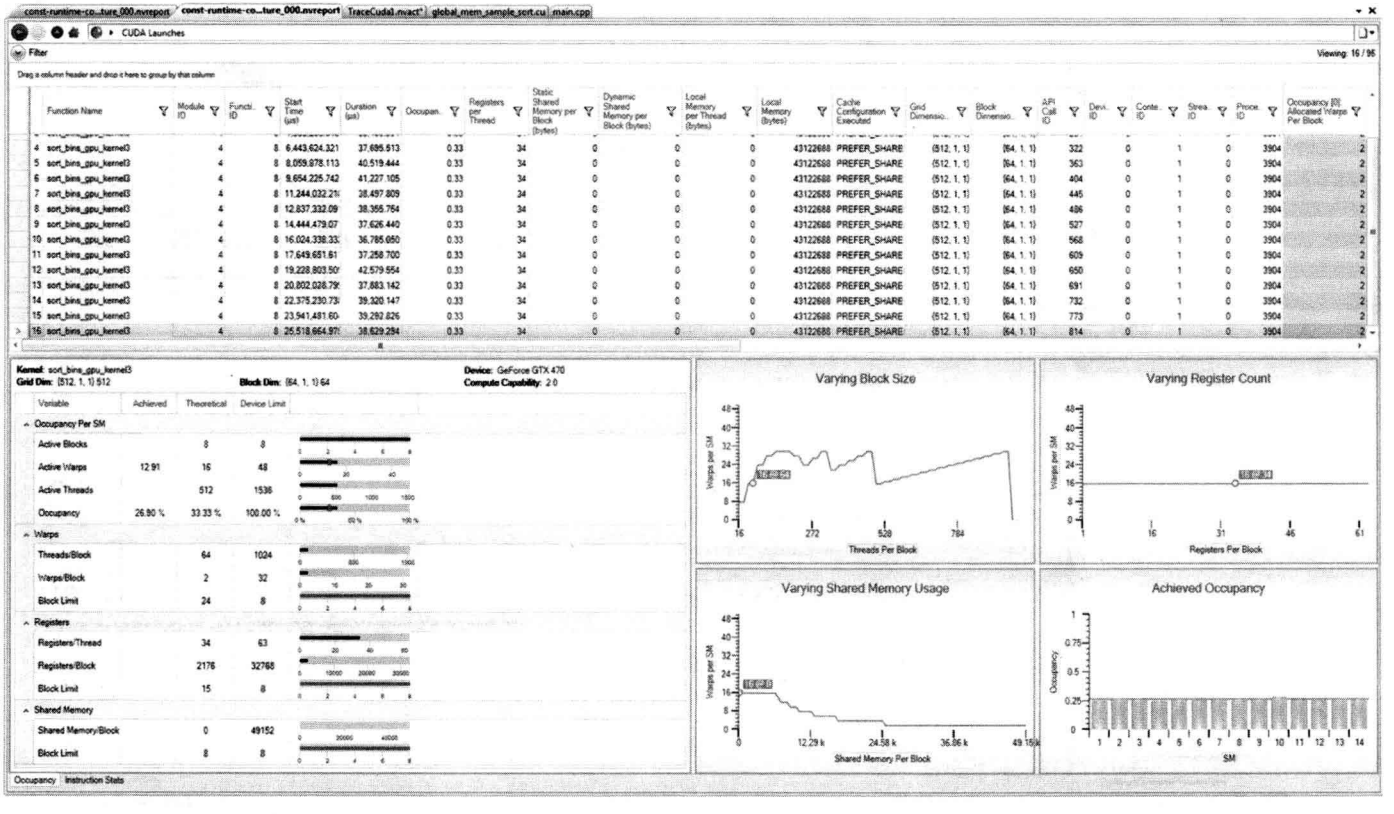


图 7-6 Parallel Nsight 分析

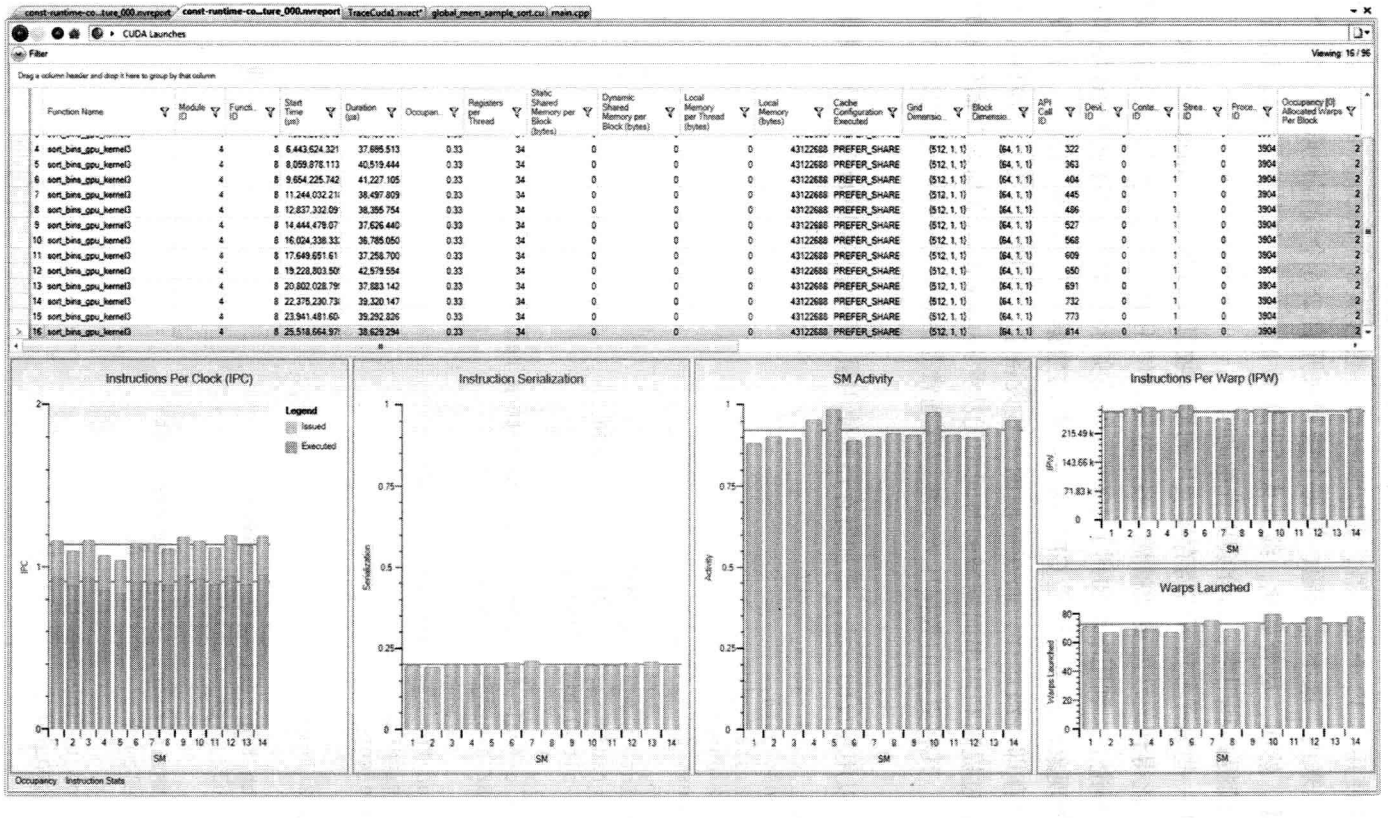


图 7-7　Parallel Nsight 分析

当每个线程块开启 256 个线程时，未执行的指令数目与已执行的指令数目比值增加了。由于每个线程使用了 32 个寄存器，调度的线程块数目从 8 个降为 3 个。尽管对于每个线程块开启 64 线程来说，这不是问题，但每个线程块开启 256 个线程则限制了每个 SM 可调度的总线程块数。然而，尽管如此，线程束的调度数目却从 16 增长到了 24，设备利用率达到 50%。如果继续提高利用率，这样会更有帮助吗？

通过使用一些编译参数可将寄存器使用的最大数目设置为 32（-maxregcount=32），但事实证明，这样优化得到的性能更加糟糕。编译器随后使用 18 个了寄存器，调度的线程块数目达到最大值 6。理论上，利用率增加到了 100%，但是实际的执行时间却从 63ms 增加到了 86ms。

产生这种现象的原因主要是 GPU 将寄存器的数据压入了本地储存器。在费米架构的设备上，本地内存即一级缓存，而在早期的一些设备上，本地内存是全局内存。对于早期的 GPU，任何通过提高利用率而缩减的时间都无法弥补由于使用全局内存而带来的时间增量。对于费米架构，其将更多的数据压入一级缓存通常是因为一些原因而有意减少可利用的缓存空间。

我们也可以用相反的方法，增加寄存器的使用量。下面这段原始 C 代码为桶排序的函数：

```
__device__ void radix_sort(
 u32 * const data,
 const u32 start_idx,
 const u32 end_idx,
 u32 * const sort_tmp_1)
{
 // Sort into num_list, lists
 // Apply radix sort on 32 bits of data
 for (u32 bit=0;bit<32;bit++)
 {
  // Mask off all but the bit we're interested in
  const u32 bit_mask = (1u << bit);

  // Set up the zero and one counter
  u32 base_cnt_0 = start_idx;
  u32 base_cnt_1 = start_idx;

  for (u32 i=start_idx; i<end_idx; i++)
  {
   // Fetch the test data element
   const u32 elem = data[i];

   // If the element is in the one list
   if ( (elem & bit_mask) > 0u )
   {
    // Copy to the one list
    sort_tmp_1[base_cnt_1++] = elem;
   }
   else
```

```
    {
     // Copy to the zero list (inplace)
     data[base_cnt_0++] = elem;
    }
   }

   // Copy data back to source from the one's list
   for (u32 i=start_idx; i<base_cnt_1; i++)
   {
    data[base_cnt_0++] = sort_tmp_1[i];
   }
  }
}
```

检查内核生成的 PTX 代码（具体步骤详见第 9 章），可以看到如下代码片段：

```
mov.s64 %rd5, %rd2;
cvt.u64.u32 %rd6, %r17;
mul.wide.u32 %rd7, %r17, 4;
add.u64 %rd8, %rd5, %rd7;
ld.u32 %r20, [%rd8+0];
mov.s32 %r21, %r20;
```

与这段代码等价的 C 代码为：

```
// Fetch the test data element
const u32 elem = data[i];
```

上述 C 代码显示出了许多问题。首先，数组下标的计算导致了一次乘法指令的使用。由于 elem 会立刻在下一条 C 指令中用来计算分支条件，因此在计算之前必须等待数据加载完成，因此线程会在此产生阻塞。乘法和除法这样的指令通常需要多个周期来完成指令流水线，并且执行这种复杂指令的执行单元是有限的。

我们可以用一个指向数组的指针来代替所有的数组下标，然后在每次用完指针后做指针增加的操作。这样，先前的代码片段变为：

```
// Fetch the test data element
const u32 elem = (*data_in_ptr);
data_in_ptr++;
```

编译器翻译的 PTX 代码如下：

```
; const u32 elem = (*data_in_ptr);
mov.s64  %rd20, %rd14;
ld.u32 %r18, [%rd20+0];
mov.s32  %r19, %r18;

; data_in_ptr++;
mov.s64   %rd21, %rd14;
add.u64   %rd22, %rd21, 4;
mov.s64   %rd14, %rd22;
```

现在的 PTX 代码仍有 6 条指令，但现在，第一部分做加载工作，第二部分做指针增加

的操作。指针的增加是一个简单的加法，比乘法简单得多，并且增加之后的结果直到下次循环迭代时才会用到。

对其他的数组操作使用相同的方法可将执行时间从 39.4ms 降到 36.3ms，降低了 3ms，接近 10%。然而，每个线程束执行的任务又是如何变化的呢？这些变化是来自于哪儿呢？

在样本排序的例子中，每个线程束将数据排序到线程块或桶中。如果将每个线程束得到的结果值输出，我们将看到一些有趣的事情。

```
Bin Usage - Max:331 Min:0 Avg:32 Zero:10275
0000:00000022 0001:00000000 0002:0000003e 0003:0000001d 0004:00000028 0005:00000000
0006:00000018 0007:0000003d
0008:00000052 0009:00000000 0010:0000001d 0011:00000000 0012:00000061 0013:00000000
0014:00000000 0015:00000000
0016:00000024 0017:0000009d 0018:00000021 0019:00000000 0020:0000002b 0021:00000021
0022:00000000 0023:00000000
0024:00000025 0025:00000000 0026:00000056 0027:00000050 0028:00000019 0029:00000000
0030:00000025 0031:0000001d
```

有大量的桶存储的项均为 0。也有其他的桶，里面存着较大的值。每个线程处理所有桶，为了遍历整个数据集，对于给定线程束，我们需要迭代最大数量的桶。显示的第一个线程束有最大值 0x9d（十进制为 157）和最小值 0。当我们迭代到 157，整个线程束中只有一个线程是活跃的。我们可以看到已发布的和我们先前看到的执行指令（每个 clock 的指令，如图 7-7）之间的巨大差别。正是有最大迭代数量的桶占据了时间。

我们看到当我们增加一倍的样本数量，基数排序的执行时间减少了，因为最大的值被分担了，分成多个桶。然而，排序样本成为了主要的问题。问题在给桶的样本分配。

大量的桶里保存着 0，这是由于样本数据集的重复性引起的。原始数据数组通过简单地调用 rand（）（返回一个伪随机数）来填充。一段时间后重复这些操作。因为两个选择样本之间有规定的距离，因此样本集包含了许多重复的部分。除去在随机数据集中的这个错误也除去了在桶计数中所有的 0，但是有一个意外的效果，就是执行时间会上升到原来的 40ms。

然而，我们可以用另外一个技术来解决这个问题，即循环展开和尾部减少，我们将在第 9 章中讲解它们。我们把下面的代码片段：

```
for (u32 i=start_idx; i<end_idx; i++)
{
// Fetch the test data element
const u32 elem = (*data_in_ptr);
data_in_ptr++;
```

替换为

```
// Unroll 4 times
u32 i=start_idx;
if ( (end_idx - start_idx) >= 4)
{
 for (; i< (end_idx-4); i+=4)
```

```
{
  // Fetch the test first and second data element
  const u32 elem_1 = (*data_in_ptr);
  const u32 elem_2 = (*(data_in_ptr+1));
  const u32 elem_3 = (*(data_in_ptr+2));
  const u32 elem_4 = (*(data_in_ptr+3));
  data_in_ptr+=4;
```

假设 start_idx 和 end_idx 之间差 32，这是一个常见的例子。在第一个循环中的迭代次数是 32，然而使用因子 4 展开循环，我们减少了操作的数量，变为了 8 次迭代。对于循环展开有一些其他的重要影响。注意我们在因子为 4 的情况下，另外需要 3 个寄存器来存储 3 个额外的数据指针。我们也需要处理最后一次循环条件，在这个条件下我们还有 0 ~ 3 个元素要处理。

我们来看 PTX 代码：

```
;const u32 elem_1 = (*data_in_ptr);
  mov.s64 %rd20, %rd14;
  ld.u32 %r23, [%rd20+0];
  mov.s32 %r24, %r23;

;const u32 elem_2 = (*(data_in_ptr+1));
  mov.s64 %rd21, %rd14;
  ld.u32 %r25, [%rd21+4];
  mov.s32 %r26, %r25;

;const u32 elem_3 = (*(data_in_ptr+2));
  mov.s64 %rd22, %rd14;
  ld.u32 %r27, [%rd22+8];
  mov.s32 %r28, %r27;

;const u32 elem_4 = (*(data_in_ptr+3));
  mov.s64 %rd23, %rd14;
  ld.u32 %r29, [%rd23+12];
  mov.s32 %r30, %r29;

;data_in_ptr+=4;
  mov.s64 %rd24, %rd14;
  add.u64 %rd25, %rd24, 16;
  mov.s64 %rd14, %rd25;
```

上述代码中我们做了很重要的操作，就是通过使每个线程使用独立的元素，引进了指令级并行。表 7-1 和图 7-8 显示了循环展开的影响。

正如在表 7-1 和图 7-8 中看到的，引进少量的线程级并行显著地减少了基数排序的执行时间。然而，我们注意到了一些其他的事情：尽管我们在费米架构中可以用 63 个寄存器，但是寄存器的数量从来没有超过 44 个。发生的事情就是编译器引进了一个调用栈，并且不再增加使用的寄存器的数量。

表 7-1 展开的层次与用到寄存器的数量和时间及比较

展开	0	2	4	6	8	10	12	14	16
时间	40	37.02	33.98	33.17	32.78	32.64	32.25	33.17	32.51
寄存器	38	38	40	42	44	44	44	44	44

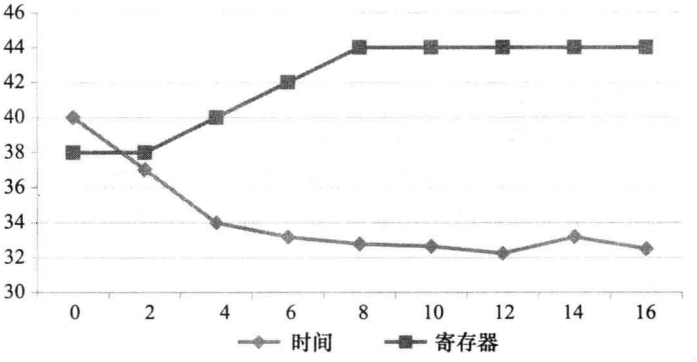


图 7-8 展开的层次和时间及用到的寄存器比较

我们已经对原始代码用了一些优化技术，你可能希望编译器可以自动地应用这些技术。这些我们都不会移除，因此获得的任何收益都将来自编译器增加的优化。让我们来看这个例子，是否通过转换为 release(发布) 模式，能使所有的编译器默认优化 (参见表 7-2 和图 7-9)。

表 7-2 debug 和 release 版时间对比

展开	0	2	4	6	8	10	12	14	16
debug	40	37.02	33.98	33.17	32.78	32.64	32.25	33.17	32.51
release	32.34	26.25	25.07	24.7	24.36	24.26	24.1	24.13	24.24

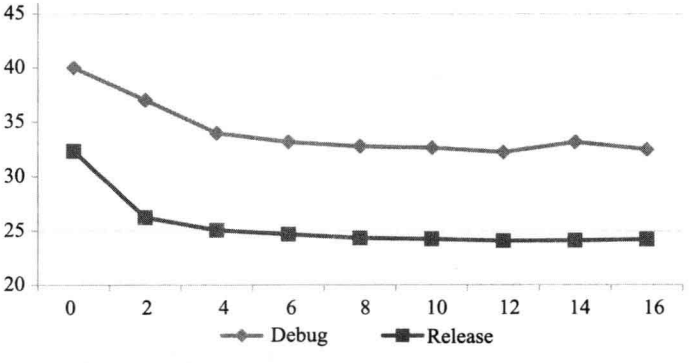


图 7-9 debug 和 release 时间对比

我们从表 7-2 和图 7-9 中看到了 release 和优化版本相似的模式，这表明我们刚刚用到的优化并不会自动地被编译器应用。我们也注意到，又看到了相同的模式，即每个线程的 4 个元素起了很大作用，但是除了这点，影响还是有限的。注意，即使使用了优化，编译器也不能自动地展开循环。这样，我们仍然需要人工地展开 4 次的循环，因为额外增加的速度却使

用了额外的寄存器，这不是一个好的折衷。

你本希望编译器来提取读操作，并且将它们放到展开循环的起始端。除了一些困难的情况，在很多情况下编译器会这么做。不幸的是，这些困难我们经常碰到。在你执行读操作，然后写操作，然后再执行读操作的地方，编译器不易知道写操作写入的数据区是否正在进行读操作。因此，必须保持读 – 写 – 读的顺序来确保正确性。然而，作为程序员，你知道读操作是否会受预处理的写操作影响并且用效率更高的读 – 读 – 写的顺序代替读 – 写 – 读的顺序。

因为我们现在已经从一个侧面，根本上地改变了执行时间，从 40ms 降到了 25ms，我们应该重新扫描问题空间来查看这个操作是否改变了每个线程的最优样本数量。

值得注意的是，快速排序的 release 版本确实更快，实际上会快 2 倍多。这样期望更快的排序就更困难了。然而，快速排序是样本排序的一个主要部分，正如我们在 CPU 样本上的预排序。用这种方法，执行时间的减少会帮很大的忙。最好的时间结果如下：

ID：0 GeForce GTX 470：Test 16-用每个线程块含有 256 个线程的 64 个线程块从 1 048 576 个元素中选出 32768 个

线程数：				32	64	128	256
选样本的时间 -	CPU：	0.38	GPU：	0.38	0.19	0.50	0.31
样本排序时间 -	CPU：	4.63	GPU：	4.63	4.69	4.31	4.31
计数桶时间 -	CPU：	64.50	GPU：	5.71	5.65	5.59	5.31
计算桶线程号时间 -	CPU：	0.06	GPU：	1.55	0.79	0.86	0.79
分类桶时间 -	CPU：	80.44	GPU：	6.25	6.08	5.96	5.71
排序桶时间 -	CPU：	62.81	GPU：	27.37	25.10	36.28	39.87
总时间 -	CPU：	212.81	GPU：	45.89	42.49	53.50	56.31
快速排序时间 -	CPU：	187.69					

ID：0 GeForce GTX 470：Test 16-用每个线程块含有 256 个线程的 64 个线程块从 1 048 576 个元素中选出 65 536 个

线程数：				32	64	128	256
选样本的时间 -	CPU：	0.50	GPU：	0.50	0.50	0.44	0.50
样本排序时间 -	CPU：	9.56	GPU：	9.56	9.63	9.56	9.63
计数桶时间 -	CPU：	95.88	GPU：	6.70	6.67	6.60	6.34
计算桶线程号时间 -	CPU：	0.06	GPU：	1.17	1.22	1.36	1.19
分类桶时间 -	CPU：	119.88	GPU：	7.27	7.06	6.94	6.73
排序桶时间 -	CPU：	52.94	GPU：	24.23	16.84	25.22	29.95
总时间 -	CPU-CPU：	278.81	GPU：	49.43	41.91	50.12	54.35
快速排序时间 -	CPU：	187.69					

因此不论 16K 还是 32K 样本都几乎得到相等的结果，它们之间仅有 0.6ms 的差别。这是基于 CPU 快序排序速度的 4.4 倍。缓存的使用是这里的关键。在第 9 章中，我们会看到缓

存带来的影响。

总之，我们用 Parallel Nsight 展示了不同数量和大小的线程块对程序的影响，我们也看到了它是如何从根本上影响整体性能的。我们随后认真研究这个数据并且最终注意到样本排序的一个设计问题。每个线程处理不同数量元素引起的串行化是这个问题的起因。尽管这是一个问题，但我们可以通过用每个线程处理多个元素的线程级并行方法来优化。启用附加编译器级别的优化为 CPU 和 GPU 的编码带来了相当大的好处。

7.5 一个使用 AES 的示例

高级加密标准（Advanced Encryption Standard，AES）是用来为诸如 WinZip、Bitlocker、TrueCrypt 等软件提供加密功能的算法。不同行业对加密技术的需要不同，你可能对它很熟悉，看起来也可能跟你毫无瓜葛。许多公司认为他们在本地机器上创建的数据已经很安全，所有的木马程序和黑客都在公司的防火墙外面，因此本地数据不需要再加密。这是一种错误的想法。这种想法是有缺陷的，因为机器、雇员以及承包商都可能在一些防火墙上制造一些漏洞，这样他们就可以在家或者在办公室外工作了。因此，安全保障需要多层次的解决方案。

加密的主要思想就是使用一种算法对需要加密的数据进行模糊处理，使其他人在非法得到数据之后无法直接读懂数据。加密之后，数据或者存储数据的机器，例如笔记本电脑，无论它们受到入侵、丢失还是被盗，都无法直接访问数据。受到入侵的机器会遭受各种可能的数据破坏。要想移除对数据的保护进而访问数据，则需要一个密钥。提供密钥后，给定的算法就可将数据解密。

加密技术也可以用在不安全的网络上，例如在互联网中，主机之间进行安全连接。如果我们想在公共网络中发布分布式应用程序，怎样才能保证我们发送到另一台机器的数据包在中途没有被截取或更改呢？诸如开源安全套接层（Open Secure Socket Layer，OpenSSL）之类的标准，可以保证当用户登录一些类似网上银行的安全服务器时，没有人监听用户的登录数据。现在，这种标准已广泛地被浏览器采用。

当设计一款软件时，我们要考虑它的安全性，以及数据以怎样的方式在服务器与客户机之间进行传输。互联网工程任务组（Internet Engineering Task Force，ITEF）是一个批准互联网新标准的组织，它要求所有的标准协议都包括安全部分。如果机构丢失客户或公司的数据将会受到巨额罚款。如果你在使用联网的计算机或者存储了一些敏感的、私人的数据，你或多或少都会对某些加密标准有一定了解。

当存储数据时，AES 是被一些美国政府机构托管的。作为一个如今经常使用的算法，我们用它作为学习的例子来看如何用 GPU 来实现基于 AES 的加密。然而，在我们详细介绍实现细节之前，我们需要分析算法，理解它的原理，并找出能够被并行计算的元素。AES 算法很复杂，但对一些不具备加密知识的人也是很好理解的。接下来，我们将详细介绍这种有用的算法，并尽量应用到目前为止所学的一些技巧。

7.5.1 算法

AES 是一种基于区块的加密算法。加密算法通常称为加密器（cipher）。将要编码的文本在未编码时叫做明文（plain text），编码之后叫做密文（cipher text）。将明文编码成密文需要一种算法和一个密钥。密钥就是简单的一串数，就像现实生活中的钥匙，而加密算法就是锁。

AES 支持很多模式的操作，最简单的就是电子密码本（Electronic Code Book，ECB）。此处，我们将讨论这种模式。AES 将需要编码的数据分成若干个长度为 128 比特位（16 个字节）的区块。ECB 模式下的每一块都基于从加密密钥中获得的一串值进行独立地编码。编码将执行若干轮，每一轮都会用一个新密钥再次加密，如图 7-10 所示。

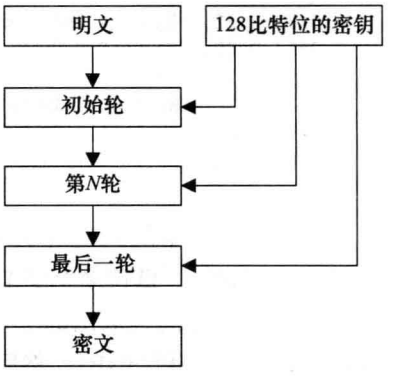


图 7-10 AES 概览

每一轮，这 128 比特位的密钥将单独调整，并且不受原始文本以及上一轮加密的影响。因此，每一轮的密钥提取可以在 AES 算法编码之外独立完成。通常，每块对应的密钥都是相同的，因此，在开始编写加密代码之前，我们就可以完成密钥提取的操作。

尽管区块的大小（明文的大小）常用 128 比特位，但 AES 也可以用 128、192 或 256 比特位的密钥。加密的轮数将根据密码长度的来确定，分别为 10 次、12 次和 14 次。

明文将以一个 4 × 4 的矩阵存储，矩阵的每个元素为一个字节，我们称这个矩阵为状态空间。

一轮加密包含如下这些步骤：

- 置换——4 × 4 矩阵中的每个字节都用查找表中的其他字节替换。
- 行循环左移——将第 1、2、3 行分别循环左移 1、2、3 个位置。第 0 行不变。
- 列混合——对每一列的值用一定规则进行扩散操作。
- 循环密钥——将数据与从原始密钥提取出的当前密钥做异或运算。

初始轮，即第 0 循轮，只包含该循环密码的操作。最后一个循环删除了混合列的操作。解密则是加密过程的逆过程，从最后一个循环开始，直到返回初始循环。

因此，为了实现算法，我们需要了解上面这 4 个步骤以及从原始 128 比特位密码提取密钥这 5 个方面的具体操作。

1. 置换

置换主要是将 4 × 4 数据块（也叫状态空间）中的每个字节与一个叫 Rijndael s-box 的常量查询表中的数值进行交换。

```
unsigned char s_box[256] =
{
/*  0    1    2    3    4    5    6    7    8    9    A    B    C    D    E    F */
0x63,0x7C,0x77,0x7B,0xF2,0x6B,0x6F,0xC5,0x30,0x01,0x67,0x2B,0xFE,0xD7,0xAB,0x76,/*0*/
0xCA,0x82,0xC9,0x7D,0xFA,0x59,0x47,0xF0,0xAD,0xD4,0xA2,0xAF,0x9C,0xA4,0x72,0xC0,/*1*/
```

```
0xB7,0xFD,0x93,0x26,0x36,0x3F,0xF7,0xCC,0x34,0xA5,0xE5,0xF1,0x71,0xD8,0x31,0x15,/*2*/
0x04,0xC7,0x23,0xC3,0x18,0x96,0x05,0x9A,0x07,0x12,0x80,0xE2,0xEB,0x27,0xB2,0x75,/*3*/
0x09,0x83,0x2C,0x1A,0x1B,0x6E,0x5A,0xA0,0x52,0x3B,0xD6,0xB3,0x29,0xE3,0x2F,0x84,/*4*/
0x53,0xD1,0x00,0xED,0x20,0xFC,0xB1,0x5B,0x6A,0xCB,0xBE,0x39,0x4A,0x4C,0x58,0xCF,/*5*/
0xD0,0xEF,0xAA,0xFB,0x43,0x4D,0x33,0x85,0x45,0xF9,0x02,0x7F,0x50,0x3C,0x9F,0xA8,/*6*/
0x51,0xA3,0x40,0x8F,0x92,0x9D,0x38,0xF5,0xBC,0xB6,0xDA,0x21,0x10,0xFF,0xF3,0xD2,/*7*/
0xCD,0x0C,0x13,0xEC,0x5F,0x97,0x44,0x17,0xC4,0xA7,0x7E,0x3D,0x64,0x5D,0x19,0x73,/*8*/
0x60,0x81,0x4F,0xDC,0x22,0x2A,0x90,0x88,0x46,0xEE,0xB8,0x14,0xDE,0x5E,0x0B,0xDB,/*9*/
0xE0,0x32,0x3A,0x0A,0x49,0x06,0x24,0x5C,0xC2,0xD3,0xAC,0x62,0x91,0x95,0xE4,0x79,/*A*/
0xE7,0xC8,0x37,0x6D,0x8D,0xD5,0x4E,0xA9,0x6C,0x56,0xF4,0xEA,0x65,0x7A,0xAE,0x08,/*B*/
0xBA,0x78,0x25,0x2E,0x1C,0xA6,0xB4,0xC6,0xE8,0xDD,0x74,0x1F,0x4B,0xBD,0x8B,0x8A,/*C*/
0x70,0x3E,0xB5,0x66,0x48,0x03,0xF6,0x0E,0x61,0x35,0x57,0xB9,0x86,0xC1,0x1D,0x9E,/*D*/
0xE1,0xF8,0x98,0x11,0x69,0xD9,0x8E,0x94,0x9B,0x1E,0x87,0xE9,0xCE,0x55,0x28,0xDF,/*E*/
0x8C,0xA1,0x89,0x0D,0xBF,0xE6,0x42,0x68,0x41,0x99,0x2D,0x0F,0xB0,0x54,0xBB,0x16 /*F*/
};
```

对状态空间中的 16 字节元素，分别提取出单字节的十六进制数，以该数值的高位半个字节作为行索引，低位半个字节作为列索引，对查询表进行查询，找出对应的十六进制数。例如，从状态空间提取出的十六进制数为 0x3E，对应的数在查询表中的第 3 行、第 E 列，查询 s_box 表，得到 0xB2，然后用 0xB2 替换 0x3E。对状态空间中的其他字节也执行相同的操作。

2. 行循环左移

在此步骤中，将第 1、2、3 行分别循环左移 1、2、3 个位置。第 0 行不变。循环左移操作是通过将行中所有的字节向左移动一个位置的方式来打乱字节的顺序。最左端的字节移动到最右端。图 7-11 详细地演示了每行字节的移位操作。

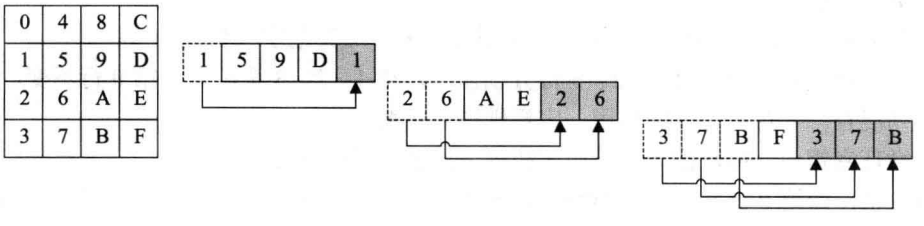


图 7-11 AES 行循环左移

3. 列混合

Rijndael 列混合的步骤比较复杂。它将状态空间的每一列 r 与一个 4×4 的列混合矩阵相乘，该矩阵如图 7-12 所示。

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

图 7-12 列混合矩阵

在列混合矩阵中的 1 表示原值不变，2 表示将原值乘以 2，3 表示将原值乘以 2 后再与原值做异或运算。在这 3 种操作中，如果得到的结果大于 0xFF，则需与 0x1B 做一次额外的异或运算。这就是简化的 Galois 乘法。以下是该算法用 C 语言实现的源代码（参见维基百科，2012 年 1 月 31 日）。

```
void mix_column(unsigned char *r)
{
 unsigned char a[4];
 unsigned char b[4];
 unsigned char c;
 unsigned char h;

 for(c=0;c<4;c++)
 {
  a[c] = r[c];
  h = r[c] & 0x80; /* hi bit */
  b[c] = r[c] << 1;

  if(h == 0x80)
   b[c] ^= 0x1b; /* Rijndael's Galois field */
 }

 r[0] = b[0] ^ a[3] ^ a[2] ^ b[1] ^ a[1];
 r[1] = b[1] ^ a[0] ^ a[3] ^ b[2] ^ a[2];
 r[2] = b[2] ^ a[1] ^ a[0] ^ b[3] ^ a[3];
 r[3] = b[3] ^ a[2] ^ a[1] ^ b[0] ^ a[0];
}
```

以上这段代码并不是最优的，但却是该标准算法最常见的一种实现方式。在编码之前，输入参数 r 是指向一个 1×4 矩阵的指针，该矩阵是状态空间的某一列，将其复制至临时数组 a 中以便后续使用。数组 b 存储原值乘以 2（<<1）后的操作结果。操作 3 实际上是在操作 2 的基础上再做异或操作。最后，根据算法的规则利用数组 a 与数组 b 中的值做一系列的异或操作 (^)，得到矩阵乘法每一维的结果。如图 7-13 所示。

由于这一部分比较耗时，稍后我们将进行更加详细的介绍。

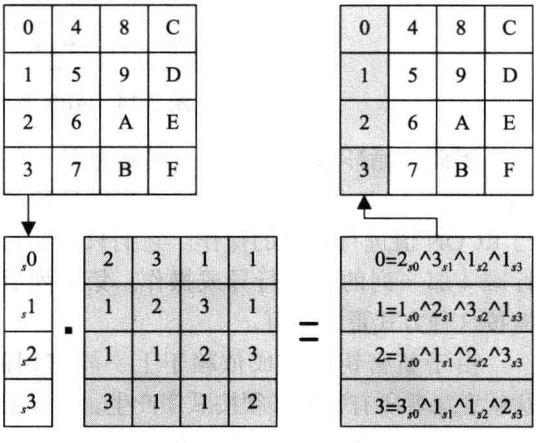


图 7-13 与第 0 列混合（对第 1、2、3 列重复操作）

4. 添加循环密钥

循环密钥是从原始密钥中抽取出的一段密码。它以 4×4 矩阵的形式存在，在加密算法每一轮循环中都不相同，并且它将与当前的结果进行简单的异或运算。

5. 提取循环密钥

AES 算法用到了一系列的循环密钥，每轮循环对应一个密钥。产生密钥是一个迭代的过程，新密钥的生成依赖于上一轮的密钥。操作之初，将现有的密钥拷贝到 0 号密钥中，产生一个 4×4 的矩阵，作为初始循环密钥。

接下来的 N 个循环密钥必须一个接一个地一次性产生。任何循环密钥的第一列都是用上

一轮密钥的最后一列作为它的起始点。新密钥第一列的生成还包含一些利用标准列生成函数的其他操作。

对于密钥的第一列，我们需要对其做一个基于列的循环移位操作，以使列中第 0 行的值移动到第 3 行。但这只针对第一列，其他列不需要这种操作。在对密文数据进行处理时我们也执行过相同的操作，行循环左移，但这里不是对行操作而是对列操作。移位操作之后，再次使用置换的方法，从 Rijndael s-box 中找出对应值将第一列的值进行替换。

之后，所有元素的操作都是相同的。新计算的值必须与列下标减 4 那一列的值进行异或操作，因此，第 1、2、3 列的值很容易就能计算出来。但对于第 0 列，还有一个附加的操作。第 0 列的第一元素必须与 0x01、0x02、0x04、0x08、0x10、0x20、0x40、0x80、0x1b 或 0x36 中的某一个值进行异或运算。这 10 个值称作 RCON 值，它的选择取决于当前循环的轮数（如图 7-14 所示）。

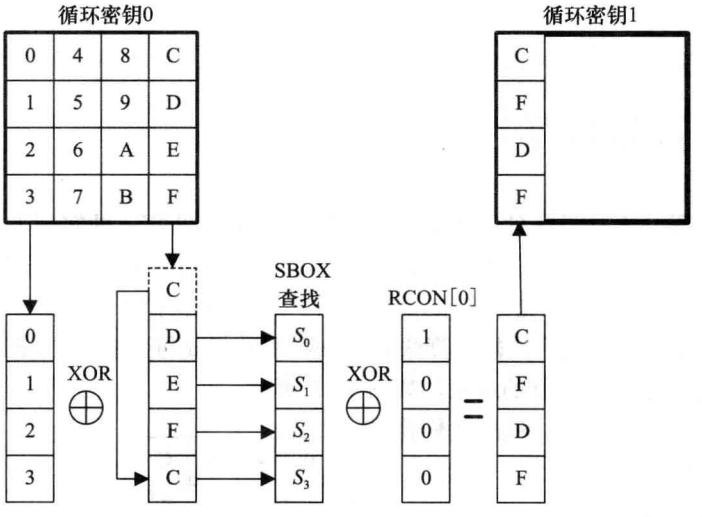


图 7-14 AES 循环密钥生成（第 1 列）

因此，循环密钥 1 的第一列成为下一个提取的列。第 1、2、3 列的计算相对简单一些（参见图 7-15），因为它们不需要列循环移位以及与 RCON 值进行异或的操作。它们只需要与列下标减 4 那一列的值进行异或操作。第 4 列后，只需按这种方式重复即可。

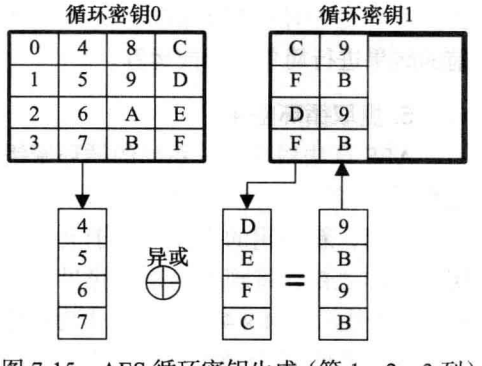


图 7-15 AES 循环密钥生成（第 1、2、3 列）

由于新密钥的生成依赖于上一轮密钥的值，这就意味着所有密钥必须按顺序产生。当需要多个密钥时，这可能成为并行实现的瓶颈。幸好，大多数算法只需要一组密码。因此，密钥可以在编码或解码之前生成并保存到一个数组中。由于单个密钥的生成并不耗时，因此使用 CPU 或 GPU 生成密钥

都可以。

7.5.2 AES 的串行实现

AES 得到了广泛研究。它可以在 8、26 或 32 位的机器上运行并且它的负载并不太大。然而，在了解算法之后，我们知道算法的实现并不简单。因此，在 GPU 上优化这个算法之前，我们需要考虑一些设计折衷。

1. 存取大小

为了支持 8 位的简单处理器，第一个问题就是存储访问方式应设计成基于字节存取方式。但所有的现代处理器都是至少 32 位的处理器。因此，如果我们使用单字节的操作，寄存器中 75% 的空间和潜在的工作就变得无用了。对于 32 位的处理器，例如 x86 或费米架构的 GPU，我们需要设计一个解决方案使其使用 32 位。

我们可以很自然地将一行数据合并到 1 个 32 位的字中，也可以将整个 4 × 4 矩阵合并成一个 16 字节（128 位）的向量。英特尔 AVX（高级向量扩展）指令集支持这种类型的向量。GPU 中的 uint4 类型也同样允许 GPU 使用单个指令从内存中存取数据。然而，与英特尔的 AVX 指令集有所不同的是，GPU 没有类似的向量指令，只能借助于内存的传入、传出达到类似目的。

如果仅对单个字节进行操作，当任何状态编码或密钥矩阵大于一个字节时，我们则不得不考虑使用位掩码以及移位操作。假设这些操作的开销并不大，减少内存读 / 写操作（尽管是大规模地读取数据）会比基于寄存器的掩码和位移操作带来更多的性能提升。

2. 内存与操作的权衡

大多数算法都通过增加内存使用量的方式来减少执行时间。但这主要取决于内存的存取速度与算术指令在开销和数量上的折衷。

一些 AES 的实现只是简单地将置换、行左移和列混合操作扩展为一系列的查找。对于 32 位的处理器，这显然需要一个 4K 的常量表格以及小规模的查询和位运算。假设 4K 的查找表保存在缓存中，在大多数处理器上的执行时间将大大减少。然而只有在完全实现整个算法之后，我们才会考虑使用这种类型的优化。

3. 硬件加速

英特尔针对 x86 处理器的 AES-NI 扩展指令集可运行在大多数英特尔沙桥 I5 和 I7 处理器上以及基于 Westmere 的 I7 Xeon 处理器和其升级产品。AES-NI 指令集主要由如下指令组成：

- AESENC（密文数据，循环密钥）——硬件中执行的标准完整的一轮加密。
- AESENCLAST（密文数据，循环密钥）——硬件中执行的最后一轮加密。
- AESKEYGENASSIST(循环密钥，上轮密钥，轮数)——协助循环密钥的生成。
- ASDEC（密文数据，循环密钥）——硬件中执行的标准的一轮解密。
- ASDECLAST（密文数据，循环密钥）——硬件中执行的最后一轮解密。

因此，整个 AES 加密和解密过程都可全部在硬件中执行。特殊的 128 位的 xmm1 和 xmm2 寄存器将被用来保存一些在单精度寄存器中的操作数。当 AES-NI 具体使用到一些真实的应用程序时，我们可以看到其性能是一个未加速处理器性能的 2 的指数幂次倍（Toms Hardware，“AES-NI 标准测试结果：Bitlocker、Everest 以及 WinZip 14”，http://www.tomshardware.co.uk/clark-aes-ni-encryption,review-31801-7.html）。当然通过手写的汇编程序以及在多核上优化调度条件可能会获得更显著的效果。但不管怎样，将该算法通过编码实现还是值得的，因为这样会让我们获益良多。

7.5.3 初始内核函数

让我们来看该算法的初始内核函数。

```
__host__ __device__ void AES_encrypt_4x4_reg(uint4 * const cipher_block,
                                             KEY_T * const cipher_key,
                                             const u32 num_rounds)
{
```

首先来看函数原型。该函数首先使用了 uint4 向量类型的指针传入需要加密的数据块。一个 uint4 向量（4 个整型数）足够用来存储 16 字节（128 位）的密文数据。函数的第二个参数传递的是密钥，它由 10 个 uint4 密钥组成，每个 uint4 向量保存的是一个循环密钥。最后一个参数是循环的轮数，在之后某一时刻，该参数会用一个固定值替换。注意，__host__ 和 __device__ 修饰符表示该函数允许 CPU 和 GPU 调用。

```
const u8 * const s_box_ptr = s_box;

// Read 4 x 32 bit values from data block
u32 w0 = cipher_block->w;
u32 w1 = cipher_block->x;
u32 w2 = cipher_block->y;
u32 w3 = cipher_block->z;
```

接着，从 uint4 型向量中提取 4 个无符号整型数。

```
register u8 a0 =  EXTRACT_D0(w0);
register u8 a1 =  EXTRACT_D0(w1);
register u8 a2 =  EXTRACT_D0(w2);
register u8 a3 =  EXTRACT_D0(w3);
register u8 a4 =  EXTRACT_D1(w0);
register u8 a5 =  EXTRACT_D1(w1);
register u8 a6 =  EXTRACT_D1(w2);
register u8 a7 =  EXTRACT_D1(w3);
register u8 a8 =  EXTRACT_D2(w0);
register u8 a9 =  EXTRACT_D2(w1);
register u8 a10 = EXTRACT_D2(w2);
register u8 a11 = EXTRACT_D2(w3);
register u8 a12 = EXTRACT_D3(w0);
register u8 a13 = EXTRACT_D3(w1);
register u8 a14 = EXTRACT_D3(w2);
register u8 a15 = EXTRACT_D3(w3);
```

然后，从 4 个整型数中分别将单个字节提取到独立的寄存器中。注意，此处使用的是数据类型 u8 而不是基本 C 数据类型，它允许重定义。此外，宏 EXTRACT 用来提取 32 位字中的各个字节，它支持大端字节序（big-endian）和小端字节序（little-endian）表示。

```
// Initial round - add key only
 u32 round_num = 0;
 // Fetch cipher key from memory
 w0 = (*cipher_key)[round_num].w;
 w1 = (*cipher_key)[round_num].x;
 w2 = (*cipher_key)[round_num].y;
 w3 = (*cipher_key)[round_num].z;
```

随后我们从密钥数组中读出 4 个值，该密钥数组中每个元素的类型仍然是 uint4，保存了 4 个 32 位的值。

```
a0 ^=  EXTRACT_D0(w0);
a4 ^=  EXTRACT_D1(w0);
 a8 ^= EXTRACT_D2(w0);
a12 ^= EXTRACT_D3(w0);

 a1 ^= EXTRACT_D0(w1);
 a5 ^= EXTRACT_D1(w1);
 a9 ^= EXTRACT_D2(w1);
a13 ^= EXTRACT_D3(w1);

a2 ^=  EXTRACT_D0(w2);
a6 ^=  EXTRACT_D1(w2);
a10 ^= EXTRACT_D2(w2);
a14 ^= EXTRACT_D3(w2);

 a3 ^= EXTRACT_D0(w3);
 a7 ^= EXTRACT_D1(w3);
a11 ^= EXTRACT_D2(w3);
a15 ^= EXTRACT_D3(w3);
round_num++;
```

第一轮密钥编码只需简单地将数据与第一轮对应的循环密钥按列做异或操作。

```
while (round_num <= num_rounds)
{
 // Fetch cipher key from memory
 w0 = (*cipher_key)[round_num].w;
 w1 = (*cipher_key)[round_num].x;
 w2 = (*cipher_key)[round_num].y;
 w3 = (*cipher_key)[round_num].z;

  // Substitution step
  a0 = s_box_ptr[a0];
  a1 = s_box_ptr[a1];
  a2 = s_box_ptr[a2];
  a3 = s_box_ptr[a3];
```

```
a4 = s_box_ptr[a4];
a5 = s_box_ptr[a5];
a6 = s_box_ptr[a6];
a7 = s_box_ptr[a7];
a8 = s_box_ptr[a8];
a9 =  s_box_ptr[a9];
a10 = s_box_ptr[a10];
a11 = s_box_ptr[a11];
a12 = s_box_ptr[a12];
a13 = s_box_ptr[a13];
a14 = s_box_ptr[a14];
a15 = s_box_ptr[a15];
```

接下来是内核函数的主循环部分。循环迭代执行 num_rounds 次。由于之后会用到密钥，而密钥需要从内存中获取，因此我们将尽可能早地从内存中读取出密钥。下一步则是进入置换操作，只需简单地从之前的 s_box 数组中找到对应的值替换当前值即可。

```
// Rotate Rows
u8 tmp0, tmp1, tmp2, tmp3;
// a0, a4, a8, a12 remains unchanged

// a1, a5, a9, a13 rotate 1
// a5, a9, a13, a1
tmp0 = a1;
a1 = a5;
a5 = a9;
a9 = a13;
a13 = tmp0;

// a2, a6, a10, a14 rotate 2
// a10, a14, a2, a6
tmp0 = a14;
tmp1 = a10;
a14 = a6;
a10 = a2;
a6 = tmp0;
a2 = tmp1;

// a3, a7, a11, a15 rotate 3
// a15, a3, a7, a11
tmp0 = a3;
tmp1 = a7;
tmp2 = a11;
tmp3 = a15;

a15 = tmp2;
a11 = tmp1;
 a7 = tmp0;
 a3 = tmp3;
```

置换之后是行循环左移的操作，分别对第 1、2、3 行进行移位操作。由于我们已经用寄

存器分别存储了 32 位字的每个字节，因此我们不能简单地对 32 位字进行循环移位操作。但事实上，GPU 也没有本地指令支持这种循环移位的操作，这样做有些不切实际。

```
if (round_num != 10)
{
 // Column Mix
 const u8 b0 =  MIX_COL(a0);
 const u8 b1 =  MIX_COL(a1);
 const u8 b2 =  MIX_COL(a2);
 const u8 b3 =  MIX_COL(a3);
 const u8 b4 =  MIX_COL(a4);
 const u8 b5  = MIX_COL(a5);
 const u8 b6  = MIX_COL(a6);
 const u8 b7  = MIX_COL(a7);
 const u8 b8  = MIX_COL(a8);
 const u8 b9  = MIX_COL(a9);
 const u8 b10 = MIX_COL(a10);
 const u8 b11 = MIX_COL(a11);
 const u8 b12 = MIX_COL(a12);
 const u8 b13 = MIX_COL(a13);
 const u8 b14 = MIX_COL(a14);
 const u8 b15 = MIX_COL(a15);

  tmp0 = XOR_5(b0, a3, a2, b1, a1 );
  tmp1 = XOR_5(b1, a0, a3, b2, a2 );
  tmp2 = XOR_5(b2, a1, a0, b3, a3 );
  tmp3 = XOR_5(b3, a2, a1, b0, a0 );

 const u8 tmp4  = XOR_5(b4, a7, a6, b5, a5 );
 const u8 tmp5  = XOR_5(b5, a4, a7, b6, a6 );
 const u8 tmp6  = XOR_5(b6, a5, a4, b7, a7 );
 const u8 tmp7  = XOR_5(b7, a6, a5, b4, a4 );
 const u8 tmp8  = XOR_5(b8, a11, a10, b9, a9 );
 const u8 tmp9  = XOR_5(b9, a8, a11, b10, a10 );
 const u8 tmp10 = XOR_5(b10, a9, a8, b11, a11 );
 const u8 tmp11 = XOR_5(b11, a10, a9, b8, a8 );
 const u8 tmp12 = XOR_5(b12, a15, a14, b13, a13 );
 const u8 tmp13 = XOR_5(b13, a12, a15, b14, a14 );
 const u8 tmp14 = XOR_5(b14, a13, a12, b15, a15 );
 const u8 tmp15 = XOR_5(b15, a14, a13, b12, a12 );

  a0 = tmp0;
  a1 = tmp1;
  a2 = tmp2;
  a3 = tmp3;
  a4 = tmp4;
  a5 = tmp5;
  a6 = tmp6;
  a7 = tmp7;
  a8 = tmp8;
  a9 = tmp9;
```

```
a10 = tmp10;
a11 = tmp11;
a12 = tmp12;
a13 = tmp13;
a14 = tmp14;
a15 = tmp15;
}
```

下一个步骤是混合列操作。除了最后一轮循环，其他每一轮循环都执行该步骤。我们通过使用MIC_COL宏将之前循环实现的列混合代码展开。此外，为了控制异或的顺序，我们写了一个名为XOR_5的宏，它的输入为5个需要异或的值。

```
// Addkey
a0 ^= EXTRACT_D0(w0);
a4 ^= EXTRACT_D1(w0);
a8 ^= EXTRACT_D2(w0);
a12 ^= EXTRACT_D3(w0);

a1 ^= EXTRACT_D0(w1);
a5 ^= EXTRACT_D1(w1);
a9 ^= EXTRACT_D2(w1);
a13 ^= EXTRACT_D3(w1);

a2 ^= EXTRACT_D0(w2);
a6 ^= EXTRACT_D1(w2);
a10 ^= EXTRACT_D2(w2);
a14 ^= EXTRACT_D3(w2);

a3 ^= EXTRACT_D0(w3);
a7 ^= EXTRACT_D1(w3);
a11 ^= EXTRACT_D2(w3);
a15 ^= EXTRACT_D3(w3);

round_num++;
}
```

随后与循环之初获取的密钥进行异或操作。

```
cipher_block->w = (ENCODE_D0(a0) | ENCODE_D1(a4) | ENCODE_D2(a8) | ENCODE_D3(a12));
cipher_block->x = (ENCODE_D0(a1) | ENCODE_D1(a5) | ENCODE_D2(a9) | ENCODE_D3(a13));
cipher_block->y = (ENCODE_D0(a2) | ENCODE_D1(a6) | ENCODE_D2(a10) | ENCODE_D3(a14));
cipher_block->z = (ENCODE_D0(a3) | ENCODE_D1(a7) | ENCODE_D2(a11) | ENCODE_D3(a15));
}
```

最后，将密文结果的各个字节合并成一个32位的值，并写回uint4类型的密文中。至此，我们一共执行了10次循环并且密文块基于10个循环密钥进行了加密。

为了让代码显得更加完整，以下列出所有宏的定义：

```
#define EXTRACT_D0(x) ( ( (x) >> 24uL ) )
#define EXTRACT_D1(x) ( ( (x) >> 16uL ) & 0xFFuL )
#define EXTRACT_D2(x) ( ( (x) >> 8uL ) & 0xFFuL )
```

```
#define EXTRACT_D3(x) ( ( (x)  ) & 0xFFuL )

#define ENCODE_D0(x) ( (x) << 24uL )
#define ENCODE_D1(x) ( (x) << 16uL )
#define ENCODE_D2(x) ( (x) << 8uL )
#define ENCODE_D3(x) ( (x)  )
#define MIX_COL(a) ( ((a) & 0x80uL) ? ((((a) << 1uL) & 0xFFuL) ^ 0x1Bu) : ((a) << 1uL) )
#define XOR_5(a,b,c,d,e) ( (((a)^(b)) ^ ((c)^(d))) ^ (e) )
```

7.5.4 内核函数性能

该内核函数的性能如何？我们又该如何衡量、理解和预测性能？首先，看一下针对计算能力为 2.x 设备上拆分的代码，我们看到一些不希望看到的东西。声明使用无符号的 8 位寄存器导致代码中的移位和掩码操作。使用提取数据的宏将 32 位字中没有用到的比特位屏蔽掉，但这完全是不必要的。事实上，如果使用 u8 类型替代 u32 类型，生成的代码量会是原先的 4 倍。

将 u8 改为 u32 意味着我们浪费了大量的寄存器空间，但是却减少了大量的指令。实际上，GPU 上 u8 寄存器的实现与 u32 寄存器是一样的，因此这样做并没有导致大量的寄存器空间的开销。

第二步，我们来看看寄存器的使用情况。初始内核函数用到了 43 个寄存器，这并不令人感到惊讶，但却让人有些失望。如果装上 CUDA 利用率计算器（它可以在 SDK 的工具（tool）目录中找到），我们可以清楚看到 43 个寄存器将会限制每个 SM 只能处理一个线程块，并且线程数不超过 320 个。每个 SM 上只有 10 个活跃的线程束，这远不及最大数目（在计算能力为 1.3 的设备上最多有 24 个活跃线程束，2.x 的设备上最多有 48 个活跃线程束，在 3.x 的设备上最多有 64 个活跃线程束）。我们需要让每个 SM 处理更多的线程块，这样会产生更多的混合指令供线程束调度。此外，一个 SM 上执行异或操作的数量（参见第 9 章）会受限制，并且 10 个线程束无法隐藏内存延迟。

为了获得更高的吞吐量，我们不希望连续地执行一系列相同的指令。如果每个 SM 能处理更多的线程块，那么当一个线程块在执行异或操作时，另一个线程块很可能在执行 s_box 置换操作。但这涉及到大量的地址计算和内存查询。我们需要用特定方法减少寄存器的使用数量。

对此，编译器提供了一个开关。可我们又该如何执行？我们将调用包含 256 个线程的 16 个线程块。使用这种方法，当每个 SM 调度更多的线程块时性能会得到一定提升。该内核函数将在基于英伟达 ION（计算能力为 1.2，包含 2 个 SM）的便携式电脑上进行测试。

```
// Encodes multiple blocks based on different key sets
__global__ void AES_encrypt_gpu(uint4 * const cipher_blocks,
                                KEY_T * const cipher_keys,
                                const u32 num_cipher_blocks,
                                const u32 num_cipher_keys,
                                const u32 num_rounds)
{
 const int idx = (blockIdx.x * blockDim.x) + threadIdx.x;
```

```
if (idx < num_cipher_blocks)
{
 AES_encrypt_4x4_reg(&(cipher_blocks[idx]),
                     &(cipher_keys[0]),
                     num_rounds);
}
}
```

由于加密函数是一个设备函数，需要一个全局函数来调用它。该全局函数提取出合适的密文数据块，所有的块使用相同的密钥进行加密。大多数编码算法都是按照这种方法执行的。

最初，内核函数用了 6.91ms 同时对 512 个明文块进行编码加密（2 个线程块，每个线程块 256 个线程，每 SM 处理一个线程块）。强制编译器仅用 32 个寄存器时，每个 SM 会处理两个线程块，一共处理了 4 个线程块。强制使用 24 个寄存器时，每个 SM 处理 3 个线程块，一共处理了 6 个线程块。事实上，当使用 32 个寄存器时，运行时间降到了 4.72ms，性能有很大的提升。然而，当使用 24 个寄存器时，运行时间增长到了 5.24ms。为什么会这样呢？

强制编译器使用更少的寄存器并不会导致它们的数据丢失。编译器有许多可用的策略。首先，它可以从内存中将数据重新加载到寄存器。这听起来有点违背常理，因为我们知道全局内存比寄存器要慢得多。然而，额外的线程块意味着有更多的线程束需要调度，轮流的线程束调度能隐藏内存延时。当线程数从 256（1 个线程块，8 个线程束）增加为 512（2 个线程块，16 个线程束）时，指令显著地进行了混合，每个 SM 能调度的线程束也增加了。

第二种策略是将寄存器的内容移到其他类型的内存中：共享内存、常量内存或本地内存。如果在编译阶段使用 -v 选项，编译器会告诉我们各种类型内存的使用情况。共享内存比寄存器慢。常量内存是高级缓存，因此也比寄存器慢。本地内存在费米（计算能力为 2.x）架构的设备上是一级缓存，在计算能力为 1.x 的设备上是全局内存。

此外，如果编译器能正确确定某个区域内寄存器的使用范围和用法，那么寄存器就可重复利用。

如果强制编译器使用更少的寄存器，最终将使用本地内存弥补寄存器的数量不足。对于费米架构的设备，这并不会带来很大的性能影响，但由于我们的测试平台是计算能力为 1.2 的设备，其本地内存是全局内存，额外增加的第 3 个线程块无法抵消全局内存带来的延迟，因此性能非常糟糕，内核函数的执行速度不增反降。

通过 5 分钟简单地对编译器进行设置，我们就让内核函数的执行时间减少了 30%，获得的性能提升还是挺可观的。然而，我们能通过优化 C 代码来获得更高的性能吗？是什么原因使编译器需要使用 43 个寄存器？我们该如何减少对寄存器的使用？

以当前代码为例，我们可以划分出一些额外需要寄存器的代码段，将所有的寄存器局部化到单独的块中。在 C 代码中我们可以通过在一段代码外加上大括号（{}）来创建一个新的范围层，以将变量或常量局部化到这段代码中。

事实证明开销最大的代码段是列混合的部分。这并不让人感到意外，因为在这段代码中，我们根据 16 个 a<n> 值计算得到 16 个 b<n> 值，还额外用到 16 个 tmp<n> 值。然而，

实际上此处一共只有 4 列参数，列与列之间的计算并没有相互依赖。当建立依存关系树时，编译器应该注意到这一点，并对执行顺序重新排序。因此，我们只需要 8 个寄存器而不是 32 个寄存器。但是，由于对如此多的参数无法高效地建立模型，因此编译器并没有对执行顺序重新排序。不管是什么原因，编译器还是用到了比需要更多的寄存器。因此，我们需要重写列混合的部分：

```
// Column Mix
const u8 b0 = MIX_COL(a0);
const u8 b1 = MIX_COL(a1);
const u8 b2 = MIX_COL(a2);
const u8 b3 = MIX_COL(a3);
const u8 tmp0 = XOR_5(b0, a3, a2, b1, a1 );
const u8 tmp1 = XOR_5(b1, a0, a3, b2, a2 );
const u8 tmp2 = XOR_5(b2, a1, a0, b3, a3 );
const u8 tmp3 = XOR_5(b3, a2, a1, b0, a0 );
a0 = tmp0;
a1 = tmp1;
a2 = tmp2;
a3 = tmp3;
```

为简单起见，我们这里只展示了其中一列的操作。这种操作让变量的使用与变量或常量的赋值更加靠近，提高了指令混合效率，也缩小了变量或常量需要存在的范围。事实上，我们让编译器识别和重用这些寄存器变得更加简单了。

此外，我们还可以改变密钥数据的读取方式。之前我们是通过计算地址来访问每一个数据：

```
w0 = (*cipher_key)[round_num].w;
```

这段代码中，首先读取 cipher_key 指针指向的数据区内容，然后用 round_num 获取下标，最后获得结构体中偏移量为 0 的成员 w 的值。当每次读取 w、x、y 和 z 时，这些操作都将重新执行一次并且还需增加结构体的偏移量。为了避免对下一条指令产生依赖，编译器在获取每个 w<n> 值时共将这些指令重复执行了 4 次。指令延迟大概是 20 个周期左右，因此这种方法可以连续地得到 4 个结果。然而，在每次计算地址及偏移量时需要用到更多的寄存器。由于更多的线程块会带来更多可供调度的线程束，从而很好地隐藏延迟，因此，这是一个很好的折衷。我们将用下面这段新代码代替原来的代码：

```
// Fetch cipher key from memory
const uint4 * const key_ptr = &((*cipher_key)[0]);
w0 = key_ptr->w;
w1 = key_ptr->x;
w2 = key_ptr->y;
w3 = key_ptr->z;
```

这里我们引入了一个新的指针变量，它只需计算一次得到基地址。当利用该指针访问结构体中 w、x、y 和 z 这些成员时，编译器只需根据基地址简单地执行加 0、4、8 和 12 的操作即可计算得到偏移地址。

另外，我们也把 unit4 类型的密钥读到 uint4 类型的局部常量中。不幸的是，这会导致编译器将 unit4 常量存放到本地内存（lmem）中，这恰恰是我们不想要的。尽管新版本的编译器可以解决这个问题。LLVM 编译器（CUDA 4.1）似乎更偏向于将向量类型存放到本地内存中而不是寄存器中。

最后，我们将 round_num 的定义从函数的开始处移动到了 while 循环的开始处，并且对第 0 轮循环直接使用 0 索引代替。

这些改动将内核函数寄存器的使用个数从 43 个减少到 25 个，并且执行时间缩短到了 4.32ms，比之前强制寄存器分配的版本快了许多。如果再次强制编译器最多使用 24 个寄存器会由于编译器对本地内存的使用而导致程序的执行速度降低。但我们最多只想使用 24 个寄存器，而不是 25 个，因为这样会增加线程块的数量，有更多的线程束可供调度，提升整体的并行度。

我们可以用

```
while (round_num <= num_rounds)
```

代替

```
while (round_num <= NUM_ROUNDS)
```

这样可以省去之前存储变量 num_rounds 的寄存器，让编译器直接使用值为 10 的宏 NUM_ROUNDS。使用字面值主要有两个目的。首先，它使得存放 num_rounds 的寄存器与一个立即值进行比较，而不是两个寄存器之间的比较。第二，这样做意味着循环的次数是已知的，这更有利于编译器正确地展开整个循环或循环中的部分区域。

通过使用宏，编译器确实只用到 24 个寄存器，这一神奇数字潜在地增加了调度一个线程块的可能性。尽管每个线程块开启 256 个线程，没有使可调度的线程块数量增加，但这种节省还是非常必要的。尽管此次 16 个线程块的运行时间减少了，但运行时间却变得很不稳定，有时候执行时间比之前还长。此时我们来看一看线程束之间的竞争。当使用一个小样本（16 个密文块）时，每次运行结果都很不稳定。因此，我们将密文块的数量增加到 2048K 并取平均值。

在分配寄存器时，CUDA 采取的策略是尽可能地让寄存器的使用数量达到最少。当我们将内核中寄存器的使用数量从 25 减少到 24 个时，每个线程块开启 256 个线程，仍只有两个线程块可供调度。然而，如果将每个线程块的线程数减半，则可以挤出另外一个有 128 个线程的线程块。这样，每个 SM 可处理 5 个线程块，每个线程块 128 个线程（一共 640 个），每个线程使用 24 个寄存器。这与每个 SM 处理 4 个线程块（512 个线程），每个线程使用 25 个寄存器相比，确实有很大不同（参见表 7-3）。

表 7-3 使用不同数量线程的效果

64 个线程	128 个线程	256 个线程
1150ms	1100ms	1220ms
100%	96%	111%

如果以 64 线程的版本作为基准，每个 SM 处理的线程块数目达到最大值 8，这也使得最多处理 512 个线程。128 线程的版本最多处理 5 个线程块，总共 640 个线程。256 个线程的版本只能处理两个线程块，总共 512 个线程。

考虑到 64 线程的版本和 256 线程的版本同时都运行了 512 个线程，我们将对它们进行对比，分析它们的执行时间为什么不同。64 线程的版本更快是因为它提供了更好的指令混合，不同线程块可以执行算法的不同部分。而 256 线程的版本中所有线程几乎同时做着相同的事情。记住在计算能力为 1.2 的设备中没有一级和二级缓存，因此只需要对指令和内存吞吐量作比较。由于调度单元更小了，对于 CUDA 运行时来说，它可以更容易地在两个 SM 间进行负载平衡。

与 256 线程的版本相比，128 线程的版本通过每个线程块开启更少的线程从而拥有更多可供调度的线程块，执行时间降低了 120ms，性能提升了 15%。但主要是因为我们将寄存器的数目限定到了 24 才达到这个效果。

对于基于 ION 的 GPU 进行测试，每秒可对大约 180 万个密文块进行编码加密，速率大概是 28MB/s（包括传输时间）。如果除去传输时间，得到的速率是现在的两倍。传输性能将在下一节中具体介绍。

7.5.5 传输性能

通过 PCI-E 数据总线将数据传输到 GPU 中是非常必要的。与访存相比，总线速度是非常慢的。第 9 章详细介绍了 PCI-E 传输速率以及使用分页或锁页内存的影响。锁页内存是一种内存块，其内容是不会被操作系统的虚拟内存管理机制换出到磁盘上的。事实上，PCI-E 传输的数据只能是锁页内存中的数据，如果应用程序并没有分配锁页内存，则 CUDA 驱动会在后台为程序分配。不幸的是，这会在常规（分页）内存和锁页内存之间产生一些不必要的拷贝操作。当然，我们也可以通过分配锁页内存来减少这些拷贝操作。

在应用程序中，我们只需将主机端分配内存的代码块

```
uint4 * cipher_data_host = (uint4 *) malloc(size_of_cipher_data_in_bytes);
KEY_T * cipher_key_host = (KEY_T *) malloc(size_of_cipher_key_size_in_bytes);
```

替换为

```
uint4 * cipher_data_host;
KEY_T * cipher_key_host;
CUDA_CALL(cudaMallocHost(&cipher_data_host, size_of_cipher_data_in_bytes));
CUDA_CALL(cudaMallocHost(&cipher_key_host, size_of_cipher_key_size_in_bytes));
```

并且将主机端最后释放内存的代码

```
free(cipher_key_host);
free(cipher_data_host);
```

替换为

```
CUDA_CALL(cudaFreeHost(cipher_data_host));
CUDA_CALL(cudaFreeHost(cipher_key_host));
```

这样做对性能影响有多大呢？通过测试，运行时间从 1100ms 减少到 1070ms，缩短了 30ms，总的执行时间仅仅下降了 3%。事实上通过这种方法获得的加速取决于处理器和使用的芯片，在一些机器上可获得高达 20% 的性能提升。然而，我们用来测试的笔记本使用的是 X1 PCI-E 2.0 总线，因此我们得到的性能并没有很大的提升，这说明在这种低端的总线上减少不必要的拷贝操作并不太重要。

尽管在此平台上使用锁页内存带来的性能提升并不大，但在后文中讲述的传输优化中我们仍需要用到锁页内存。

7.5.6 单个执行流版本

第 8 章将详细介绍流，通常利用多个 GPU 解决问题时才会用到流。此处，我们将在单个 GPU 问题上使用流，因为它允许内存传输与内核执行同时进行。实际上，我们应尽量尝试让传输时间与内核函数的执行窗口重叠。如果传输时间少于或等于内核计算的时间，这样，传输时间就可以有效被多个流的执行时间隐藏。

流是虚拟的工作队列，在这里，我们将以一种相对简单的方式使用流。最初我们将建立一个单一的流，并从相对于 CPU 的同步操作转变为异步操作。这种方法可以减少异步操作中需要的同步次数，从而使性能有少量的提升。但这只是次要的。只有当使用多个流时，性能才会有显著的提升。

如果不指定一个流，则 0 号流就是默认的流。这是一个同步流，可以帮助调试应用程序，但不是 GPU 最高效的使用方式。因此，我们必须首先创建另外一个流，然后把内存拷贝、事件和内核操作压入流中。

首先是创建一个流。代码如下：

```
cudaStream_t aes_async_stream;
CUDA_CALL(cudaStreamCreate(&aes_async_stream));
```

相反，执行完毕后需要在主机端程序末尾销毁流。

```
CUDA_CALL(cudaStreamDestroy(aes_async_stream));
```

然后，将所需拷贝操作和事件操作压入流中，我们需要将下面的代码

```
// Copy to GPU and then zero host cipher memory
CUDA_CALL(cudaEventRecord(start_round_timer));

CUDA_CALL(cudaMemcpyAsync(cipher_data_device, cipher_data_host,
size_of_cipher_data_in_bytes, cudaMemcpyHostToDevice));

CUDA_CALL(cudaMemcpyAsync(cipher_key_device, cipher_key_host,
size_of_cipher_key_size_in_bytes, cudaMemcpyHostToDevice));
```

改为

```
// Copy to GPU and then zero host cipher memory
CUDA_CALL(cudaEventRecord(start_round_timer, aes_async_stream));
```

```
CUDA_CALL(cudaMemcpyAsync(cipher_data_device, cipher_data_host,
size_of_cipher_data_in_bytes, cudaMemcpyHostToDevice, aes_async_stream));

CUDA_CALL(cudaMemcpyAsync(cipher_key_device, cipher_key_host,
size_of_cipher_key_size_in_bytes, cudaMemcpyHostToDevice, aes_async_stream));
```

请注意，如何将每个调用的新创建的流作为最后一个参数来使用。流参数是一个可选参数。然后，我们需要再次通过指定流将内核启动到正确的流中。流参数作为内核启动的第 4 个参数，第 3 个参数为 0。第 3 个参数表示内核用到的动态共享内存的大小。由于内核中并没有动态地分配共享内存，因此我们将这个值设为零。代码由原先的

```
AES_encrypt_gpu<<<num_blocks,num_threads>>>(cipher_data_device,cipher_key_device,
num_cipher_blocks,num_cipher_keys);
```

变为

```
AES_encrypt_gpu<<<num_blocks,num_threads,0,aes_async_stream>>>(cipher_data_
device,cipher_key_device, num_cipher_blocks, num_cipher_keys);
```

接着，我们需要将数据拷贝回来，并停止计时器事件。由于是在内核函数的末尾停止计时器事件，因此需要确保等待这个事件。

```
CUDA_CALL(cudaEventSynchronize(stop_round_timer));
```

现在内核函数、拷贝操作和事件操作完全异步，在内核函数未完全执行完毕返回数据之前，不要使用内核函数返回的数据，这一点至关重要。当向异步转化时，忘记在数据拷贝回主机端之后添加同步操作通常会导致错误。

通过使用流，性能会有多少提升？运行测试程序显示，时间从 1070ms 下降到 940ms，执行时间减少了 12%。考虑到我们所要做的只是移除 CUDA 驱动程序默认使用 0 号流时带来的隐式同步，因此这种优化得到的效果已经相当显著了。

7.5.7 如何与 CPU 比较

英特尔为 AVX 指令集提供了一种名为 AES-NI 的特殊扩展。这是基于 128 位宽的 AES 密钥状态和密钥扩展处理的支持。这相当于我们在加载或存储内存时使用的 u4 类型。AES-NI 具有硬件支持的编码 / 解码以及扩展密钥操作。因此，让我们来看看如何利用这些扩展。

英特尔提供了一个 AES-NI 样例库，它可以在 http://software.intel.com/en-us/articles/download-the-intel-aesni-sample-library/ 下载。当下载完库之后，需要进行生成，因为其中并没有可以直接链接的预编译二进制库。这需要通过一个旧的命令行接口。如果使用 Microsoft Visual Studio 生成，则需要运行一个名为 vcvars32.bat 的命令文件，这个文件为命令行版本设置了一系列的命令行环境变量。事实上它只是映射到了一个叫 vsvars32.bat 的文件，由该文件设置所有的环境变量。

库生成之后，我们需要在 Visual Studio 的工程中添加库包含路径、头文件包含路径以及额外需要链接的库。

Intel版本的AES和我们目前开发的GPU版本的AES有一个关键的区别。原始的AES描述中，数据是以列方式进行排列的，A、B、C和D位于同一列中。而英特尔ASE-NI则期望对其进行转置，让A、B、C和D位于同一行中。同时由于Intel的字节表示顺序，AES-NI要求字节在内存中按与当前顺序相反的顺序排列。

因此，我们有两个选择：要么调整代码以满足英特尔AES-NI的顺序，或对数据进行变换，转换成其需要的形式。为了让内存块能够直接在主机端进行比较，我们调整当前的解决方案，以符合AES-NI规定的格式。为了获取AES-NI支持，之后所有的开发将移到搭载有GTX470显卡的沙桥-E（酷睿i7 3930K @3.2GHz）平台上。因此，之后所有的测试时间将不再与当前所使用的基于原子操作的ION系统相比较。

此时，另一个需要注意的重要问题是uint4类型在GPU上的编码顺序为x、y、z、w而不是w、x、y、z。无论是GPU版本还是CPU版本得到的结果都是错误的，因为它们都是基于相同的错误代码。但如果理解了uint4类型这种奇怪的排列方式之后（通常对应红、绿、蓝和阿尔法这4个通道，其中w代表阿尔法通道），这个问题就很容易纠正。很明显，我们应该基于CPU版本，根据现有库或使用AES-NI库尽早检测这类问题。AES-NI代码很简单，如下所示：

```
void aes_encode_block_aes_ni(const u32 num_cipher_blocks,
                             const u32 num_cipher_keys,
                             const u8 * initial_keys,
                             u8 * src_data_blocks,
                             u8 * dest_data_blocks)
{
  // Encode the data blocks
  TIMER_T encode_key_time = get_time();

  // Encode using one or more blocks and single key
  intel_AES_enc128( (_AES_IN UCHAR *) src_data_blocks,
              (_AES_OUT UCHAR *) dest_data_blocks,
              (_AES_IN UCHAR *) initial_keys,
              (_AES_IN size_t) num_cipher_blocks );
  encode_key_time = (get_time() - encode_key_time);
  if (num_cipher_blocks > 1)
  {
   printf("\n\nEncrypting using AES-NI : %.u blocks", num_cipher_blocks);
  printf("\nEncrypt Encode : %.3fms", encode_key_time);
 }
}
```

AES代码的接口是基于字节的。此处我们展示了基于单个密钥来对单个数据块num_cipher_blocks进行编码的示例代码。解码部分采用类似的代码集，如下所示：

```
void aes_decode_block_aes_ni(const u32 num_src_cipher_blocks,
              const u32 num_cipher_keys,
              const u8 * key,
```

```
                const u8 * src_data_blocks,
                u8 * const dest_data_blocks)
{
  // Decode one or more blocks using a single key
  TIMER_T decode_key_time = get_time();
  intel_AES_dec128( (_AES_IN UCHAR *) src_data_blocks,
                    (_AES_OUT UCHAR *) dest_data_blocks,
                    (_AES_IN UCHAR * ) key,
                    (_AES_IN size_t) num_src_cipher_blocks );
  decode_key_time = (get_time() - decode_key_time);

  if (num_src_cipher_blocks > 1)
  {
   printf("\n\nDecrypting using AES-NI : %.u blocks", num_src_cipher_blocks);
   printf("\nDecrypt Decode   :%.3fms", decode_key_time);
  }
}
```

密钥扩展是隐式的操作，在此操作中，我们传递了一个未展开的、只有 16 字节的密钥。然而，在内部每一次加密 / 解密阶段只有一次操作。

我们将开发一个程序，生成一组 400 万个随机数据块（约 64MB 的数据），并使用一个单一密钥对其进行编码。随后，我们解码该数据，并检查解码后的数据和原来是否一样。我们将分别运行 AES-NI 库、串行版本和对应这些操作的 CUDA 版本，并交叉检查结果，以确保所有的实现都一致。

一旦 GPU 和 CPU 的版本与 AES-NI 库相匹配，我们能够看到 AES-NI 指令集有多快。在我们的沙桥 -E 系统上，基于软件的串行扩展密钥和解码块操作用了 3880ms，而启用硬件的 AES-NI 版本只用了 20ms。相比较而言，不包括任何与设备的传入或传出时间，CUDA 版本用了 103ms。事实上，传入和传出设备的操作分别用了 27ms 和 26ms。由于我们使用 GTX470 而不是 lesla 作为我们的测试设备，我们不能够重叠传入和传出，因为这个设备只有单一的内存传输引擎。因此，最好的情况是，在某个传输的同时，完全隐藏内核执行时间，这将有效地消除执行时间。但是，要做到这一点，我们就需要将内核的执行时间加速 5 倍。让我们来看看解码内核函数，它已修订了其形式以便兼容 AES-NI 按字节的输出。

```
#define AES_U8_DECODE u32
__host__ __device__ void AES_decrypt_4x4_reg(const uint4 * const src_cipher_block,
                                             uint4 * const dest_cipher_block,
                                             KEY_T * const cipher_key)
{

 // Read 4 x 32 bit values from data block as 128 bit read
 uint4 key = *src_cipher_block;

 // Store into four 32 bit registers
 u32 w0 = key.x;
 u32 w1 = key.y;
```

```
u32 w2 = key.z;
 u32 w3 = key.w;

 // Allocate room for sixteen 32 bit registers
 register AES_U8_DECODE a0, a4, a8, a12;
 register AES_U8_DECODE a1, a5, a9, a13;
 register AES_U8_DECODE a2, a6, a10, a14;
 register AES_U8_DECODE a3, a7, a11, a15;

 // Expand the 32 bit words into 16 registers
 EXTRACT_WORD(w0, a0, a1, a2, a3);
 EXTRACT_WORD(w1, a4, a5, a6, a7);
 EXTRACT_WORD(w2, a8, a9, a10, a11);
 EXTRACT_WORD(w3, a12, a13, a14, a15);

 // Always start at round ten
 u32 round_num = NUM_ROUNDS;

 // Setup some pointers to the lookup gmul tables
 const GMUL_U8 * const gmul_14_ptr = gmul_tab_14;
 const GMUL_U8 * const gmul_09_ptr = gmul_tab_09;
 const GMUL_U8 * const gmul_13_ptr = gmul_tab_13;
 const GMUL_U8 * const gmul_11_ptr = gmul_tab_11;

 // Define either a host or device point for the s_box function
#ifdef __CUDA_ARCH__
  const S_BOX_U8 * const s_box_ptr = s_box_inv_device;
#else
  const S_BOX_U8 * const s_box_ptr = s_box_inv_host;
#endif
 // Count down from round ten to round one
 while (round_num > 0)
 {
  // Add Round Key
  {
   // Fetch cipher key from memory as a 128 bit read
   key = ((*cipher_key)[round_num]);

   // Convert to four 32 bit values
   w0 = key.x;
   w1 = key.y;
   w2 = key.z;
   w3 = key.w;

   // Extract the key values, XOR'ing them with
   // the current values
   EXTRACT_WORD_XOR(w0, a0, a1, a2, a3);
   EXTRACT_WORD_XOR(w1, a4, a5, a6, a7);
   EXTRACT_WORD_XOR(w2, a8, a9, a10, a11);
   EXTRACT_WORD_XOR(w3, a12, a13, a14, a15);
```

```
}
// Invert Column Mix on every round except the first
if (round_num != 10)
{
 AES_U8_DECODE tmp0, tmp1, tmp2, tmp3;

 // Invert mix column operation on each column
 INV_MIX_COLUMN_PTR(a0, a1, a2, a3,
            tmp0, tmp1, tmp2, tmp3,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

 INV_MIX_COLUMN_PTR(a4, a5, a6, a7,
            tmp0, tmp1, tmp2, tmp3,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

 INV_MIX_COLUMN_PTR(a8, a9, a10, a11,
            tmp0, tmp1, tmp2, tmp3,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);
 INV_MIX_COLUMN_PTR(a12, a13, a14, a15,
            tmp0, tmp1, tmp2, tmp3,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);
}

// Invert Shift Rows
{
 // a0, a4, a8, a12 remains unchanged

 // a1, a5, a9, a13 rotate right 1
 AES_U8_DECODE tmp0;
 ROTR_1(a1, a5, a9, a13, tmp0);

 // a2, a6, a10, a14 rotate right 2
 AES_U8_DECODE tmp1;
 ROTR_2(a2, a6, a10, a14, tmp0, tmp1);

 // a3, a7, a11, a15 rotate right 3
 ROTR_3(a3, a7, a11, a15, tmp0);
}

// Invert Substitute bytes
{
 SBOX_SUB(s_box_ptr, a0, a4, a8, a12);
 SBOX_SUB(s_box_ptr, a1, a5, a9, a13);
 SBOX_SUB(s_box_ptr, a2, a6, a10, a14);
 SBOX_SUB(s_box_ptr, a3, a7, a11, a15);
}

  // Decrement the round counter
  round_num--;
 }
```

```
    // Execute round zero - only an XOR
    // Read ahead of time, round zero of the cipher key
    key = ((*cipher_key)[0]);

    // Pack the values back into registers
    w0 = ENCODE_WORD( a0, a1, a2, a3 );
    w1 = ENCODE_WORD( a4, a5, a6, a7 );
    w2 = ENCODE_WORD( a8, a9, a10, a11 );
    w3 = ENCODE_WORD( a12, a13, a14, a15 );

    // XOR the results with the last key
    key.x ^= w0;
    key.y ^= w1;
    key.z ^= w2;
    key.w ^= w3;
    // Use a 128 bit memory write to store the decoded block
    *dest_cipher_block = key;
}
```

该函数首先读取加密的数据，然后解码存入一个 16 个寄存器组里。该解码函数是编码函数的逆过程。因此，我们从 10 倒序循环到 0。

解码侧比编码更复杂，这主要是因为使用了 Galois 乘法。乘法预先计算到一个表中。因此，异或操作的序列需要执行一些依赖于数据的查找。每次查找需使用 4 个表中的一个。这些表都是 1K 字节大小。然而，这会产生糟糕的分散内存访问模式。

然后我们循环行中的值，并最终像前面那样，执行 s_box 置换。正如反转的混合列操作，s_box 函数生成一个分散的内存访问模式。最后，用一个 128 字节的写操作将数据写入到全局内存。

这个初始实现方式的另一个重要的问题是，使用了 44 个寄存器，这确实太多了。这是一个复杂的内核函数。直到最后一刻，我们都成功地保持在寄存器中计算。强制寄存器数为 42 个寄存器（通过设置编译器标志 maxrregcount=42），将 SM 可允许调度的线程块增加 1 个。这反过来又把执行时间降为 97ms。强制降低寄存器的使用，意味着超出的存储需求将溢出到全局内存。在这种情况下，我们看到内存带宽的需求跃增 25%。这表明通过减少寄存器的使用仍然有提高的空间，但它需要通过其他手段来完成。

我们可以减少每个线程块的线程数来获得更多可供调度的线程块，从而达到预期的效果。从 128 个线程减少到每块 96 个线程，使我们能够调度像以前一样数量的线程束，但是用 8 个线程块而不是 6 个。这样执行时间降低到 96ms。由于内核函数没有使用同步点，时间上的降低完全来自两方面。一个是增加的线程块带来的更好的指令混合，另一个是缓存的作用。

如果我们看一下图 7-16 的内存视图，它来自 Parallel Nsight 得到的一个实验结果。我们可以看到，我们有非常高的一级缓存的使用率，但仍然有 281MB 的数据溢出到二级缓存中。更糟糕的是，有 205MB 溢出到全局内存中。内核函数的读取和写入将会通过全局内存，这

将导致需要一些全局内存流量。但是我们应该对此如何预期呢？我们有 4 195 328 个线程块，每块使用 16 字节大小的数据，因此，我们要读取 67 125 248 字节或正好为 64MB 的数据。同样，我们要写入一个解密块，所以我们有 64MB 的数据写出。图 7-16 中显示了整个设备上全局内存的统计数据，表明我们的读 / 写量共 205MB。因此，我们产生了所需全局内存流量的 160%，这反过来限制了性能。

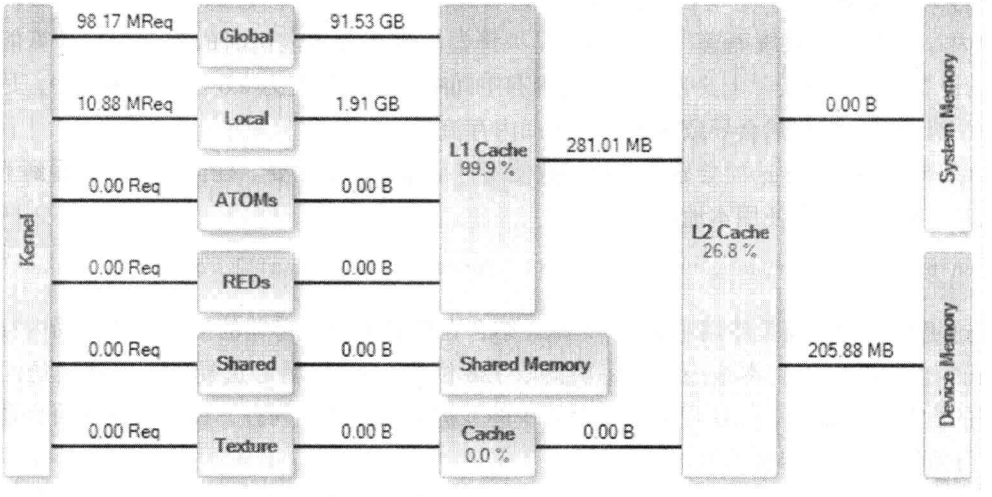


图 7-16 初始内存带宽视图

目前，一级缓存以峰值的效率运行，但有 16K 的共享内存我们没使用。它不用像全局内存那样要求合并访问，因此这对于小数据区域上分散内存访问模型来说，是很不错的选择。然而，与一级缓存不同，共享内存只是线程块内可见，这意味着，对每个 SM 上的线程块都要复制一份数据。

常量内存缓存未在图 7-16 中显示，但它也大到足以容纳 Galios 乘法（gmul）或 s_box 表。但是，常量缓存的每个时钟仅具有一个 32 位的元素带宽，并且是专为所有线程访问同一个元素设计的。因此，共享内存是更好的选择。

不过，让我们先来看看这两个表，s_box 和 gmul 表。它们声明为 32 位的无符号类型，以避免大量添加指令，来完成对 32 位字的移位和掩码操作。考虑我们产生的内存流量，这很可能不是一个好的选择。改成 u8 类型，我们可以看到片外内存访问从 205MB 降到了 183MB，执行时间从 96ms 下降到 63ms。显然，这引起了全局内存上过多的存取操作，减少它会有很大的帮助。

减少了内存使用量，每个 gmul 表是 256 个字节的大小，这样的 4 个表正好 1K 字节。我们可以在每个 SM 上设置最多 8 个线程块，所以 8K 的共享内存足以容纳 gmul 表。

然而，执行此共享内存的优化有一个问题。事实上，我们将 18G 的内存带宽从一级缓存移到共享内存，主内存带宽下降 7MB。但是，我们必须在每个块的起始位置移动 1K 字节的

数据，因为共享内存是不持久的，也不能在线程块之间共享。然而，一级缓存却在块之间是共享的，鉴于表完全驻留在缓存内，所以能对目前这种分散内存访问模式有很好的处理。对于我们使用 8K 共享内存的方式，总计的速度改善几乎是零，所以移除了这一优化，而是让表驻留在一级缓存中。注意，与全局内存访问相比，使用共享内存将对计算能力为 1.x 的设备性能带来很大的改善，因为那里有没有一级 / 二级缓存。

再回来看图 7-16，你有没有注意到一些有趣的事情？是否注意到我们在使用 1.91GB 的本地内存？对内存系统而言，本地内存是编译器处理寄存器溢出时的首选。在计算能力为 2.0 的设备之前，这实际上是溢出到全局内存空间中。在计算能力 2.0 及后续版本中，尽可能使用一级缓存。但仍然会导致不必要的全局内存流量。

编译过程中，-v 选项会显示内核函数使用寄存器的一个摘要。任何时候当你看到下面的消息时，说明你正在使用本地内存：

```
nn bytes stack frame, nn bytes spill stores, nn bytes spill loads
```

这里的主要问题是使用了 uint4 类型。结合在别处大量寄存器的使用，从全局内存加载的 uint4 会立即溢出到本地内存。每次读取 128 位的 uint4 是精心选择的，以便使访问全局内存的次数达到最低。无法保持在寄存器中的数据溢出到本地内存，编译器将对缓存造成污染，导致数据写回全局内存。

我们可以通过将其声明为一个 __shared__ 类型的数组和并使用 threadIdx.x 索引，明确地将其移动到共享内存，而不是本地内存中。共享内存是每个线程块的本地内存，我们可以将溢出的寄存器明确地放到共享内存中。移动此参数，生成的内存视图如图 7-17 所示。

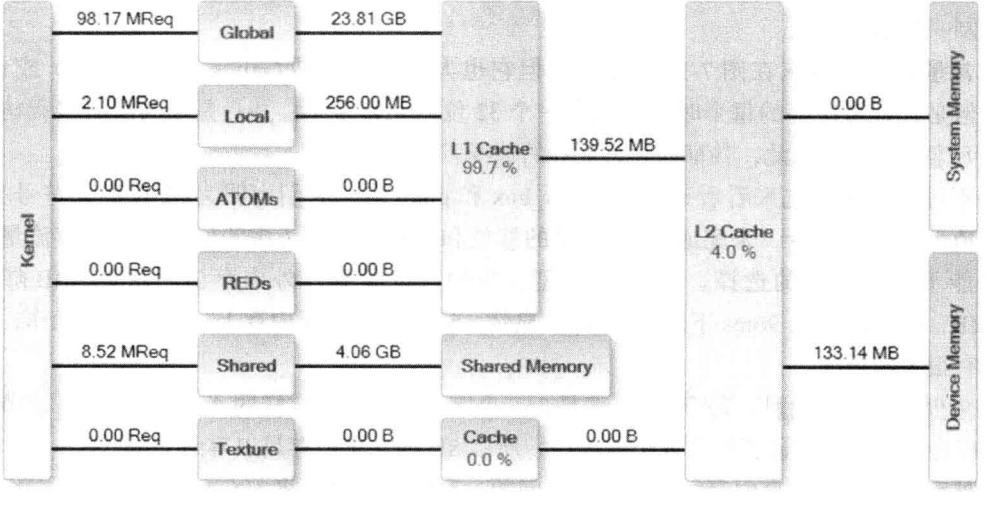


图 7-17 使用共享内存后的内存传输视

请注意，简单地将数据移到共享内存中会使本地内存的使用从 1.91GB 降到只有 256MB，全局内存流量从 183MB 降低到 133MB。我们的共享内存流量比它以前的一级缓存流量增加约一倍，这在很大程度上是由于共享内存的存储片冲突。这些都是由将一个 128 位（16 字节）的值放置到一个 32 位（4 个字节）的共享内存系统中引起的。编译器仍然坚持创建一个栈帧，虽然会比以前更小，但它仍然存在。总的执行时间仍然固执地停留在 63ms。

为了查看究竟是什么参数溢出，我们需要查看 PTX 代码，它们是来自给定内核的汇编代码。任何 PTX 指令，如 st.local 或 ld.local 都是操纵本地数据。本地数据也是以 local 作为前缀的。它表明其余的本地数据实际来自 __global__ 调用者和 __device__ 函数之间使用的参数数据。

```
__global__void AES_decode_kernel_multi_block_single_key(uint4 * const src_block,
                                                         uint4 * const dest_blocks,
                                                         KEY_T * const expanded_key,
                                                         const u32 num_cipher_blocks)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_cipher_blocks)
  AES_decrypt_4x4_reg( &(src_block[tid]),&(dest_blocks[tid]),&(expanded_key[0]) );
}

__host__ __device__ void AES_decrypt_4x4_reg(
 const uint4 * const src_cipher_block,
 uint4 * const dest_cipher_block,
 KEY_T * const cipher_key)
{
...
}
```

事实上，我们已经传递了许多参数到设备函数上，这反过来又允许它被一些主机函数和全局函数调用，使编译器插入一个栈帧。我们很少希望编译器调用栈，而是希望它内联该调用到设备函数中，从而消除了任何使用栈的需求。当声明该函数时，我们可以直接使用 __forceinline__，如下所示：

_forceinline__ directive when declaring the function as shown here:

```
__host__ __device__ __forceinline__ void AES_decrypt_4x4_reg(
 const uint4 * const src_cipher_block,
 uint4 * const dest_cipher_block,
 KEY_T * const cipher_key)
{
...
}
```

重新编译代码不再产生栈帧信息。由于函数现在是一个紧密的整体，因此编译器可以把更好的优化技术应用到它上面。寄存器的使用下降到只有 33 个，而不是像先前那样通过强制使用 42 个寄存器达到启用 8 个线程块。通过查看图 7-18 中的内存概要信息，我们可以验证本地内存不再被使用。

我们可以看到，在图 7-18 中，本地内存流量下降到零。少量的二级缓存使用也消除了。全局内存的使用量又下降了 5MB，总使用量为 128MB，鉴于我们所处理的数据量，这正是我们期望的、最理想的全局内存带宽。执行时间减少一些，但仍保持在 63ms。

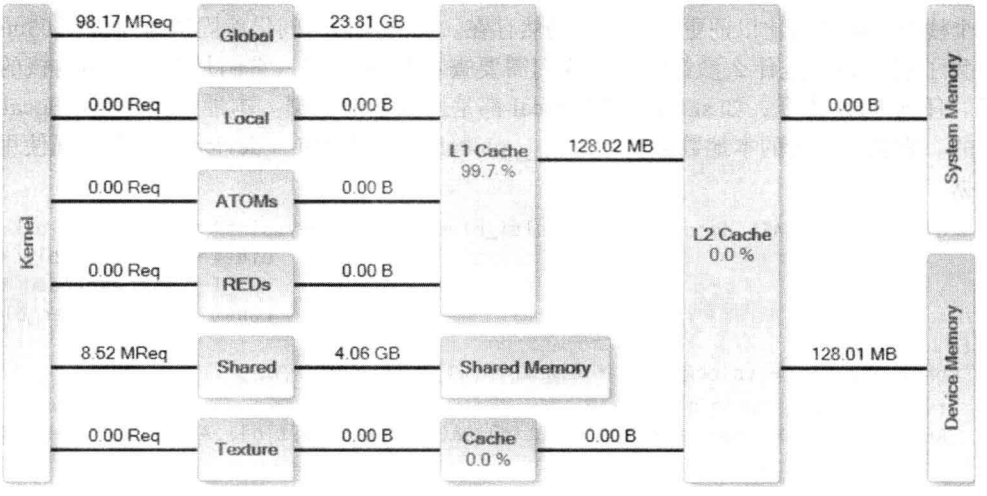


图 7-18 消除栈使用后的内存使用

内核函数会导致大量使用异或操作，这一指令在设备上是无法全速使用的。因此，通过确保我们开启了 SM 的最大线程块数，我们确保了良好的指令混合，也确保在执行异或操作时一切都没有倒退。

在之前的情形，即每线程块使用 96 个线程，每个线程占用 42 个寄存器，我们可以安排 8 个线程块，共计 24 个线程束。就 SM 可以运行的线程束数目而言，这大约是 SM50%的可用容量。但是，我们从 Parallel Nsight 的“Issue Stalls”（事件失速）实验中可以看到，实际上正在使用的 SM 容量。这里的失速仅有 0.01%的时间，这表示 SM 已经几乎达到峰值容量。因此，通过增加线程束数目来增大占用率的做法，很难有显著的帮助。线程数从 96 增加到 128，使我们能够将可用的线程束数量从 24 增加到 28。这消除了余下部分的少量失速问题，同时增加了两个线程束调度器存在足够线程束供应的时间，我们又获得了 1.5ms 的提速。这样总的执行时间为 61.5ms。

7.5.8 考虑在其他 GPU 上运行

已经开发出的针对单一现代 GPU 的程序，它会在其他 GPU 上工作的如何呢？通常情况下，特别是当你正在编写商业应用程序时，你的程序需要在市场中每个级别的硬件上很好地工作。虽然程序将运行在大多数 GPU 上，但你应该知道采取什么策略能在该硬件上达到良好的性能。在已经开发的 AES 程序中，我们将对此进行检查。

我们第一个目标是 GTX460 卡，它是基于费米架构的计算能力 2.1 的设备。主要的差异包括计算能力 2.1 架构（7SM × 48CUDA 核与 14SM × 32CUDA 核）、变小的二级缓存（512K

与 640K），减少的每 CUDA 核的一级缓存（48K 一级缓存共享于 48 个 CUDA 核与 48K 一级缓存共享于 32 个 CUDA 核），以及降低了的内存带宽（115GB/ 秒与 134GB/ 秒）。

仅根据总体 CUDA 核数量（336 个与 448 个），我们预计大约是之前 75%的性能。然而，通过平衡时钟速度上的差异，这两个设备之间的性能差异不会超过 10%。GTX460 的内存带宽下降了 15%。

解密函数的实际测量时间是 100ms，跟之前的 61.5ms 相比有点令人失望。通过分析执行性能文件，我们看到 GTX460 上的 SM，每个时钟可执行更多的指令。因此，所需的数据变得可用的时刻发生了变化。我们同时看到，SM 中有少量的失速。每个线程块使用 128 个线程，我们成功地让 7 个线程块得到调度机会（28 个线程束）。如果我们能稍微减少寄存器的使用，则可以另外执行一个线程块并更充分地利用 SM。因此，我们采用在编码操作时使用的相同技术，并且将逆混合列操作移到与解码操作更接近的地方。因此，下面的代码：

```
// Add Round Key
{
 // Fetch cipher key from memory as a 128 bit read
 *key_ptr = ((*cipher_key)[round_num]);

 // Extract the key values, XOR'ing them with
 // the current values
 EXTRACT_WORD_XOR2((key_ptr->x), a0, a1, a2, a3);
 EXTRACT_WORD_XOR2((key_ptr->y), a4, a5, a6, a7);
 EXTRACT_WORD_XOR2((key_ptr->z), a8, a9, a10, a11);
 EXTRACT_WORD_XOR2((key_ptr->w), a12, a13, a14, a15);
}

// Invert Column Mix on every round except the first
if (round_num != 10)
{
 INV_MIX_COLUMN_PTR2(a0, a1, a2, a3,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

 INV_MIX_COLUMN_PTR2(a4, a5, a6, a7,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

 INV_MIX_COLUMN_PTR2(a8, a9, a10, a11,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

 INV_MIX_COLUMN_PTR2(a12, a13, a14, a15,
            gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);
}
```

变为

```
// Add Round Key
{
 // Fetch cipher key from memory as a 128 bit read
 *key_ptr = ((*cipher_key)[round_num]);
```

```
    // Extract the key values, XOR'ing them with
    // the current values
    EXTRACT_WORD_XOR2((key_ptr->x), a0, a1, a2, a3);
    if (round_num != 10)
     INV_MIX_COLUMN_PTR2(a0, a1, a2, a3,
                gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

    EXTRACT_WORD_XOR2((key_ptr->y), a4, a5, a6, a7);
    if (round_num != 10)
     INV_MIX_COLUMN_PTR2(a4, a5, a6, a7,
                gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

    EXTRACT_WORD_XOR2((key_ptr->z), a8, a9, a10, a11);
    if (round_num != 10)
     INV_MIX_COLUMN_PTR2(a8, a9, a10, a11,
                gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);

    EXTRACT_WORD_XOR2((key_ptr->w), a12, a13, a14, a15);
    if (round_num != 10)
     INV_MIX_COLUMN_PTR2(a12, a13, a14, a15,
                gmul_14_ptr, gmul_09_ptr, gmul_13_ptr, gmul_11_ptr);
}
```

这种操作的合并允许寄存器的使用下降到神奇的 31 个，从而使我们能够另外调度一个线程块，从而每个 SM 包含 32 个线程束。相比计算能力 2.0，计算能力 2.1 的设备具有更高的计算与加载单元比率，这对后者来说，是种补偿。我们看到了从 100ms 到 98ms 的小幅降低。然而，我们的计算能力 2.0 设备（GTX470）已经全部利用了它的计算核。这种变化，引入了更多的测试，花费 0.5 毫秒。在计算能力 2.0 的设备上，我们倒退到 62ms。有时你可能会发现，在一个 SM 中执行单元的平衡是不同的，尤其是对计算能力 2.0 或 2.1 的设备。

第二个目标是 GTX260，它是计算能力 1.3 的设备。这里最为重要的区别是 GTX260 完全缺乏一级和二级缓存。其他方面：SM 架构上有 27 个 SM（相较 14 个）；CUDA 核总数含 216 个（相较 448 个 CUDA 核）；内存带宽是 112GB/s（相较 134GB/s，约少 16%），与 GTX460 相同。

解码函数的初始运行耗费 650ms，比 GTX470 慢 10 倍以上。为什么？其中一个关键的原因是计算能力 1.x 的平台不支持统一的寻址模式。因此，预期的内存使用量必须要显式声明。针对 gmul 表的情况，它们通过一个小的计算内核在设备上生成。同样的，这些表存在于全局内存中。在计算能力 2.x 的平台上对全局内存进行缓存，而在计算 1.x 的平台必须显式地让它可以进行缓存。为此，我们有几种实现方式。

首先，我们需要指定，用于 gmul 的内存是常量类型，反过来这又意味着，我们不能从设备上改写它。由于我们在主机端有一份数据，我们可以通过调用 cudaMemcpyToSymbol 将它复制到该设备或简单地在设备上将它声明为常量内存并静态地初始化它。因此，计算 gmul 表的代码被替换为一个易于扩充定义的查找表。把它存放在常量内存中。重新运行该代码，我们看到从 650ms 降到 265ms，下降近 60%的执行时间。然而，GTX260 仍然比 GTX470 慢

4.2 倍，比 GTX460 慢 2.7 倍。

最后，老的 GT9800 卡约有 GTX260 一半数量的 CUDA 核和一半的内存带宽。正如所预期的，它的运行时间约是 265ms 的 2 倍（实际为 1.8 倍），达到 478ms。

GTX260 和 GT9800 共有的问题在于数据组织。让数据与 AES-NI 相匹配，意味着单个密钥值的数据在内存中顺序排列。为了实现更好的性能，我们需要组织内存，以使每个来自密钥的连续 32 位值以列的形式保存在内存中，而不是以行的形式。通常，对 CPU 理想的顺序排列方式对 GPU 来讲可能并不理想。

AES 加密 / 解密的实际输出如下所示：

```
Intel AES NI support enabled.
Logical CUDA device 0 mapped to physical device 0.Device ID: GeForce GTX 470 on PCI-E 5
Logical CUDA device 1 mapped to physical device 1.Device ID: GeForce GTX 470 on PCI-E 4
Logical CUDA device 2 mapped to physical device 2.Device ID: GeForce GTX 470 on PCI-E 3
Logical CUDA device 3 mapped to physical device 3.Device ID: GeForce GTX 470 on PCI-E 2
Logical CUDA device 4 mapped to physical device 4.Device ID: GeForce GTX 470 on PCI-E 1

test_single_block_single_key_encode_decode
AES NI Key           :  2b, 7e, 15, 16, 28, ae, d2, a6, ab, f7, 15, 88, 09, cf, 4f, 3c,
AES NI Plaintext     :  6b, c1, be, e2, 2e, 40, 9f, 96, e9, 3d, 7e, 11, 73, 93, 17, 2a,
AES NI Ciphertext    :  3a, d7, 7b, b4, 0d, 7a, 36, 60, a8, 9e, ca, f3, 24, 66, ef, 97,
Expected Ciphertext  :  3a, d7, 7b, b4, 0d, 7a, 36, 60, a8, 9e, ca, f3, 24, 66, ef, 97,
Single block single key AES-NI decode Passed
GPU Intial Key       :  16157e2b,  a6d2ae28,  8815f7ab,  3c4fcf09,
GPU Plaintext        :  e2bec16b,  969f402e,  117e3de9,  2a179373,
CPU Ciphertext       :  b47bd73a,  60367a0d,  f3ca9ea8,  97ef6624,
GPU Ciphertext       :  b47bd73a,  60367a0d,  f3ca9ea8,  97ef6624,
Expected Ciphertext  :  b47bd73a,  60367a0d,  f3ca9ea8,  97ef6624,
Single block single key serial decode Passed
Single block single key parallel decode Passed
Single block single key parallel decode and AES-NI match Passed

Encrypting on GPU         :  4194304 blocks (32768 Blocks x 128 Threads)
Encrypt Copy To Device    :  28.469ms
Encrypt Expand Key Kernel :   0.025ms
Encrypt Encode Key Kernel :  45.581ms
Encrypt Copy From Device  :  25.428ms
Encrypt Total Time        :  99.503ms
Encrypting on CPU        :  4194304 blocks
Encrypt Encode           :  3900.000ms
Encrypting using AES-NI  :  4194304 blocks
Encrypt Encode           :  20.000ms
CPU and GPU encode result Passed.
CPU and GPU AES-NI encode result Passed.

Decrypting on GPU         :  4194304 blocks (32768 Blocks x 128 Threads)
Decrypt Copy To Device    :  27.531ms
Decrypt Expand Key Kernel :   0.028ms
```

```
Decrypt Decode Key Kernel :  62.027ms
Decrypt Copy From Device  :  25.914ms
Decrypt Total Time        : 115.500ms

Decrypting on CPU  :  4194304 blocks
Decrypt Decode     :  2760.000ms

Decrypting using AES-NI  :  4194304 blocks
Decrypt Decode           :  20.000ms
CPU and GPU decode result Passed.
CPU and AES-NI decode result Passed.
```

注意，对加密部分，我们已经设法让其不超过AES-NI硬件运行时间的2倍，对应的解密部分大约在3倍以内。我们这里使用的GTX470是跟常规沙桥CPU硬件的时间相匹配的，而无法跟更先进的沙桥 –E CPU相比较。基于常规沙桥硬件的AES-NI的性能大约为沙桥 –E的一半。如果比较GPU执行的时间与沙桥上AES-NI的时间，大概处于同一级别。而实际上，基于开普勒架构的GTX680才是能与沙桥 –ECPU配对的典型设备。这将带来2倍量级的性能提高。这样，GPU上的性能就能与基于硬件的AES-NI上的性能旗鼓相当。

支持什么GPU的议题是棘手的。在消费级市场上有很多古老的GPU，如果你面向的是消费级市场，那么你的应用程序必须能在老GPU上运行良好。而在大范围使用条件下，但就能耗账单而言就没有必要继续使用老显卡，尽早更换为新卡更合算。开普勒的出现将极大加速淘汰较旧的Tesla板。

如果你需要支持旧的硬件，那么最好的办法是在该硬件上开发。然后对后生代显卡也会有较好的表现。对这些老卡所做的优化在后生代卡上可能获益没那么明显。然而，几乎所有的优化都会表现出一定的好处，只是你即刻收到什么成效不太确定。

7.5.9 使用多个流

在第8章里提供了一个多个流和多个流 / 多GPU编程的例子。因此我们不再详细介绍如何实现这个算法的多流版本。然而，我们将要讨论一些实现这个算法或者实现算法过程中需要思考的问题。

由于允许重叠进行内核执行与PCI-E传输，多个流是有用的。但它们的用处严重地受限于PCI-E传输引擎的数目。在消费级显卡上，仅启用了一个引擎。只有特斯拉系列卡的双传输引擎均启用了，使得可以同时进行双向传输。

我们通常想传输数据到显卡上，处理其中一些数据，然后把数据从显卡上传输出去。在仅启用单个PCI-E传输引擎的情况下，我们对硬件里所有的内存传输都用单个队列。尽管处于不同的流中，内存传输请求在费米和更早的硬件上，存入单个队列。因此，典型的主机和设备间的工作流模式（分别经历从主机传入设备、调用内核、从设备传回主机）会存在失速问题。从设备传回的操作会阻塞从下一个流中传入设备的操作。因此，所有的流实际上按顺序运行。

使用多个流时，下一个我们需要考虑的问题资源使用情况。你需要 N 组主机和设备的内存，其中 N 是你想要运行的流的数量。当你有多个 GPU 的时候，这会很有意义。因为每个 GPU 都非常有助于整体的结果。然而，仅使用单个消费级 GPU 卡，其收益不容易量化。如果 GPU 工作量在输入输出上比较均衡，或者整体传输时间小于内核执行时间，单卡能较好地运行。

在我们的应用程序中，传入了一组待编码为单个密钥集的数据块，传出了编码完成的数据块。传入和传出的大小几乎相同。内核执行的时间大约是传输时间的两倍。这意味着我们有机会隐藏数据输入阶段的传输时间，用户只付出输出阶段的传输时间。

单个 GPU 可以支持多达 16 个硬件流（开普勒架构是 32 个），所以在设备和主机的内存界限内，它能够支持 16 个传入操作、16 个内核执行操作和 16 个传出操作。如第 9 章将要讨论的，传输导致的问题更明显。由于主机上的资源竞争，跨越 PCI-E 总线的并行传输越多，本身所用的传输时间也会变得越长。

7.5.10 AES 总结

这里有一些有关 AES 的问题，值得总结一下。

- 针对 CPU 与 GPU 的理想内存模式是不同的。优化 GPU 的访存模式收益甚大，费米架构上通常提速 1 倍以上，对更早的 GPU，内存模式的优化更为关键。
- 对于计算能力 1.X 设备，申请只读内存需显式声明为常量内存，而不是由编译器自动指定。
- 为了使编译器更容易自动优化性能，内核程序的重新排序或者变换是必不可少的。
- 有效的寄存器使用方式以及合适的使用数量，对高性能的 CUDA 程序设计至关重要。
- 你可以通过一级缓存在线程块间共享只读数据，也可以使用共享内存保存这些数据。但后者需要保存 N 份数据，这里 N 是可用的线程块数。
- 复杂的、含线程分支的算法，如 gmul 函数的解码，可以使用没有线程分支的内存查找代替，查找表驻留于缓存或共享内存。缓存就是专门为此类数据驱动方式的分散内存访问模式而增添的。
- 确认变量的分配没有溢出寄存器，尽早消除栈或本地内存的使用。
- 务必尽早在解决方案之处验证其正确性。最好相互独立地开发代码的不同版本。
- 务必查看程序的实际执行时间。你的心智模型 (mental model) 关于事情运转方式的认识未必正确，常常会忽略一些东西。经常查看数据，观测每次变化后的效果。

7.6 本章小结

我们已经考察了 GPU 技术的几个应用程序，刻意避免在很多 CUDA 编程示例中讨论泛滥的简单矩阵乘法。我们着眼于利用 GPU 来过滤数据，不论是从寻找数据中有趣的事实这一角度看，还是从一个纯粹的信号处理的角度看，这都是非常有用的。我们也讨论了，如何

在GPU上实现一个标准的AES加密算法。即使你不需要在CUDA中实现它，也应该对这个算法和其实现过程有所理解，并乐于去使用它。

你应该通过具体操作过程，学会面向多种计算能力的硬件时的权衡之术和设计要点。你也可以看到，项目开发过程中早期的设计决策是如何影响到结果的。在你开始书写代码之处，你应该全盘考虑和理解寄存器、共享内存、缓存的用法以及全局内存的访问模式等设计的主要方面。

今天的程序员有一个最大的弱势：他们对程序所依赖的硬件缺乏了解。要实现优异的性能表现，而不只是平均表现，必须要理解，而且是彻底理解，你正在开发的环境。诸如不同内存层次结构的概念在传统编程语言中并不存在。C语言发明于20世纪70年代初，直到C11标准（在2011年），我们才看到了线程和本地线程存储概念的出现。CUDA和其母语C语言所遵循的原则是信任程序员。它会把很多硬件资源暴露给你，因此，你有责任去理解它们的特性并充分用好它们。

利用已经介绍的几个例子，我们后面将继续讨论多GPU的使用和应用程序的优化策略。只需在单个节点PCI-E总线上插入更多的显卡并将我们的应用改写为多GPU的，我们就能实现大规模的加速。开普勒架构的Tesla K10产品是第一Tesla型号的双GPU解决方案。在未来几年，也许我们可以看到很多类似方案。多GPU编程，在CUDA 4.0版本以后，其实并不难。你会在随后的章节看到。

问题

1. AES应用程序在GTX260和GT9800卡上明显慢于GTX460和GTX470卡的主要原因是什么？你会怎样来解决这个问题呢？
2. 在AES应用程序中，把s_box和gmul表从u32改变为u8，为什么会提高性能？
3. 什么是线程级并行？它是否有帮助？如果有帮助，为什么呢？
4. 使用原子操作，会带来什么问题？

答案

1. GTX260和GT9800卡的计算能力分别是1.3和1.1的。因此，它们有没有一级或二级缓存。只有到计算能力2.x，才有这两种缓存。在内存视图中显示，我们使用的一级缓存有99%的命中率。从一级缓存溢出到全局内存，意味着我们从万亿字节级带宽降低到数亿字节级带宽。

内存合并上也有根本性改变。计算能力2.x的硬件获取内存数据存入128字节的缓存行。如果某线程读取一个单独的128字节的值，例如uint4，硬件可以提供此项服务。而在计算能力1.x的硬件上，内存合并的要求更为严格。

目前编译的uint4类型对算法有副作用。在计算能力2.x的硬件上，一个4字向量加载操作会紧随于一个4字向量的共享内存存储操作。在计算能力1.x的硬件上，CUDA 4.1编译器生成的代码，会独自加载每个32位的字，从而在每个方向上都产生4倍的流量。加密的

密文数据需要按照特定形式存放，以便达到内存合并。

常量缓存是有帮助的。移除 uint4 类型的共享内存变量，代以寄存器类型的 u32，并把 gmul 和 s_box 表分配在共享内存里会更为有利。还应该考虑到，纹理缓存是老设备上难得的重要资源，值得花费功夫去研究和尝试。

2. s_box 和 gmul 表的访问采用数据依赖的模式。我们总共有 4 个表，其中每一个表包含 256 个条目。使用 u8 类型，意味着我们使用 5K 的内存，该内存是能够放入一级缓存和常量缓存的。使用 u32 的值会移除一些 cvt（转换类型）指令，但一级缓存或常数缓存中数据的移动增加为 4 倍。额外的计算开销比移动这么多数据更合算。作为 u32 类型，缓存需要存储 20K 的数据，超过了一级缓存通常的 16K 容量和常量缓存的 8K 工作集。

3. 线程级并行充分利用了大多数硬件的流水线结构，从而能够在连续的时钟里，接受非依赖性指令，而不会发生阻塞。每个线程使用 4 个独立的条目，通常能较好地实现线程级并行，我们将在第 9 章讨论。

4. 有两个主要问题要考虑。首先，原子操作，如果超量使用，会导致串行化。因此，一个线程束中 32 个线程向相同的内存地址执行写入操作，不管是共享内存还是全局内存，都将导致串行化。原子操作，至少在费米架构上，是线程束宽度的操作。因此，如果线程束中的每个线程的原子操作针对的寻址位置互相独立，将导致 32 个原子操作而不会产生串行化。

第二个问题是原子写入顺序。如果线程束中所有线程都要写入同一个地址，那么操作的顺序是没有定义的。对于某个给定的设备，你可以观察到明显的顺序，而且顺序很可能保持一致。然而，换一个设备，其顺序就可能不同。因此，在需要利用这种知识时，你要在你的应用程序中建立一个故障点。

参考文献

Wikipedia, Rijndael Mix Columns. Available at: *http://en.wikipedia.org/wiki/Rijndael_mix_columns*, accessed Jan. 31, 2012

Federal Information Processing Standards Publication 197, Advanced Encryption Standard (AES). Available at: *http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf*, accessed Feb. 5, 2012.

Toms Hardware, AES-NI Benchmark Results: Bitlocker, Everest, and WinZip 14. Available at: *http://www.tomshardware.co.uk/clarkdale-aes-ni-encryption, review-31801–7.html*, accessed Apr. 26, 2012

第 8 章

多 CPU 和多 GPU 解决方案

8.1 简介

在现代计算系统中通常有多个设备：CPU 和 GPU。在 CPU 方面我们将讨论卡槽和核。卡槽是指主板上用来放置 CPU 的物理插槽。一个 CPU 可能包含一个或多个核。每个核实际上是一个独立的实体。许多 CPU 和 GPU 卡槽都位于单个节点或计算机系统。

了解核、卡槽以及节点的物理布局，会使得我们更有效地调度和分配任务。

8.2 局部性

局部性原理用在 GPU 和 CPU 上是相当不错的。接近设备的内存（GPU 上的共享内存或 CPU 上的高速缓存）会更快地访问。一个卡槽内（比如核之间）的通信远远快于和不同卡槽上的另一核的通信。访问另一个节点核的通信方式至少比该节点内的访问慢一个数量级。

显然，拥有能够意识到这一点的软件可以对任何系统的整体性能产生巨大影响。这种卡槽感知软件可以按照硬件布局分割数据，确保一个核在一致的数据集上工作，并且需要多个核合作于同一个卡槽或节点上。

8.3 多 CPU 系统

最常见的多 CPU 系统是单插槽、多核台式机。如今你购买的任何 PC 都将有一个多核 CPU。即使是笔记本电脑和媒体 PC，你也会发现多核 CPU。如果我们看一下 Steam 定期的硬件（消费级 / 游戏级）调查报告，该调查表明在 2012 年约 50% 的用户有双核系统，另外 40% 用户拥有 4 核或更高级的系统。

你能遇到的第二种类型的多 CPU 系统是工作站和低端服务器。这些通常是双插槽的机器，通常是由多核 Xeon 或 Opteron 处理器驱动的。

最后一种你能遇到的多 CPU 系统是数据中心的服务器，它通常有 4、8 或 16 个插槽，

每一个插槽都有一个多核 CPU。这样的硬件通常被用来创建一个虚拟的机器集合，使企业能够通过一个大的服务器集中支持大量的虚拟 PC。

任何多处理器系统的主要问题之一是内存的一致性。CPU 和 GPU 将内存分配给单个设备。对于 GPU，这是每个 GPU 卡上的全局内存。对于 CPU，这是主板上的系统内存。

当你有独立的程序时可以只使用单一核，使用这种方法能够很好地进行扩展，因为每个程序都可以限制在一个给定的核。然后程序访问自己的数据，并能充分利用 CPU 核的缓存。然而，当你需要两个核互相合作的时候，就会遇到问题。

为了加快内存的访问速度，CPU 大量使用高速缓存。一个参数的值被更新时（例如，x++），x 真地被写入内存了吗？假设两个核都需要更新 x，因为一个核分配给一个债务处理任务和另一个核分配给信用处理任务。两个核从存储参数 x 的内存位置读出的内容必须是一致的。

这是缓存一致性问题，就是这个问题限制了可以在单个节点上实际合作的核的最大数量。实际上硬件是这样工作的：当核 1 写入 x 时，它会通知所有其他的核 x 的值已经改变，然后缓慢地写到主存中，而不是快速回写到高速缓存。

在一个简单的一致性模型中，其他核标记条目表明 x 在它们的缓存中无效。下一次访问 x 时，会从缓慢的主内存中读取 x 值。随后的核写入 x 时，重复该过程，下一个核访问参数 x 必须再次从主内存中读取并且重新写回。实际上，参数 x 没有被缓存，这对于 CPU 意味着巨大的性能损失。

在更复杂的一致性模型中，用更新请求替换无效请求，而不是替换无效的参数 x。因此，每一个写操作被分配到 N 个高速缓存中。随着 N 的增长，同步这些缓存所用的时间变得不切实际。这往往限制了可以放入对称多处理器（SMP）系统的节点的实际数目。

现在请记住，高速缓存都应该高速运行。在一个单独的卡槽中，这是不难的。然而，只要你用到多个卡槽，高时钟速率难以维持，因此一切都开始放缓。用的卡槽越多，保持一切同步就变得更加困难。

下一个重要的问题是内存访问时间。为了使在这样的机器中编程更容易，往往内存在逻辑上被安排成一个巨大的线性地址空间。然而，只要卡槽 1 的核试图访问卡槽 2 的一个内存地址，就必须要卡槽 2（作为唯一可以实际访问该内存地址的卡槽）提供服务。这就是所谓的非一致性内存访问（Nonuniform Memory Access，NUMA）。尽管在概念上，它使程序员工作更轻松，但是在实践中，你还是需要考虑内存地址的问题，否则你写的程序执行速度会非常缓慢。

8.4 多 GPU 系统

就像 CPU 一样，现在有很多系统都是多 GPU 的。从拥有 3 或 4 “速力”（Scalable Link Interface，SLI）系统或类似 9800GX2、GTX295、GTX590 和 GTX690 的爱好者，到用专用的 GPU 显卡升级他的低功耗的 ION 台式机的人，许多人都有多 GPU 系统。作为一个程序

员，你应该总是努力生产出在任何硬件上都能运行的、最好的产品。

如果用户有一个双GPU系统，而你只使用一个GPU，那么你就和那些不愿意学习如何使用一个以上核的CPU程序员一样懒。有很多程序监视GPU负载。精通技术的用户或审查员会批评你的产品，因为你没有进一步地改进、提升。

如果你正在编写科学应用程序或者在已知的硬件上工作，而不是消费者应用程序，你也应该调查多GPU解决方案。几乎所有的个人电脑都支持至少两个PCI-E插槽，这允许几乎所有的PC都至少能有两个GPU显卡。CUDA不使用也不需要“速力”连接器，所以没有“速力”认证的主板不阻碍在CUDA应用程序中使用多个GPU。添加一个额外的GPU卡，你通常会看到性能水平增加了一倍，当前执行时间减半。你很少能这么容易地获得这样的加速。

8.5 多GPU算法

CUDA环境本身不支持多GPU模式的合作。该模型更多的是基于单核、单GPU的关系。对于相互独立的任务。这一点确实很好。但是，如果你想编写一个需要GPU以某种方式合作的任务，这却是很痛苦的。

这种模式能让如BOINC的应用程序运行良好。BOINC是这样一个应用程序，它允许用户贡献出闲置的计算能力来解决世界性的问题。在一个多GPU系统上，它产生N个任务，其中N等于系统中GPU的数目。每个任务从一个中央服务器获得一个单独的数据包或工作。当GPU完成任务时，它只是简单地要求从中央服务器（任务调度程序）中得到额外的任务。

现在，如果你看看不同的例子，比如说我们需要合作，那么情况就不一样了。在最简单的层次，我们通常通过将JPEG型算法应用到每一帧，然后再寻找帧与帧之间的运动矢量完成视频编码。因此，我们对帧的一个操作可以分布到N个GPU中，但随后的操作需要GPU共享数据，并依赖于第一个任务（JPEG压缩）的完成。

有几种方式处理这个问题。最简单的方法是使用两个内核程序，一个内核简单地做N个独立帧的JPEG压缩，第二个内核以运动矢量的分析为基础进行压缩。我们之所以可以这样做是因为基于运动矢量压缩使用有限的窗口帧，所以帧1不影响帧1000。因此，我们可以把工作分割成N个独立的部分。与其他的多遍算法一样，这种方法的不足之处在于我们需要多次读取数据。由于数据集通常是相当大的，这将涉及速度较慢的大容量存储设备，因此这通常不是一个好的办法。

单通的方法更加有效，但是编程却更加困难。我们可以通过把做运动向量压缩的帧看作数据集，从而将问题进行转化。每个帧组是独立的并且可以被调度到一个单独的GPU显卡。GPU内核首先把提供给它的所有帧做JPEG压缩。然后对那些相同的帧，做运动矢量计算。通过使用这种方法，你就能成功地将数据保存在GPU显卡上。这种做法消除了这类问题的主要瓶颈——系统中的数据移动。

在这种情况下，我们重组算法使它可以被分解成独立的数据块。然而这并不总是可能的，许多类型的问题需要至少从其他GPU中获得少量数据。当你需要另一个GPU的数据，